

Fiches de Cours

Master IMA

Axel PIGEON

axel.pigeon@univ-tlse3.fr

Université Toulouse III – Paul Sabatier

Années universitaires 2025–2027

20 septembre 2026

Table des matières

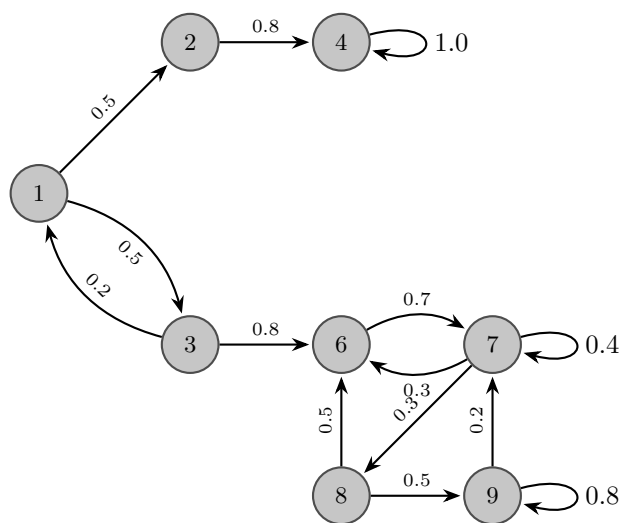
I Probabilités	6
1 Chaînes de Markov	7
1.1 Définitions et Généralités	7
1.2 Propriétés de Markov	12
1.3 Mesures Invariantes	15
1.4 Classification des états	22
1.5 Mesures invariantes dans le cas dénombrable	25
1.6 Ergodicité et convergence en loi	29
2 Espérances Conditionnelles	31
2.1 Espérances conditionnelles sur un espace d'états discret	31
2.2 Définition de l'espérance conditionnelle	33
2.3 Propriétés de l'espérance conditionnelle	35
2.4 Processus aléatoires - Filtrations	37
3 Martingales	39
3.1 Généralités	39
3.2 Stratégies et théorème d'arrêt	43
3.3 Inégalités maximales	46
3.4 Convergence des martingales	47
II Simulation Aléatoire	50
1 Simulation de variables aléatoires	51
1.1 Simulation de variables aléatoires discrètes	52
1.2 Simulation de variables aléatoires à densité	55
1.3 Méthodes ad-hoc	59
2 Simulation de chaînes de Markov	66
2.1 Généralités	66
2.2 Comportement asymptotique d'une chaîne de Markov	69
3 Processus de Markov en temps continu	74
3.1 Lois exponentielles et lemme des révéls	75
3.2 Processus de Markov en temps continu	78
4 Algorithmes de Metropolis-Hasting et Recuit Simulé	83
4.1 Cas d'un espace d'états fini	83
4.2 Cas d'un espace d'états continu	86
4.3 Mesures de Gibbs sur un espace d'états fini	87

4.4	Algorithme du recuit simulé	88
5	Algorithme de Robbins Monro	90
5.1	Cadre Général	90
5.2	Convergence de l'algorithme	91
III	Statistiques	96
1	Théorie de l'Information	97
1.1	Cadre	97
1.2	Informations Qualitatives	102
1.3	Information de Kullback-Leibler	104
1.4	Informations Quantitatives	104
2	Estimation et Risque	108
2.1	Risque	108
2.2	Estimateurs	109
2.3	Risque Quadratique	112
2.4	Bornes inférieures de variance et Efficacité	114
IV	Algorithmique	116
1	Flot maximal dans un graphe	117
1.1	Rappels sur les graphes	117
1.2	Cycle et flot	119
1.3	Flot et coupure	120
1.4	Flot maximal : principe de Ford-Fulkerson	122
2	Programmation Linéaire	124
2.1	Programmation Linéaire	124
2.2	Programmation Linéaire en Nombres Entiers	133
3	Méta-heuristiques, CSP et SAT	137
4	Théorie de la complexité	138
4.1	Généralités	138
4.2	Problèmes de décisions, classes P, NP et NP-complet	139
V	Apprentissage Automatique	141
1	Fondements, Réseaux Denses et Optimisation	142
1.1	Généralités	142
1.2	Du Perceptron au MLP	144
1.3	Fonctions d'Activation et Classification Multiclasse	146
1.4	Fonctions de Perte (<i>Loss Functions</i>)	147
1.5	Rétropropagation du Gradient et Dérivation Automatique	147
2	Réseaux Convolutifs	150
2.1	L'Opérateur de Convolution en Vision Artificielle	150
2.2	Sous-Échantillonnage, Champ Réceptif et Calcul	151
2.3	Techniques de Normalisation et Réseaux Résiduels	152

3 Réseaux Récurrents	154
3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN)	154
3.2 Rétropropagation à travers le Temps (<i>BPTT</i>) et Pathologies	155
3.3 Cellules à Portes de Contrôle : LSTM et GRU	155
3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention	156
4 Modèles de Markov Caché	158
4.1 Modèles de Langage et Hypothèses Markoviennes	158
4.2 Formalisme des Modèles de Markov Cachés (HMM)	160
4.3 Résolution Algorithmique des Trois Problèmes de Rabiner	161
VI Mathématiques du Machine Learning	167
1 Introduction à l'Apprentissage Supervisé	168
1.1 Des données et des prédictions	168
1.2 Formalisation mathématique	169
1.3 Risque et prédicteur de Bayes	170
1.4 Paradigmes classiques d'apprentissage	171
2 Régression Linéaire et Régression Ridge	175
2.1 Moindres Carrés	175
2.2 Analyse statistique – design fixe	176
2.3 Régression Ridge – design fixe	178
3 Méthodes à Noyaux	180
3.1 De la similarité au Théorème du Représentant	180
3.2 Noyaux définis positifs, Théorème de Bochner, et Théorème d'Aronszajn	182
3.3 Séparateurs à Vaste Marge (SVM)	186
4 Théorie de l'Apprentissage et Minimisation du Risque Empirique	189
4.1 Convexification du risque et substituts convexes	189
4.2 Décomposition du Risque et Classes Finies	191
4.3 Complexité de Rademacher	195
VII Informatique Graphique	197
1 Modélisation Géométrique	198
1.1 Cadre mathématique et prérequis géométriques	198
1.2 Représentations polynomiales	200
VIII Planification	205
1 Formalisation de la Planification Classique	206
1.1 Le cadre classique	206
1.2 Algorithmes et Langages de représentation	209
2 Le Planificateur GRAPHLAN	212
2.1 Définitions et fonctionnement	212
2.2 Améliorations	214

IX	Traitement du signal	215
1	Signaux à temps discrets et Systèmes	216
1.1	Systèmes et signaux à temps discret	216
1.2	Convolution et corrélation	219
2	Transformée de Fourier et TFD	222
2.1	Transformée de Fourier des signaux à temps discret (TFTD)	222
2.2	Transformée de Fourier Discrète (TFD) et Algorithme FFT	224
2.3	Troncature spectrale et Fenêtres d'apodisation	225
3	Filtrage par Transformée de Fourier	226
3.1	Transformée en Z	226
3.2	Description des SLIT et Fonctions de Transfert	227
3.3	Filtres sélectifs et Synthèse de filtres RIF	228
4	Traitement d'images 2D	229
4.1	Numérisation, Représentation de l'Image 2D et Traitements Points-à-Points . . .	230
4.2	Premiers filtres spatiaux	232
4.3	Analyse fréquentielle bidimensionnelle (TFD 2D)	234
4.4	Filtrage Sélectif et Réhaussement fréquentiel 2D	236
4.5	Modélisation des Dégradations et Filtres de Débruitage	237
4.6	Bruit périodique, Problèmes Inverses et Défloutage	239
5	Analyse du son et Reconnaissance vocale	242
5.1	Architecture des Systèmes de Reconnaissance Vocale (RAP)	242
5.2	Paramétrisation Acoustique : Coefficients MFCC	243
5.3	Modélisation Statistique et Modèles de Mélanges de Gaussiennes (GMM)	243
5.4	Alignement Temporel Dynamique (DTW)	244
5.5	Modèles de Markov Cachés (HMM / MMC)	245
5.6	Modèles Connexionnistes (Réseaux de Neurones Récurents)	245

Probabilités



Chapitre 1

Chaînes de Markov

Contents

1.1 Définitions et Généralités	7
1.2 Propriétés de Markov	12
1.2.1 Propriété de Markov Faible	12
1.2.2 Temps d'arrêt et propriété de Markov Forte	13
1.3 Mesures Invariantes	15
1.3.1 Définition et propriétés	15
1.3.2 Existence des mesures invariantes	16
1.3.3 Unicité des mesures de probabilité invariantes et irréductibilité	17
1.3.4 Construction de mesures de probabilité invariantes	19
1.3.5 Réversibilité	21
1.4 Classification des états	22
1.4.1 Nombre de visites en un point	22
1.4.2 État transiant et état récurrent	23
1.5 Mesures invariantes dans le cas dénombrable	25
1.5.1 États récurrents positif ou nul	25
1.5.2 Propriétés des chaînes irréductibles	25
1.5.3 Critère de récurrence positive	27
1.6 Ergodicité et convergence en loi	29
1.6.1 Ergodicité	29
1.6.2 Convergence en loi	29

Jusqu'à maintenant, nous avons étudié des suites de variables aléatoires dites iid (indépendantes et identiquement distribuées). Ces propriétés nous ont permis de développer beaucoup d'outils pour les étudier tels que la loi forte des grands nombres ou le théorème central limite.

Or nous n'utilisons pas beaucoup ce type de suite de variables aléatoires en simulation. En effet, ces propriétés plutôt fortes ne permettent pas de modéliser des phénomènes dont l'évolution est plus complexe. Nous allons donc continuer à étudier des suites de variables aléatoires mais en rajoutant des relations de dépendance entre celles-ci. Cela nous permettra de modéliser des systèmes dynamiques plus complexes.

1.1 Définitions et Généralités

On se place ici dans un espace probabilisé $(E, \mathcal{F}, \mathbb{P})$ et E un *espace d'état* fini ou dénombrable. On s'intéresse à une suite de variables aléatoires $(X_n)_{n \in \mathbb{N}}$ ou X_k est l'état du système au temps k .

Définition (Trajectoire) . On appelle *trajectoire du système* jusqu'au temps $n \in \mathbb{N}^*$ la donnée $(X_1, \dots, X_n)$.

Définition (Chaîne de Markov) . On dit qu'une suite $(X_n)_{n \geq 0}$ est une *chaîne de Markov* si elle vérifie la *propriété de Markov* :

$$\forall n \in \mathbb{N}, \forall i_0, i_1, \dots, i_{n-1} \in E,$$

$$\boxed{\mathbb{P}(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = \mathbb{P}(X_{n+1} = j \mid X_n = i)}$$

Remarque Conditionnellement à l'état présent $\{X_n = i\}$, la probabilité d'arriver à l'état $\{X_{n+1} = j\}$ à l'itération suivante ne dépend pas des événements du passé $\{X_0 = i_0, \dots, X_{n-1} = i_{n-1}\}$. On parle d'*absence de mémoire*.

Définition (Chaîne de Markov Homogène) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov. On dit qu'elle est *homogène* si :

$$\forall n \in \mathbb{N}, \forall i, j \in E, \quad \mathbb{P}(X_{n+1} = j \mid X_n = i) = \mathbb{P}(X_1 = j \mid X_0 = i)$$

On se placera toujours dans ce cadre.

Remarque Une chaîne de Markov homogène est une chaîne de Markov où la probabilité de passer d'un état à un autre ne dépend pas du temps.

Exemple (Marche Aléatoire) On modélise le mouvement d'une particule sur une grille par une chaîne de Markov où X_n représente la position de la particule au temps $n \in \mathbb{N}$. On considère que la particule se déplace sur une seule dimension. Elle peut donc avancer d'une case ou reculer d'une case à chaque itération. On notera $p \in [0, 1]$ la probabilité qu'elle avance et $1 - p$ la probabilité qu'elle recule indépendamment du passé.

Soit $(\varepsilon_i)_{i \geq 0}$ à valeurs dans $\{-1, 1\}$ telle que :

$$\mathbb{P}(\varepsilon_i = 1) = 1 - \mathbb{P}(\varepsilon_i = -1) = p \in [0, 1]$$

la variable aléatoire qui représente le mouvement de la particule. Déterminons la loi de X_n :

$$X_{n+1} = X_n + \varepsilon_{n+1} = X_{n-1} + \varepsilon_n + \varepsilon_{n+1} = \dots = \sum_{i=1}^n \varepsilon_i$$

Montrons que c'est bien une chaîne de Markov :

$$\begin{aligned} X_{n+1} &= X_n + \varepsilon_{n+1} \\ &= \left(\sum_{i=1}^n \varepsilon_i \right) + \varepsilon_{n+1} \\ &= \sum_{i=1}^{n+1} \varepsilon_i \end{aligned}$$

où tous les $\varepsilon_i, i \in \llbracket 1, n+1 \rrbracket$ sont indépendants. On a donc :

$$\begin{aligned} \mathbb{P}(X_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) &= \mathbb{P}(X_n + \varepsilon_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) \\ &= \mathbb{P}(i + \varepsilon_{n+1} = j \mid X_n = i, \dots, X_0 = i_0) \\ &= \mathbb{P}(\varepsilon_{n+1} = j - i) \end{aligned}$$

On pourra alors s'intéresser au nombre de fois où une trajectoire passe par un point $n \in \mathbb{N}$. La théorie des chaînes de Markov permet de répondre à ce genre de questions.

Plus généralement, on peut construire récursivement des chaînes de Markov en définissant un X_0 fixé ou tiré par une loi π_0 puis en posant :

$$X_{n+1} = f(X_n, \varepsilon_{n+1})$$

où (ε_i) est une suite de variables aléatoires iid et f une fonction mesurable.

Définition (Matrice de Transition) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov. On définit la *matrice de transition* P de la chaîne comme la matrice, potentiellement infinie, telle que :

$$P(i, j) = \mathbb{P}(X_1 = j | X_0 = i)$$

Ainsi le coefficient de la i -ème ligne et de la j -ème colonne correspond à la probabilité de passer de l'état i à l'état j en une étape.

Exemple Soit $P \in \mathcal{M}_3(\mathbb{R})$ la matrice de transition suivante :

$$P = \begin{pmatrix} 1/4 & 3/4 & 0 \\ 0 & 1/2 & 1/2 \\ 1/2 & 0 & 1/2 \end{pmatrix}$$

On peut alors la représenter par un graphe pondéré orienté dont les noeuds représentent les états au temps i de la chaîne de Markov et les poids les probabilités de passer d'un état à l'autre en une étape.

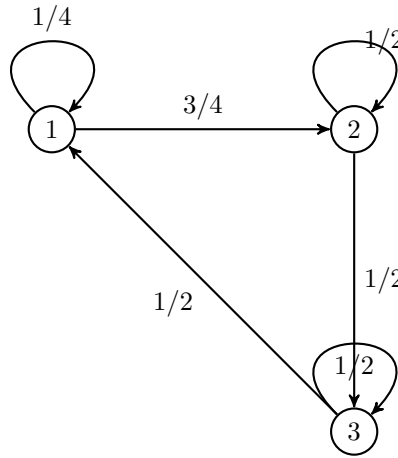


FIGURE 1.1 – Graphe Orienté Pondéré de la matrice de transition P

Définition (Matrice Stochastique) . La matrice de transition P est dite *stochastique* si

$$\forall i \in E, \quad \sum_{j \in E} P(i, j) = 1$$

(i.e si la somme des poids des arrêtes de tout noeud du graphe associé est égale à 1)

Proposition La loi d'une chaîne de Markov homogène $(X_n)_{n \geq 0}$ est complètement déterminée par la loi de π_0 de la condition initiale X_0 et par la matrice de transition P . On a :

$$\forall i_0, i_1, \dots, i_n \in E, \quad \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) = \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)$$

Démonstration Démontrons ce résultat par récurrence sur $\mathbb{N}$.

- Initialisation : Pour $n = 0$, par définition, on a :

$$\mathbb{P}(X_0 = i_0) = \pi_0(i_0)$$

- Hérédité : Soient $i_0, \dots, i_n \in E$, supposons que :

$$\mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) = \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)$$

Soit $i_{n+1} \in E$ alors :

$$\begin{aligned} & \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n, X_{n+1} = i_{n+1}) \\ &= \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) \times \mathbb{P}(X_{n+1} = i_{n+1} | X_0 = i_0, \dots, X_n = i_n) \\ &= \mathbb{P}(X_0 = i_0, X_1 = i_1, \dots, X_n = i_n) \times \mathbb{P}(X_{n+1} = i_{n+1} | X_n = i_n) \\ &= \pi_0(i_0)P(i_0, i_1)P(i_1, i_2) \dots P(i_{n-1}, i_n)P(i_n, i_{n+1}) \end{aligned}$$

On s'intéresse maintenant à la loi de X_n au temps $n \in \mathbb{N}$.

Propriété (Relation de Chapman-Kolmogorov) . Soit $(X_n)_{n \geq 0}$ une chaîne de markov homogène de matrice P et de loi initiale π_0 . Pour tout $i, j \in E$, on a :

$$\mathbb{P}(X_{n+m} = j | X_0 = i_0) = \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k) \times \mathbb{P}(X_n = k | X_0 = i_0)$$

En particulier :

$$\mathbb{P}(X_n = j | X_0 = i_0) = P^n(i_0, j)$$

Démonstration Prouvons les résultats précédents.

Rappelons tout d'abord la formule des probabilités totales. Soit $(B_k)_{k \in \mathbb{N}}$ une partition de E , alors pour tout $A \in E$, on a :

$$\mathbb{P}(A) = \sum_{k \in \mathbb{N}} \mathbb{P}(A \cap B_k)$$

ou en conditionnel sur C :

$$\mathbb{P}(A | C) = \sum_{k \in \mathbb{N}} \mathbb{P}(A \cap B_k | C)$$

Or ici, $\{X_k | k \geq 0\}$ forme bien une partition de E puisque à chaque instant k , le processus est dans un unique état X_k . D'après la formule des probabilités totales, on a donc :

$$\begin{aligned} \mathbb{P}(X_{n+m} = j | X_0 = i_0) &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j, X_n = k | X_0 = i_0) \\ &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k, X_0 = i_0) \times \mathbb{P}(X_n = k | X_0 = i_0) \\ &= \sum_{k \in E} \mathbb{P}(X_{n+m} = j | X_n = k) \mathbb{P}(X_n = k | X_0 = i_0) \end{aligned}$$

On remarque de $\mathbb{P}(X_{n+m} = j | X_n = k)$ est la probabilité de passer de l'état k à j en m pas d'où :

$$\mathbb{P}(X_{n+m} = j | X_n = k) = \mathbb{P}(X_m = j | X_0 = k)$$

Déduisons maintenant la seconde formule de la première :

$$\begin{aligned}\mathbb{P}(X_2 = j | X_0 = i_0) &= \sum_{k \in E} \mathbb{P}(X_2 = j | X_1 = k) \mathbb{P}(X_1 = k | X_0 = i_0) \\ &= \sum_{k \in E} P(i, k) P(k, i) \\ &= P^2(i, j) \quad \text{etc} \dots\end{aligned}$$

Remarque (Notation) Si X_0 est distribué selon la loi π_0 , on notera :

$$\begin{aligned}\mathbb{P}_{\pi_0}(X_n = j) &= \sum_{i_0 \in E} \mathbb{P}(X_n = j | X_0 = i_0) \pi_0(i_0) \\ &= \sum_{i_0 \in E} \mathbb{P}(X_n = j | X_0 = i_0) \mathbb{P}(X_0 = i_0) \\ &= \sum_{i_0 \in E} P(i_0, j) \pi_0(i_0) \\ &= \pi_0 P^n(j)\end{aligned}$$

où $P^n(j)$ est la i ème colonne de P^n et π_0 un vecteur ligne.

Exemple (Simple) Soit $(X_n)_{n \geq 0}$ une suite de variables aléatoires réelles iid de loi π .

$$\text{i.e } \forall i \in E, \mathbb{P}(X_n = i) = \pi(i)$$

On a donc :

$$P(i, j) = \mathbb{P}(X_1 = j | X_0 = i) = \mathbb{P}(X_1 = j) = \pi(j)$$

C'est un cas très simple et peu intéressant.

Exemple (Ruine du joueur) Soit $n \geq 1$ et $p \in [0, 1]$ et une suite de variables aléatoires $(X_n)_{n \in \mathbb{N}}$ à valeurs dans $\llbracket 1, n \rrbracket$. On définit X_m comme le montant gagné par le joueur après m parties. On considère que le joueur ne peut pas jouer avec un montant supérieur à n . On peut donc représenter son évolution dans le jeu par le graphe ci-dessous :

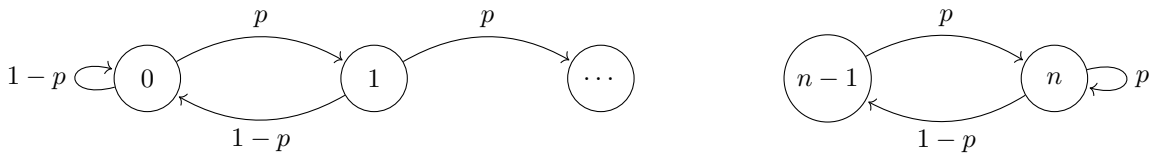


FIGURE 1.2 – Ruine du Joueur

où :

$$\begin{cases} P(k, k+1) = p & \text{si } k \in \{0, \dots, n-1\} \\ P(k, k+1) = 1-p & \text{si } k \in \{1, \dots, n\} \end{cases}$$

En pratique, on s'intéressera souvent à :

$$\mathbb{P}(X_n \text{ atteint } 0 \text{ avant } m | X_0 = i)$$

Exemple (File d'attente) On s'intéresse au nombre de personnes dans une file d'attente où, à chaque itération, une personne rentre ou sort de la file. Soit $(X_n)_{n \in \mathbb{N}}$ une suite à valeur dans $\mathbb{N}$. Ici, un pas de temps n représente un événement qui se passe. On va s'intéresser au fait que la chaîne tende vers 0 ou $+\infty$ en fonction de $i_0 \in \llbracket 1, n \rrbracket$.

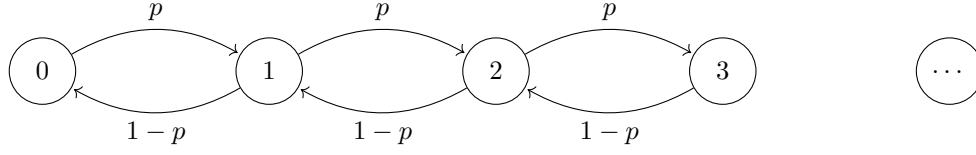


FIGURE 1.3 – File d'attente

Exemple (Processus de Galton-Watson) Ici $(X_n)_{n \geq 1}$ compte le nombre d'individus d'une population dans la génération n . On suppose que chaque individu vivant a des descendants indépendamment des autres. On appelle $(\zeta_{n,k})_{n \leq 1, k \leq 1}$ la suite donnant le nombre de descendants de l'individu k vivant à la génération n .

On peut alors en déduire que $(\zeta_{n,k})_{n \leq 1, k \leq 1}$ est une suite de var iid et que :

$$X_{n+1} = \sum_{k=1}^{X_n} \zeta_{n,k}$$

1.2 Propriétés de Markov

1.2.1 Propriété de Markov Faible

Théorème (Propriété de Markov Faible) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène de matrice de transition P et de loi initiale π_0 . Soit $m \in \mathbb{N}$ fixé. Alors, conditionnellement à l'événement $\{X_m = i\}$, la chaîne décalée $(X_{m+k})_{k \in \mathbb{N}}$ est une chaîne de Markov homogène de même matrice de transition P et de loi initiale δ_i .

En particulier, le *futur* $(X_{m+1}, X_{m+2}, \dots)$ est indépendant du *passé* $(X_0, X_1, \dots, X_{m-1})$ conditionnellement à X_m . Autrement dit, pour tous événements A mesurables par rapport à $\sigma(X_{m+1}, X_{m+2}, \dots)$ et B mesurables par rapport à $\sigma(X_0, X_1, \dots, X_{m-1})$, on a :

$$\mathbb{P}(A \cap B \mid X_m = i) = \mathbb{P}(A \mid X_m = i) \mathbb{P}(B \mid X_m = i).$$

Démonstration Démontrons ce théorème en deux parties :

1. Égalité des lois :

Montrons que, pour tout $n \in \mathbb{N}$ et toute suite d'états $(y_1, \dots, y_n)$,

$$\begin{aligned} \mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n \mid X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0) \\ = \mathbb{P}(X_1 = y_1, \dots, X_n = y_n \mid X_0 = i) \end{aligned}$$

On a :

$$\begin{aligned} & \mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n \mid X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0) \\ &= \frac{\mathbb{P}(X_{m+1} = y_1, \dots, X_{m+n} = y_n, X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0)}{\mathbb{P}(X_m = i, X_{m-1} = i_{m-1}, \dots, X_0 = i_0)} \\ &= \frac{\pi_0(i_0)P(i_0, i_1) \cdots P(i_{m-1}, i)P(i, y_1) \cdots P(y_{n-1}, y_n)}{\pi_0(i_0)P(i_0, i_1) \cdots P(i_{m-1}, i)} \\ &= P(i, y_1)P(y_1, y_2) \cdots P(y_{n-1}, y_n) \\ &= \mathbb{P}(X_1 = y_1, \dots, X_n = y_n \mid X_0 = i). \end{aligned}$$

Ainsi, la chaîne décalée $(X_{m+k})_{k \in \mathbb{N}}$, conditionnellement à $X_m = i$, suit bien la loi d'une chaîne de Markov homogène de matrice P et de loi initiale δ_i .

2. Indépendance conditionnelle :

Soient A un événement mesurable par rapport au futur $\sigma(X_{m+1}, X_{m+2}, \dots)$ et B un événement mesurable par rapport au passé $\sigma(X_0, X_1, \dots, X_{m-1})$.

Montrons que :

$$\mathbb{P}(A \cap B \mid X_m = i) = \mathbb{P}(A \mid X_m = i) \mathbb{P}(B \mid X_m = i).$$

Par définition de la probabilité conditionnelle :

$$\mathbb{P}(A \cap B \mid X_m = i) = \frac{\mathbb{P}(A \cap B, X_m = i)}{\mathbb{P}(X_m = i)}.$$

D'après la propriété de Markov, conditionnellement à $X_m = i$, le futur $(X_{m+1}, X_{m+2}, \dots)$ ne dépend pas du passé $(X_0, \dots, X_{m-1})$, donc :

$$\mathbb{P}(A \cap B, X_m = i) = \mathbb{P}(A, X_m = i) \mathbb{P}(B, X_m = i) / \mathbb{P}(X_m = i).$$

Ainsi :

$$\begin{aligned} \mathbb{P}(A \cap B \mid X_m = i) &= \frac{\mathbb{P}(A, X_m = i) \mathbb{P}(B, X_m = i)}{\mathbb{P}(X_m = i)^2} \\ &= \mathbb{P}(A \mid X_m = i) \mathbb{P}(B \mid X_m = i). \end{aligned}$$

Ce qui montre bien l'indépendance conditionnelle du passé et du futur, sachant X_m .

Remarque Cette propriété de Markov est aussi vraie de façon plus générale quand on conditionne par l'état de la chaîne en un temps aléatoire appelé temps d'arrêt. Ce sera exactement le propos de la propriété de Markov forte que nous verrons plus tard.

1.2.2 Temps d'arrêt et propriété de Markov Forte

Définition (Tribu $\mathcal{F}_n$) . On définit $\mathcal{F}_n$ la tribu engendrée par les variables aléatoires $(X_1, \dots, X_n)$ la tribu :

$$\begin{aligned} \mathcal{F}_n &= \sigma(X_0, \dots, X_n) \\ &= \sigma(\{X_0 = x_0, \dots, X_n = x_n\}, x_0, \dots, x_n \in E) \end{aligned}$$

Ainsi, la tribu engendrée par l'ensemble $\{X_0 = x_0, \dots, X_n = x_n\}$ est la plus petite tribu contenant cette ensemble.

Définition (Temps d'arrêt) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov à valeurs dans un espace d'états E , définie sur un espace probabilisé $(\Omega, \mathcal{F}, \mathbb{P})$, avec

$$\forall n \in \mathbb{N}, \quad X_n : \Omega \longrightarrow E$$

On note $\mathcal{F}_n = \sigma(X_0, X_1, \dots, X_n)$ la tribu engendrée par les n premières variables de la chaîne. Une application

$$\tau : \Omega \longrightarrow \mathbb{N} \cup \{\infty\}$$

est appelée *temps d'arrêt* de la chaîne de Markov (ou pour la filtration $(\mathcal{F}_n)_{n \geq 0}$) si elle

vérifie :

$$\boxed{\forall n \in \mathbb{N}, \quad \{\tau \leq n\} \in \mathcal{F}_n}$$

Donc τ est mesurable pour la tribu $\mathcal{F}_n$. Autrement dit, le fait que l'évènement $\{\tau \leq n\}$ ne dépend que de l'évolution de la chaîne jusqu'au temps n .

Exemple Soit (X_n) une chaîne de Markov sur $\mathbb{Z}$. On définit le temps d'attente du dépassement de $j \in \mathbb{Z}$ comme la variable aléatoire :

$$\tau_j = \min\{i \geq 0 \mid X_i > j\}$$

τ_j est bien un temps d'arrêt pour la CMH car :

$$\{\tau_j \leq n\} = \{\exists k \in \mathbb{N}, X_k \geq j\}$$

donc τ_j ne dépend que de $X_0, \dots, X_n$. De façon générale, soit $A \subset E$, le temps d'attente de A défini par :

$$\tau_A = \min\{i \geq 0, X_i \in A\}$$

est un temps d'arrêt. Si cet ensemble est vide (i.e la chaîne de Markov n'atteint jamais A), alors $\tau_A = +\infty$. Par contre, $S_j = \max\{i \geq 0, X_i > j\}$ n'est pas un temps d'arrêt car $\{S_j \leq n\} = X_{n+1} < j, X_{n+2} < j, \dots$ dépend de ce qui s'est passé "après".

Théorème (Propriété de Markov forte) . Soit $(X_n)_{n \geq 0}$ une chaîne de Markov homogène et τ un temps d'arrêt. Alors conditionnellement à $\{\tau < \infty\}$ et $\{X_\tau = x\}$ la chaîne décalée $(X_{\tau+n})_{n \geq 0}$ est encore une chaîne de Markov de condition initiale x .
De plus, conditionnellement à $\{\tau < \infty\} \cap \{X_\tau = x\}$ on a :

$$(X_{\tau+n})_{n \geq 0} \text{ est indépendante de } (X_0, \dots, X_{\tau-1})$$

Remarque Ici, la chaîne de Markov est décalée conditionnellement à une variable aléatoire, c'est beaucoup plus fort que la propriété précédente.

Exemple (Application - Ruine du Joueur) Soit l'espace d'état $E = \{0, \dots, m\}$ et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur E telle que $X_0 = k \in \{1, \dots, m\}$. On s'intéresse à deux temps d'arrêts :

- $\tau_0 = \inf\{n \geq 0 \mid X_n = 0\}$
- $\tau_m = \inf\{n \geq 0 \mid X_n = m\}$

On souhaite estimer le temps de jeu moyen d'un joueur. Pour cela, on considère le temps d'arrêt :

$$\tau = \min(\tau_0, \tau_m)$$

On veut savoir quand $t < t_0$ (i.e quand le joueur gagne avant de perdre). Soit $m > 1$, alors τ_0 est distinct de τ_m si l'un des deux est fini. En conséquence si $\tau < \infty$, on a :

- soit $\tau = \tau_0 < \tau_m$
- soit $\tau = \tau_m < \tau_0$.

On cherche la "probabilité de ruine" :

$$f(k) = \mathbb{P}(\tau < \infty, \tau = \tau_0 \mid X_0 = k)$$

Pour cela, nous devons donc déterminer toutes les valeurs de f . On utilise donc les propriétés de Markov pour en déduire des relations de récurrence entre $f(k)$, $f(k+1)$ et $f(k-1)$. On remarque ainsi que :

$$\text{si } X_0 = 0 \text{ alors } \tau = \tau_0 = 0 \text{ donc } f(0) = 1$$

si $X_0 = m$ alors $\tau = \tau_m = 0$ donc $f(m) = 0$

Si $X_0 \notin \{0, m\}$, on doit au moins faire un pas pour atteindre τ_0 ou τ_m . On en déduit donc que :

$$\tau_0((X_n)_{n \in \mathbb{N}}) = 1 + \tau_0((X_{n+1})_{n \in \mathbb{N}})$$

$$\tau_m((X_n)_{n \in \mathbb{N}}) = 1 + \tau_m((X_{n+1})_{n \in \mathbb{N}})$$

On a donc, via l'analyse à un pas :

$$\begin{aligned} f(k) &= \mathbb{P}(\tau((X_n)_{n \in \mathbb{N}}) = \tau_0((X_n)_{n \in \mathbb{N}}) < \infty \mid X_0 = k) \\ &= \mathbb{P}(\tau((X_{n+1})_{n \in \mathbb{N}}) = \tau_0((X_{n+1})_{n \in \mathbb{N}}) < \infty \mid X_0 = k) \end{aligned}$$

En appliquant la formule des probabilités totales à X_1 , on a donc :

$$\begin{aligned} f(k) &= \mathbb{P}(X_1 = k + 1 \mid X_0 = k) \times \mathbb{P}(\tau((X_{n+1})_{n \in \mathbb{N}}) = \tau_0((X_{n+1})_{n \in \mathbb{N}}) < \infty \mid X_1 = k + 1, X_0 = k) \\ &\quad + \mathbb{P}(X_1 = k - 1 \mid X_0 = k) \times \mathbb{P}(\tau((X_{n+1})_{n \in \mathbb{N}}) = \tau_0((X_{n+1})_{n \in \mathbb{N}}) < \infty \mid X_1 = k - 1, X_0 = k) \end{aligned}$$

Or, conditionnellement à $\{X_1 = k + 1\}$, la chaîne $(X_{n+1})_{n \in \mathbb{N}}$ est une chaîne de Markov partant de $k + 1$. On a donc :

$$\begin{aligned} f(k) &= p\mathbb{P}(\tau_0((X_n)_{n \in \mathbb{N}}) = \tau((X_n)_{n \in \mathbb{N}}) \mid X_0 = k + 1) \\ &\quad + (1 - p)\mathbb{P}(\tau_0((X_n)_{n \in \mathbb{N}}) = \tau((X_n)_{n \in \mathbb{N}}) \mid X_0 = k - 1) \\ &= pf(k + 1) + (1 - p)f(k - 1) \end{aligned}$$

Résolvons cette relation de récurrence pour $p = 1/2$. On a donc $\forall k \in \llbracket 1, m \rrbracket, k \notin \{0, m\}$:

$$f(k) = \frac{f(k + 1) + f(k - 1)}{2}$$

$$\iff f(k + 1) - f(k) = f(k) - f(k - 1) = \Delta$$

On peut donc poser :

$$\begin{aligned} f(m) - f(0) &= \sum_{k=0}^{m-1} f(k + 1) - f(k) = (m - 1)\Delta \\ \iff \Delta &= -\frac{1}{m} \end{aligned}$$

D'où :

$$f(k) = 1 + k\Delta = 1 - \frac{k}{m}$$

1.3 Mesures Invariantes

Dans cette section, nous supposons tout d'abord que l'espace d'états E est fini. Au début du cours on a vu que, pour une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ de matrice P et de loi initiale π_0 , la relation de Chapman-Kolmogorov nous donne :

$$\mathbb{P}(X_n = j \mid X_0 = i_0) = P^n(i_0, j)$$

Nous allons ici chercher des mesures de probabilités telles que :

$$\pi P = \pi$$

Nous appellerons de telles mesures des *mesures de probabilités invariantes*.

1.3.1 Définition et propriétés

Définition (Mesures Invariantes) . Soit une mesure positive π sur E . On dit que π est *invariante* pour une matrice de transition P si :

$$\forall x \in E, \quad \pi(x) = \pi P(x) = \sum_{y \in E} \pi(y) P(y, x)$$

On dit alors que π est un *vecteur propre à gauche* de P pour la valeur propre 1. Par convention, les mesures sont écrites comme des vecteurs ligne.

Remarque Attention, une mesure invariante n'est pas forcément une mesure de probabilité. Pour que ce soit le cas, il faut que sa masse totale soit égale à 1.

$$\text{i.e.} \quad \sum_{x \in E} \pi(x) = 1$$

Or ici, puisque E est fini, cette somme l'est aussi donc toute mesure invariante peut être normalisée en mesure de probabilités. Nous verrons plus tard que ce n'est pas toujours le cas.

Lemme Soit π une mesure invariante pour une matrice de transition P . Alors :

$$\forall n \in \mathbb{N}^*, \quad \pi = \pi P^n$$

De plus si X_0 est distribuée selon la loi π alors la loi de X_n l'est également pour tout $n > 0$. En conséquence :

$$\forall n, m \in \mathbb{N}, \quad (X_0, X_1, \dots, X_m) \stackrel{\text{loi}}{=} (X_n, X_{n+1}, \dots, X_{n+m})$$

Propriété (Mesure Invariante et Loi) . Soit π une mesure de probabilité invariante. Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène de matrice P et de loi initiale π_0 alors, pour tout $n \in \mathbb{N}$, $X_n \sim \pi$.

Ce lemme justifie toute l'utilisation que nous ferons des mesures invariantes. En effet, une mesure invariante permet de déterminer le comportement asymptotique de la chaîne de Markov sans avoir à la simuler en entier un grand nombre de fois.

1.3.2 Existence des mesures invariantes

Théorème (Mesure invariance, existence) . Toute chaîne de Markov homogène sur un espace d'état fini admet une mesure de probabilité invariante.

Démonstration Soit ψ une mesure de probabilité quelconque sur E où $|E| = d$. Posons :

$$\forall n \in \mathbb{N}, \quad \psi_n = \frac{1}{n} \sum_{k=0}^{n-1} \psi P^k$$

On sait que pour tout $k \in \mathbb{N}$, ψP^k est une mesure de probabilité sur E . Donc ψ_n l'est aussi une par construction.

De plus, pour tout $x \in E$, $(\psi_n(x))_{n \in \mathbb{N}}$ est une suite à valeurs dans $[0, 1]^d$. Par compacité, il existe une sous-suite convergente $(\psi_{\varphi(n)}(x))_{n \in \mathbb{N}}$. Comme E est fini, il existe une extraction telle que :

$$\forall x \in E, \quad \psi_{\varphi(n)}(x) \xrightarrow{n \rightarrow \infty} \pi(x)$$

Montrons maintenant que π est une mesure de probabilités invariante pour P . On a :

$$\forall n \in \mathbb{N}, \quad \sum_{x \in E} \psi_n(x) = 1 \quad \text{et} \quad \forall x \in E, \quad \psi_n(x) \geq 0$$

Donc lorsque $n \rightarrow \infty$, on a :

$$\sum_{x \in E} \pi(x) = 1 \quad \text{et} \quad \forall x \in E, \pi(x) \geq 0$$

Donc π est une mesure de probabilités sur E . De plus, pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned} \psi_n P &= \frac{1}{n} \sum_{k=1}^n \psi P^k P = \frac{1}{n} \sum_{k=1}^n \psi P^{k+1} = \frac{1}{n} \sum_{k=2}^{n+1} \psi P^k \\ &= \frac{1}{n} \frac{n+1}{n+1} \left(\sum_{k=1}^{n+1} \psi P^k - \psi P \right) \\ &= \frac{n+1}{n} \psi_{n+1} - \frac{1}{n} \psi P \end{aligned}$$

Donc lorsque $n \rightarrow \infty$, on obtient :

$$\pi P = \pi$$

Remarque Il existe pleins de méthodes numériques pour approcher une mesure invariante d'une chaîne de Markov homogène. Ce n'est pas un problème facile car il faut en général résoudre un système de d équations à d inconnues. Nous nous intéresserons donc aux *mesures réversibles*, plus faciles à calculer et qui nous donnent une mesure invariante quand elle existe.

Exemple Soit le graphe suivant et $p, q \in]0, 1[$. Soit $\pi = (\pi_1, \pi_2)$ une mesure invariante. On a

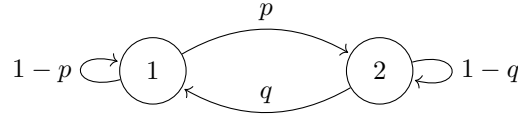


FIGURE 1.4 – Mesure Invariante

alors $\pi P = \pi$.

$$\begin{aligned} &\iff (\pi_1, \pi_2) \begin{pmatrix} 1-p & p \\ q & 1-q \end{pmatrix} = (\pi_1, \pi_2) \\ &\iff \begin{cases} \pi_1(1-p) + q\pi_2 = \pi_1 \\ \pi_1 p + \pi_2(1-q) = \pi_2 \end{cases} \\ &\iff \pi_1 = \frac{q}{p} \pi_2 \end{aligned}$$

Or π est une mesure de probabilité ssi :

$$p\pi_1 + \pi_2 = 1 \iff \pi_1 = \frac{q}{p+q} \text{ et } \pi_2 = \frac{p}{p+q}$$

1.3.3 Unicité des mesures de probabilité invariantes et irréductibilité

Définition (Communication) . Soient $x, y \in E$. On dit que x *communique avec* y s'il existe un chemin de probabilité positive de x à y dans E . On le note alors $x \rightsquigarrow y$. Plus formellement :

$$\begin{aligned} x \rightsquigarrow y &\iff \exists x, x_1, \dots, x_{n-1}, y \in E, P(x, x_1), P(x_1, x_2), \dots, P(x_{n-1}, y) > 0 \\ &\iff \forall n \geq 1, \quad P^n(x, y) > 0 \end{aligned}$$

Propriété (Transitivité) . La relation *communique* $\rightsquigarrow$ est transitive.

Définition (Classe fermée) . Soit $E_0 \subset E$ un sous-ensemble de E que l'on appellera *classe de* E . On dit que la classe E_0 est *fermée* si :

$$\forall x \in E_0, \quad x \rightsquigarrow y \implies y \in E_0$$

Définition (Classe irréductible) . Une classe E_0 de E est dite *irréductible* si elle est fermée et si :

$$\forall x, y \in E_0, \quad x \rightsquigarrow y \text{ et } y \rightsquigarrow x$$

Par extension, on dira qu'une chaîne de Markov homogène est irréductible si son espace d'état l'est aussi.

Théorème (Unicité des mesures invariantes) . Toute chaîne de Markov homogène irréductible sur un espace d'état fini admet une *unique mesure de probabilité invariante* π et :

$$\forall x \in E, \quad \pi(x) > 0$$

Démonstration Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène d'espace d'état E . Supposons que E est irréductible et soit π une mesure invariante de $(X_n)_{n \in \mathbb{N}}$.

1. Positivité : Montrons que π est positive. On sait que π est une mesure de probabilités donc :

$$\sum_{x \in E} \pi(x) = 1$$

donc il existe nécessairement $x_0 \in E$ tel que $\pi(x_0) > 0$. Comme E est irréductible, alors pour tout $x \in E, x_0 \rightsquigarrow x$. Donc il existe $n \in \mathbb{N}$ tel que :

$$P^n(x_0, x) > 0$$

De plus, on sait que π vérifie :

$$\begin{aligned} \pi(x) &= \sum_{x \in E} \pi(x) P(x, y) = \pi P \\ \implies \pi(y) &= \sum_{x \in E} \pi(x) P^n(x, y) \\ &= \underbrace{\pi(x_0) P^n(x_0, y)}_{>0} + \underbrace{\sum_{x \in E, x \neq x_0} \pi(x) P^n(x, y)}_{\geq 0} \end{aligned}$$

Donc $\forall y \in E, \pi(y) > 0$.

2. Unicité : voir TD

1.3.4 Construction de mesures de probabilité invariantes

Nous avons vu que les mesures de probabilité invariantes jouent un rôle fondamental dans l'étude des chaînes de Markov. Elles permettent notamment de décrire le comportement stationnaire du système. Nous allons à présent présenter quelques outils permettant de construire de telles mesures.

Soit E un espace d'états fini, et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène et irréductible sur E . L'idée est de chercher à faire converger la loi de X_n lorsque n tend vers l'infini, de façon à obtenir une mesure de probabilité invariante pour la matrice de transition associée.

Définition (Temps de retour) . Soit $x \in E$, on définit le temps de retour en x comme :

$$\tau_x^+ := \inf\{n \geq 1 \mid X_n = x\} \in \overline{\mathbb{N}}$$

remarquons que c'est un *temps d'arrêt*.

Remarque Nous pouvons facilement montrer que tout temps de retour est un temps d'arrêt. En effet, pour rappel nous avons défini les temps d'arrêt comme :

Une application

$$\tau : \Omega \longrightarrow \mathbb{N} \cup \{\infty\}$$

est appelée temps d'arrêt de la chaîne de Markov (ou pour la filtration $(\mathcal{F}_n)_{n \geq 0}$) si elle vérifie :

$$\forall n \in \mathbb{N}, \quad \{\tau \leq n\} \in \mathcal{F}_n$$

Soit $n \in \mathbb{N}$ on a alors :

$$\begin{aligned} \{\tau_x^+ \leq n\} &= \{\exists k \in \llbracket 1, n \rrbracket \mid X_k = x\} \\ &= \bigcup_{k=1}^n \{X_k = x\} \end{aligned}$$

or, pour tout $k \in \llbracket 1, n \rrbracket$, l'évènement $\{X_k = x\} \in \mathcal{F}_k \subseteq \mathcal{F}_n$. Donc par union dénombrable, on a :

$$\{\tau_x^+ \leq n\} \in \mathcal{F}_n$$

Donc tout temps d'attente est un temps d'arrêt. De plus, on peut remarquer que la différence avec un temps de retour est que si $\{X_0 = x\}$ alors :

$$\tau_x = 0 \quad \text{et} \quad \tau_x^+ \geq 1$$

Lemme Pour tout $x, y \in E$, on a :

$$\mathbb{E}(\tau_y^+ \mid X_0 = x) < \infty$$

Théorème (Mesure Invariante) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène sur un espace d'états E fini. Alors son unique mesure de probabilités invariante est :

$$\pi(x) = \frac{1}{\mathbb{E}(\tau_x^+ \mid X_0 = x)} \quad \forall x \in E$$

Remarque Cette mesure charge plus les points $x \in E$ tels que $\mathbb{E}(\tau_x^+ \mid X_0 = x)$ est petit. C'est à dire les points que la chaîne de Markov visite souvent.

Démonstration Soit $x \in E$ une condition initiale. Soit $\bar{\pi} : E \rightarrow \mathbb{N}$ une application qui compte le nombre de fois que la chaîne visite un point $y \in E$.

$$\begin{aligned} \forall y \in E, \quad \bar{\pi}(y) &:= \mathbb{E} \left(\sum_{n=0}^{\tau_x^+ - 1} \mathbb{1}_{X_n=y} \mid X_0 = x \right) \\ &= \mathbb{E} \left(\sum_{n \geq 0} \mathbb{1}_{X_n=y} \times \mathbb{1}_{\tau_x^+ > n} \mid X_0 = x \right) \\ &= \sum_{n \geq 0} \mathbb{P}(X_n = y, \tau_x^+ > n \mid X_0 = x) \end{aligned}$$

On veut montrer que $\bar{\pi}$ est invariante. On veut donc :

$$\forall y \in E, \quad \bar{\pi}(y) = \sum_{z \in E} \bar{\pi}(z) P(z, y)$$

(On va également montrer que $\bar{\pi}$ ne dépend pas de la condition initiale x .) Pour tout $z \in E$, calculons donc :

$$\sum_{z \in E} \bar{\pi}(z) P(y, z) = \sum_{z \in E} \sum_{n \geq 0} \mathbb{P}(X_n = z, \tau_x^+ > n \mid X_0 = x) \times P(z, y)$$

Or on a :

$$\begin{aligned} &\mathbb{P}(X_n = z, \tau_x^+ > n, X_{n+1} = y \mid X_0 = x) \\ &= \mathbb{P}(X_{n+1} = y \mid X_n = z, \tau_x^+ > n, X_0 = x) \times \mathbb{P}(X_n = z, \tau_x^+ > n, X_0 = x) \\ &= \mathbb{P}(X_{n+1} = y \mid X_n = z) \times \mathbb{P}(X_n = z, \tau_x^+ > n, X_0 = x) \quad (\text{Propriété de Markov}) \\ &= P(z, y) \times \mathbb{P}(X_n = z, \tau_x^+ > n, X_0 = x) \end{aligned}$$

On en déduit alors que :

$$\begin{aligned} \sum_{z \in E} \bar{\pi}(z) P(y, z) &= \sum_{z \in E} \sum_{n \geq 0} \mathbb{P}(X_n = z, \tau_x^+ > n, X_{n+1} = y \mid X_0 = x) \\ &= \sum_{n \geq 0} \mathbb{P}(\tau_x^+ > n, X_{n+1} = y \mid X_0 = x) \quad (\text{Probabilités Totales}) \\ &= \sum_{n \geq 0} \mathbb{P}(\tau_x^+ \geq n+1, X_{n+1} = y \mid X_0 = x) \\ &= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ \geq n, X_n = y \mid X_0 = x) \quad (\text{Changement d'indices}) \\ &= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ > n, X_n = y \mid X_0 = x) + \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_n = y \mid X_0 = x) \\ &= \bar{\pi}(y) - \mathbb{P}(X_0 = y, \tau_x^+ > 0 \mid X_0 = x) + \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_n = y \mid X_0 = x) \end{aligned}$$

Or :

$$\begin{aligned}
& \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_n = y \mid X_0 = x) \\
&= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ = n, X_{\tau_x^+} = y \mid X_0 = x) \\
&= \sum_{n \geq 1} \mathbb{P}(\tau_x^+ < \infty, X_{\tau_x^+} = y \mid X_0 = x) \\
&= \mathbb{P}(X_{\tau_x^+} = y \mid X_0 = x)
\end{aligned}$$

D'où :

$$\sum_{z \in E} \bar{\pi}(z) P(z, y) = \bar{\pi}(y) - \underbrace{\mathbb{P}(X_0 = y, \tau_x^+ > 0 \mid X_0 = x)}_{1 \text{ si } y=x \text{ et } 0 \text{ sinon}} + \underbrace{\mathbb{P}(X_{\tau_x^+} = y \mid X_0 = x)}_{-1 \text{ si } y=x \text{ et } 0 \text{ sinon}} = \bar{\pi}(y)$$

Donc $\bar{\pi}$ est bien une mesure invariante de masse totale :

$$\sum_{y \in E} \bar{\pi}(y) = \mathbb{E}(\tau_x^+ \mid X_0 = x)$$

Or, d'après le lemme précédent, $\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$. Donc l'application :

$$\forall y \in E, \quad \pi(y) = \frac{\bar{\pi}(y)}{\mathbb{E}(\tau_x^+ \mid X_0 = x)}$$

est bien une mesure de probabilités invariante. D'après le théorème d'unicité, c'est donc la seule. De plus, on a $\pi(x) = 1$ donc :

$$\forall y \in E, \quad \pi(y) = \frac{\bar{\pi}(y)}{\mathbb{E}(\tau_x^+ \mid X_0 = x)} = \frac{1}{\mathbb{E}(\tau_x^+ \mid X_0 = x)}$$

1.3.5 Réversibilité

Trouver la mesure de probabilité invariante en résolvant le système est compliqué en général. La *réversibilité* est une propriété suffisante pour qu'une mesure soit invariante.

Définition (Mesure réversible) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène de matrice P . Une mesure μ est dite réversible pour la matrice P si :

$$\boxed{\forall x, y \in E, \quad \mu(x)P(x, y) = \mu(y)P(y, x)}$$

Remarque La réversibilité peut être vue comme le fait d'être capable de parcourir les trajectoires d'une chaîne de Markov dans le sens inverse.

$$\text{i.e } \mathbb{P}(X_0 = x_0, X_1 = x_1, \dots, X_n = x_n) = \mathbb{P}(X_0 = x_n, X_1 = x_{n-1}, \dots, X_n = x_0)$$

Ce résultat provient de la relation de Chapman-Kolmogorov. En effet, si X_0 est choisie selon μ réversible, alors :

$$\begin{aligned}
\mathbb{P}(X_0 = x_0, X_1 = x_1, \dots, X_n = x_n) &= \mu(x_0)P(x_0, x_1) \dots P(x_{n-1}, x_n) \\
&= P(x_1, x_0)\mu(x_1)P(x_1, x_2) \dots P(x_{n-1}, x_n) \\
&= P(x_1, x_0)P(x_2, x_1) \dots P(x_n, x_{n-1})\mu(x_n) \\
&= \mu(x_n)P(x_n, x_{n-1}) \dots P(x_2, x_1)P(x_1, x_0) \\
&= \mathbb{P}(X_0 = x_n, X_1 = x_{n-1}, \dots, X_n = x_0)
\end{aligned}$$

Proposition Une mesure réversible pour une matrice de transition est invariante.

L'étude des mesures réversibles, plus simples que les mesures invariantes, sera donc privilégiée pour déterminer des mesures invariantes.

1.4 Classification des états

Ici, on suppose maintenant que E est un espace d'états fini ou dénombrable et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène sur E de matrice de transition P .

1.4.1 Nombre de visites en un point

Définition (Nombre de visites) . Soit $x \in E$, on note V_x le nombre de visites de la chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ dans x . Plus formellement :

$$V_x := |\{n \geq 1, X_n = x\}|$$

V_x est donc une variable aléatoire à valeurs dans $\overline{\mathbb{N}}$.

Théorème (Nombre de visites et temps de retour) . Soit τ_x^+ le temps de retour de la chaîne de Markov en $x \in E$. On a alors :

- si $\mathbb{P}(\tau_x^+ < \infty \mid X_0 = x) = 1$ alors $V_x = \infty$ presque sûrement.
- si $\mathbb{P}(\tau_x^+ < \infty \mid X_0 = x) = 1 - p < 1$ alors le nombre de visites est fini presque sûrement et il suit une loi géométrique.

$$\text{i.e } \mathbb{P}(V_x = k) = p(1 - p)^k \quad \forall k \geq 0$$

Démonstration Pour démontrer ce théorème, il serait suffisant de montrer que $\forall k \in \mathbb{N}$, on ait :

$$\begin{aligned} \mathbb{P}(V_x \geq k \mid X_0 = x) &= \mathbb{P}(\tau_x^+ < \infty \mid X_0 = x)^k \\ \iff \mathbb{P}_x(V_x \geq k + 1) &= \mathbb{P}_x(\tau_x^+) \mathbb{P}_x(V_x \geq k) \end{aligned}$$

Montrons donc que :

$$\forall k \geq 0, \quad \mathbb{P}_x(V_x \geq k + 1) = \mathbb{P}_x(\tau_x^+ < \infty) \mathbb{P}_x(V_x \geq k)$$

Pour l'évènement $\{V_x \geq k + 1\}$ on a $V_x \geq 1$ donc $\tau_x^+ < \infty$ d'où :

$$\begin{aligned} \mathbb{P}_x(V_x \geq k + 1) &= \mathbb{P}_x(V_x \geq k + 1, \tau_x^+ < \infty) \\ &= \sum_{m \geq 1} \mathbb{P}_x(V_x \geq k + 1, \tau_x^+ = m) \\ &= \sum_{m \geq 1} \mathbb{P}_x(V_x \geq k + 1 \mid \tau_x^+ = m) \mathbb{P}_x(\tau_x^+ = m) \end{aligned}$$

Or on a :

$$\begin{aligned} &\mathbb{P}_x(V_x(X_n) \geq k + 1 \mid \tau_x^+ = m) \\ &= \mathbb{P}_x(V_x(X_{n+m}) + 1 \geq k + 1 \mid \tau_x^+ = m) \\ &= \mathbb{P}_x(V_x(X_n) \geq k) \end{aligned}$$

Donc :

$$\begin{aligned}\mathbb{P}_x(V_x \geq k+1) &= \sum_{m \geq 1} \mathbb{P}_x(V_x \geq k) \mathbb{P}_x(\tau_x^+ = m) \\ &= \mathbb{P}_x(V_x \geq k) \sum_{m \geq 1} \mathbb{P}_x(\tau_x^+ = m) \\ &= \mathbb{P}_x(V_x \geq k) \times \mathbb{P}_x(\tau_x^+ < \infty)\end{aligned}$$

D'où :

$$\mathbb{P}_x(V_x \geq k) = \mathbb{P}_x(\tau_x^+ < \infty)^k$$

Donc si $\mathbb{P}(\tau_x^+ < \infty) = 1$ alors $\mathbb{P}(V_x \geq k) = 1$ pour tout $k \geq 0$ donc $V_x = \infty$. D'autre part, si $\mathbb{P}(\tau_x^+ < \infty) = 1 - p < 1$ alors $\mathbb{P}(V_x \geq k) = (1 - p)^k$.

Remarque Précédemment, nous avons vu que dans le cas où E est fini et $(X_n)_{n \in \mathbb{N}}$ irréductible, alors :

$$\begin{aligned}\mathbb{E}(\tau_x^+ \mid X_0 = x) &< \infty \\ \implies \mathbb{P}(\tau_x^+ < \infty \mid X_0 = x) &= 1\end{aligned}$$

Donc on visitera le point x infiniment de fois.

Nous allons maintenant chercher à découper une trajectoire en "excursions" entre deux passages successifs par le point x . D'après la propriété de Markov forte, ces excursions auront la même loi et seront indépendantes conditionnellement.

Proposition Conditionnellement à l'évènement $\{\tau_x^+ = m\}$, $(X_{n+m})_{n \in \mathbb{N}}$ est une chaîne de Markov de condition initiale x et indépendante de $X_0, \dots, X_m$.

1.4.2 État transiant et état récurrent

Définition (État récurrent/transiant) . Si $x \in E$ tel que $\mathbb{P}_x(\tau_x^+ < \infty) = 1$, on dit alors que x est *récurrent* et $V_x = \infty$. Sinon, on dit que x est *transiant* et $V_x < \infty$ presque sûrement.

Proposition On dit qu'un état $x \in E$ est *récurrent* si :

$$\sum_{n \geq 0} P^n(x, x) = \infty$$

Or $P^n(x, x) = \mathbb{P}(X_n = x \mid X_0 = x)$ et $V_x = \sum_{n \geq 1} \mathbb{1}_{X_n = x}$ donc :

$$\mathbb{E}(V_x) = \sum_{n \geq 1} \mathbb{E}(\mathbb{1}_{X_n = x} \mid X_0 = x) = \sum_{n \geq 1} P^n(x, x)$$

Donc si x est *récurrent* alors $V_x = \infty$ presque sûrement. D'où :

$$\mathbb{E}(V_x) = \infty \implies \sum_{n \geq 1} P^n(x, x) = \infty$$

D'autre part, si x est *transiant*, alors $V_x \sim \mathcal{G}(p)$ où :

$$\mathbb{E}(V_x) < \infty \implies \sum_{n \geq 1} P^n(x, x) < \infty$$

Théorème (Récurrence et Transitivité) . Soient $x, y \in E$. Alors, si x est récurrent et que $x \rightsquigarrow y$ alors y est récurrent.

Proposition Donc $\rightsquigarrow$ est une relation d'équivalence pour les points récurrents. On peut donc partitionner l'ensemble des points récurrents de E en classes d'équivalences pour $\rightsquigarrow$ (i.e en parties connexes dans le graphe de la chaîne).

$$E = \mathcal{R} \sqcup \mathcal{C}$$

où $\mathcal{R}$ est l'ensemble des états récurrents de E et $\mathcal{C}$ l'ensemble des états transients. On a donc :

$$\mathcal{R} = \bigsqcup_{x \in E} \mathcal{R}_x$$

avec $\mathcal{R}_x$ la classe de x pour la relation $\rightsquigarrow$

$$\text{i.e } \forall x \in E, \quad \mathcal{R}_x = \{y \in E, x \rightsquigarrow y\}$$

Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur E .

- Si $X_0 = x \in \mathcal{R}$ alors pour tout $n \in \mathbb{N}$, $X_n \in \mathcal{R}_x$ presque sûrement. Donc la chaîne visitera tout les états de cette classe un nombre infini de fois.
- En revanche si $x \in \mathcal{C}$ on peut distinguer deux cas :
 - Si $\mathcal{C}$ est infini, alors pour tout $n \in \mathbb{N}$, $X_n \in \mathcal{C}$ et chaque état de $\mathcal{C}$ est visité un nombre fini de fois.
 - Si $\mathcal{C}$ est fini, alors il va exister $n \in \mathbb{N}$ tel que $X_n \in \mathcal{R}$ et on se retrouve dans le premier cas.

Exemple Soit E un espace d'état fini et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur E définie par le graphe suivant :

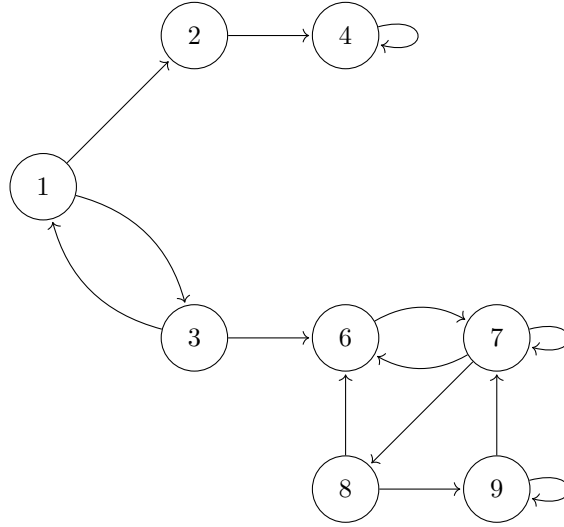


FIGURE 1.5 – Espace d'état fini

On a alors une partition de E :

$$E = \underbrace{\{1, 2, 3\}}_{\text{états transients}} \cup \underbrace{\{\{4\} \cup \{6, 7, 8, 9\}\}}_{\substack{\mathcal{R}_4 \quad \mathcal{R}_2 \\ \text{états récurrents}}}$$

1.5 Mesures invariantes dans le cas dénombrable

1.5.1 États récurrents positif ou nul

De puis le début de ce chapitre, nous nous plaçons dans un ensemble d'états E fini. Or ce n'est pas toujours le cas. En général, E est infini et dénombrable. Nous pouvons donc définir une mesure de probabilité invariante π telle que :

$$\forall x \in E, \quad \pi(x) = \frac{1}{\mathbb{E}(\tau_x^+ \mid X_0 = x)}$$

Du fait de la dénombrabilité de E , nous aurons plusieurs classes d'états qui communiquent entre eux. On peut donc redéfinir la notion d'*irréductibilité* pour ces chaînes.

Définition (Chaîne irréductible (cas dénombrable)) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov homogène sur un espace d'état E dénombrable. On dira que $(X_n)_{n \in \mathbb{N}}$ est irréductible :

- si tous les états de E sont transients
- si il existe une unique classe de récurrence dans E

$$\text{i.e } \forall x, y \in E, \quad x \text{ et } y \text{ récurrent et } x \rightsquigarrow y$$

Remarque Si on considère une chaîne de Markov irréductible et récurrente alors :

$$\forall x \in E, \mathbb{P}_x(\tau_x^+ < \infty) = 1$$

Or quand E est fini, on n'a pas forcément :

$$\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$$

Cela va donc nous mener à introduire une classification plus fine des états en distinguant les états récurrents selon que $\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$ ou pas.

Définition (État récurrent positif) . On dit qu'un état $x \in E$ est *récurrent positif* s'il est récurrent et que $\mathbb{E}(\tau_x^+ \mid X_0 = x) < \infty$. On dira en revanche que x est *récurrent nul* s'il est récurrent et que :

$$\mathbb{P}_x(\tau_x^+ < \infty) = 1 \text{ et } \mathbb{E}_x(\tau_x^+) = \infty$$

La notion de récurrence positive ou nulle permet de quantifier la vitesse de retour de la chaîne dans un état x . Un état récurrent positif est donc un état où la chaîne revient souvent alors qu'un état récurrent nul est un état où la chaîne revient en temps fini mais potentiellement très long. C'est ce que traduit le fait que $\mathbb{E}_x(\tau_x^+)$ soit fini ou pas.

Remarque Dans le cas des espaces d'états finis, lors de la construction de la mesure de probabilités invariante, nous avons posé :

$$\bar{\pi}(y) = \mathbb{E} \left(\sum_{n=0}^{\tau_x^+ - 1} \mathbb{1}_{X_n = y} \right)$$

en tant que mesure invariante de masse totale $\mathbb{E}_x(\tau_x^+)$. Or dans le cas actuel si $(X_n)_{n \in \mathbb{N}}$ est récurrente positive, on aura $\mathbb{E}_x(\tau_x^+) < \infty$ donc on pourra construire notre mesure de probabilités invariante. Dans le cas contraire, notre mesure ne sera pas de probabilité.

1.5.2 Propriétés des chaînes irréductibles

Propriété (Positivité) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible récurrente. Si π est une mesure de probabilités invariante non nulle, alors :

$$\forall x \in E, \quad \pi(x) > 0.$$

Démonstration Comme π est non nulle, il existe $x_0 \in E$ tel que $\pi(x_0) > 0$. Comme la chaîne est irréductible, pour tout $y \in E$, il existe $n_0 \geq 0$ tel que :

$$P^{n_0}(x_0, y) > 0$$

En utilisant l'invariance de π et en isolant le terme $z = x_0$, on obtient :

$$\pi(y) = \sum_{z \in E} \pi(z) P^{n_0}(z, y) \geq \pi(x_0) P^{n_0}(x_0, y) > 0$$

Ainsi, chaque état $y \in E$ a une mesure strictement positive sous π .

Théorème (Unicité) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible et récurrente. Il existe alors une unique mesure invariante à constante multiplicative près.

- De plus, si tous les états sont *récurrents nuls* alors toute mesure invariante non nulle est de masse infinie.
- En revanche, si $(X_n)_{n \in \mathbb{N}}$ est *récurrente positive*, alors toutes les mesures invariantes sont de masse finie donc il existe une unique mesure de probabilités invariante telle que :

$$\forall x \in E, \quad \pi(x) = \frac{1}{\mathbb{E}_x(\tau_x^+)}$$

Exemple (Modèle de file d'attente) Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov définie par le graphe suivant :

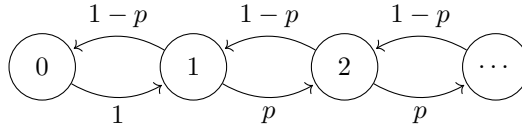


FIGURE 1.6 – Graphe d'un modèle de file d'attente

Donc les transitions sont décrites par la matrice de transition P :

$$\begin{cases} P(k, k+1) = p & \text{si } k \geq 1 \\ P(k, k-1) = 1-p & \text{si } k \geq 1 \\ P(0, 1) = 1 \end{cases}$$

Cette chaîne de Markov est irréductible car :

$$\forall x, y \in \mathbb{N}, \exists n \in \mathbb{N}, \quad \mathbb{P}_x(X_n = y) > 0$$

$$\iff \forall x, y \in \mathbb{N}, \exists n \in \mathbb{N}, \quad P^n(x, y) > 0$$

Déterminons une mesure invariante pour cette chaîne de Markov. Soit π une mesure invariante pour P . Alors elle vérifie $\pi P = \pi$ d'où :

$$\begin{cases} \pi(0) = \sum_{k \in \mathbb{N}} \pi(k) P(k, 0) = (1-p)\pi(1) \\ \pi(1) = \pi(0) + (1-p)\pi(2) \end{cases}$$

et $\forall k \geq 1$, on a $\pi(k) = p\pi(k-1) + (1-p)\pi(k+1)$. Nous avons donc une relation de récurrence d'ordre 2 :

$$Q = (1-p)X^2 - X + p$$

de racines évidentes 1 et $p/(1-p)$. Les solutions de la relation de récurrence sont donc de la forme :

$$\forall k \geq 1, \quad \pi(k) = a1^k + b\left(\frac{p}{1-p}\right)^k$$

Déterminons a et b . On a :

$$\begin{cases} \pi(1) = \frac{\pi(0)}{1-p} = a + \frac{bp}{1-p} \\ \pi(2) = \frac{\pi(0)}{1-p} \left(\frac{1}{1-p} - 1 \right) = \frac{\pi(0)p}{(1-p)^2} = a + b\frac{p^2}{(1-p)^2} \end{cases}$$

Par identification, on obtient donc :

$$a = 0 \quad \text{et} \quad b = \frac{\pi(0)}{p}$$

Donc la solution de l'équation de récurrence s'écrit :

$$\forall k \geq 1, \quad \pi(k) = \frac{\pi(0)}{p} \left(\frac{p}{1-p} \right)^k$$

Pour aller plus loin, en considérant la série $\sum_{k \geq 0} \pi(k)$ on peut affirmer que si $p < 1/2$ alors la mesure peut être de probabilité. Dans le cas contraire, toutes les mesures invariantes seront de masse infinie.

1.5.3 Critère de récurrence positive

Il peut être très difficile de calculer $\mathbb{E}_x(\tau_x^+)$ car il faut très bien connaître les propriétés des excursions de la chaîne. En pratique, on utilisera le critère suivant :

Théorème (Réciproque) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible. Si il existe une mesure invariante π non nulle et de masse finie alors la chaîne est récurrente positive.

Ce théorème traduit le fait qu'il y a équivalence entre le fait qu'une chaîne de Markov soit récurrente positive et qu'elle admette une mesure de probabilités invariante.

Démonstration Revenons à notre théorème d'unicité des mesures invariantes. Le principe de la démonstration est le même que dans le cas discret. Soit π une mesure invariante non nulle de masse finie. On veut comparer π avec $\bar{\pi}$ qui compte le nombre moyen de passage en $y \in \mathbb{E}$.

$$\forall x \in E, \quad \bar{\pi}(x) = \mathbb{E}_x \left(\sum_{n=0}^{\tau_x^+-1} \mathbb{1}_{X_n=y} \right) = \sum_{n \geq 0} \mathbb{P}_x(X_n = y, \tau_x^+ > n)$$

or π est invariante donc $\forall y \in E$, on a :

$$\begin{aligned}
 \pi(y) &= \sum_{z \in E} \pi(z)P(z, y) \\
 &= \pi(x)P(x, y) + \sum_{z_1 \neq x} \pi(z_1)P(z_1, y) \\
 &= \pi(x)P(x, y) + \sum_{z_1 \neq x} \left(\sum_{z_2 \in E} \pi(z_2)P(z_2, z_1) \right) P(z_1, y) \\
 &= \pi(x)P(x, y) + \underbrace{\pi(x) \sum_{z_1 \neq x} P(x, z_1)P(z_1, y)}_{\text{terme } z_2=x} + \sum_{z_1 \neq x} \left(\sum_{z_2 \neq x} \pi(z_2)P(z_2, z_1)P(z_1, y) \right)
 \end{aligned}$$

où $\pi(x) \sum_{z_1 \neq x} P(x, z_1)P(z_1, x) = \mathbb{P}(X_2 = y, X_1 \neq x, X_0 = x)$. Si on itère $l \in \mathbb{N}$ fois ce processus, on aura donc :

$$\begin{aligned}
 \pi(y) &= \pi(x) \left(P(x, y) + \sum_{k=1}^{l-1} \sum_{\substack{z_1 \neq x \\ \vdots \\ z_k \neq x}} P(x, z_k)P(z_k, z_{k-1}) \dots P(z_1, y) \right) \\
 &\quad + \sum_{\substack{z_1 \neq x \\ \vdots \\ z_l \neq x}} \pi(z_l)P(z_l, z_{l-1}) \dots P(z_1, y)
 \end{aligned}$$

Donc

$$\pi(y) \geq \pi(x) \left[\sum_{k=1}^{l-1} \mathbb{P}_x(X - k = y, \tau_x^+ > k) \right]$$

or ceci est valable pour tout $l \in \mathbb{N}$ donc on peut faire tendre l vers ∞ d'où :

$$\pi(y) \geq \pi(x)\bar{\pi}(y)$$

Si on choisit $x \in E$ tel que $\pi(x) > 0$ alors :

$$\infty > \frac{\pi(y)}{\pi(x)} \geq \bar{\pi}(y)$$

donc $\bar{\pi}$ est bien définie. Or $\bar{\pi}$ est définie de telle sorte que :

$$\sum_{y \in E} \bar{\pi}(y) = \mathbb{E}_x(\tau_x^+) \leq \sum_{y \in E} \frac{\pi(y)}{\pi(x)}$$

or $\sum_{y \in E} \frac{\pi(y)}{\pi(x)} < \infty$ car π est finie donc :

$$\forall x \in E, \quad \mathbb{E}_x(\tau_x^+) < \infty$$

i.e x est récurrent positif donc la chaîne des récurrente positive irréductible. On peut donc construire l'unique mesure de probabilité invariante.

1.6 Ergodicité et convergence en loi

1.6.1 Ergodicité

On cherche ici à généraliser la loi forte des grands nombres valable pour une suite iid de variables aléatoires $(Y_i)_{i \in \mathbb{N}}$ telles que $\mathbb{E}(Y_1) < \infty$ telle que :

$$\frac{1}{n} \sum_{i=1}^n Y_i \xrightarrow[n \rightarrow \infty]{} \mathbb{E}(Y_1) \quad \text{p.s.}$$

Le théorème suivant est son équivalent pour les chaînes de Markov.

Théorème (Birhoff - Ergodique) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible récurrente positive et π sa mesure invariante. Pour toute fonction $f : E \rightarrow \mathbb{R}$ telle que :

$$\pi(|f|) = \sum_{x \in E} |f(x)| \pi(x) < \infty$$

on a :

$$\boxed{\frac{1}{n} \sum_{i=1}^n f(X_i) \xrightarrow[n \rightarrow \infty]{} \pi(f)} \quad \text{p.s.}$$

$$\stackrel{\text{def}}{\iff} \mathbb{P} \left(\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n f(X_i) = \pi(f) \right) = 1$$

Remarque On traitera souvent le cas où $f = \mathbb{1}_{x=y}$ pour $y \in E$. On aura alors :

$$\frac{1}{n} \sum_{i=1}^n \mathbb{1}_{X_i=y} \xrightarrow[n \rightarrow \infty]{} \pi(y)$$

donc le nombre moyen de passages de $(X_n)_{n \in \mathbb{N}}$ en y tend vers la mesure invariante en y . C'est très utile pour déterminer numériquement une mesure invariante.

Proposition Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov. On définit le nombre de visites de la chaîne en $y \in E$ avant le temps n comme :

$$V_{y,n} = \sum_{i=0}^n \mathbb{1}_{X_i=y}$$

Si $(X_n)_{n \in \mathbb{N}}$ est irréductible et transiente ou récurrente nulle, alors :

$$\forall y \in E, \quad \frac{V_{y,n}}{n} \xrightarrow[n \rightarrow \infty]{} 0$$

Si $(X_n)_{n \in \mathbb{N}}$ est récurrente positive, on a :

$$\forall y \in E, \quad \frac{V_{y,n}}{n} \xrightarrow[n \rightarrow \infty]{} \pi(y)$$

1.6.2 Convergence en loi

Une conséquence du théorème ergodique est que pour $(X_n)_{n \in \mathbb{N}}$ irréductible récurrente positive on ait :

$$\frac{1}{n} \sum_{i=1}^n \mathbb{1}_{X_i=y} \xrightarrow[n \rightarrow \infty]{} \pi(y)$$

d'où, en passant à l'espérance :

$$\frac{1}{n} \sum_{i=1}^n \mathbb{P}(X_i = y) \xrightarrow{n \rightarrow \infty} \pi(y)$$

On veut donc savoir sous quelles conditions on aura la convergence en loi suivante $\lim_{n \rightarrow \infty} \mathbb{P}(X_n = y) = \pi(y)$.

Définition (apériodicité) . Une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ irréductible est *apériodique* si :

$$\boxed{\forall x, y \in E, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, P^n(x, y) > 0}$$

$$\iff \text{pgcd}(\{n \in \mathbb{N} \mid P^n(x, y) > 0\}) = 1$$

Lemme Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne irréductible. S'il existe $x_0 \in E$ et $n \in \mathbb{N}$ tel que $P^n(x_0, x_0) > 0$ alors la chaîne est apériodique.

Démonstration Soient $y, z \in E$, puisque $(X_n)_{n \in \mathbb{N}}$ est irréductible, il existe $r, s \in \mathbb{N}$ tels que $P^r(x, y) > 0$ et $P^s(y, x) > 0$ On en déduit donc que pour n suffisamment grand :

$$P^{r+n+s}(y, z) \geq P^r(y, x)P^n(x, x)P^s(x, z) > 0$$

Théorème (Convergence en Loi) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov irréductible et apériodique. Distinguons deux cas :

1. Si $(X_n)_{n \in \mathbb{N}}$ est *récurrente positive*, alors quelque soit la loi initial π_0 , on a :

$$\mathbb{P}_{\pi_0}(X_n = x) \xrightarrow{n \rightarrow \infty} \pi(x)$$

(i.e X_n converge en loi vers π).

2. Si $(X_n)_{n \in \mathbb{N}}$ est *transiente* ou *récurrente nulle*, alors :

$$\mathbb{P}_{\pi_0}(X_n = x) \xrightarrow{n \rightarrow \infty} 0$$

Remarque On peut donc approcher π la mesure invariante de deux façons. Soit par le théorème ergodique :

$$\frac{1}{n} \sum_{i=1}^n \mathbb{P}(X_i = y) \xrightarrow{n \rightarrow \infty} \pi(y)$$

grâce à une seule réalisation de $(X_1, \dots, X_n)$. Soit grâce au théorème de convergence en loi : pour n grand, X_n aura une loi proche de $\pi(y)$. Ainsi pour m réalisations de X_n ($X_n^1, \dots, X_n^m$) on aura :

$$\frac{1}{m} \sum_{i=1}^m \mathbb{1}_{X_n^i = y} \xrightarrow{n \rightarrow \infty} \pi(y)$$

Chapitre 2

Espérances Conditionnelles

Contents

2.1	Espérances conditionnelles sur un espace d'états discret	31
2.1.1	Exemple Introductif	31
2.2	Définition de l'espérance conditionnelle	33
2.2.1	Généralités	33
2.2.2	Espérance conditionnelle	34
2.2.3	Pour les variables à densité	35
2.3	Propriétés de l'espérance conditionnelle	35
2.4	Processus aléatoires - Filtrations	37

Dans ce court chapitre nous allons nous attarder sur la notion d'espérance. Jusqu'ici, ce n'était qu'un outil pour décrire la tendance d'une série de réalisations d'une variable aléatoire. Nous allons ici nous permettre de la conditionner pour en déduire des informations sur notre variable de départ.

2.1 Espérances conditionnelles sur un espace d'états discret

2.1.1 Exemple Introductif

Une entreprise souhaite effectuer un questionnaire de satisfaction en vue d'améliorer ses services. Pour chaque client, le résultat du sondage peut être représenté par une variable aléatoire X à valeurs dans $E = \{1, \dots, 100\}$. le résultat du sondage sera la moyenne empirique des réponses et il donnera une bonne estimation de $\mathbb{E}(X)$.

Or, pour permettre de personnaliser un peu plus ses services, l'entreprise souhaite connaître les caractéristiques des clients ayant donné le même score au sondage. Par exemple, elle souhaite étudier les réponses moyennes des clients en fonction de leur âge, de leur sexe, etc... Par exemple si Y est la variable aléatoire qui représente l'âge d'un client, on souhaite obtenir la probabilité qu'un client d'âge y attribue la note de x . Ce sera donc la quantité suivante :

$$\mathbb{P}(X = x \mid Y = y) = \frac{\mathbb{P}(X = x, Y = y)}{\mathbb{P}(Y = y)}$$

que l'on appelle *probabilité conditionnelle* de x sachant y . On peut ainsi définir une nouvelle

mesure de probabilités appelée *probabilité conditionnelle* telle que, pour y fixé, on ait :

$$\mathbb{P}(X = \bullet \mid Y) = \frac{\mathbb{P}(X = \bullet, Y = y)}{\mathbb{P}(Y = y)}$$

$$\sum_{x \in E} \mathbb{P}(X = x \mid Y = y) = 1$$

Ainsi, dans notre exemple, la note moyenne des clients d'âge y sera *l'espérance conditionnelle* de X sachant $Y = y$ définie par :

$$\mathbb{E}(X \mid Y = y) = \sum_{x \in E} x \mathbb{P}(X = x \mid Y = y)$$

On peut donc définir une fonction $h : E \mapsto \overline{\mathbb{R}}$ telle que :

$$h = \mathbb{E}(X \mid Y) \quad \text{i.e.} \quad \forall y \in E, h(y) = \mathbb{E}(X \mid Y = y)$$

À noter que X et Y ne sont pas nécessairement à valeurs dans le même ensemble.

Exemple Considérons l'expérience aléatoire suivante. On lance un dé équilibré à 6 faces. Soit y son résultat. On lance ensuite y fois une pièce de monnaie équilibrée. On cherche à déterminer le nombre moyen de fois où on tombe sur Pile.

Pour cela, on pose Y une variable aléatoire à valeurs dans $\llbracket 1, 6 \rrbracket$ et, conditionnellement à $\{Y = y\}$ on définit X la variable aléatoire de loi $\mathcal{B}(y, p)$ où p est la probabilité de faire Pile. On a alors :

$$\forall k \in \llbracket 0, y \rrbracket, \quad \mathbb{P}(X = k \mid Y = y) = \binom{y}{k} p^k (1-p)^{y-k}$$

d'où $\mathbb{E}(X \mid Y = y) = yp$ donc $\mathbb{E}(X \mid Y) = pY$.

Propriété (Composition) . Soient deux variables aléatoires discrètes X et Y on a l'égalité suivante :

$$\mathbb{E}(\mathbb{E}(X \mid Y)) = \mathbb{E}(X)$$

Démonstration Soient X et Y deux variables aléatoires respectivement à valeurs dans E et E' . On a :

$$\begin{aligned} \mathbb{E}(\mathbb{E}(X \mid Y)) &= \sum_{y \in E'} \mathbb{E}(X \mid Y = y) \mathbb{P}(Y = y) \quad (\text{Théorème de transfert}) \\ &= \sum_{y \in E'} \mathbb{P}(Y = y) \left(\sum_{x \in E} x \mathbb{P}(X = x \mid Y = y) \right) \\ &= \sum_{y \in E'} \sum_{x \in E} x \mathbb{P}(X = x, Y = y) \\ &= \sum_{x \in E} \sum_{y \in E'} x \mathbb{P}(X = x, Y = y) \\ &= \sum_{x \in E} x \mathbb{P}(X = x) \quad (\text{Probabilités Totales}) \\ &= \mathbb{E}(X) \end{aligned}$$

Proposition Pour reprendre l'exemple précédent, on se donne maintenant $y \in E'$ l'âge d'une personne et on cherche à savoir quelle réponse $x \in E$ cette personne a donnée au sondage. On

souhaite donc déterminer X à partir des informations fournies par Y . On cherche donc $h(Y)$, une variable aléatoire fonction de Y , qui minimise l'erreur quadratique moyenne :

$$\inf_h \mathbb{E}[(X - h(Y))^2] = \inf_h \{ \mathbb{E}(X^2) - 2\mathbb{E}[h(Y)X] + \mathbb{E}[h(Y)^2] \}.$$

On a, par le théorème de transfert :

$$\begin{aligned} \mathbb{E}[h(Y)\mathbb{E}(X|Y)] &= \sum_{y \in E'} h(y)\mathbb{E}(X | Y = y) \mathbb{P}(Y = y) \\ &= \sum_{x \in E} \sum_{y \in E'} x h(y) \mathbb{P}(X = x, Y = y) \\ &= \mathbb{E}[h(Y)X]. \end{aligned}$$

On en déduit que

$$\mathbb{E}[h(Y)(X - \mathbb{E}[X|Y])] = 0.$$

Cette relation peut être interprétée comme une *orthogonalité* : si l'on considère le produit scalaire défini sur les variables aléatoires par

$$\langle U, V \rangle = \mathbb{E}[UV],$$

alors $X - \mathbb{E}[X|Y]$ est orthogonal à toute variable aléatoire de la forme $h(Y)$. Cette interprétation sera approfondie dans la partie suivante.

Ainsi :

$$\begin{aligned} \inf_h \{ \mathbb{E}(X^2) - 2\mathbb{E}[h(Y)X] + \mathbb{E}[h(Y)^2] \} &= \inf_h \{ \mathbb{E}(X^2) - 2\mathbb{E}[h(Y)\mathbb{E}(X|Y)] + \mathbb{E}[h(Y)^2] \} \\ &= \inf_h \{ \mathbb{E}(X^2) - \mathbb{E}[\mathbb{E}(X|Y)^2] + \mathbb{E}[(h(Y) - \mathbb{E}(X|Y))^2] \} \\ &= \mathbb{E}(X^2) - \mathbb{E}[\mathbb{E}(X|Y)^2] + \inf_h \mathbb{E}[(h(Y) - \mathbb{E}(X|Y))^2]. \end{aligned}$$

Le terme d'espérance est minimal lorsque $h(Y) = \mathbb{E}(X|Y)$, ce qui prouve que l'espérance conditionnelle est la meilleure approximation quadratique de X sachant Y .

2.2 Définition de l'espérance conditionnelle

Cette section fait appel à beaucoup de connaissances de théorie de la mesure. Des rappels peuvent s'imposer. On se place ici dans un espace de variables aléatoires de carré intégrable $L^2(\Omega, \mathcal{A}, \mathbb{P})$. C'est un espace de Hilbert doté d'un produit scalaire :

$$\langle \cdot, \cdot \rangle : \begin{cases} (L^2(\Omega, \mathcal{A}, \mathbb{P}))^2 \longrightarrow \mathbb{R} \\ (X, Y) \longmapsto \mathbb{E}(XY) \end{cases}$$

et muni d'une norme quadratique :

$$\|\cdot\|_2 : \begin{cases} L^2(\Omega, \mathcal{A}, \mathbb{P}) \longrightarrow \mathbb{R} \\ X \longmapsto \sqrt{\mathbb{E}(X^2)} \end{cases}$$

2.2.1 Généralités

Définition (Tribu engendrée) . Soit $(\Omega, \mathcal{A}, \mathbb{P})$ un espace de probabilité et $X : \Omega \rightarrow \mathbb{R}$ une variable aléatoire. On définit la tribu engendrée par X , notée $\sigma(X)$, est la tribu engendrée par les sous-ensembles de Ω de la forme $\{X \leq a\}, \forall a \in \mathbb{R}$.

De même, $\sigma(X)$ contient tout les évènement de la forme $\{X \in B\}$ avec $B \in \mathcal{B}_{\mathbb{R}}$. Plus généralement, on peut définir $\sigma(X_1, \dots, X_n)$ pour une collection de n variables aléatoire $X_1, \dots, X_n$. Intuitivement, $\sigma(X)$ contient toute l'information associée à la variable X .

Définition (Variable aléatoire mesurable par rapport à X) . Soit $(\Omega, \mathcal{A}, \mathbb{P})$ un espace de probabilité et $X : \Omega \rightarrow \mathbb{R}$ une variable aléatoire. Une variable aléatoire $Z : \Omega \rightarrow \mathbb{R}$ est dite *mesurable par rapport à X* si elle est mesurable pour la tribu $\sigma(X)$, c'est-à-dire si :

$$\forall B \in \mathcal{B}_{\mathbb{R}}, \quad Z^{-1}(B) \in \sigma(X).$$

Lemme Soient Y, W deux variables aléatoires sur $(\Omega, \mathcal{A}, \mathbb{P})$ telles que Y est à valeurs dans $(E, \mathcal{E})$ et W dans $(\mathbb{R}, \mathcal{B}_{\mathbb{R}})$. W est mesurable par rapport à $\sigma(Y)$ si et seulement si il existe $f \in \mathcal{M}(E, \mathbb{R})$ telle que $W = f(Y)$.

2.2.2 Espérance conditionnelle

Nous allons maintenant définir l'espérance conditionnelle comme une projection orthogonale.

Proposition Soient $X, Y \in L^2(\Omega, \mathcal{A}, \mathbb{P})$. On souhaite approcher X par une variable aléatoire mesurable par rapport à Y . D'après le lemme précédent, toute variable aléatoire mesurable par rapport à Y est de la forme $h(Y)$ avec une fonction borélienne $h : \mathbb{R} \rightarrow \mathbb{R}$.

On cherche donc à approcher X à partir des variables du sous-espace :

$$\mathcal{H} = \{h(Y) \mid h \in L^2(\mathbb{R}, \mathcal{B}_{\mathbb{R}}, \mathbb{P}_Y)\} \subset L^2(\Omega, \mathcal{A}, \mathbb{P}).$$

La variable aléatoire de $\mathcal{H}$ la plus proche de X est, par définition, le *projeté orthogonal* de X sur $\mathcal{H}$.

Définition (Espérance conditionnelle comme projection) . Dans le même contexte que précédemment, l'espérance conditionnelle $\mathbb{E}(X \mid Y)$ est le *projeté orthogonal* de X sur le sous-espace $\mathcal{H}$ défini ci-dessus. Elle est caractérisée par la relation d'orthogonalité :

$$\forall h(Y) \in \mathcal{H}, \quad \mathbb{E}[(X - \mathbb{E}(X \mid Y))h(Y)] = 0.$$

On peut étendre cette définition au cas L^1 grâce au théorème suivant.

Théorème (Existence) . Soit $X \in L^1(\Omega, \mathcal{A}, \mathbb{P})$ et $\mathcal{F}$ une sous-tribu de $\mathcal{A}$. Alors il existe une unique variable aléatoire Z , mesurable par rapport à $\mathcal{F}$, telle que :

- i) $\mathbb{E}(|Z|) < \infty$,
- ii) pour tout $W \in L^\infty(\Omega, \mathcal{F}, \mathbb{P})$, on a

$$\mathbb{E}(XW) = \mathbb{E}(ZW).$$

On appelle Z l'*espérance conditionnelle* de X sachant $\mathcal{F}$ et on la note $\mathbb{E}(X \mid \mathcal{F})$.

2.2.3 Pour les variables à densité

Rappel (Loi jointe et lois marginales) Soit (X, Y) un couple de variables aléatoires à densité de loi jointe $f_{X,Y}$. La *loi marginale* de y par rapport à x est obtenue par :

$$f_Y(y) = \int_{\mathbb{R}} f_{X,Y}(x, y) dx$$

de même pour la *loi marginale* de x :

$$f_X(x) = \int_{\mathbb{R}} f_{X,Y}(x, y) dy$$

Définition (Probabilité Conditionnelle) . De même que pour les variables discrètes, on définit la probabilité conditionnelle de X par sa densité :

$$f_{X|Y=y} = \frac{f_{X,Y}(x, y)}{f_Y(y)}$$

Ainsi pour $y \in \mathbb{R}$ fixé tel que $f_Y(y) > 0$, $f_{X|Y=y}$ définit une densité sur $\mathbb{R}$ et on peut lui associer :

$$\mathbb{E}(X | Y = y) = \int_{\mathbb{R}} x f_{X|Y=y}(x) dx$$

Définition (Espérance Conditionnelle (Cas continu)) . Soient deux variables aléatoires X et Y à densité de loi jointe $f_{X,Y}$. On définit l'espérance conditionnelle de X par rapport à Y comme la fonction :

$$h : y \mapsto \mathbb{E}(X | Y = y) = \int_{\mathbb{R}} x f_{X|Y=y}(x) dx$$

Remarque La condition d'orthogonalité du théorème précédent se vérifie de la même façon. Pour $h : \mathbb{R} \rightarrow \mathbb{R}$ bornée, on a :

$$\begin{aligned} \mathbb{E}(\mathbb{E}(X | Y)h(Y)) &= \int_{\mathbb{R}} f_Y(y)h(y)\mathbb{E}(X | Y = y) dy \\ &= \int_{\mathbb{R}} f_Y(y)h(y) \left(\int_{\mathbb{R}} x f_{X|Y=y}(x) dx \right) dy \\ &= \int_{\mathbb{R}^2} xh(y)f_{X|Y=y}(x)f_Y(y) dxdy \\ &= \int_{\mathbb{R}^2} xh(y)f_{X,Y}(x, y) dxdy \\ &= \mathbb{E}(Xh(Y)) \end{aligned}$$

2.3 Propriétés de l'espérance conditionnelle

Voyons à présent quelques propriétés de l'espérance conditionnelle. Soit $(\Omega, \mathcal{A}, \mathbb{P})$ un espace de probabilités.

Propriété (Calcul) . Soit $\mathcal{F}$ une sous-tribu de $\mathcal{A}$ et $X \in L^1(\mathcal{A}, \mathbb{P})$ une variable aléatoire. On a :

- i) $\mathbb{E}(\bullet \mid \mathcal{F})$ est *linéaire*.
- ii) si $X \geq 0$ alors $\mathbb{E}(X \mid \mathcal{F}) \geq 0$ presque sûrement.
- iii) si X est $\mathcal{F}$ -mesurable alors $\mathbb{E}(X \mid \mathcal{F}) = X$ presque sûrement.

Propriété (Indépendance) . Si X est *indépendante* de $\mathcal{F}$ alors :

$$\mathbb{E}(X \mid \mathcal{F}) = \mathbb{E}(X)$$

Remarque Soit Y, X deux variables aléatoires sur $(\Omega, \mathcal{A}, \mathbb{P})$. On pose $\mathcal{F} = \sigma(Y)$. Dire que X est *indépendante* de $\mathcal{F}$ revient à dire que X est indépendante de Y . On aura alors :

$$\mathbb{E}(Xh(Y)) \stackrel{!}{=} \mathbb{E}(X)\mathbb{E}(h(Y)) = \mathbb{E}(\mathbb{E}(X)h(Y))$$

Les théorèmes de convergence (monotone et dominée) de l'intégration sont toujours valables pour l'espérance conditionnelle.

Théorème (Convergence Monotone) . Soit $(X_n)_{n \in \mathbb{N}}$ une suite de variables aléatoires et X dans $L^1(\mathcal{A}, \mathbb{P})$. Si $X_n \xrightarrow[n \rightarrow \infty]{} X$ p.s en croissant, alors :

$$\lim_{n \rightarrow \infty} \mathbb{E}(X_n \mid \mathcal{F}) = \mathbb{E}(X \mid \mathcal{F}) \quad \text{p.s}$$

Théorème (Convergence Dominée) . Soit $(X_n)_{n \in \mathbb{N}}$ une suite de variables aléatoires et X dans $L^1(\mathcal{A}, \mathbb{P})$.

$$\begin{aligned} X_n \xrightarrow[n \rightarrow \infty]{} X \text{ p.s} \quad \text{et} \quad \sup_n |X_n| \leq w \in L^1 \\ \implies \mathbb{E}(X_n \mid \mathcal{F}) \xrightarrow[n \rightarrow \infty]{} \mathbb{E}(X \mid \mathcal{F}) \end{aligned}$$

Théorème (Inégalité de Jensen) . Soit $g : \mathbb{R} \rightarrow \mathbb{R}$ une fonction *convexe* et $X \in L^1(\mathcal{A}, \mathbb{P})$. Alors :

$$\mathbb{E}(g(X) \mid \mathcal{F}) \geq g(\mathbb{E}(X \mid \mathcal{F}))$$

Proposition Soient $X \in L^1(\mathcal{A}, \mathbb{P})$ et $\mathcal{F}, \mathcal{G}$ deux sous-tribus de $\mathcal{A}$. Alors :

- i) Si $\mathcal{F} \subset \mathcal{G}$ (i.e $\mathcal{G}$ contient plus d'informations que $\mathcal{F}$) alors :

$$\mathbb{E}(\mathbb{E}(X \mid \mathcal{G}) \mid \mathcal{F}) = \mathbb{E}(X \mid \mathcal{F})$$

(Puisque $\mathcal{F}$ contient moins d'information que $\mathcal{G}$, projeter X sur $\mathcal{G}$ puis sur $\mathcal{F}$ revient directement à projeter X sur $\mathcal{F}$).

- ii) Si $\mathcal{F}$ est indépendante de $\mathcal{G}$ et $X \perp \mathcal{G}$, alors :

$$\mathbb{E}(X \mid \sigma(\mathcal{F}, \mathcal{G})) = \mathbb{E}(X \mid \mathcal{F})$$

(Rajouter une information $\mathcal{G}$ qui n'a rien à voir avec l'expérience ne permet pas d'améliorer la prédiction $\mathbb{E}(X \mid \mathcal{F})$).

- iii) Soit Y une variable aléatoire mesurable par rapport à $\mathcal{F}$ et que $XY \in L^1(\mathcal{A}, \mathbb{P})$, alors :

$$\mathbb{E}(XY \mid \mathcal{F}) = Y\mathbb{E}(X \mid \mathcal{F})$$

(Si Y ne dépend que de l'information contenue dans $\mathcal{F}$, alors sa meilleure estimation $\mathbb{E}(Y \mid \mathcal{F})$ est elle-même.)

2.4 Processus aléatoires - Filtrations

Les seuls processus que nous avons vu jusqu'à présent sont des suites de variables aléatoires indexées par le temps tels que les chaînes de Markov. Nous allons ici essayer de généraliser cette notion en introduisant la notion de *filtration*.

Définition (Filtration) . Soit $(\Omega, \mathcal{A}, \mathbb{P})$ un espace probabilisé. Une *filtration* de cet espace est une suite *croissante* de tribus $(\mathcal{F}_n)_{n \in \mathbb{N}}$ telles que :

$$\mathcal{F}_1 \subset \mathcal{F}_2 \subset \dots \subset \mathcal{A}$$

Une filtration d'un processus sera donc l'information disponible jusqu'au temps n de ce processus. Sa croissance vient du fait que le processus accumule de l'information au cours de son évolution.

On parlera ainsi d'espace de probabilité filtré, noté $(\Omega, \mathcal{A}, \mathcal{F}, \mathbb{P})$.

Dans le cadre précédent de l'étude des chaînes de Markov, nous avons étudié des suites $(X_n)_{n \in \mathbb{N}}$ à valeurs dans $(\mathbb{R}, \mathcal{B}_{\mathbb{R}})$. On se concentrera donc sur l'étude des filtrations de cet espace mesurable. Nous pouvons bien sûr réexprimer les chaînes de Markov à l'aide des filtrations en définissant la notion de *filtration naturelle* d'une chaîne de Markov.

Définition (Filtration Naturelle) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur $(\Omega, \mathcal{A}, \mathbb{P})$ à valeurs dans $(E, \mathcal{E})$. Sa filtration naturelle est la suite croissante de sous-tribus $\mathcal{F}_n$ de $\mathcal{A}$ telles que :

$$\forall n \in \mathbb{N}, \quad \mathcal{F}_n = \sigma(X_0, \dots, X_n) \quad (2.4.1)$$

dans le cas où E est discret, on aura donc que :

$$\forall n \in \mathbb{N}, \quad \mathcal{F}_n = \sigma(\{X_0 = x_0, \dots, X_n = x_n \mid \forall x_0, \dots, x_n \in E\})$$

Remarque Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur un espace d'états discret. On a donc défini sa filtration naturelle comme :

$$\forall n \in \mathbb{N}, \quad \mathcal{F}_n = \sigma(\{X_0 = x_0, \dots, X_n = x_n \mid \forall x_0, \dots, x_n \in E\})$$

D'où $\forall B \in \mathcal{E}$, par propriété de Markov, on a :

$$\mathbb{P}(X_{n+1} \in B \mid \mathcal{F}_n) = \mathbb{P}(X_{n+1} \in B \mid X_n)$$

qui est une variable aléatoire à valeurs dans $(\mathbb{R}, \mathcal{B}_{\mathbb{R}})$ qui dépend des valeurs prises par X_n et dont la valeur est $\mathbb{P}(X_{n+1} \in B \mid X_n = x_n)$ si $X_n = x_n$.

De plus, pour toute fonction mesurable $h : (E, \mathcal{E}) \rightarrow (\mathbb{R}, \mathcal{B}_{\mathbb{R}})$, l'espérance conditionnelle de $h(X_{n+1})$ sachant X_n est une variable aléatoire $\sigma(X_n)$ -mesurable définie par :

$$\mathbb{E}[h(X_{n+1}) \mid X_n] = g(X_n)$$

où la fonction $g : (E, \mathcal{E}) \rightarrow (\mathbb{R}, \mathcal{B}_{\mathbb{R}})$ est donnée, dans le cas discret, par :

$$g(x_n) = \sum_{y \in E} h(y) \mathbb{P}(X_{n+1} = y \mid X_n = x_n)$$

De même que précédemment, on pourra dire qu'un processus est *adapté* à une filtration...

Définition (Processus Adapté - Processus Prévisible) . Soit $(\Omega, \mathcal{A}, \mathbb{P})$ un espace de probabilité et $(\mathcal{F}_n)_{n \in \mathbb{N}}$ une filtration de cet espace. Soit $(Y_n)_{n \in \mathbb{N}}$ un processus à valeurs dans $(E, \mathcal{E})$.

- i) On dira que le processus $(Y_n)_{n \in \mathbb{N}}$ est *adapté* à la filtration $(\mathcal{F}_n)_{n \in \mathbb{N}}$ si pour tout $n \in \mathbb{N}$, Y_n est $\mathcal{F}_n$ mesurable.

$$\text{i.e } \forall n \in \mathbb{N}, \quad \forall A \in \mathcal{E}, \quad Y_n^{-1}(A) \in \mathcal{F}_n$$

- ii) On dira que le processus $(Y_n)_{n \in \mathbb{N}}$ est *prévisible* par la filtration $(\mathcal{F}_n)_{n \in \mathbb{N}}$ si pour tout $n \in \mathbb{N}$, Y_n est $\mathcal{F}_{n-1}$ mesurable.

$$\text{i.e } \forall n \in \mathbb{N}, \quad \forall A \in \mathcal{E}, \quad Y_n^{-1}(A) \in \mathcal{F}_{n-1}$$

par convention, on dira que $\mathcal{F}_{-1} = \{\emptyset, \Omega\}$.

La notion de *prévision* signifie que l'on peut connaître Y_n avant le temps n grâce aux informations contenues dans $\mathcal{F}_{n-1}$.

La notion de filtration permet aussi de généraliser les temps d'arrêts.

Définition (Temps d'arrêt) . Un temps d'arrêt T pour une filtration $\mathcal{F}_n$ sur $(\Omega, \mathcal{A}, \mathbb{P})$ est une variable aléatoire à valeurs dans $(\bar{\mathbb{N}}, \mathcal{P}(\bar{\mathbb{N}}))$ telle que :

$$\forall n \in \mathbb{N}, \quad \{T \leq n\} \in \mathcal{F}_n$$

Pour être plus précis, cela revient à $T^{-1}\{0, 1, 2, \dots, n\} = \{\omega \in \Omega \mid T(\omega) \leq n\} \in \mathcal{F}_n$.

Chapitre 3

Martingales

Contents

3.1 Généralités	39
3.1.1 Définition et exemples	39
3.1.2 Inégalité de Jensen et Décomposition de Doob	41
3.2 Stratégies et théorème d'arrêt	43
3.3 Inégalités maximales	46
3.4 Convergence des martingales	47
3.4.1 Critères de convergences dans L^2	47

Nous définissons dans ce chapitre un nouveau type de processus aléatoires plus théorique que les chaînes de Markov : les martingales. Celles-ci possèdent de bonnes propriétés calculatoires et de convergence. Elles ont donc beaucoup d'application, que ce soit dans la finance, ou pour l'étude de la convergence d'algorithmes d'apprentissage.

Dans tout ce chapitre, nous considérerons des processus aléatoires à valeurs dans $\mathbb{R}$.

3.1 Généralités

3.1.1 Définition et exemples

Définition (Martingale) . Soit $(\Omega, \mathcal{A}, \mathcal{F}, \mathbb{P})$ un espace de probabilité filtré et $(X_n)_{n \in \mathbb{N}}$ un processus aléatoire intégrable. On dira que $(X_n)_{n \in \mathbb{N}}$ est

i) une *martingale* par rapport à $(\mathcal{F}_n)_{n \in \mathbb{N}}$ si :

$$\forall n \geq 1, \quad \mathbb{E}(X_n | \mathcal{F}_{n-1}) = X_{n-1} \quad (3.1.1)$$

ii) une *sur-martingale* par rapport à $(\mathcal{F}_n)_{n \in \mathbb{N}}$ si :

$$\mathbb{E}(X_n | \mathcal{F}_{n-1}) \leq X_{n-1} \quad (3.1.2)$$

iii) une *sous-martingale* par rapport à $(\mathcal{F}_n)_{n \in \mathbb{N}}$ si :

$$\mathbb{E}(X_n | \mathcal{F}_{n-1}) \geq X_{n-1} \quad (3.1.3)$$

Exemple (Marche aléatoire simple) Soit $(\varepsilon_n)_{n \in \mathbb{N}}$ une suite indépendante de variables aléatoires à valeurs dans $(E, \mathcal{E})$ telles que $\mathbb{E}(\varepsilon_n) = 0$ pour tout n . On définit une marche aléatoire sur E comme la suite $(S_n)_{n \in \mathbb{N}}$ telle que :

$$S_0 = 0 \quad \forall n \geq 1, S_n = \sum_{k=1}^n \varepsilon_k$$

$(S_n)_{n \in \mathbb{N}}$ est une martingale pour la filtration $\mathcal{F}_n = \sigma(\varepsilon_1, \dots, \varepsilon_n)$. En effet, on a :

$$\begin{aligned} \mathbb{E}(S_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}(S_n + \varepsilon_{n+1} \mid \mathcal{F}_n) \\ &= \mathbb{E}(S_n \mid \mathcal{F}_n) + \mathbb{E}(\varepsilon_{n+1} \mid \mathcal{F}_n) \end{aligned}$$

Or $\varepsilon_{n+1} \perp \varepsilon_0, \dots, \varepsilon_n$ donc $\mathbb{E}(\varepsilon_{n+1} \mid \mathcal{F}_n) = 0$ d'où

$$\mathbb{E}(S_{n+1} \mid \mathcal{F}_n) = \mathbb{E}(S_n) = S_n$$

c'est donc bien une martingale par définition. On peut remarquer que si $\mathbb{E}(S_n) \geq 0$, alors $(S_n)_{n \in \mathbb{N}}$ est une sous-martingale.

Remarque Si $\mathbb{E}(\varepsilon_n) \geq 0, \forall n \in \mathbb{N}$ alors :

$$\mathbb{E}(S_{n+1} \mid \mathcal{F}_n) = S_n + \mathbb{E}(\varepsilon_{n+1}) \geq S_n$$

donc $(S_n)_{n \in \mathbb{N}}$ serait une sous-martingale.

Exemple (Multiplicatif) Soit $(\varepsilon_i)_{i \in \mathbb{N}}$ une suite indépendante de variables aléatoires telles que $\mathbb{E}(\varepsilon_i) = 1$ pour tout $i \in \mathbb{N}$. Posons :

$$M_0 = 1 \quad \text{et} \quad \forall n \in \mathbb{N}, M_n = \prod_{i=1}^n \varepsilon_i$$

et $\mathcal{F}_n = \sigma(\varepsilon_1, \dots, \varepsilon_n)$. Montrons que $(M_n)_{n \in \mathbb{N}}$ est une martingale. Pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned} \mathbb{E}(M_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}(\varepsilon_{n+1} \times M_n \mid \mathcal{F}_n) \\ &= M_n \times \mathbb{E}(\varepsilon_{n+1} \mid \mathcal{F}_n) \\ &= M_n \times \mathbb{E}(\varepsilon_{n+1}) \\ &= M_n \end{aligned}$$

Par définition, $(M_n)_{n \in \mathbb{N}}$ est donc une martingale.

Exemple (Lien avec les chaînes de Markov) On peut généraliser les chaînes de Markov en tant que Martingales grâce à l'utilisation de fonctions harmoniques. Soit $(X_n)_{n \in \mathbb{N}}$ sur un espace d'états discret E et de matrice de transition P . Une fonction harmonique $h : E \rightarrow \mathbb{R}$ vérifie :

$$\forall x \in E, \quad h(x) = \sum_{y \in E} P(x, y) h(y)$$

c'est donc un *vecteur propre à droite* pour P . Supposons de plus que $\forall n \in \mathbb{N}, h(X_n) \in L^1(\mathbb{R})$. Alors $(h(X_n))_{n \in \mathbb{N}}$ sera une martingale pour la filtration $(\mathcal{F}_n)_{n \in \mathbb{N}}$ associée à $(X_n)_{n \in \mathbb{N}}$. En effet,

pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned}
 \mathbb{E}(h(X_{n+1}) \mid \mathcal{F}_n) &= \mathbb{E}(h(X_{n+1}) \mid X_n) \\
 &= \sum_{y \in E} \mathbb{E}(h(X_{n+1}) \mathbb{1}_{X_{n+1}=y} \mid X_n) \\
 &= \sum_{y \in E} \mathbb{P}(X_{n+1} = y \mid X_n) \\
 &= \sum_{y \in E} P(X_n, y) \\
 &= h(X_n)
 \end{aligned}$$

Proposition (Généralisation de la définition) Soit $(M_n)_{n \geq 0}$ une martingale pour une filtration $\mathcal{F}_n$ alors :

$$\forall n \in \mathbb{N}, \forall k \geq 1, \quad \boxed{\mathbb{E}(M_{n+k} \mid \mathcal{F}_n) = M_n} \quad (3.1.4)$$

Démonstration Par récurrence sur k , on a :

- *Initialisation* : pour $k = 1$ c'est la définition.
- *Hérédité* : soit $k \geq 1$ tel que :

$$\mathbb{E}(M_{n+k} \mid \mathcal{F}_n) = M_n$$

alors :

$$\begin{aligned}
 \mathbb{E}(M_{n+k+1} \mid \mathcal{F}_n) &= \mathbb{E}(\mathbb{E}(M_{n+k+1} \mid \mathcal{F}_n) \mid \mathcal{F}_n) \\
 &= \mathbb{E}(M_{n+1} \mid \mathcal{F}_n) = M_n
 \end{aligned}$$

Remarque La proposition précédente nous donne une relation de récurrence pour les espérances :

$$\forall n \in \mathbb{N}, \forall k \geq 1, \quad \mathbb{E}(\mathbb{E}(M_{n+k} \mid \mathcal{F}_n)) = \mathbb{E}(M_n)$$

or $\mathbb{E}(\mathbb{E}(M_{n+k} \mid \mathcal{F}_n)) = \mathbb{E}(M_{n+k})$ d'où :

$$\begin{aligned}
 \mathbb{E}(M_{n+k}) &= \mathbb{E}(M_n) \\
 \iff \boxed{\mathbb{E}(M_n) &= \mathbb{E}(M_0)}
 \end{aligned}$$

Attention, la réciproque est fausse. En effet, n'importe quel processus d'espérance constante n'est pas nécessairement une martingale. Nous essaierons par la suite de généraliser cette remarque pour des temps aléatoires.

3.1.2 Inégalité de Jensen et Décomposition de Doob

Pour rappel, une sous-martingale est un processus (Y_n) adapté à une filtration $(\mathcal{F}_n)_{n \geq 0}$ tel que :

$$\mathbb{E}(Y_{n+1} \mid \mathcal{F}_n) \geq Y_n$$

De plus, pour une fonction convexe $g : \mathbb{R} \rightarrow \mathbb{R}$, pour $X \in L^1(\mathcal{A}, \mathbb{P})$ et $\mathcal{F}$ une sous-tribu de $\mathcal{A}$, l'inégalité de Jensen nous donne :

$$\mathbb{E}(g(X) \mid \mathcal{F}) \geq g(\mathbb{E}(X \mid \mathcal{F})).$$

Elle nous permet donc faire le lien entre les martingales et les sous-martingales.

Corollaire (Inégalité de Jensen) . Soit $(M_n)_{n \in \mathbb{N}}$ une martingale pour une filtration $(\mathcal{F}_n)_{n \geq 0}$. Soit $g : \mathbb{R} \rightarrow \mathbb{R}$ convexe telle que $g(M_n) \in L^1$ pour tout $n \in \mathbb{N}$. Alors $(g(M_n))_{n \in \mathbb{N}}$ est une sous-martingale.

Démonstration On a facilement $\mathbb{E}(g(M_{n+1}) | \mathcal{F}_n) \geq g(\mathbb{E}(M_{n+1} | \mathcal{F}_n)) = g(M_n)$.

Exemple (Marche aléatoire) Soit $(\varepsilon_i)_{i \geq 0}$ une suite iid de variables aléatoires telles que :

$$\forall i \in \mathbb{N}, \quad \mathbb{E}(\varepsilon_i) = 0 \text{ et } \varepsilon_i^2 \in L^1$$

Posons $S_n = \sum_{i=1}^n \varepsilon_i$ et $\mathcal{F}_n = \sigma(\varepsilon_1, \dots, \varepsilon_n)$. $(S_n)_{n \geq 0}$ est donc une martingale pour $\mathcal{F}_n$ (voir exemple précédent). D'après le corollaire de l'inégalité de Jensen, $(S_n^2)_{n \geq 0}$ est une sous-martingale.

On a donc :

$$\mathbb{E}(S_{n+1}^2 | \mathcal{F}_n) \geq \mathbb{E}(S_n^2) \iff \mathbb{E}(S_{n+1}^2) \geq \mathbb{E}(S_n^2)$$

On a donc une suite croissante d'espérances et :

$$\mathbb{E}(S_n^2) = \mathbb{E} \left(\left(\sum_{i=1}^n \varepsilon_i \right)^2 \right) = n\mathbb{E}(\varepsilon_1^2)$$

Essayons donc de construire une martingale en posant :

$$\forall n \in \mathbb{N}, \quad M_n = S_n^2 - \mathbb{E}(S_n^2) = S_n^2 - n\mathbb{E}(\varepsilon_1^2)$$

On a donc, par définition :

$$\mathbb{E}(M_{n+1} | \mathcal{F}_n) = \mathbb{E}(S_{n+1}^2 | \mathcal{F}_n) - (n+1)\mathbb{E}(\varepsilon_1^2 | \mathcal{F}_n)$$

or

$$\begin{aligned} \mathbb{E}(S_{n+1}^2 | \mathcal{F}_n) &= \mathbb{E}[(S_n + \varepsilon_{n+1})^2 | \mathcal{F}_n] \\ &= \mathbb{E}(S_n^2 | \mathcal{F}_n) + 2\mathbb{E}(S_n \varepsilon_{n+1} | \mathcal{F}_n) + \mathbb{E}(\varepsilon_{n+1}^2 | \mathcal{F}_n) \\ &= S_n^2 + 2S_n \mathbb{E}(\varepsilon_{n+1}) + \mathbb{E}(\varepsilon_{n+1}^2) \\ &= S_n^2 + \mathbb{E}(\varepsilon_1^2) \end{aligned}$$

d'où :

$$\mathbb{E}(M_{n+1} | \mathcal{F}_n) = S_n^2 - n\mathbb{E}(\varepsilon_1^2) = M_n$$

On peut donc décomposer notre sous-martingale S_n^2 en une somme de martingale et ce que l'on appellera un processus croissant en n tel que $S_n^2 = M_n + n\mathbb{E}(\varepsilon_1^2)$.

Théorème (Décomposition de Doob) . Soit $(X_n)_{n \geq 0}$ une sous-martingale par rapport à une filtration $\mathcal{F}_n$. Alors, il existe un unique couple de processus $(M_n)_{n \geq 0}$ et $(Z_n)_{n \geq 0}$ tels que :

- i) $(M_n)_{n \geq 0}$ est une martingale pour $\mathcal{F}_n$
- ii) $Z_0 = 0$ et $Z_{n+1} \geq Z_n$ et $(Z_n)_{n \geq 0}$ est prévisible pour $(\mathcal{F}_n)_{n \geq 0}$.

et :

$$\forall n \in \mathbb{N}, \quad \boxed{X_n = M_n + Z_n} \tag{3.1.5}$$

Une sous-martingale se décompose donc en une somme d'une martingale et d'un processus croissant prévisible pour la même filtration.

Démonstration Soit $(X_n)_{n \in \mathbb{N}}$ une sous-martingale. par rapport à $(\mathcal{F}_n)_{n \geq 0}$. On définit les écarts :

$$\forall n \in \mathbb{N}, \quad \Delta_n = X_{n+1} - X_n$$

Posons $M_0 = X_0$ et $Z_0 = 0$. On construit récursivement $(M_n)_{n \geq 0}$ et $(Z_n)_{n \geq 0}$ tels que :

$$\forall n \in \mathbb{N}, \quad Z_n = \sum_{i=1}^n \mathbb{E}(\Delta_i \mid \mathcal{F}_{i-1}), \quad M_n = X_n - Z_n$$

Montrons que des deux processus vérifient les hypothèses :

- i) Par construction, pour tout $n \in \mathbb{N}$, on a $X_n = M_n + Z_n$.
- ii) Pour tout $i \in \llbracket 1, n \rrbracket$, $\mathbb{E}(\Delta_i \mid \mathcal{F}_{i-1})$ est $\mathcal{F}_{i-1}$ mesurable et Z_n est une somme de i variables $\mathcal{F}_{i-1}$ mesurables. Donc $(Z_n)_{n \geq 0}$ est prévisible pour $(\mathcal{F}_n)_{n \geq 0}$.
- iii) Pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned} Z_{n+1} - Z_n &= \mathbb{E}(\Delta_{n+1} \mid \mathcal{F}_n) \\ &= \mathbb{E}(X_{n+1} \mid \mathcal{F}_n) - \mathbb{E}(X_n \mid \mathcal{F}_n) \\ &= \mathbb{E}(X_{n+1} \mid \mathcal{F}_n) - X_n \end{aligned}$$

or X_n est une sous-martingale donc :

$$\mathbb{E}(X_{n+1} \mid \mathcal{F}_n) \geq X_n$$

Puisque $Z_0 = 0$, $(Z_n)_{n \in \mathbb{N}}$ est un processus croissant.

Montrons maintenant que $(M_n)_{n \in \mathbb{N}}$ est une martingale.

$$\begin{aligned} \forall n \in \mathbb{N}, \mathbb{E}(M_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}(X_{n+1} \mid \mathcal{F}_n) - \mathbb{E}(Z_{n+1} \mid \mathcal{F}_n) \\ &= \mathbb{E}(X_{n+1} \mid \mathcal{F}_n) - Z_{n+1} \\ &= \mathbb{E}(X_{n+1} \mid \mathcal{F}_n) - Z_n - \mathbb{E}(\Delta_{n+1} \mid \mathcal{F}_n) \\ &= \mathbb{E}(X_{n+1} \mid \mathcal{F}_n) - Z_n - (\mathbb{E}(\Delta_{n+1} \mid \mathcal{F}_n) - X_n) \\ &= X_n - Z_n = M_n \end{aligned}$$

L'unicité se montre par récurrence sur n . Nous ne la ferons pas ici.

3.2 Stratégies et théorème d'arrêt

Exemple (Ruine du joueur) Considérons un joueur lors d'un jeu aléatoire constitué de N parties. Lors de la partie $n \leq N$, il mise une somme Θ_n et il remporte la somme $\varepsilon_n \Theta_n$ où $\varepsilon_n = -1$ s'il perd et $\varepsilon_n = 2$ s'il gagne.

Les variables $(\varepsilon_n)_{n \geq 0}$ dépendent du jeu, donc on peut construire une filtration $(\mathcal{F}_n)_{n \geq 0}$ définie par $\mathcal{F}_n = \sigma(\varepsilon_1, \dots, \varepsilon_n)$. On peut considérer que $(\varepsilon_n)_{n \in \mathbb{N}}$ sont iid. Par construction, Θ_n est $\mathcal{F}_{n-1}$ -mesurable, donc prévisible. On définit alors le gain total du joueur après n parties comme la variable :

$$\forall n \leq N, \quad S_n = S_0 + \sum_{k=1}^n \Theta_k \varepsilon_k$$

Dans cette section, nous allons essayer de généraliser cette structure au travers des stratégies. Nous donnerons des outils pour étudier la convergence de la suite $(S_n)_{n \geq 0}$ et nous généraliserons la proposition 3.1.4 grâce au théorème d'arrêt.

Proposition Soit $(M_n)_{n \geq 0}$ une martingale pour la filtration $(\mathcal{F}_n)_{n \geq 0}$, $(\Phi_n)_{n \geq 0}$ un processus prévisible et borné. Le processus défini par :

$$X_0 = 0 \text{ et } \forall n \in \mathbb{N}, \quad X_n = \sum_{k=1}^n \Phi_k(M_k - M_{k-1}) \quad (3.2.1)$$

Est une martingale pour $(\mathcal{F}_n)_{n \geq 0}$.

Cette proposition est fondamentale dans l'interprétation des martingales en tant que stratégies de jeu. En effet, elle nous montre que pour n'importe quel jeu initial d'espérance nulle $(M_n)_{n \in \mathbb{N}}$ (gain moyen), il n'existe aucune stratégie $(\Phi_n)_{n \in \mathbb{N}}$ permettant de modifier le gain moyen. Cette notion est traduite par le fait que $(X_n)_{n \in \mathbb{N}}$ reste une martingale.

Remarque Les martingales nous permettent de modéliser la plupart des stratégies. Par exemple, si l'on veut modéliser une stratégie du type "le joueur mise jusqu'à avoir accumulé une certaine somme λ ", alors on peut définir T le temps d'arrêt tel que :

$$T = \inf\{n \in \mathbb{N} \mid S_n \geq \lambda\}$$

On posera donc :

$$\forall n \in \mathbb{N}, \forall k \in \llbracket 1, n \rrbracket, \quad \Phi_k = \mathbb{1}_{T \geq k} \quad \text{et} \quad X_n = \sum_{k=1}^n \Phi_k(M_k - M_{k-1})$$

Où $(M_n)_{n \in \mathbb{N}}$ représente, comme précédemment, le jeu initial. On souhaiterait aussi généraliser la notion de chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ arrêtée par un temps d'arrêt T . Pour cela, on considèrera la martingale suivante :

$$\forall n \in \mathbb{N}, \quad X_n = \begin{cases} X_n & \text{si } n \leq T \\ X_T & \text{sinon.} \end{cases}$$

Définition (Martingale arrêtée) . Ainsi, soit $(M_n)_{n \geq 0}$ une martingale pour la filtration $(\mathcal{F}_n)_{n \geq 0}$. Soit T un temps d'arrêt. On pose :

$$\forall k \in \mathbb{N}, \quad \Phi_k = \mathbb{1}_{\{T \geq k\}}$$

L'évènement $\{T \leq n\}$ est mesurable par rapport à $\mathcal{F}_{n-1}$ car il se décompose comme le complémentaire d'une intersection dénombrable d'évènements $\mathcal{F}_{n-1}$ mesurables :

$$\{T \leq n\} = \overline{\{T \geq n\}} = \bigcap_{k=1}^{n-1} \{T \neq k\}$$

Le processus $(\Phi_k)_{k \geq 0}$ est donc prévisible pour la filtration $(\mathcal{F}_n)_{n \geq 0}$. D'après la proposition 3.2.1, le processus défini comme :

$$\boxed{\forall n \in \mathbb{N}, \quad M_n^T = \begin{cases} M_n & \text{si } n \leq T \\ M_T & \text{sinon} \end{cases}} \quad (3.2.2)$$

est donc aussi une martingale pour $(\mathcal{F}_n)_{n \in \mathbb{N}}$ appelé *martingale arrêtée*.

Cette définition des martingales arrêtées nous permet donc d'énoncer un des théorèmes phares de ce chapitre. Celui-ci permet de généraliser la proposition 3.1.4 au temps d'arrêts pour les martingales.

Théorème (Arrêt - Doob) . Soit $(M_n)_{n \geq 0}$ une martingale pour la filtration $(\mathcal{F}_n)_{n \geq 0}$. Soit T un temps d'arrêt pour $(\mathcal{F}_n)_{n \geq 0}$. Si une des trois assertions suivante est vraie :

- i) T est borné p.s
- ii) $T < \infty$ p.s et $(M_n)_{n \in \mathbb{N}}$ est borné.
- iii) Si $T \in L^1$ et :

$$\sup_n |M_n - M_{n-1}| = \sup_n |\Delta_{M_n}| \leq C \quad \text{p.s} \quad (3.2.3)$$

Alors $\mathbb{E}(M_T) = \mathbb{E}(M_0)$.

Ce théorème nous permet donc de généraliser aux temps d'arrêt la proposition de généralisation de la définition des martingales 3.1.4.

Démonstration

Remarque (Limites) Ce théorème n'est pas vrai dans le cas général. Fournissons un contre exemple. Soit $(\varepsilon_i)_{i \in \mathbb{N}}$ une suite de variables aléatoires iid telles que :

$$\forall i \in \mathbb{N}, \quad \mathbb{P}(\varepsilon_i = -1) = \mathbb{P}(\varepsilon_i = 1) = \frac{1}{2}$$

Posons alors :

$$S_0 = 0, \quad \text{et} \quad S_n = \sum_{i=1}^n \varepsilon_i$$

Soit $T_1 = \inf\{n \geq 0 \mid S_n = 1\}$ le temps d'arrêt à l'état 1. Alors $(S_n^T) = S_{\min(n, T_1)}$ est une martingale arrêtée. On a alors :

$$\mathbb{E}(S_{\min(n, T_1)}) = \mathbb{E}(S_0) = 0$$

Nous avons montré que $(S_n)_{n \geq 0}$ est une chaîne de Markov sur $\mathbb{N}$ irréductible récurrente nulle. Donc T_1 est fini p.s. On ne rentre donc dans aucune des hypothèses du théorème. De plus :

$$\mathbb{E}(S_{T_1}) = 1$$

car par définition, $S_{T_1} = 1$ p.s.

Exemple (Application - Modèle de Wright-Fisher) On considère une population de N individus dont chacun porte un allèle A ou a . la suite $(X_n)_{n \in \mathbb{N}}$ compte le nombre d'individus portant l'allèle A au temps N . À chaque itération, les individus de l'étape $n + 1$ choisissent un parent uniformément dans la génération n et prennent leur allèle. La propriété importante du modèle est :

$$\mathbb{P}(\text{choisir un parent de type } A) = \frac{X_n}{N}$$

On peut aussi voir l'évolution du modèle comme une chaîne de Markov. Comme chaque descendant choisit un parent uniformément et indépendamment dans la génération n , on a :

$$X_{n+1} \sim \mathcal{B}\left(n, \frac{X_n}{N}\right)$$

$(X_n)_{n \in \mathbb{N}}$ est donc une chaîne de Markov à valeurs dans $\llbracket 1, N \rrbracket$ et 0 et N sont des points absorbants. L'état 0 correspond à la disparition de l'allèle A dans la population et l'état N correspond à la disparition de l'allèle a dans la population (i.e fixation de l'allèle A). On souhaiterait avoir des informations sur :

$$\tau = \inf(T_0, T_N) \quad \text{et} \quad \mathbb{P}(\tau = T_0)$$

Or $(X_n)_{n \geq 0}$ est une martingale car :

$$\mathbb{E}(X_{n+1} \mid \mathcal{F}_n) = N \times \frac{X_n}{N} = X_n$$

Elle est même bornée par N sur un espace d'états fini. Donc τ est fini p.s car c'est un temps d'atteinte d'une chaîne de Markov sur un espace discret.

On peut aussi remarquer que les états $\{1, \dots, N-1\}$ sont transients (car on peut en sortir avec probas positive). En revanche, 0 et N sont récurrents.

On peut donc appliquer le théorème d'arrêt de Doob, on a donc que :

$$\mathbb{E}(X_\tau) = \mathbb{E}(X_0)$$

or $\mathbb{E}(X_\tau) = 0 \times \mathbb{P}(\tau = T_0) + N\mathbb{P}(\tau = T_N)$ donc on en déduit que si $X_0 = i \in \llbracket 1, N \rrbracket$, alors :

$$i = N\mathbb{P}(\tau = T_N) \iff \mathbb{P}(\tau = T_N) = \frac{i}{N}$$

Les martingales permettent donc d'avoir facilement des informations sur des temps d'arrêts. En effet, le théorème d'arrêt nous donne le même résultat qu'une analyse à un pas. On peut également proposer en une réciproque au théorème d'arrêt.

Théorème (Réciproque au théorème d'arrêt) . Soit $(X_n)_{n \in \mathbb{N}}$ un processus adapté à une filtration $(\mathcal{F}_n)_{n \geq 0}$ tel que $X_n \in L^1$ pour tout $n \in \mathbb{N}$. Si, pour tout temps d'arrêt borné T , on a $\mathbb{E}(X_T) = \mathbb{E}(X_0)$, alors $(X_n)_{n \in \mathbb{N}}$ est une martingale.

3.3 Inégalités maximales

On souhaite contrôler le sup d'un processus sur une trajectoire à partir de la valeur terminale du processus. On considère donc une sous-martingale $(M_n)_{n \in \mathbb{N}}$. Par définition, elle vérifie :

$$\mathbb{E}(M_{n+1} \mid \mathcal{F}_n) \geq M_n$$

Soit $(M_n^*)_{n \in \mathbb{N}}$ le processus des sup des valeurs de $(M_n)_{n \in \mathbb{N}}$ définit par :

$$\forall n \in \mathbb{N}, \quad M_n^* := \max\{M_k \mid 0 \leq k \leq n\}$$

Il existe différentes inégalités pour avoir des informations sur ce processus.

Proposition (Inégalités Maximales) Dans le même contexte que précédemment, on a :

- i) $\forall c \geq 0, \quad \mathbb{P}(M_n^* \geq c) \leq \frac{1}{c} \mathbb{E}(M_n \mathbb{1}_{M_n^* \geq c})$
- ii) Soit $p > 1$ si $(M_n)_{n \in \mathbb{N}}$ est une sous-martingale positive et $M_n \in L^p$ pour tout $n \in \mathbb{N}$, alors :

$$\|M_n^*\|_p \leq \frac{p}{p-1} \|M_n\|_p \quad (3.3.1)$$

où $\|X\|_p = \mathbb{E}(|X|^p)^{1/p}$ pour toute variable aléatoire X .

Remarque Si $(M_n)_{n \in \mathbb{N}}$ est une martingale dans L^p alors $(|M_n|)_{n \in \mathbb{N}}$ est une sous-martingale positive dans L^p . Les inégalités précédentes nous permettent de montrer que :

$$\mathbb{E} \left(\left(\sup_{0 \leq k \leq n} |M_k| \right)^p \right) \leq \left(\frac{p}{p-1} \right)^p \mathbb{E}(M_n^p)$$

3.4 Convergence des martignales

Dans cette section, nous nous intéressons à la convergence des martignales lorsque $n \rightarrow \infty$. Nous essaierons d'énoncer des théorèmes similaires au TCL dans le cadre des martignales.

3.4.1 Critères de convergences dans L^2

Rappel Une martignale $(M_n)_{n \in \mathbb{N}}$ est bornée dans L^2 si $\sup_n \mathbb{E}(M_n^2) < \infty$.

L'idée est donc d'exprimer une martignale comme une somme de variables aléatoires. Pour cela, on posera :

$$\forall n \in \mathbb{N}, \quad M_n - M_0 = \sum_{k=1}^n M_k - M_{k-1} = \sum_{k=1}^n \Delta_{M_k}$$

On appellera les Δ_{M_k} les *incrément*s de la martignale. Pour rappel, le produit scalaire dans L^2 est de la forme :

$$\langle X, Y \rangle = \mathbb{E}(XY)$$

Si $(M_n)_{n \in \mathbb{N}}$ est une martignale L^2 , ses incréments sont centrés et orthogonaux par rapport au produit scalaire de L^2 . On a :

$$\mathbb{E}(\Delta_{M_k}) = \mathbb{E}(M_k - M_{k-1}) \quad (3.4.1)$$

$$= \mathbb{E}(\mathbb{E}(M_k - M_{k-1} \mid \mathcal{F}_{k-1})) \quad (3.4.2)$$

$$= \mathbb{E}(\mathbb{E}(M_k \mid \mathcal{F}_{k-1}) - M_{k-1}) = 0 \quad (3.4.3)$$

Montrons l'orthogonalité, soient $j, k \in \mathbb{N}, j < k$, on a :

$$\begin{aligned} \mathbb{E}(\Delta_{M_j} \Delta_{M_k}) &= \mathbb{E}(\mathbb{E}(\Delta_{M_j} \Delta_{M_k} \mid \mathcal{F}_{k-1})) \\ &= \mathbb{E}(\Delta_{M_j} \mathbb{E}(\Delta_{M_k} \mid \mathcal{F}_{k-1})) \\ &= \mathbb{E}(\Delta_{M_j} \times 0) \quad (\text{d'après 3.4.3}) \\ &= 0 \end{aligned}$$

En général, on suppose que $M_0 = 0$, on aura donc :

$$\mathbb{E}(M_n^2) = \mathbb{E} \left(\left(\sum_{k=1}^n \Delta_{M_k} \right)^2 \right) = \sum_{k=1}^n \mathbb{E}(\Delta_{M_k}^2)$$

Or $\mathbb{E}(M_n^2)$ est une suite croissante en n et si M_n est bornée dans L^2 , cette suite est majorée, donc elle converge vers une quantité positive.

Théorème (Convergence) . Soit $(M_n)_{n \in \mathbb{N}}$ une martignale bornée dans L^2 . Alors il existe une variable aléatoire $M_\infty \in L^2$ telle que :

i) $\mathbb{E}((M_n - M_\infty)^2) \xrightarrow{n \rightarrow \infty} 0$

ii) $M_n \xrightarrow{n \rightarrow \infty} M_\infty$

Si $p \in [1, 2]$ alors $\|M_n - M_\infty\|_p \leq \|M_n - M_\infty\|_2$ donc $\mathbb{E}(|M_n - M_\infty|) \xrightarrow{n \rightarrow \infty} 0$.

Exemple On peut considérer l'exemple d'une série de variables aléatoires :

$$M_n = \sum_{k=1}^n \frac{\varepsilon_k}{i^\alpha} \quad \text{où } \varepsilon_k \sim \mathcal{U}([-1, 1])$$

Un telle série converge dès que $\alpha > 1$ car :

$$M_n = \sum_{k=1}^n \frac{\varepsilon_k}{k^\alpha} \leq \sum_{k=1}^n \frac{|\varepsilon_k|}{k^\alpha} \leq \sum_{k=1}^n k^{-\alpha}$$

- $(M_n)_{n \geq 0}$ est une martingale car :

$$\mathbb{E}(|M_n|) < \infty \quad \text{si} \quad \alpha > 1$$

et :

$$\begin{aligned} \mathbb{E}(M_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}\left(M_n + \frac{\varepsilon_{n+1}}{(n+1)^\alpha} \mid \mathcal{F}_n\right) \\ &= M_n + \frac{\mathbb{E}(\varepsilon_{n+1})}{(n+1)^\alpha} = M_n \end{aligned}$$

- Montrons que les $(M_n)_{n \in \mathbb{N}}$ sont bornés dans L^2 :

$$\mathbb{E}(M_n^2) = \sum_{k=1}^n \mathbb{E}\left[\left(\frac{\varepsilon_k}{k^\alpha}\right)^2\right] \leq \sum_{k=1}^n \frac{1}{k^{2\alpha}} < \infty$$

Donc dès que $\alpha > 1/2$, M_n est bornée dans L^2 donc elle converge vers $M_\infty \in L^2$.

Démonstration 1. Convergence dans L^2 : On sait que la suite $\mathbb{E}(M_n^2)$ est croissante et majorée par hypothèse, donc elle converge. Montrons que cette suite est de Cauchy ce qui nous assurera qu'il existera une variable aléatoire limite (si l'on suppose que L^2 est complet). Pour tout $n, p \geq 0$, on a :

$$\begin{aligned} &\mathbb{E}((M_{n+p} - M_n)^2) \\ &= \mathbb{E}(M_{n+p}^2 - 2M_n M_{n+p} + M_n^2) \\ &= \mathbb{E}(M_{n+p}^2) + \mathbb{E}(M_n^2) - 2\mathbb{E}(\mathbb{E}(M_n M_{n+p} \mid \mathcal{F}_n)) \\ &= \mathbb{E}(M_{n+p}^2) + \mathbb{E}(M_n^2) - 2\mathbb{E}(M_n^2) \\ &= \mathbb{E}(M_{n+p}^2) - \mathbb{E}(M_n^2) \xrightarrow{n \rightarrow \infty} 0 \end{aligned}$$

2. Convergence presque sûre : On souhaiterait utiliser le lemme de Borel-Cantelli. On souhaite donc montrer que :

$$\mathbb{P}(\{A_n\}) = \mathbb{P}\left(\sup_{k \geq n} |M_k - M_\infty| > \varepsilon\right)$$

Alors la $\mathbb{P}(\limsup A_n) = 0 \iff \mathbb{P}(\liminf A_n^c) = 1$ d'où :

$$\mathbb{P}(\exists n \in \mathbb{N}, \forall k \geq n, |M_k - M_\infty| \leq \varepsilon) = 1$$

$$\begin{aligned} \mathbb{P}(\sup_{k \geq n} |M_k - M_\infty| \geq \varepsilon) &\leq \frac{\mathbb{E}(\sup_{k \geq n} |M_k - M_\infty|^2)}{\varepsilon^2} \\ &\leq \frac{1}{\varepsilon^2} \left(\mathbb{E}(|M_n - M_\infty|^2) + \mathbb{E}(\sup_{k \geq n} |M_k - M_n|^2) \right) \end{aligned}$$

Par limite monotone :

$$\mathbb{E}(\sup_{k \geq n} |M_k - M_n|^2) = \lim_{n \rightarrow \infty} \mathbb{E}\left(\sup_{N \geq k \geq n} |M_k - M_n|^2\right)$$

or pour tout $k \geq n$, $X_k = |M_k - M_n|$ est une sous-martingale positive. Alors, d'après l'inégalité maximale de Doob :

$$\mathbb{E} \left(\sup_{N \geq k \geq n} |M_k - M_n| \right)^{1/2} \leq 2 \mathbb{E}(|M_N - M_n|^2)^{1/2}$$

Donc :

$$\begin{aligned} \mathbb{E}(\sup_{k \geq n} |M_k - M_n|^2) &\leq 4 \lim_{N \rightarrow \infty} \mathbb{E}(|M_N - M_n|^2) \\ &\leq 4 \mathbb{E}(|M_\infty - M_n|^2) \end{aligned}$$

En revenant à notre inégalité du départ, on a donc :

$$\mathbb{P}(\sup_{k \geq n} |M_k - M_\infty|^2 \geq \varepsilon) \leq \frac{c}{\varepsilon^2} \mathbb{E}(|M_\infty - M_n|^2) \xrightarrow{n \rightarrow \infty} 0$$

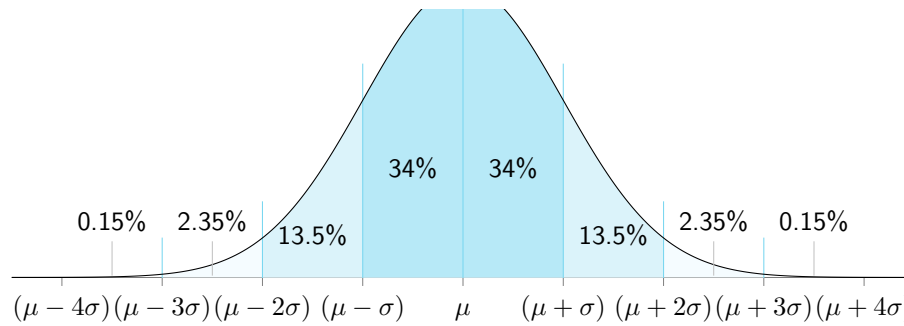
Quitte à extraire, on peut construire une suite $n_L \rightarrow \infty$ telle que $\sum \mathbb{E}(|M_\infty - M_{n_L}|^2) < \infty$. Le théorème de Borel-Cantelli nous donne donc que :

$$\mathbb{P}(\limsup_{n_L} A_{n_L}) = 0$$

$$\iff \mathbb{P}(\liminf_{n_L} A_{n_L}^c) = 1$$

ce qui suffit à donner la convergence presque sûre.

Simulation Aléatoire



Chapitre 1

Simulation de variables aléatoires

Contents

1.0.1	Introduction	51
1.1	Simulation de variables aléatoires discrètes	52
1.1.1	Exemples Introductifs	52
1.1.2	Méthode Générale - Inverse de la fonction de masse	53
1.1.3	Méthode du rejet (Cas discret)	54
1.2	Simulation de variables aléatoires à densité	55
1.2.1	Méthode de l'inverse généralisée	55
1.2.2	Méthode du rejet (Cas continu)	57
1.3	Méthodes ad-hoc	59
1.3.1	Loi Normale - Méthode de Box-Muller	60
1.3.2	Lois de Poisson	61
1.3.3	Mélange de lois	63
1.3.4	Méthode de Monte-Carlo	63

1.0.1 Introduction

La simulation peut se définir comme la prédiction de l'évolution d'un système en résolvant des équations venues de domaines appliqués. La notion d'aléatoire vient du fait que l'on simule des phénomènes dont nous n'aurons que des informations imparfaites tels que des phénomènes de désintégration nucléaire ou de prévention des risques. Les utilisations possibles de la simulation sont :

- L'étude d'un phénomène (ex : la trajectoire d'un avion dans des turbulences)
- L'estimation de caractéristiques d'un système (ex : déterminer les caractéristiques d'une simulation météo)
- Tester des hypothèses (ex : en médecine, on peut utiliser la simulation aléatoire pour déterminer à l'avance les effets d'un médicament sur un patient.)
- La prise de décision dans un environnement incertain.
- Réaliser des expériences coûteuses voire impossibles (ex : effectuer des essais nucléaires informatiquement en fonction des essais passés ou simuler une propagation virale dans une population).

La simulation aléatoire est donc un outil rapide économique et contrôlé.

Définition (Simuler une variable aléatoire) . Simuler une variable aléatoire de loi μ consiste à produire un algorithme dont les réalisations donnent des copies indépendantes d'une variable de loi μ .

La simulation aléatoire est impossible sans l'accès à des *données aléatoires*. En pratique, on se contentera de données *pseudo-aléatoires* tel que le nombre de secondes écoulées depuis le 01/01/1970, la température d'un processeur ou les dernières touches entrées avant l'exécution du programme. Ceci est obtenu en **Python** avec la fonction `rand()`. Dans ce cours, on considère que chaque appel à cette fonction nous renvoie une variable iid de loi Uniforme sur $[0, 1]$. Simuler une variable aléatoire consiste donc à construire un algorithme qui transforme un nombre fini de variables aléatoires iid de loi $\mathcal{U}([0, 1])$ en une variable aléatoire de loi μ :

$$\mathcal{U}([0, 1]) \xrightarrow{\text{simulation}} \mu$$

1.1 Simulation de variables aléatoires discrètes

1.1.1 Exemples Introductifs

Exemple (Simuler une variable aléatoire de loi $\mathcal{B}(p)$) Soit $X \sim \mathcal{U}([0, 1])$ avec $p \in [0, 1]$. Alors la variable $Y = \mathbb{1}_{X \leq p}$ suit une loi de Bernoulli de paramètre p .

$$\mathbb{P}(Y = 1) = \mathbb{P}(\mathbb{1}_{X \leq p} = 1) = \mathbb{P}(X \leq p) = p$$

$$\mathbb{P}(Y = 0) = 1 - p$$

On peut donc développer l'algorithme suivant :

```

1 def bernouilli(p):
2     x = rand()
3     if x <= p:
4         return 1
5     return 0
6 
```

Exemple (Simuler une loi binomiale $\mathcal{B}(n, p)$) Soient $(X_1, \dots, X_n)$ une suite de variables aléatoires iid de loi $\mathcal{B}(n, p)$. La variable $Y := \sum_{i=1}^n X_i \sim \mathcal{B}(n, p)$ telle que :

$$\mathbb{P}(Y = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

Il nous faut donc seulement générer la somme de n variables aléatoires suivant une loi de Bernoulli. On a donc :

```

1 def binom(n, p):
2     s = 0
3     for i in range(n):
4         s += 1 * (rand() <= p)
5     return s
6 
```

On cherchera à développer des programmes qui terminent en temps fini. On préférera ceux qui terminent en temps borné (comme `binom(n, p)`).

1.1.2 Méthode Générale - Inverse de la fonction de masse

Proposition Soit $(p_n)_{n \geq 1}$ une suite réelle positive telle que :

$$\sum_{k=1}^{\infty} p_k = 1$$

et (x_n) une suite d'éléments d'un ensemble E . On définit la loi μ par :

$$\mu = \sum_{k=1}^{\infty} p_k \delta_{x_k}$$

si une variable aléatoire X est de loi μ on aura alors $\mathbb{P}(X = x_k) = p_k$. On pose $f_0 = 0$ et $s_k = p_1 + \dots + p_k$. Soit U une variable aléatoire de loi $\mathcal{U}([0, 1])$. Alors la variable :

$$Y = \sum_{k=1}^{\infty} x_k \times \mathbb{1}_{s_{k-1} \leq U \leq s_k}$$

suit la loi μ . L'algorithme suivant simule une variable aléatoire de loi μ .

```

1 def simulation_mu([p1, ..., pk], [x1, ..., xk]):
2     u = rand()
3     s = p1
4     while u > s :
5         i++
6         s += pi
7     return xi
8

```

Démonstration Pour tout $n \in \mathbb{N}$, on a :

$$\mathbb{P}(s_{n-1} \leq U \leq s_n) = s_n - s_{n-1} = p_n$$

donc $\mathbb{P}(Y = x_n) = \mathbb{P}(s_{n-1} \leq U \leq s_n) = p_n$.

Exemple (Simuler une loi de Poisson $\mathcal{P}(\lambda)$) Pour rappel, si $X \sim \mathcal{P}(\lambda)$ alors :

$$\forall k \in \mathbb{N}, \quad \mathbb{P}(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}$$

Posons $p_k := e^{-\lambda} \frac{\lambda^k}{k!}$. On remarque que :

$$p_{k+1} = e^{-\lambda} \frac{\lambda^{k+1}}{(k+1)!} = p_k \frac{\lambda}{k+1}$$

On en déduit donc l'algorithme suivant :

```

1 def poisson(lambda):
2     x = 0
3     p = exp(-lambda)
4     s = p
5     u = rand()
6     while u > s :
7         x += 1
8         p = p * lambda / x
9         s += p
10    return x
11

```

Exemple (Loi uniforme) Simulons une loi uniforme sur $\{1, \dots, n\}$. On a donc l'algorithme suivant :

```

1 def uniforme(n):
2     x = 1
3     s = x/n
4     u = rand()
5     while u > s :
6         x += 1
7         s += 1/n
8     return x
9
10 def uniforme(n):
11     return ceil(n * rand())
12

```

1.1.3 Méthode du rejet (Cas discret)

Proposition (Algorithme du Rejet) Soit $(E, \mathcal{E}, \mu)$ un espace probabilisé et $A \subset E$ un événement tel que $\mu(A) > 0$. La loi μ conditionnellement à A , notée μ_A , est définie par :

$$\mu_A(B) = \frac{\mu(A \cap B)}{\mu(A)}, \quad B \in \mathcal{E}.$$

Soit $(X_n)_{n \geq 1}$ une suite de variables aléatoires i.i.d. de loi μ . On pose :

$$T := \inf\{n \geq 1 \mid X_n \in A\},$$

c'est-à-dire le rang du premier tirage appartenant à A . Alors :

1. $T \sim \mathcal{G}(\mu(A))$, c'est-à-dire

$$\mathbb{P}(T = k) = (1 - \mu(A))^{k-1} \mu(A), \quad k = 1, 2, \dots$$

2. $X_T \sim \mu_A$ (loi conditionnelle sur A)
3. X_T et T sont indépendantes.

Démonstration Soit $B \in \mathcal{E}$ et $k \geq 1$. On calcule :

$$\begin{aligned}
 \mathbb{P}(X_T \in B, T = k) &= \mathbb{P}(X_k \in B, T = k) \\
 &= \mathbb{P}(X_1 \notin A, \dots, X_{k-1} \notin A, X_k \in A, X_k \in B) \\
 &= \mathbb{P}(X_1 \notin A)^{k-1} \times \mathbb{P}(X_1 \in A \cap B) \\
 &= (1 - \mu(A))^{k-1} \mu(A) \frac{\mu(A \cap B)}{\mu(A)} \\
 &= \underbrace{(1 - \mu(A))^{k-1} \mu(A)}_{\mathbb{P}(T=k)} \underbrace{\frac{\mu(A \cap B)}{\mu(A)}}_{\mu_A(B)}.
 \end{aligned}$$

On en déduit :

- $\mathbb{P}(T = k) = (1 - \mu(A))^{k-1} \mu(A)$ (loi géométrique)
- $\mathbb{P}(X_T \in B) = \mu_A(B)$
- $\mathbb{P}(X_T \in B, T = k) = \mathbb{P}(T = k) \mathbb{P}(X_T \in B)$, donc T et X_T sont indépendantes.

L'algorithme du rejet décrit la procédure suivante :

1. Tirer $X_1, X_2, \dots$ de manière indépendante selon la loi μ .

2. Rejeter tout tirage $X_n \notin A$ et recommencer.
3. Accepter le premier $X_T \in A$: sa loi est exactement μ_A .

Ainsi :

- T représente le nombre de tirages nécessaires avant d'obtenir un succès.
- X_T est la valeur de ce premier succès, distribuée comme μ conditionnellement à A .
- T et X_T sont indépendants : le temps d'attente n'influence pas la valeur obtenue.

Exemple Si E est l'ensemble des notes d'étudiants, μ la distribution des notes et $A = \{\text{note} > 15\}$, alors :

- T = nombre d'étudiants observés avant d'en trouver un avec une note > 15 ,
- X_T = note de ce premier étudiant (loi conditionnelle sur A).

La loi de T est géométrique et la loi de X_T est la loi des notes sachant qu'elles sont > 15 .

Exemple On veut simuler une variable aléatoire U de loi uniforme sur les nombres premiers inférieurs à n . D'après la proposition précédente, on peut donc utiliser l'algorithme suivant :

```

1 def unif_prem(n):
2     x = uniforme(n)
3     while not premier(x):
4         x = uniforme(n)
5     return x
6

```

Ainsi le premier nombre premier sur lequel on tombe est uniformément distribué sur l'ensemble des nombres premiers inférieurs ou égaux à n .

1.2 Simulation de variables aléatoires à densité

Dans cette section, nous allons voir comment simuler des variables aléatoires continues. Nous découvrirons la méthode d'inversion de la fonction de répartition, une autre version de la méthode du rejet ainsi que d'autres exemples.

Rappel (Fonction de répartition) Soit μ une mesure de probabilité sur $\mathbb{R}$. La fonction de répartition de μ est la fonction :

$$F : \begin{cases} \mathbb{R} \rightarrow \mathbb{R} \\ x \mapsto \mu(]-\infty, x]) \end{cases}$$

Proposition La fonction de répartition F d'une loi admet plusieurs propriétés :

- F est croissante
- F est continue à droite, elle tend vers 1 en $+\infty$ et vers 0 en $-\infty$.

De plus, si μ admet une densité f par rapport à la mesure de Lebesgue, alors F est dérivable presque partout et $\forall x \in \mathbb{R}, F'(x) = f(x)$.

1.2.1 Méthode de l'inverse généralisée

La fonction de répartition F d'une variable aléatoire X nous donne la probabilité que X soit inférieure à $x \in \mathbb{R}$ notée $\mathbb{P}(X < x) \in [0, 1]$. Puisque l'on sait facilement générer un nombre aléatoire entre 0 et 1 avec la fonction `rand()`, il serait intéressant de voir si on ne peut pas, à partir de celui-ci, déterminer une valeur aléatoire de la variable X . C'est ce que fait la méthode de l'inverse généralisée.

Définition (Inverse Généralisée) . Soit $F : U \longrightarrow V$ une fonction croissante définie à droite. L'inverse généralisée de F est la fonction :

$$F^{-1} : \begin{cases} V \longrightarrow U \\ y \longmapsto \inf \{x \in U | F(x) > y\} \end{cases}$$

Proposition Si $F : U \longrightarrow V$ est inversible sur $I \subset U$ alors :

$$\forall y \in F(I), \quad F^{-1}(y) = (F|_I)^{-1}(y)$$

Exemple Soit F la fonction de répartition de la loi uniforme $\mathcal{U}([a, b])$. On sait que F est une bijection de $[a, b]$ sur $[0, 1]$ comme fonction continue strictement croissante.

$$F|_{[a,b]} = \varphi : x \longmapsto \frac{x-a}{b-a}$$

d'inverse φ^{-1} telle que $\forall y \in [0, 1]$, on ait :

$$\begin{aligned} \varphi^{-1}(y) = x &\iff \varphi(x) = y \\ &\iff y = \frac{x-a}{b-a} \\ &\iff x = (b-a)y + a \end{aligned}$$

D'où $\forall y \in [0, 1]$, $\varphi^{-1}(y) = (b-a)y + a$.

Théorème (Inverse de la fonction de répartition) . Soit μ une mesure de probabilité sur $\mathbb{R}$. On note F sa fonction de répartition et U une variable aléatoire de loi Uniforme sur $[0, 1]$. On a alors :

$$F^{-1}(U) \sim \mu$$

Démonstration Soit F la fonction de répartition d'une loi μ et $U \sim \mathcal{U}([0, 1])$. Montrons que $F^{-1}(U) \sim \mu$. Soit $x \in \mathbb{R}$, on a :

$$\mathbb{P}(F^{-1}(U) < x) = \mathbb{P}(U < F(x)) = F(x)$$

Ce théorème nous assure que la méthode de l'inverse de la fonction de répartition nous permet de générer des valeurs d'une loi voulue, en appliquant simplement la fonction de répartition au résultat de la fonction `rand()`. Pour les variables aléatoires discrètes, cette méthode revient au même que le premier algorithme vu dans ce cours.

Proposition Soit X une variable aléatoire de densité f .

1. Calculer $F(x) = \int_{-\infty}^x f(t) dt$ la fonction de répartition de X .
2. Calculer F^{-1} l'inverse généralisé de F sur un intervalle bien choisi.
3. Écrire l'algorithme :

```
1 def simulerX():
2     return invF(rand())
3
```

Exemple Simulons une variable aléatoire X dont la densité est :

$$f_X(x) = \frac{1}{\pi} \frac{1}{\sqrt{1-x^2}}, \quad \forall x \in]-1, 1[$$

Calculons sa fonction de répartition :

$$\begin{aligned} F_X(x) &= \int_{-\infty}^x \frac{1}{\pi} \frac{1}{\sqrt{1-t^2}} dt \\ &= \int_{-1}^x \frac{1}{\pi} \frac{1}{\sqrt{1-t^2}} dt \\ &= \frac{1}{\pi} [\arcsin t]_{-1}^x \\ &= \frac{1}{\pi} \arcsin x - \frac{1}{2} \end{aligned}$$

or F est strictement croissante et continue sur $] -1, 1[$ donc elle réalise une bijection de $[-1, 1]$ dans $[0, 1]$. Soient $x \in [-1, 1]$ et $y \in [0, 1]$ tels que :

$$\begin{aligned} F(x) = y &\iff \frac{1}{\pi} \arcsin x + \frac{1}{2} = y \\ &\iff x = \sin \left(\pi \left(y + \frac{1}{2} \right) \right) \end{aligned}$$

Donc F est bijective d'inverse :

$$F^{-1} = x \mapsto \sin \left(\pi \left(y + \frac{1}{2} \right) \right)$$

1.2.2 Méthode du rejet (Cas continu)

On souhaite simuler une variable aléatoire X de fonction de densité f que l'on ne sait pas bien simuler. On va donc utiliser une autre fonction auxiliaire g que l'on sait bien simuler.

Théorème (Méthode du rejet) . Soient f, g deux fonctions de densité telles qu'il existe $c > 0$ vérifiant :

$$f \leq cg$$

Soient $(Y_i)_{i \geq 1}$ des variables iid de densité g et $(U_i)_{i \geq 1}$ des variables iid de loi $\mathcal{U}([0, 1])$.

Posons $T = \inf \left\{ n \geq 1 \mid U_n \leq \frac{f(Y_n)}{cg(Y_n)} \right\}$. Alors :

1. Y_T a pour densité f
2. T est de loi géométrique $\mathcal{G}(1/c)$
3. T et Y_T sont indépendantes.

Proposition On peut donc ensuite simuler la variable aléatoire de densité f grâce à la densité de g avec l'algorithme suivant :

```

1 def simul_f():
2     y = simul_g()
3     u = rand()
4     while u > f(y) / (c * g(y)):
5         y = simul_g()
6         u = rand()
7     return y
8 
```

Démonstration Soient φ, f deux fonctions continues et bornées sur $\mathbb{R}$. Montrons que Y_T est une variable aléatoire de densité f :

$$\begin{aligned}
\mathbb{E}(\varphi(Y_T)) &= \mathbb{E}\left(\sum_{n=1}^{\infty} \mathbb{1}_{T=n} \varphi(Y_T)\right) \\
&= \sum_{n=1}^{\infty} \mathbb{E}(\varphi(Y_T) \times \mathbb{1}_{T=n}) \\
&= \sum_{n=1}^{\infty} \mathbb{E}(\varphi(Y_n) \times \mathbb{1}_{T=n}) \\
&= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}} \times \mathbb{1}_{U_{n-1} > \frac{f(Y_{n-1})}{cg(Y_{n-1})}} \times \cdots \times \mathbb{1}_{U_1 > \frac{f(Y_1)}{cg(Y_1)}}\right) \\
&= \sum_{n=1}^{\infty} \left(\mathbb{E}\left(\varphi(Y_n) \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}}\right) \mathbb{P}\left(U_{n-1} > \frac{f(Y_{n-1})}{cg(Y_{n-1})}\right) \cdots \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}\right)\right)
\end{aligned}$$

or on a :

$$\begin{aligned}
\mathbb{P}(U_1 > \frac{f(Y_1)}{cg(Y_1)}) &= \mathbb{E}\left(\mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)} \mid Y_1\right)\right) \\
&= \mathbb{E}\left(1 - \frac{f(Y_1)}{cg(Y_1)}\right) = 1 - \mathbb{E}\left(\frac{f(Y_1)}{cg(Y_1)}\right) \\
&= 1 - \int_{\mathbb{R}} \frac{f(y)}{cg(y)} \times g(y) dy = 1 - \frac{1}{c}
\end{aligned}$$

de plus :

$$\begin{aligned}
&\mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}}\right) \\
&= \mathbb{E}\left(\varphi(Y_n) \frac{f(Y_n)}{cg(Y_n)}\right) = \int_{\mathbb{R}} g(y) \varphi(y) \frac{f(y)}{cg(y)} dy \\
&= \frac{1}{c} \int_{\mathbb{R}} \varphi(y) f(y) dy
\end{aligned}$$

enfin :

$$\begin{aligned}
\mathbb{E}(\varphi(y)) &= \sum_{n=1}^{\infty} \mathbb{E}\left(\varphi(Y_n) \times \mathbb{1}_{U_n \leq \frac{f(Y_n)}{cg(Y_n)}}\right) \times \left(1 - \frac{1}{c}\right)^{n-1} \\
&= \left(\int_{\mathbb{R}} \varphi(y) f(y) dy\right) \times \frac{1}{c} \times \left(\sum_{n=1}^{\infty} \left(1 - \frac{1}{c}\right)^{n-1}\right) = \int_{\mathbb{R}} \varphi(y) f(y) dy
\end{aligned}$$

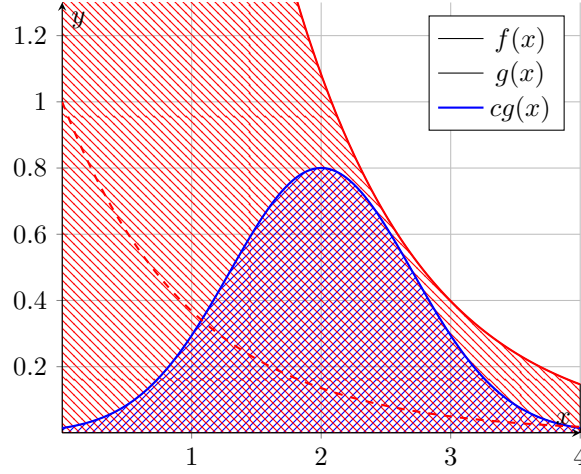
Donc Y_n est une variable aléatoire de densité f . Calculons la loi de T :

$$\begin{aligned}
\mathbb{P}(T > n) &= \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}, \dots, U_n > \frac{f(Y_n)}{cg(Y_n)}\right) \\
&= \mathbb{P}\left(U_1 > \frac{f(Y_1)}{cg(Y_1)}\right)^n = \left(1 - \frac{1}{c}\right)^n
\end{aligned}$$

d'où $T \sim \mathcal{G}(1/c)$. Pour finir, montrons que T et Y_T sont indépendantes :

$$\begin{aligned}
\mathbb{E}(\varphi(Y_T) \times \mathbb{1}_{T=n}) &= \mathbb{E}(\varphi(Y_T)) \times \frac{1}{c} \times \left(1 - \frac{1}{c}\right)^{n-1} \\
&= \mathbb{E}(\varphi(Y_T)) \times \mathbb{P}(T = n)
\end{aligned}$$

Proposition (Méthode du rejet : Principe) Comme dit précédemment, la méthode du rejet nous permet de simuler une densité f à partir d'une autre densité g que l'on sait simuler. Pour cela, on supposera qu'il existe $c > 0$ tel que $f \leq cg$.



Soit $X \sim g$ et $U \sim \mathcal{U}([0, 1])$. L'idée est donc de tirer un point aléatoirement selon la densité g , $(X, U \times cg(X))$ dans la zone rouge. Si la valeur $U \times cg(X)$ est sous la courbe de f (i.e dans la zone bleu), on accepte le point, sinon on recommence. La probabilité d'acceptation d'un point est donc de $1/c$.

Exemple Soit f la fonction suivante :

$$f : \begin{cases} \mathbb{R}_+^* \rightarrow \mathbb{R} \\ x \mapsto \frac{\sqrt{2}}{\pi} e^{x^2/2} \end{cases}$$

et g la fonction $g : x \mapsto -e^x$ définie sur $\mathbb{R}_+^*$. Pour tout $x \in \mathbb{R}_+^*$, on a :

$$f(x) \leq cg(x) \quad \text{où} \quad c = \frac{\sqrt{2}}{\pi} e$$

On peut donc simuler f via l'algorithme suivant :

```

1 def simul_f():
2     u = rand()
3     y = - log(1 - rand())
4     while u > exp(-y**2/2) / exp(1 - y):
5         u = rand()
6         y = - log(1 - rand())
7     return y
8

```

1.3 Méthodes ad-hoc

Dans cette section, nous allons détailler un ensemble de méthodes destinées à des usages précis. Nous verrons comment simuler efficacement une loi normale, une loi de poisson, ainsi qu'un mélange de lois. Nous finirons par aborder la méthode de Monte-Carlo pour estimer des quantités qui s'expriment à partir d'intégrales.

1.3.1 Loi Normale - Méthode de Box-Muller

Rappel (Loi Normale) Une variable aléatoire X suit une *loi normale* de moyenne μ et de variance σ^2 si la densité de la loi sur $\mathbb{R}$ est :

$$x \mapsto \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

On note alors $X \sim \mathcal{N}(\mu, \sigma^2)$ et on a :

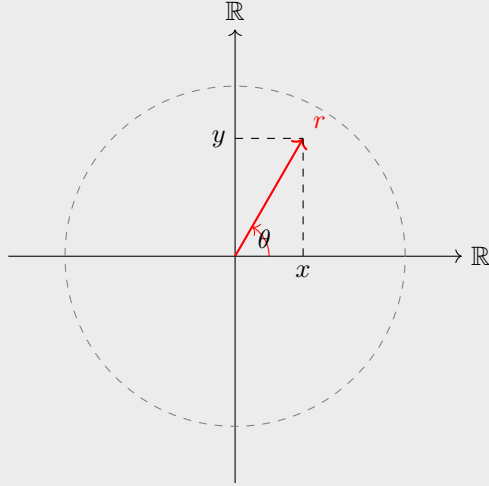
$$\mathbb{E}(X) = \mu \quad \text{et} \quad \mathbb{V}(X) = \sigma^2$$

Propriété (Loi centrée réduite.) . Si $Y \sim \mathcal{N}(0, 1)$, alors :

$$X = \mu + \sigma Y \sim \mathcal{N}(\mu, \sigma^2)$$

On dit alors que Y suit une *loi normale centrée réduite*.

Théorème (Méthode de Box-Muller) . Soit (X, Y) un couple de variables aléatoires iid de loi $\mathcal{N}(0, 1)$. Alors $X^2 + Y^2$ est une variable aléatoire de loi $\mathcal{E}(1/2)$ indépendante de $\arctan(Y/X) \sim \mathcal{U}([-\frac{\pi}{2}; \frac{\pi}{2}])$.



Le théorème nous dit que θ est indépendant de r .

Démonstration Soient (X, Y) deux variables aléatoires indépendantes de loi $\mathcal{N}(0, 1)$. Calculons l'espérance du couple :

$$\mathbb{E}(g(X, Y)) = \frac{1}{2\pi} \int_{\mathbb{R}^2} g(x, y) e^{-\frac{x^2}{2}} e^{-\frac{y^2}{2}} dx dy$$

Pour cela, utilisons la formule de changement de variable. Posons :

$$\Phi : \begin{cases} U = R_+ \times [0, 2\pi[\longrightarrow V = \mathbb{R}^2 \\ (r, \theta) \mapsto (x, y) = (r \cos \theta, r \sin \theta) \end{cases}$$

Φ est bijective de $R_+^* \times [0, 2\pi[$ dans $\mathbb{R}^2 / \{(0, 0)\}$ donc c'est un $\mathcal{C}^1$ difféomorphisme. On peut donc appliquer la formule de changement de variable :

$$\int_V f(x, y) dx dy = \int_U f(\Phi(r, \theta)) \times |\det J_\Phi(r, \theta)| dr d\theta$$

On a donc :

$$\Phi(r, \theta) = \begin{pmatrix} r \cos \theta \\ r \sin \theta \end{pmatrix} \quad \text{d'où} \quad J_\Phi(r, \theta) = \begin{pmatrix} \cos \theta & -r \sin \theta \\ \sin \theta & r \cos \theta \end{pmatrix}$$

Avec $D_\Phi(r, \theta) = \det J_\Phi(r, \theta) = r$. D'où :

$$\begin{aligned} \mathbb{E}(g(X, Y)) &= \frac{1}{2\pi} \int_{\mathbb{R}^2} g(x, y) e^{-\frac{x^2}{2}} e^{-\frac{y^2}{2}} dx dy \\ &= \frac{1}{2\pi} \int_0^{2\pi} \int_0^\infty g(r \cos \theta, r \sin \theta) \times e^{-\frac{r^2}{2}} \times r dr d\theta \end{aligned}$$

Posons donc $z = r^2$, on a donc $dz = 2r dr$ d'où :

$$\begin{aligned} \mathbb{E}(g(X, Y)) &= \frac{1}{4\pi} \int_0^{2\pi} \int_0^\infty g(\sqrt{z} \cos \theta, \sqrt{z} \sin \theta) e^{-z/2} dz d\theta \\ &= \int_0^{2\pi} \int_0^\infty g(\sqrt{z} \cos \theta, \sqrt{z} \sin \theta) f_z(z) f_\theta(\theta) dz d\theta \end{aligned}$$

où :

$$\begin{cases} f_z(z) = \frac{1}{2} e^{-z/2} \\ f_\theta(\theta) = \frac{1}{2\pi} \end{cases} \implies \begin{cases} Z \sim \mathcal{E}(1/2) \\ \Theta \sim \mathcal{U}([0, 2\pi]) \end{cases}$$

On en déduit donc que $\Theta \perp Z$.

Proposition On peut donc simuler deux lois normales à partir d'une loi exponentielle et d'une loi uniforme via l'algorithme suivant :

```
1 def simuler_N():
2     u = 2 * pi * rand()
3     e = -2 * log(rand())
4     return sqrt(e) * cos(u)
5
```

Ou plus généralement :

```
1 def simuler_N(mu, sigma):
2     u = 2 * pi * rand()
3     e = -2 * log(rand())
4     return sqrt(sigma**2 * e) * cos(u + mu)
5
```

1.3.2 Lois de Poisson

Rappel (Loi de Poisson) Une variable aléatoire X suit une loi de Poisson de paramètre λ si $\forall k \in \mathbb{N}$:

$$\mathbb{P}(X = k) = e^{-\lambda} \frac{\lambda^k}{k!}$$

Propriété (Loi de Poisson et loi Exponentielle) . Soit $(E_n)_{n \geq 1}$ une suite de variables aléatoires iid de loi $\mathcal{E}(1)$. On note $S_n = \sum_{k=1}^n E_k$. Posons :

$$N = \max \{k \in \mathbb{N} \mid S_k < \lambda\}$$

Alors N suit une loi de Poisson de paramètre λ .

Démonstration Démontrons ce résultat par récurrence sur $\mathbb{N}$:

- Initialisation : On observe que :

$$\mathbb{P}(N = 0) = \mathbb{P}(E_1 > \lambda) = e^{-\lambda}$$

- Hérédité : Soit $k \geq 1$, on suppose que :

$$\mathbb{P}(N = k - 1) = \frac{\lambda^{k-1}}{(k-1)!} e^{-\lambda} \quad \forall \lambda \in \mathbb{R}_+$$

En utilisant le fait que pour tout évènement A et pour toute partie mesurable $\mathcal{F}$, on a :

$$\mathbb{P}(A) = \mathbb{E}(\mathbb{1}_A) = \mathbb{E}(\mathbb{E}(\mathbb{1}_A | \mathcal{F})) = \mathbb{E}(\mathbb{P}(A | \mathcal{F}))$$

alors :

$$\mathbb{P}(N = k) = \mathbb{P}(S_k < \lambda < S_{k+1}) = \mathbb{E}(\mathbb{P}(S_k < \lambda < S_{k+1} | E_1))$$

or :

$$\begin{aligned} \mathbb{P}(S_k < \lambda < S_{k+1}) &= \mathbb{P}(E_1 + \dots + E_k < \lambda < E_1 + \dots + E_{k+1} | E_1) \\ &= \mathbb{P}(E_2 + \dots + E_k < \lambda - E_1 < E_2 + \dots + E_{k+1} | E_1) \\ &= \mathbb{P}(S'_{k-1} < \lambda - E_1 < S_{k+1} | E_1) \\ &= \mathbb{P}(N' = k - 1) \end{aligned}$$

où S' est une copie indépendante de S et indépendante de E_1 , d'où :

$$\mathbb{P}(N' = k - 1) = \frac{(\lambda - E_1)^{k-1}}{(k-1)!} e^{-(\lambda - E_1)}$$

On a donc :

$$\begin{aligned} \mathbb{P}(N = k) &= \mathbb{E} \left(\frac{(\lambda - E_1)^{k-1}}{(k-1)!} e^{-(\lambda - E_1)} \times \mathbb{1}_{E_1 \leq \lambda} \right) \\ &= \int_0^\lambda \frac{(\lambda - e)^{k-1}}{(k-1)!} \underbrace{e^{-(\lambda - e)} e^{-e}}_{e^{-\lambda}} de \\ &= \frac{e^{-\lambda}}{(k-1)!} \int_0^\lambda (\lambda - e)^{k-1} de \\ &= \frac{e^{-\lambda}}{(k-1)!} \times \frac{\lambda^k}{k} = \frac{\lambda^k}{k!} e^{-\lambda} \end{aligned}$$

D'où $N \sim \mathcal{P}(\lambda)$.

Proposition Pour simuler une loi de poisson de paramètre λ , on a donc l'algorithme suivant :

```

1 def simuler_poisson(lambda):
2     s = 1 * log(rand())
3     k = 0
4     while s <= lambda :
5         k += 1
6         s = -log(rand())
7     return k
8 
```

Algorithme dont la complexité dépend linéairement de λ .

Remarque Quand λ revient très grand, on peut approcher une loi de Poisson de paramètre λ par une loi normale $\mathcal{N}(\lambda, \lambda)$.

1.3.3 Mélange de lois

On cherche ici à déterminer la loi d'une combinaison linéaire de loi μ_1 et μ_2 de la forme :

$$\mu = t\mu_1 + (1 - t)\mu_2$$

Pour simuler ce genre de variables aléatoires, on utilise le résultat suivant.

Lemme Soit $X \sim \mu_1$ et $Y \sim \mu_2$ et $B \sim \mathcal{B}(t)$ avec $B \perp (X, Y)$. On a alors :

$$BX + (1 - B)Y \sim t\mu_1 + (1 - t)\mu_2$$

Attention, ici X et Y ne sont pas forcément indépendantes.

Démonstration Calculons la loi de $BX + (1 - B)Y$. Soit $f \in \mathcal{M}_+(\mathbb{R}, \mathcal{B}_{\mathbb{R}})$ on a :

$$\begin{aligned} \mathbb{E}(g(BX + (1 - B)Y)) &= \mathbb{E}[\mathbb{E}(g(BX + (1 - B)Y)) \mid B] \\ &= \mathbb{E}[g(X) \times \mathbb{1}_{B=1} + g(Y) \times \mathbb{1}_{B=0}] \\ &= t\mathbb{E}(g(X)) + (1 - t)\mathbb{E}(g(Y)) \\ &= t \int g \, d\mu_1 + (1 - t) \int g \, d\mu_2 \\ &= \int g \, d_{t\mu_1 + (1-t)\mu_2} \end{aligned}$$

d'où $BX + (1 - B)Y \sim t\mu_1 + (1 - t)\mu_2$.

Proposition On a donc l'algorithme suivant :

```

1 def simuler_mu(t):
2     if rand() < t :
3         return simuler_mu1()
4     else :
5         return simuler_mu2()
6 
```

1.3.4 Méthode de Monte-Carlo

Pour la fin de ce chapitre, on souhaite simuler des variables aléatoires pour estimer des quantités déterministes pouvant s'écrire sous forme d'intégrale.

Proposition Soit $I = \int_a^b f(x) \, dx$ une intégrale que l'on ne sait pas calculer algébriquement, mais que l'on souhaite estimer. On peut alors réécrire :

$$I = \int_a^b \frac{f(x)}{g(x)} g(x) \, dx$$

où g est une densité de probabilité d'une variable que l'on sait simuler. Or, d'après la loi des grands nombres, on a :

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n \frac{f(X_i)}{g(X_i)} \xrightarrow[n \rightarrow \infty]{\text{p.s.}} \mathbb{E} \left(\frac{f(X)}{g(X)} \right)$$

où $(X_1, \dots, X_n)$ sont des variables aléatoires iid de densité g . Ainsi, on va pouvoir estimer toute intégrale de la forme :

$$I \simeq \mathbb{E} \left(\frac{f(X_1)}{g(X_1)} \right)$$

par un estimateur convergent $\hat{I}_n$.

Théorème (Rappel - Loi des grands nombres) . Soit (X_n) une suite de variables aléatoires iid et intégrables. On a alors :

$$\frac{1}{n} \sum_{i=1}^n X_i \xrightarrow[n \rightarrow \infty]{} \mathbb{E}(X_1)$$

Remarque On peut aussi approcher I par une méthode de Riemann :

$$I = \frac{1}{n} \sum_{i=1}^n f\left(a + \frac{\tau}{n}\right)$$

or elle ne nous permet d'estimer l'intégrale que pour des fonctions Riemann-intégrables, alors que la Loi des grands nombres (LGN) nous permet d'élargir notre méthode aux fonctions intégrables au sens de Lebesgue. De plus, les méthodes déterministes sont très lentes en plusieurs dimensions comparé aux méthodes stochastiques.

Théorème (Rappel - Théorème Central Limite) . Soit (X_n) une suite de variables iid de carré intégrable. On a alors :

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n X_i - \mathbb{E}(X_1) \right) \xrightarrow[n \rightarrow \infty]{\text{Loi}} \mathcal{N}(0, \mathbb{V}(X_1))$$

Autrement dit :

$$\lim_{n \rightarrow \infty} \mathbb{P} \left(\left| \frac{1}{n} \sum_{i=1}^n X_i - \mathbb{E}(X_1) \right| > \frac{t}{\sqrt{n}} \right) = \int_t^\infty \frac{e^{-\frac{y^2}{2\mathbb{V}(X_1)}}}{\sqrt{2\pi\mathbb{V}(X_1)}} dy$$

Rappel Soit (X_n) une suite de variables iid de carré intégrable. On note $\overline{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ et on pose :

$$S_n^2 := \frac{1}{n} \sum_{i=1}^n (X_i - \overline{X}_n)^2$$

D'après la loi des grands nombres, on a :

$$\overline{X}_n \xrightarrow[n \rightarrow \infty]{} \mathbb{E}(X_1)$$

D'après le théorème central limite, on a :

$$\sqrt{n} \frac{\overline{X}_n - \mathbb{E}(X_1)}{\sqrt{S_n^2}} \xrightarrow[n \rightarrow \infty]{} \mathcal{N}(0, 1)$$

avec $\mathbb{P}(I \in IC_n) \xrightarrow[n \rightarrow \infty]{} \alpha$.

Proposition On peut donc utiliser le TCL pour proposer des intervalles de confiance asymptotiques pour la valeur de I . On a donc, pour $I = \mathbb{E}(X_1)$, l'intervalle asymptotique de niveau α :

$$IC_n = \left[\overline{X}_n \pm \frac{q_{1-\alpha/2}}{\sqrt{n}} \sqrt{S_n^2} \right]$$

Exemple On souhaite calculer :

$$I = \int_{[0,1]^3} \mathbb{1}_{\substack{x_1 \leq \cos x_2 \sin x_3 \\ x_2 \leq x_3^2}} dx_1 dx_2 dx_3$$

D'après la méthode de Monte-Carlo, soit $f \in \mathcal{M}_+(\mathbb{R}, \mathcal{B}_{\mathbb{R}})$ telle que :

$$I = \mathbb{E}(f(Z))$$

où $f(Z) = \mathbb{1}_{\substack{z_1 \leq \cos z_2 \sin z_3 \\ z_2 \leq z_3^2}}$ et $Z = (z_1, z_2, z_3) \sim \mathcal{U}([0, 1]^3)$. Autrement dit :

$$I = \mathbb{P}(z_1 \leq \cos z_2 \sin z_3, z_2 \leq z_3^2)$$

On peut donc approcher I par :

$$\frac{1}{n} \sum_{i=1}^n \mathbb{1}_{\substack{z_1 \leq \cos z_2 \sin z_3 \\ z_2 \leq z_3^2}} = \hat{I}_n$$

D'après le TCL, on a :

$$\lim_{n \rightarrow \infty} \sqrt{n} \frac{\hat{I}_n - I}{\sqrt{I(1-I)}} = \mathcal{N}(0, 1)$$

i.e $\lim_{n \rightarrow \infty} \sqrt{n} \frac{\hat{I}_n - I}{\sqrt{\hat{I}(1-\hat{I})}} = \mathcal{N}(0, 1)$

On peut donc construire l'intervalle de confiance de niveau α :

$$\left[\hat{I}_n \pm \frac{\sqrt{\hat{I}_n(1-\hat{I}_n)}}{\sqrt{n}} q_\alpha \right] \subseteq \left[\hat{I}_n \pm \frac{q_\alpha}{2\sqrt{n}} \right]$$

Chapitre 2

Simulation de chaînes de Markov

Contents

2.1 Généralités	66
2.1.1 Définition et exemples	66
2.1.2 La simulation en pratique	67
2.2 Comportement asymptotique d'une chaîne de Markov	69
2.2.1 Méthode MCMC	71

Dans ce chapitre, nous nous intéresserons à la simulation de processus de Markov grâce à l'étude de celles-ci. Pour rappel, une chaîne de Markov décrit l'évolution d'une quantité soumise à des fluctuations aléatoires. C'est une notion plus générale que les variables i.i.d.

2.1 Généralités

2.1.1 Définition et exemples

Définition (Chaîne de Markov) . Une chaîne de Markov est un processus aléatoire $(X_n)_{n \geq 1}$ défini sur un espace E tel qu'il existe $f \in \mathcal{M}(E \times [0, 1], E)$ et $(u_n)_{n \geq 0}$ une suite iid de loi $\mathcal{U}([0, 1])$ telle que :

$$\forall n \geq 1, \quad X_{n+1} = f(X_n, u_{n+1})$$

avec :

$$X_0 \perp (u_n)_{n \geq 0}$$

Exemple Soit $P \in \mathcal{M}_n(\mathbb{R})$ une matrice stochastique. Soit une chaîne de Markov sur un espace $E = \llbracket 1, n \rrbracket$ de matrice de transition P . Alors c'est bien une chaîne de Markov au sens de la définition précédente. En effet, on pose :

$$\forall i, j \in E, \quad f(i, x) = j \quad \text{avec} \quad x \in \left[\sum_{k=1}^{j-1} P(i, k); \sum_{k=1}^j P(i, k) \right]$$

On aura donc :

$$\begin{aligned}\mathbb{P}(X_0 = x_0, \dots, X_n = x_n) \\ &= \mathbb{P}(X_0 = x_0, f(x_0, u_1) = x_1, \dots, f(x_{n-1}, u_n) = x_n) \\ &= \mathbb{P}(X_0 = x_0) \mathbb{P}(f(x_0, u_1) = x_1) \dots \mathbb{P}(f(x_{n-1}, u_n) = x_n) \\ &= \mathbb{P}(X_0 = x_0) P(x_0, x_1) \times \dots \times P(x_{n-1}, x_n)\end{aligned}$$

Exemple Soit le processus $(X_n)_{n \in \mathbb{N}}$ décrivant l'évolution de la température locale au cours du temps défini par :

$$X_0 = 25 \quad X_{n+1} = \frac{X_n + 25}{2} + Z_{n+1}$$

où $(Z_n)_{n \in \mathbb{N}}$ est une suite de variables aléatoires i.i.d de loi $\mathcal{N}(0, 10)$. Alors ce processus est une chaîne de Markov avec :

$$f(x, u) = \frac{x + 25}{2} + \Phi^{-1}(u)$$

avec $N \sim \mathcal{N}(0, 10)$ et :

$$\Phi(u) = \mathbb{P}(N \leq u) = \left(\int_{-\infty}^u e^{-x^2/20} dx \right) \times \frac{1}{\sqrt{20\pi}}$$

Remarque Cet exemple nous pose une problématique. En effet, nous avons défini notre chaîne de Markov avec seulement une suite de variables i.i.d de loi uniforme et chaque terme de la chaîne ne dépend que d'une seule réalisation de cette loi uniforme. Pour nous permettre de faire dépendre un X_n d'une loi normale, nous devrions pouvoir utiliser deux variables uniformes pour la simuler (par la méthode de Box-Muller).

Le lemme suivant nous l'assure donc en affirmant que l'on peut produire autant de loi uniformes souhaité à partir d'une seule.

2.1.2 La simulation en pratique

Lemme (Doublement de la loi uniforme) Il existe $g : [0, 1] \rightarrow [0, 1]^2$ mesurable telle que si $U \sim \mathcal{U}([0, 1])$ et en posant $(V, W) = g(U)$ alors :

$$V \perp W \quad \text{et} \quad W, V \sim \mathcal{U}([0, 1])$$

Corollaire (Génération de lois uniformes) . Il existe une fonction mesurable $h : [0, 1] \rightarrow [0, 1]^{\mathbb{N}}$ telle que $h(U)$ est une suite de variables iid de loi $\mathcal{U}([0, 1])$.

Remarque Ces deux derniers résultats nous permettent d'écrire :

$$X_{n+1} = f(X_n, Z_{n+1,1}, \dots, Z_{n+1,k})$$

avec $(Z_{n,k})_{\substack{k \geq 1 \\ n \geq 1}}$ une matrice de suites i.i.d. On a alors que $(X_n)_{n \in \mathbb{N}}$ est une chaîne de Markov.

Exemple (Processus de Galton-Watson) Soit $(X_{n,k})_{n \geq 1, k \geq 1}$ un tableau de variables aléatoires i.i.d à valeurs dans $\mathbb{N}$. Le processus défini par :

$$Z_0 = 1 \quad \text{et} \quad Z_{n+1} = \sum_{j=1}^{Z_n} X_{n+1,j}$$

est une chaîne de Markov. C'est un exemple de processus représentant l'évolution d'une population au cours du temps. Ici, Z_n représente le nombre d'individus à la génération n et $X_{n+1,j}$ le nombre de descendants de j -ème individu de cette génération. Z_{n+1} est donc le nombre d'individus créés à la génération $n + 1$.

Soit $(X_n)_{n \geq 1}$ une chaîne de Markov. Pour tout $x \in E$, on note $\mathbb{P}_x$ la loi de $(X_n)_{n \geq 1}$ conditionnellement à $X_0 = x$. On a donc :

Propriété (Markov faible) . Soit $(X_n)_{n \geq 1}$ une chaîne de Markov et $N \in \mathbb{N}$, alors :

$$\mathbb{E}(h(X_{N+k}, \dots, X_{N+1}) \mid X_N, \dots, X_0) = \mathbb{E}_{X_N}(h(X_k, \dots, X_2, X_1))$$

Autrement dit, à partir du moment où l'on connaît X_N , les événements $X_{N+1}, \dots, X_{N+k}$ ne dépendent plus du passé.

Démonstration Cette propriété est juste une reformulation de la propriété de Markov vue en probabilités. En effet, d'après la propriété de Markov faible, on sait que, étudier la chaîne $X_{N+k+1}, \dots, X_{N+1}$ sachant que l'on est passé par $X_n, \dots, X_0$ revient à étudier la chaîne décalée $X_k, \dots, X_1$ sachant que l'on est parti de X_n . Plus formellement :

$$\begin{aligned} \mathbb{P}(X_{N+k} = i_{N+k}, \dots, X_{N+1} = i_{N+1} \mid X_N = i_N, \dots, X_0 = i_0) \\ = \mathbb{P}(X_k = i_k, \dots, X_1 = i_1 \mid X_N = i_N) \end{aligned}$$

Donc en terme d'espérance :

$$\begin{aligned} \mathbb{E}(h(X_{N+k}, \dots, X_{N+1}) \mid X_N, \dots, X_0) \\ = \sum_{i_1, \dots, i_k \in E} h(i_k, \dots, i_1) \times \mathbb{P}(X_{N+k} = i_k, \dots, X_{N+1} = i_1 \mid X_N, \dots, X_0) \\ = \sum_{i_1, \dots, i_k \in E} h(i_k, \dots, i_1) \times \mathbb{P}(X_k = i_k, \dots, X_1 = i_1 \mid X_N) \\ = \mathbb{E}(h(X_k, \dots, X_1) \mid X_N) \\ = \mathbb{E}_{X_N}(h(X_k, \dots, X_1)) \end{aligned}$$

Exemple On définit par récurrence :

$$F_0 = 1, F_1 = 1 \quad \text{et} \quad F_{n+1} = F_n + F_{n-1} + \varepsilon_{n+1}$$

avec $(\varepsilon_n)_{n \in \mathbb{N}}$ une suite de variables aléatoires iid de loi $\mathcal{U}(\{-1, 0, 1\})$. La suite $(F_n)_{n \in \mathbb{N}}$ représente la suite de Fibonacci pour laquelle on ferait des erreurs de calculs. Cette suite n'est pas une chaîne de Markov car :

$$\mathbb{P}(F_4 = 2 \mid F_3 = 1) \neq \mathbb{P}(F_4 = 2 \mid F_3 = 1, F_2 = 1)$$

elle ne satisfait pas la propriété de Markov. Par contre, le couple $(F_n, F_{n-1})_{n \geq 1}$ est une chaîne de Markov avec :

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = f \left(\begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}, \varepsilon_{n+1} \right)$$

avec :

$$f : \begin{cases} \mathbb{R}^2 \times \mathbb{R} \longrightarrow \mathbb{R}^2 \\ (x, y, \varepsilon) \longmapsto (x + y + \varepsilon, x) \end{cases}$$

Remarque Soit $(Z_n)_{n \geq 1}$ une suite de variables aléatoires iid et :

$$S_n = Z_1 + \dots + Z_n$$

alors :

- S_n est une chaîne de Markov
- $\max\{S_k, k \leq n\}$ n'est pas une chaîne de Markov.

- $(S_n, \max\{S_k, k \leq n\})$ est une chaîne de Markov.

Proposition (Algorithme de simulation) Comme dit précédemment, une chaîne de Markov est définie telle que :

$$\begin{cases} X_0 = x_0 \\ X_{n+1} = f(X_n, u_{n+1}) \end{cases}$$

On peut donc utiliser l'algorithme suivant pour les simuler :

```
1 def simul_chaine_Markov(n, x0, f):
2     traj = [X0]
3     for i in range(n):
4         traj.append(f(traj[-1], rand()))
5     return traj
```

Remarque La principale difficulté est donc de définir la fonction f qui caractérise notre chaîne de Markov.

2.2 Comportement asymptotique d'une chaîne de Markov

L'utilisation des chaînes de Markov en simulation est liée au théorème suivant.

Théorème (Ergodique de Birkhoff) . Soit $(X_n)_{n \geq 1}$ une chaîne de Markov. Si cette chaîne admet une mesure de probabilités π invariante et ergodique alors $\forall f \in L^1(\pi)$, on a :

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n f(X_i) = \int f d\pi \quad \text{p.s.}$$

Par conséquent, pour calculer $I = \int f d\pi$ avec π une mesure coûteuse à simuler mais une mesure invariante et ergodique d'une chaîne de Markov, on peut approcher I par $\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n f(X_i)$. Évidemment, puisque les variables X_i ne sont pas i.i.d, la convergence a lieu à un rythme plus lent. Sous certaines conditions, on peut avoir :

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n f(X_i) - \int f d\pi \right) \xrightarrow[n \rightarrow \infty]{} \mathcal{N}(0, 1)$$

Ce théorème peut être vu comme une généralisation de la loi des grands nombres aux chaînes de Markov.

Définition (Mesure Invariante) . Soient un espace mesurable $(E, \mathcal{F})$ et $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov et $u \sim \mathcal{U}([0, 1])$. Soit π une mesure sur $(E, \mathcal{F})$. On dit que π est *invariante* si pour tout $A \in \mathcal{F}$ on a :

$$\begin{aligned} \int_E \mathbb{P}_x(X_1 \in A) d\pi(x) &= \pi(A) \\ \iff \int_E \mathbb{P}(f(x, u_1) \in A) d\pi(x) &= \pi(A) \end{aligned}$$

Si π est une *mesure de probabilité*, cette définition est équivalente à :

$$X_0 \sim \pi \quad \text{et} \quad U_1 \sim \mathcal{U}([0, 1]) \implies X_1 = f(X_0, U_1) \sim \pi$$

Exemple Reprenons notre exemple de simulation de l'évolution de la température :

$$X_{n+1} = \frac{X_n + 25}{2} + Z_n \quad \text{avec} \quad Z_n \sim \mathcal{N}(0, 1)$$

Si $X_0 \sim \mathcal{N}(\mu, \sigma^2)$ alors :

$$X_1 = \frac{X_0 + 25}{2} + Z_1 \sim \mathcal{N}$$

comme combinaison de gaussiennes indépendantes. De plus on a :

$$\mathbb{E}(X_1) = \frac{25 + \mu}{2} \quad \text{et} \quad \mathbb{V}(X_1) = \frac{\sigma^2}{4} + 10$$

On a donc $X_1 \sim \mathcal{N}\left(\frac{25+\mu}{2}, \frac{\sigma^2}{4} + 10\right)$. La loi $\mathcal{N}(\mu, \sigma^2)$ est donc invariante si :

$$\begin{aligned} \mu &= \frac{25 + \mu}{2} \quad \text{et} \quad \sigma^2 = \frac{\sigma^2}{4} + 10 \\ \iff \mu &= 25, \quad \sigma^2 = \frac{40}{3} \end{aligned}$$

Remarque Attention, si $X_0 \sim \pi$ avec π une mesure invariante, alors $\forall n \in \mathbb{N}^*, X_n \sim \pi$ mais $X_n \not\sim X_{n-1}$ en général.

Définition (Mesure Ergodique) . Une mesure π sur un espace mesurable $(E, \mathcal{F})$ est dite *ergodique* pour une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ si pour tout $A \in \mathcal{F}$ on a :

$$\forall x \in E, \quad \mathbb{P}_x(X_1 \in A) = \mathbb{1}_{x \in A} \implies \pi(A) = 0 \text{ ou } \pi(\bar{A}) = 0$$

Une mesure ergodique est une mesure sur E qui ne peut pas "partager" sa masse entre différents sous-ensembles invariants pour la chaîne. Autrement dit, pour tout $A \subset E$ tel que, partant de A , la chaîne reste dans A et partant de $\bar{A}$, la chaîne reste dans $\bar{A}$, la mesure ergodique charge soit A , soit $\bar{A}$, mais pas les deux.

Pour une chaîne de Markov dans E , cela implique que la chaîne ne peut pas rester bloquée entre plusieurs sous-ensembles invariants : elle explore tous les états accessibles dans l'ensemble invariant où elle a démarré, et finit par "oublier" son point de départ.

Remarque Si une chaîne de Markov n'est pas ergodique, cela signifie qu'il existe au moins un sous-ensemble invariant non trivial $A \subset E$ tel que :

- partant de A , la chaîne reste dans A ;
- partant de A^c , la chaîne reste dans A^c .

Une mesure non invariante peut alors mettre du poids à la fois sur A et sur A^c . En pratique, cela signifie que la chaîne peut rester confinée dans différents "blocs" selon l'état initial, et donc qu'elle ne se mélange pas complètement.

Propriété (Ergodicité et Irréductibilité) . Pour les chaînes de Markov sur des espaces d'états discrets, être *ergodique* est équivalent à être *irréductible*.

Corollaire (Sur les espaces d'états discrets) . Si E est un espace d'états dicret, alors toute chaîne de Markov irréductible est μ -ergodique pour toute mesure de probabilité μ .

Théorème (Birkhoff - Variante) . Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov de mesure invariante et ergodique π . Soient $f, g \in L^1(\pi)$ on a alors :

$$\lim_{n \rightarrow \infty} \frac{\sum_{i=1}^n f(X_i)}{\sum_{i=1}^n g(X_i)} = \frac{\int f d\pi}{\int g d\pi} \quad \text{p.s.}$$

Intuitivement, puisque qu'une chaîne ergodique se "mélange bien" dans son espace d'état, alors les fréquences empiriques (la moyenne des trajectoires) reflètent bien les fréquences théoriques (la mesure π) de la chaîne de Markov.

Exemple Soit $(S_n)_{n \geq 1}$ la marche aléatoire simple sur $\mathbb{Z}$. Cette chaîne de Markov n'admet pas de mesure de probabilité invariante. En revanche, la mesure de comptage sur $\mathbb{Z}$ est invariante et ergodique. Par conséquent :

$$\forall A, B \subset \mathbb{Z}, \quad \lim_{n \rightarrow \infty} \frac{\sum_{k=1}^n \mathbb{1}_{S_k \in A}}{\sum_{k=1}^n \mathbb{1}_{S_k \in B}} = \frac{|A|}{|B|}$$

alors que :

$$\frac{1}{n} \sum \mathbb{1}_{S_k \in A} = 0$$

2.2.1 Méthode MCMC

Les méthodes *Monte-Carlo Markov Chain* sont des méthodes qui généralisent la méthode de Monte-Carlo classique pour calculer des intégrales en utilisant le théorème ergodique. Si on souhaite calculer $I = \int f d\pi$ avec π une mesure difficile à simuler, mais qui est une mesure invariante et ergodique d'une chaîne de Markov, on peut alors estimer :

$$\hat{I}_n = \frac{1}{n} \sum_{i=1}^n f(X_i)$$

Exemple (Sac à dos) On souhaite charger un sac à dos avec des objets de poids w_i . On définit :

$$C = \{u \in \{0, 1\}^n \mid \langle u, w \rangle \leq 25\}$$

l'ensemble des remplissages possibles du sac, et π la loi uniforme sur C :

$$\forall x \in C, \quad \pi(x) = \frac{1}{|C|}$$

Si $|C|$ est très grand, le dénombrement direct devient impossible, et la méthode du rejet est inefficace si $|C| \ll 2^n$. On peut alors définir une chaîne de Markov $(X_n)_{n \in \mathbb{N}}$ sur C telle que :

- Pour tout $x \in C$, on choisit uniformément un indice $i \in \{1, \dots, n\}$ et on propose de "flipper" x_i pour obtenir y :

$$y_j = \begin{cases} x_j & j \neq i \\ 1 - x_i & j = i \end{cases}$$

- Si $y \in C$, alors on accepte la transition $X_{n+1} = y$, sinon on reste en X_n .

Cette chaîne de Markov est *ergodique* (on peut atteindre tout état depuis n'importe quel autre) et π est sa *mesure invariante* :

$$\pi(x) = \sum_{y \in C} \pi(y) P(y \rightarrow x), \quad \forall x \in C$$

où $\mathbb{P}(y \rightarrow x)$ est la probabilité de transition de y vers x . Ainsi, pour n grand, la loi de X_n est approximativement π , et on peut estimer des quantités d'intérêt :

$$\mathbb{E}_\pi[f(X)] \approx \frac{1}{N} \sum_{n=1}^N f(X_n).$$

Ce procédé permet également d'estimer $|C|$ de façon approchée par des techniques de comptage MCMC.

Théorème (Vitesse de convergence du théorème ergodique) . Sous certaines hypothèses de régularité (que l'on ne détaillera pas ici), si $(X_n)_{n \geq 1}$ est une chaîne de Markov de mesure de probabilité π invariante et ergodique, on a :

$$\lim_{n \rightarrow \infty} \sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n f(X_i) - \int f d\pi \right) = \mathcal{N}(0, \sigma_f^2) \quad \text{en loi}$$

où :

- $\sigma_f^2 = \mathbb{V}_\pi(f(X_0)) + 2 \sum_{k \geq 1} \text{cov}_\pi(f(X_0), f(X_k))$
- $f \in L^2(\pi)$.

Remarque Supposons que $X_0 \sim \pi$, posons $S_n = \frac{1}{n} \sum_{i=1}^n f(X_i)$, alors :

$$\begin{aligned} \mathbb{E}(S_n) &= \mathbb{E} \left(\frac{1}{n} \sum_{i=1}^n f(X_i) \right) = \frac{1}{n} \sum_{i=1}^n \mathbb{E}(f(X_i)) \\ &= \frac{1}{n} \sum_{i=1}^n \left(\sum_{x \in E} f(x) d\pi \right) = \int f d\pi \end{aligned}$$

$$\begin{aligned}
\mathbb{V}(S_n) &= \mathbb{E}(S_n^2) - \mathbb{E}(S_n)^2 \\
&= \mathbb{E} \left[\left(\frac{1}{n} \sum_{i=0}^n f(X_i) \right)^2 \right] - \left(\mathbb{E} \left[\frac{1}{n} \sum_{i=0}^n f(X_i) \right] \right)^2 \\
&= \frac{1}{n^2} \left(\sum_{i=0}^n \sum_{j=0}^n \mathbb{E}(f(X_i)f(X_j)) - \sum_{i=0}^n \sum_{j=0}^n \mathbb{E}(f(X_i))\mathbb{E}(f(X_j)) \right) \\
&= \frac{1}{n^2} \sum_{i=0}^n \sum_{j=0}^n \mathbb{E}(f(X_i)f(X_j)) - \mathbb{E}(f(X_i))\mathbb{E}(f(X_j)) \\
&= \frac{1}{n^2} \sum_{i=0}^n \sum_{j=0}^n \text{Cov}(f(X_i), f(X_j)) \\
&= \frac{1}{n^2} \left(2 \sum_{\substack{i,j=0 \\ i \neq j}}^n \text{Cov}(f(X_i), f(X_j)) + \sum_{i=0}^n \mathbb{V}(f(X_i)) \right) \\
&= \frac{1}{n^2} \sum_{i=0}^n \mathbb{V}(f(X_i)) + \frac{2}{n^2} \sum_{i=0}^{n-1} \text{Cov}(f(X_i), f(X_{i+1})) \\
&\quad \frac{2}{n^2} \sum_{i=0}^{n-2} \text{Cov}(f(X_i), f(X_{i+2})) + \cdots + \frac{2}{n^2} \text{Cov}(f(X_1), f(X_n)) \\
&= \frac{1}{n^2} (n\mathbb{V}(X_0) + 2(n-1)\text{Cov}(f(X_0), f(X_1)) + \cdots + \text{Cov}(f(X_0), f(X_n)))
\end{aligned}$$

Un obstacle à l'utilisation de méthodes MCMC est l'estimation de σ_f^2 par construire des intervalles de confiance. Plusieurs méthodes existent pour s'en affranchir.

Chapitre 3

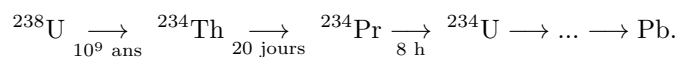
Processus de Markov en temps continu

Contents

3.1	Lois exponentielles et lemme des réveils	75
3.1.1	Rappels et Absence de mémoire	75
3.1.2	Lemme des réveils et exemples	76
3.2	Processus de Markov en temps continu	78
3.2.1	Définition, semi-groupe et générateur	78
3.2.2	Construction	79
3.2.3	Simulation	82

Pour certains modèles, la description d'un processus étape par étape à l'aide d'une chaîne de Markov peut masquer des informations importantes, comme le temps passé dans chaque état. Illustrons cela par un exemple.

Exemple (Désintégration radioactive) Considérons la désintégration radioactive de l'uranium 238, représentée par le processus suivant :



Si l'on modélisait ce processus par une chaîne de Markov classique, on ne connaîtrait que la succession des atomes traversés avant la formation du plomb, mais pas le temps passé dans chaque état, pourtant souvent l'information la plus pertinente.

Nous allons donc construire un processus de Markov en temps continu $(X_t)_{t \in \mathbb{R}_+}$ qui nous donnera cette information.

Dans la suite de ce cours, on cherchera donc à construire des processus de Markov $(X_t)_{(t \in \mathbb{R}_+)}$ sur des espaces d'états finis ou dénombrables qui vérifient une propriété de Markov. Autrement dit, tels que la variable :

$$T = \inf\{t > 0 \mid X_t \neq X_0\}$$

satisfasse une propriété d'absence de mémoire. On veut donc un équivalent à la propriété de Markov mais pour les processus à temps continu.

3.1 Lois exponentielles et lemme des réveils

3.1.1 Rappels et Absence de mémoire

Rappel (Loi exponentielle) Une variable aléatoire E est de loi exponentielle de paramètre $\lambda > 0$, noté $X \sim \mathcal{E}(\lambda)$ si pour tout $x \geq 0$, on a :

$$\mathbb{P}(X \geq x) = e^{-\lambda x} \iff \mathbb{P}(X \leq x) = 1 - e^{-\lambda x}$$

Cette variable aléatoire a donc pour densité la fonction :

$$x \mapsto \lambda e^{-\lambda x} \mathbb{1}_{x \geq 0}$$

par rapport à la mesure de Lebesgue.

Propriété (Loi exponentielle) . Soit $\lambda > 0$ et $X \sim \mathcal{E}(1)$ alors $X/\lambda \sim \mathcal{E}(\lambda)$. De plus, si $U \sim \mathcal{U}[0, 1]$ alors $-\log U \sim \mathcal{E}(1)$. On a aussi, pour $X \sim \mathcal{E}(\lambda)$:

$$\mathbb{E}(X) = \frac{1}{\lambda} \quad \text{et} \quad \mathbb{V}(X) = \frac{1}{\lambda^2}$$

Propriété (Absence de Mémoire) . On dit qu'une variable aléatoire T satisfait la propriété d'absence de mémoire si :

$$\boxed{\forall t, s > 0, \quad \mathbb{P}(T > t + s \mid T > t) = \mathbb{P}(T > s)}$$

Les seules variables satisfaisant cette propriété sont les variables aléatoires de loi exponentielle.

Démonstration Soit T une variable aléatoire satisfaisant la propriété d'absence de mémoire. Montrons que T est de loi exponentielle. Posons :

$$f : \begin{cases} \mathbb{R}_+ \longrightarrow [0, 1] \\ t \longmapsto \mathbb{P}(T > t) \end{cases}$$

On a donc $\forall t, s \geq 0$:

$$\begin{aligned} & \mathbb{P}(T > t + s \mid T > t) = \mathbb{P}(T > s) \\ \iff & \frac{\mathbb{P}(T > t + s, T > t)}{\mathbb{P}(T > t)} = \mathbb{P}(T > s) \\ \iff & \frac{\mathbb{P}(T > t + s)}{\mathbb{P}(T > t)} = \mathbb{P}(T > s) \\ \iff & \frac{f(t + s)}{f(t)} = f(s) \\ \iff & f(t + s) = f(t)f(s) \end{aligned}$$

En particulier, pour tout $n \in \mathbb{N}$, on a :

$$f(n) = f(n - 1 + 1) = f(1)f(n - 1) = f(1)^n$$

Soient $p, q \in \mathbb{N}$, on a alors :

$$f(p) = f\left(\underbrace{\frac{p}{q} + \dots + \frac{p}{q}}_{q \text{ fois}}\right) \leq f\left(\frac{p}{q}\right)^q$$

or $f(p) = f(1)^p$ donc pour tout $q \in \mathbb{Q}_+$, on a :

$$f(q) = f(1)^q = e^{q \log(f(1))}$$

Soit $x \in \mathbb{R}_+$, par décroissance de f , pour tout $n \in \mathbb{N}$, on a :

$$\begin{aligned} f\left(\frac{\lceil nx \rceil + 1}{n}\right) &\leq f(x) \leq f\left(\frac{\lceil nx \rceil}{n}\right) \\ \Leftrightarrow f(x) &\in \left[\exp\left(\frac{\lceil nx \rceil + 1}{n} \log f(1)\right); \exp\left(\frac{\lceil nx \rceil}{n} \log f(1)\right) \right] \end{aligned}$$

d'où $f(x) = \exp(x \log f(1))$. Par conséquent $T \sim \mathcal{E}(-\log f(1))$.

3.1.2 Lemme des réveils et exemples

Lemme (Réveils) Soient $T_1, \dots, T_n$ des variables aléatoires indépendantes de lois respectives $\mathcal{E}(\lambda_1), \dots, \mathcal{E}(\lambda_n)$. On pose :

$$T = \min_{1 \leq i \leq n} T_i \quad \text{et} \quad I \in \llbracket 1, n \rrbracket \text{ tel que } T_I = T$$

On a donc :

$$\{I = k\} = \{T_k < T_1, T_k < T_2, \dots, T_k < T_n\}$$

Alors :

1. $T \sim \mathcal{E}(\lambda_1 + \dots, \lambda_n)$
2. $\mathbb{P}(I = k) = \frac{\lambda_k}{\lambda_1 + \dots + \lambda_k}$
3. $T \perp I$.

Démonstration Soient $T_1, \dots, T_n$ des variables aléatoires indépendantes de loi $\mathcal{E}(\lambda_1), \dots, \mathcal{E}_{\lambda_n}$. Posons :

$$T = \min_{1 \leq i \leq n} T_i \quad I \in \llbracket 1, n \rrbracket, T_I = T$$

Soient $x \geq 0$ et $k \in \llbracket 1, n \rrbracket$, on a :

$$\mathbb{P}(T > x, I = k) = \mathbb{P}(x \leq T_k \leq \underbrace{\min\{T_j \mid j \leq n, j \neq k\}}_{z_k})$$

on a donc $z_k = \min\{T_1, T_2, \dots, T_{k-1}, T_{k+1}, \dots, T_n\}$ de plus :

$$\begin{aligned} \mathbb{P}(z_k \geq y) &= \prod_{\substack{k=1 \\ j \neq k}}^n \mathbb{P}(T_j \geq y) = \prod_{\substack{k=1 \\ j \neq k}}^n e^{\lambda_j y} \\ &= e^{-(\lambda_1 + \dots + \lambda_n - \lambda_k)y} \end{aligned}$$

Par suite :

$$\begin{aligned}
\mathbb{P}(T > x, I = k) &= \mathbb{E}(\mathbb{P}(x \leq T_k \leq z_k \mid T_k)) \\
&= \mathbb{E}\left(\mathbb{1}_{T_k \geq x} \times e^{-((\lambda_1 + \dots + \lambda_n) - \lambda_k)T_k}\right) \\
&= \int_x^\infty e^{-((\lambda_1 + \dots + \lambda_n) - \lambda_k)t} \times f_{T_k}(t) dt \\
&= \lambda_k \int_x^\infty e^{-\lambda_k t} e^{-((\lambda_1 + \dots + \lambda_n) - \lambda_k)t} dt \\
&= \lambda_k \int_x^\infty e^{-(\lambda_1 + \dots + \lambda_n)t} dt \\
&= \lambda_k \left[-\frac{e^{-(\lambda_1 + \dots + \lambda_n)t}}{\lambda_1 + \dots + \lambda_n} \right]_x^\infty \\
&= \lambda_k \frac{e^{-(\lambda_1 + \dots + \lambda_n)x}}{\lambda_1 + \dots + \lambda_n}
\end{aligned}$$

En particulier, on a :

$$\mathbb{P}(I = k) = \mathbb{P}(I = k, T > 0) = \frac{\lambda_k}{\lambda_1 + \dots + \lambda_n}$$

Par indépendance, on a donc :

$$\begin{aligned}
\mathbb{P}(T \geq x) &= \prod_{i=1}^n \mathbb{P}(T_i \geq x) \\
&= \prod_{i=1}^n e^{-\lambda_i x} = e^{-(\lambda_1 + \dots + \lambda_n)x}
\end{aligned}$$

Enfin, on a donc :

$$\mathbb{P}(T \geq x, I = k) = \mathbb{P}(T \geq x) \mathbb{P}(I = k)$$

Remarque Si on souhaite tirer un entier x proportionnellement à un poids w_x sans calculer $\sum_{i=1}^n w_i$, le lemme des réveils nous donne une solution. On peut donc utiliser l'algorithme suivant :

```

1 def tirer(poids):
2     x = -1
3     val = inf
4     for k in range(len(poids)):
5         e = -log(rand())/poids[k]
6         if e < val:
7             val = e
8             x = k
9     return x
10

```

Exemple (Sélection sans remplacement) Soient $w_1, \dots, w_n$ des poids positifs associés aux entiers $\llbracket 1, n \rrbracket$. On souhaite tirer sans remplacement k entiers avec des probabilités proportionnelles à w_k . On procédera donc comme suit :

1. On sélectionne I_1 avec :

$$\mathbb{P}(I_1 = j) = \frac{w_j}{w_1 + \dots + w_n}$$

2. Conditionnellement à I_1 , on sélectionne I_2 avec :

$$\mathbb{P}(I_2 = j \mid I_1) = \frac{w_j \mathbb{1}_{I_1 \neq j}}{\sum_{i=1}^n w_i - w_{I_1}}$$

Par généralisation, du lemme des réveils, les variables $I_1, \dots, I_n$ sont de même loi que les variables construites comme suit. Soient $T_1, \dots, T_n$ des variables de loi $\mathcal{E}(w_k)$ on pose I_j l'indice correspondant à la j ème plus petite valeur de $\{T_1, \dots, T_n\}$.

$$\text{i.e. } |\{k \in \llbracket 1, n \rrbracket \mid T_k \leq T_{I_j}\}|$$

Exemple Soient $w_1 = w_2 = \dots = w_{n-1} = 1$ et $w_n = a < 1$. Alors :

$$\begin{aligned} \mathbb{P}(\text{dernier choisi soit } n) &= \mathbb{P}(I_n = n) \\ &= \mathbb{P}(T_n \geq \max(T_1, \dots, T_{n-1})) \\ &= \int_0^\infty a e^{-at} \mathbb{P}(\max(T_1, \dots, T_{n-1}) \leq t) dt \\ &= a \int_0^\infty e^{-at} (1 - e^{-t})^{n-1} dt \end{aligned}$$

3.2 Processus de Markov en temps continu

3.2.1 Définition, semi-groupe et générateur

Nous considérons ici que E est un espace d'états fini ou dénombrable.

Définition (Processus de Markov en temps continu) . Un processus $(X_t)_{t \geq 0}$ sur E est un processus de Markov en temps continu si :

$$\forall j \in E, \forall t, s \geq 0, \quad \mathbb{P}(X_{t+s} = j \mid \mathcal{F}_t) = \mathbb{P}_s(X_t, j)$$

où $\mathcal{F}_t$ est la tribu engendrée par les événements $\{X_r = x_r\}, 0 \leq r \leq t$. En particulier, pour tout $i, j \in E$ et $\forall t \geq 0$, on a :

$$P_t(i, j) = \mathbb{P}(X_t = j \mid X_0 = i)$$

où (P_t) est le *semi-groupe* du processus de Markov.

Propriété (Semi-groupe) . $(P_t)_{t \geq 0}$ est un *semi-groupe* si :

$$\forall t, s \in E, \quad P_{t+s} = P_t P_s$$

si E est fini, alors $(P_t)_{t \geq 0}$ est une matrice et $P_t P_s$ est un produit matriciel. Dans un cas plus général, on a :

$$\begin{aligned} P_t P_s(i, j) &= \sum_{k \in E} P_t(i, k) P_s(k, j) = P_{t+s}(i, j) \\ &= \sum_{k \in E} \mathbb{P}(X_t = k \mid X_0 = i) \mathbb{P}(X_{t+s} = j \mid X_t = k) \\ &= \sum_{k \in E} \mathbb{P}(X_t = k, X_{t+s} = j \mid X_0 = i) \\ &= \mathbb{P}(X_{t+s} = j \mid X_0 = i) \end{aligned}$$

Donc le semi-groupe caractérise la loi du processus de Markov. De même, grâce à la cette propriété, il n'est pas nécessaire de connaître tout le semi-groupe.

Propriété (Génération du semi-groupe) . En particulier, $(P_t)_{t \geq 0}$ est uniquement déterminé par $(P_s)_{0 \leq s \leq a}$ avec a arbitrairement petit.

Définition (Générateur Infinitésimal) . Le *générateur infinitésimal* d'un processus de Markov $(X_t)_{t \geq 0}$ est défini par :

$$\begin{aligned} Q_{i,j} &= \lim_{t \rightarrow 0} \frac{P_t(i,j) - \mathbb{1}_{i=j}}{t} \\ &= \lim_{t \rightarrow 0} \frac{\mathbb{P}_i(X_t = j) - \mathbb{1}_{i=j}}{t} \end{aligned}$$

Il correspond à la dérivée de $P_t(i,j)$ par rapport à t en 0.

Lemme Soit $(X_t)_{t \geq 0}$ un processus de Markov sur E tel que $X_0 = i$ p.s. On pose :

$$T = \inf\{t > 0 \mid X_t \neq i\} \quad \text{et} \quad Y = X_T$$

La variable T suit une loi exponentielle et est indépendante de Y .

Intuitivement, cela signifie que le temps que prend le processus pour sortir d'un état suit une loi exponentielle et ne dépend pas de l'état suivant.

Remarque Le générateur infinitésimal caractérise la loi du processus de Markov. Donc $-Q_{i,i}$ donne le paramètre de la loi exponentielle correspondant au temps de sortie de l'état i par X . De même, $\frac{Q_{i,j}}{Q_{i,i}}$ donne la probabilité de finir en j après un saut hors de i .

3.2.2 Construction

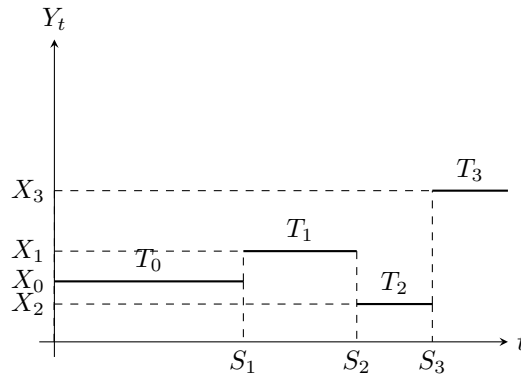
Proposition Un processus de Markov de générateur Q peut donc être construit comme suit :

- On fixe X_0 .
- On génère T_0 une variable aléatoire de loi $\mathcal{E}(-Q_{X_0, X_0})$ ainsi que X_1 indépendant de X_0 tel que :

$$\mathbb{P}(X_1 = j) = -\frac{Q_{X_0, X_1}}{Q_{X_0, X_0}}$$

- Par récurrence, X_n étant donné, on génère T_{n+1} de loi $\mathcal{E}(-Q_{X_n, X_n})$ et X_{n+1} de loi :

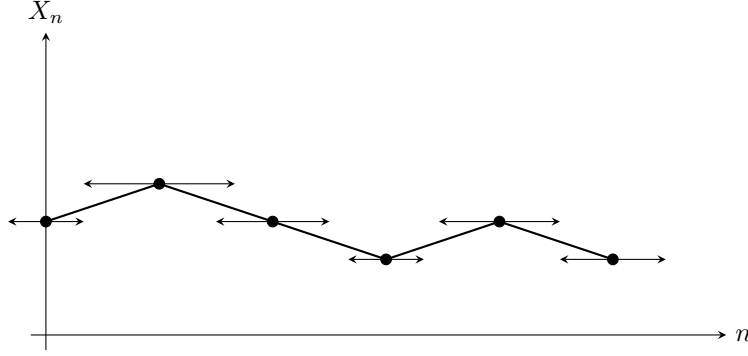
$$\mathbb{P}(X_{n+1} = j \mid X_n) = -\frac{Q_{X_n, j}}{Q_{X_n, X_n}}$$



Un processus de Markov en temps continu est donc un processus qui reste un temps exponentiel dans un état avant de sauter dans un autre. On peut alors poser :

$$S_k = T_0 + T_1 + \cdots + T_{k-1} \quad \text{et} \quad \forall t \in [S_k, S_{k+1}], Y_t = X_k$$

Les variables $S_1, S_2, \dots$ définissent les temps de saut du processus Y et $(X_n)_{n \geq 0}$ est une chaîne de Markov (à temps discret) appelée le *squelette* du processus de Markov.



Un processus de Markov en temps continu est donc une chaîne de Markov en temps discret auquel on a rajouté des durées aléatoires (exponentielles) entre le passage d'un état à l'autre. On peut observer une limite au processus. En effet, le processus de Markov est défini jusqu'au temps $\bar{S} = \lim_{k \rightarrow \infty} S_k$. Ainsi un processus tel que $\mathbb{P}(\bar{S} < \infty) > 0$ est dit *explosif*. Sinon il est dit *non explosif*.

Formalisons maintenant toutes ces nouvelles notions.

Définition (Squelette d'un processus de Markov) . Soit $(Y_t)_{t \geq 0}$ un processus de Markov à temps continu, et $(S_k)_{k \geq 0}$ la suite croissante de ses temps de saut, définie par :

$$S_0 = 0, \quad S_{k+1} = S_k + T_k$$

où les T_k sont les durées passées dans chaque état avant le saut suivant. On appelle *squelette* du processus (Y_t) la suite $(X_n)_{n \geq 0}$ définie par :

$$X_n = Y_{S_n}, \quad \text{pour tout } n \geq 0$$

Le processus $(X_n)_{n \geq 0}$ est alors une *chaîne de Markov à temps discret*, dont la matrice de transition P est donnée par :

$$P_{ij} = \mathbb{P}(X_{n+1} = j \mid X_n = i) = -\frac{Q_{ij}}{Q_{ii}}, \quad i \neq j.$$

Définition (Processus explosif) . Soit $(Y_t)_{t \geq 0}$ un processus de Markov à temps continu et $(S_k)_{k \geq 0}$ la suite de ses temps de saut. On définit :

$$\bar{S} = \lim_{k \rightarrow \infty} S_k$$

- Le processus (Y_t) est dit *explosif* si

$$\mathbb{P}(\bar{S} < \infty) > 0$$

c'est-à-dire qu'il effectue une infinité de sauts en un temps fini.

- Il est dit *non explosif* si

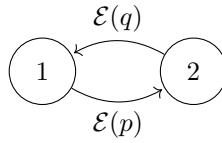
$$\mathbb{P}(\bar{S} = \infty) = 1$$

Vocabulaire. On appelle $Q_{i,j}$ le temps de saut de l'état i vers j . On a aussi $Q_{i,j} = \lambda_i \mathbb{P}_i(Y = j)$.

Exemple (Espace d'états fini) Soit $E = \{1, 2\}$ et $(X_t)_{t \geq 0}$ un processus de Markov sur E donné par le générateur infinitésimal :

$$Q = \begin{pmatrix} -p & p \\ q & -q \end{pmatrix}$$

On a donc le graphe suivant pour cette chaîne de Markov :



La durée de séjour du processus dans l'état 1 suit donc une loi $\mathcal{E}(p)$ et celle dans l'état 2 suit $\mathcal{E}(q)$.

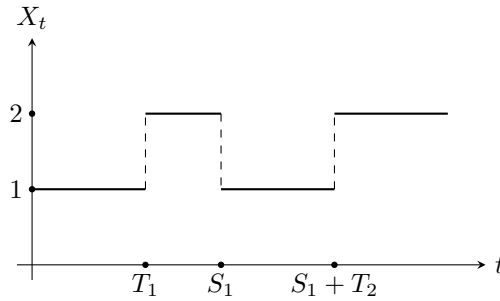
La probabilité de passer d'un état à l'autre suit une loi exponentielle. Soient $(T_k)_{k \in \mathbb{N}}$ des variables iid de loi $\mathcal{E}(p)$ et $(Q_k)_{k \in \mathbb{N}}$ des variables iid de loi $\mathcal{E}(q)$. Posons :

$$S_0 = 0, \quad \text{et} \quad S_n = \sum_{k=1}^n T_k + Q_k$$

On a donc :

$$\begin{cases} X_t = 1 & \text{si } t \in \bigcup_{n \in \mathbb{N}} [S_n, S_n + T_{n+1}[\\ X_t = 2 & \text{si } t \in \bigcup_{n \in \mathbb{N}} [S_n + T_{n+1}, S_{n+1}[\end{cases}$$

On peut donc représenter la trajectoire de ce processus par le graphe suivant :



Exemple (Espace d'états infini) Posons maintenant $E = \mathbb{N}^*$ et soit $(X_t)_{t \geq 0}$ un processus de Markov sur E de générateur :

$$Q_{n,n} = -n^2, \quad \text{et} \quad Q_{n,n+1} = n^2$$

Le processus passe donc de l'état n à $n + 1$ à taux n^2 . Le squelette de la chaîne est donc le processus

$$(1, 2, 3, 4, \dots)$$

avec $S_n = T_1 + T_2 + \dots + T_n$ où $(T_n)_{n \in \mathbb{N}}$ sont indépendantes et de loi $\mathcal{E}(k^2)$. On peut donc en déduire que :

$$S_n = E_1 + \frac{E_2}{2^2} + \dots + \frac{E_n}{n^2}$$

où les $(E_i)_{i \in \mathbb{N}}$ sont iid de loi $\mathcal{E}(1)$. On a alors :

$$\bar{S} = \lim_{n \rightarrow \infty} S_n = \sum_{k=1}^{\infty} \frac{E_k}{k^2}$$

$$\mathbb{E}(\bar{S}) = \sum_{k=1}^{\infty} \frac{\mathbb{E}(E_k)}{k^2} = \sum_{k=1}^{\infty} \frac{1}{k^2} < \infty$$

d'où $\bar{S} < \infty$ presque sûrement. Cette chaîne est donc *explosive*.

3.2.3 Simulation

Proposition La construction des processus de Markov donné ici est parfois appelé *processus de Gillespie*. On peut définir l'algorithme suivant pour les simuler :

```

1 def procMarkov(t,x,Q):
2     tau = - 1 * log(rand()) / (- Q(x,x))
3     if tau > t :
4         return x
5     # On tire x selon la loi -Qx,. / -Qx,x
6     return procMarkov(t - tau, y, Q)
```

Exemple (Processus de naissance et de mort) Soit $(X_t)_{t \geq 0}$ un processus de Markov sur $\mathbb{N}^*$ et pour tout $i \geq 0$ soit b_i le taux de naissance en i et d_i le taux de mort en i .

$$\text{i.e. } \begin{cases} X_t \text{ saute de } i \text{ en } i+1 \text{ à taux } b_i \\ X_t \text{ saute de } i \text{ en } i-1 \text{ à taux } d_i \end{cases}$$

On ne simule évidemment pas ce processus à partir de son générateur, mais plutôt par un tirage successif des durées de séjour exponentielles et des transitions, comme dans le code ci-dessous :

```

1 def simulMortNaissance(x,t,b,d):
2     etat = x
3     tps = - log(rand()) / (b[etat] + d[etat])
4     while tps < t :
5         if rand() < b[etat] / (b[etat] + d[etat]):
6             etat += 1
7         else :
8             etat -= 1
9         tps += - log(rand()) / (b[etat] + d[etat])
10    return etat
```

Proposition (Construction par le lemme des réveils) Il existe une deuxième construction des processus de Markov, fondée sur le *lemme des réveils*.

Lorsque le processus (X_t) est en état $x \in E$, on associe à chaque $y \in E \setminus \{x\}$ une variable aléatoire indépendante T_y de loi exponentielle $\mathcal{E}(Q_{x,y})$. On pose alors :

$$T = \min_{y \neq x} T_y, \quad Y = \arg \min_{y \neq x} T_y.$$

Le processus reste dans l'état x pendant une durée T , puis saute en l'état Y .

Ainsi, la durée de séjour en x suit une loi exponentielle de paramètre $-Q_{x,x} = \sum_{y \neq x} Q_{x,y}$, et la probabilité de transition vérifie :

$$\mathbb{P}(X_{t+T} = y \mid X_t = x) = -\frac{Q_{x,y}}{Q_{x,x}}.$$

Chapitre 4

Algorithmes de Metropolis-Hasting et Recuit Simulé

Contents

4.1	Cas d'un espace d'états fini	83
4.2	Cas d'un espace d'états continu	86
4.3	Mesures de Gibbs sur un espace d'états fini	87
4.4	Algorithme du recuit simulé	88

L'algorithme de Metropolis-Hasting est une procédure permettant de transformer une chaîne de Markov par le rejet sélectif de certains de ses pas en une nouvelle chaîne admettant une mesure invariante fixée.

Proposition (Algorithme de Metropolis-Hastings) Soit $(U_n)_{n \geq 1}$ et $(V_n)_{n \geq 0}$ deux suites de variables aléatoires indépendantes et identiquement distribuées, telles que $U_n \sim \mathcal{U}([0, 1])$ et $V_n \sim \mathcal{U}([0, 1])$.

Soit $f : E \times [0, 1] \rightarrow E$ une fonction qui génère un candidat $y = f(x, U)$ à partir d'un état $x \in E$ et d'une variable uniforme U .

On définit la chaîne $(Y_n)_{n \geq 0}$ de Metropolis-Hastings par :

$$Y_0 \in E, \quad Y_{n+1} = \begin{cases} y = f(Y_n, U_{n+1}), & \text{si } V_n < \alpha(Y_n, y), \\ Y_n, & \text{sinon,} \end{cases}$$

où la probabilité d'acceptation est

$$\alpha(x, y) = \min \left(1, \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right)$$

et

$$Q(x, A) = \mathbb{P}(f(x, U) \in A), \quad A \subset E.$$

4.1 Cas d'un espace d'états fini

Proposition (Matrice de transition) Soit $(X_n)_{n \in \mathbb{N}}$ une chaîne de Markov sur un espace d'états fini E . On note :

$$\forall x, y \in E, \quad Q(x, y) = \mathbb{P}(X_1 = y \mid X_0 = x)$$

Supposons que $(X_n)_{n \in \mathbb{N}}$ est irréductible. Alors l'algorithme de Metropolis-Hasting transforme la chaîne $(X_n)_{n \in \mathbb{N}}$ en une chaîne $(Y_n)_{n \in \mathbb{N}}$ de matrice de transition :

$$P(x, y) = \begin{cases} Q(x, y) \times \min \left(1, \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right) & \text{si } x \neq y \\ 1 - \sum_{z \neq x} P(x, z) & \text{si } x = y \end{cases}$$

Démonstration Soient $x, y \in E$ tels que $x \neq y$. Calculons $\mathbb{P}(Y_1 = y \mid Y_0 = x)$. On a :

$$\begin{aligned} \mathbb{P}(Y_1 = y \mid Y_0 = x) &= \mathbb{P}_x \left(f(x, U_1) = y, V_1 < \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right) \\ &= \mathbb{P}_x(X_1 = y) \mathbb{P} \left(V_1 < \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right) \\ &= Q(x, y) \min \left(1, \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right) \end{aligned}$$

De même :

$$\begin{aligned} \mathbb{P}(Y_1 = x \mid Y_0 = x) &= 1 - \sum_{y \neq x} \mathbb{P}(Y_1 = y \mid Y_0 = x) \\ &= 1 - \sum_{y \neq x} P(x, y) \end{aligned}$$

Proposition Si pour tout $x, y \in E$, on a :

$$Q(x, y) > 0 \iff Q(y, x) > 0$$

alors la mesure π est réversible pour la chaîne $(Y_n)_{n \in \mathbb{N}}$.

Démonstration Soient $X, y \in E$. On souhaite vérifier que :

$$\pi(x)P(x, y) = \pi(y)P(y, x)$$

Distinguons deux cas :

- si $x = y$ alors on a évidemment $\pi(x)P(x, x) = \pi(x)P(x, x)$.
- si $x \neq y$ alors :

$$\pi(x)P(x, y) = \pi(x)Q(x, y) \min \left(1, \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right)$$

de même :

$$\pi(y)P(y, x) = \pi(y)Q(y, x) \min \left(1, \frac{\pi(x)Q(x, y)}{\pi(y)Q(y, x)} \right)$$

Supposons que $\pi(x)Q(x, y) \leq \pi(y)Q(y, x)$ alors :

$$\pi(x)P(x, y) = \pi(x)Q(x, y)$$

et

$$\begin{aligned} \pi(y)P(y, x) &= \pi(y)Q(y, x) \frac{\pi(x)Q(x, y)}{\pi(y)Q(y, x)} \\ &= \pi(x)P(x, y) \end{aligned}$$

Si $\pi(x)Q(x, y) > \pi(y)Q(y, x)$, le raisonnement est le même.

Exemple Soit $(X_n)_{n \in \mathbb{N}}$ la marche aléatoire simple sur $\mathbb{Z}$. Soit π une mesure de probabilité sur $\mathbb{Z}$. On cherche à construire une chaîne de Markov $(Y_n)_{n \in \mathbb{N}}$ sur $\mathbb{Z}$ avec l'algorithme de Metropolis-Hasting de mesure réversible π . On a donc les algorithmes suivants :

```

1 def etapeMa(x):
2     if rand() < 1/2:
3         return x+1
4     return x-1
5

```

```

1 def algoMetHast(x):
2     y = etapeMa(x)
3     if rand() < pi(y) * (1/2) / pi(x) * (1/2):
4         return y
5     return x
6

```

On a donc :

$$Q(x, x+1) = Q(x, x-1) = \frac{1}{2}$$

$$\forall x, y \in E, |x-y| \neq 1, \quad Q(x, y) = 0$$

Et :

$$P(x, x+1) = \frac{1}{2} \min\left(1, \frac{\pi(x+1)}{\pi(x)}\right) \quad \text{et} \quad P(x, x-1) = \frac{1}{2} \min\left(1, \frac{\pi(x-1)}{\pi(x)}\right)$$

Propriété (Irréductibilité et apériodicité) . Soit Q une matrice de transition irréductible sur un espace d'états E tel que

$$\forall x, y \in E, \quad Q(x, y) > 0 \iff Q(y, x) > 0,$$

et soit π une distribution cible strictement positive sur E . Soit P la matrice de transition de la chaîne de Markov construite par l'algorithme de Metropolis-Hastings à partir de Q et π .

Alors :

- i) La chaîne de transition P est irréductible.
- ii) Si $P(x, x) > 0$ pour au moins un $x \in E$, alors P est apériodique.

Démonstration i) **Irréductibilité.** Soient $x, y \in E$. Comme Q est irréductible, il existe un chemin fini

$$x = z_1, z_2, \dots, z_k = y \in E$$

tel que $Q(z_i, z_{i+1}) > 0$ pour tout $i = 1, \dots, k-1$. Or par définition de Metropolis-Hastings :

$$P(z_i, z_{i+1}) = Q(z_i, z_{i+1}) \min\left(1, \frac{\pi(z_{i+1})Q(z_{i+1}, z_i)}{\pi(z_i)Q(z_i, z_{i+1})}\right) > 0,$$

donc $z_1, \dots, z_k$ constitue un chemin admissible pour P . Ainsi, P est irréductible.

ii) **Apériodicité.** Pour tout état $x \in E$, la probabilité de rester en x est

$$P(x, x) = 1 - \sum_{y \neq x} P(x, y) \geq 0.$$

Si $P(x, x) > 0$ pour au moins un x , alors la chaîne peut rester en x avec une probabilité strictement positive, ce qui assure l'apériodicité de P .

Remarque (Cas des chaînes symétriques) Si Q est une matrice symétrique.

$$\text{i.e } \forall x, y \in E, \quad Q(x, y) = Q(y, x)$$

et si elle est irréductible, alors l'algorithme de Metropolis-Hasting s'exprime de façon simplifiée :

$$P(x, y) = Q(x, y) \min \left(1, \frac{\pi(y)}{\pi(x)} \right) \quad \forall x, y \in E$$

et :

$$P(x, y) = 1 - \sum_{z \neq x} P(x, z) \quad \text{si } x = y$$

Dans ce cas, la probabilité d'acceptation de la transition $x \rightarrow y$ est $1 - \min(1, \pi(y)/\pi(x))$.

4.2 Cas d'un espace d'états continu

Proposition Dans le cas où E est un espace continu, tout fonctionne de manière analogue. On note $Q(x, y)$ la densité de la variable X_1 sachant $X_0 = x$. Alors :

$$\mathbb{E}(\varphi(X_1) \mid X_0 = x) = \int_E \varphi(y) Q(x, y) dy, \quad \forall \varphi \text{ mesurable.}$$

Pour tout petit intervalle $[y, y + \Delta_y]$, on a :

$$\mathbb{P}(X_1 \in [y, y + \Delta_y] \mid X_0 = x) \simeq Q(x, y) \Delta_y.$$

On note $\pi(x)$ la densité de la mesure cible et on définit la transition de la chaîne $(Y_n)_{n \in \mathbb{N}}$ par :

$$P(x, dy) = Q(x, y) \min \left(1, \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right) dy + \left[1 - \int_E Q(x, y) \min \left(1, \frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} \right) dy \right] \delta_x(dy),$$

où δ_x est la mesure de Dirac en x . On a alors :

$$\int_E P(x, dy) = 1.$$

L'algorithme reste lui aussi inchangé. On génère $y = X_1$ à partir de la chaîne Q . On accepte la transition avec probabilité $\frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)}$. Si on refuse la transition, on reste sur place. À nouveau, la chaîne de Markov sera, sous de bonnes hypothèses, irréductible et la mesure π sera réversible pour cette chaîne.

Exemple (Marche aléatoire à pas gaussiens) Soit $(Z_n)_{n \in \mathbb{N}}$ une suite de variables iid de loi $\mathcal{N}(0, 1)$. On définit le processus $(X_n)_{n \in \mathbb{N}}$ par :

$$X_0 \in \mathbb{R}, \quad \forall n \geq 1, X_{n+1} = X_n + Z_{n+1}$$

On aura donc :

$$\forall x, y \in \mathbb{R}, \quad Q(x, y) = \frac{1}{\sqrt{2\pi}} \exp \left(-\frac{(y-x)^2}{2} \right)$$

Donc si π est une densité cible quelconque, alors :

$$\frac{\pi(y)Q(y, x)}{\pi(x)Q(x, y)} = \frac{\pi(y)}{\pi(x)} \quad \text{car } Q(x, y) = Q(y, x)$$

On peut alors définir la nouvelle chaîne $(Y_n)_{n \in \mathbb{N}}$ par :

$$\forall n \in \mathbb{N}, Y_{n+1} = Y_n + Z_{n+1} \mathbb{1}_{\frac{\pi(Y_n + Z_{n+1})}{\pi(Y_n)} < V_{n+1}}$$

avec $(V_n)_{n \in \mathbb{N}}$ une suite iid de variables de loi $\mathcal{U}([0, 1])$. Cette nouvelle chaîne de Markov admet alors π pour mesure invariante, réversible et ergodique. De plus :

$$Y_n \xrightarrow[n \rightarrow \infty]{L} \pi$$

L'algorithme de Metropolis-Hasting permet donc d'approcher n'importe quelle densité sur $\mathbb{R}$. Or il peut être très lent dans certains cas.

4.3 Mesures de Gibbs sur un espace d'états fini

Soit E un espace d'états fini. On définit :

$$H : E \longrightarrow \mathbb{R} \quad (4.3.1)$$

une fonction d'énergie sur E . En physique statistique, elle représenterait l'énergie d'un état du système.

Définition (Mesure de Gibbs) . La mesure de Gibbs sur E de température inverse β est la mesure de probabilité μ_β définie par :

$$\mu_\beta(x) = \frac{1}{z_\beta} e^{\beta H(x)} \quad (4.3.2)$$

où $z_\beta = \sum_{x \in E} e^{\beta H(x)}$.

Proposition (Fondamentale) Par construction de la mesure de Gibbs, lorsque β est petit (le système a beaucoup d'énergie), la mesure a tendance à s'uniformiser sur E . Or, lorsque β est petit (i.e le système a peu d'énergie), alors μ_β ne charge plus que l'ensemble $\mathcal{M} = \{x \in E \mid \mu_\beta(x) = \mu_0\}$ où μ_0 est le maximum de μ_β .

L'algorithme du recuit simulé, que nous verrons plus tard, cherche donc à déterminer de telles solutions en simulant une marche aléatoire sur E de loi μ_β . En refroidissant le système petit à petit (i.e en augmentant β), la marche se stabilisera sur un maximum. Mais pour cela, nous devons savoir simuler μ_β . Pour cela, on utilisera l'algorithme de Metropolis-Hastings.

Or lorsque E est grand, simuler une variable aléatoire de loi μ_β est difficile pour plusieurs raisons :

- z_β devient très grand : lors de l'approximation numérique, on peut donc avoir des valeurs de $e^{\beta H(x)}/z_\beta$ encodées comme 0.
- z_β devient difficile à calculer du fait de la taille de E .

Les méthodes directes de simulation de mesure de Gibbs fonctionnent très mal. Heureusement, on utilise alors l'algorithme de Metropolis-Hasting.

Remarque (Adaptation de l'algorithme) Soit Q la matrice de transition d'une chaîne de Markov irréductible sur E telle que $Q(x, y) > 0 \iff Q(y, x) > 0$ pour tout $x, y \in E$. Pour simuler la loi μ_β , on utilise la matrice de transition définie pour tout $x, y \in E$ par :

$$P_\beta(x, y) = Q(x, y) \min \left(1, \frac{\mu_\beta(y)Q(y, x)}{\mu_\beta(x)Q(x, y)} \right) \quad (4.3.3)$$

$$= Q(x, y) \min \left(\frac{\frac{e^{\beta H(y)}}{z_\beta} Q(y, x)}{\frac{e^{\beta H(x)}}{z_\beta} Q(x, y)} \right) \quad (4.3.4)$$

$$= Q(x, y) \min \left(1, e^{\beta(H(y) - H(x))} \frac{Q(y, x)}{Q(x, y)} \right) \quad (4.3.5)$$

On pourra donc appliquer l'algorithme sans problème d'approximation de très petites valeurs, car z_β n'apparaît plus.

Exemple (Depinning Transition) Soit l'espace d'états suivant :

$$E = \{\text{chemins au plus proche voisin sur } \mathbb{Z} \text{ de longueur } n\}$$

Un élément de E est donc une marche aléatoire sur E . On définit la fonctionnelle d'énergie H sur E telle que :

$$\forall s \in E, \quad H(s) = \text{Card}(\{0 \leq i \leq n \mid s_i = 0\})$$

On souhaite simuler un élément de E sous la loi μ_β . On peut proposer différentes transformations pour un chemin de E :

1. Si $s \in E$, on tire $i \in \llbracket 1, n \rrbracket$ on remplace ensuite le saut en i par le saut opposé. (*C'est une marche réversible.*).
2. Si $s \in E$, on tire $i \in \llbracket 1, n \rrbracket$ et on remplace un schéma $\vee$ en i par un $\wedge$ en i . (*Celle-ci n'est pas réversible.*).

D'après 4.3.5, on peut donc définir l'algorithme suivant :

```

1 def etapeP(x,b):
2   y = etapeQ(x)
3   if rand() < (Q(y,x) / Q(x,y)) * exp(b (H(y) - H(x))):
4       return y
5   return x
6

```

Le comportement de la chaîne ainsi produite sera donc déterminé par le paramètre β . Si β sera faible (i.e température basse), la chaîne aura un comportement de marche aléatoire symétrique. En revanche, autour d'une certaine valeur de β appelée β_c , dit critique, la chaîne orbitera autour de l'axe des abscisses. C'est ce que l'on appelle une *transition de phase*.

Remarque Si Q est une matrice *réversible et symétrique* alors

$$\forall x, y \in E, P_\beta(x, y) = Q(x, y) \min \left(1, e^{\beta(H(y) - H(x))} \right)$$

Dans ce cas, plutôt que de tester $\text{rand}() < (Q(y,x) / Q(x,y)) * \exp(b (H(y) - H(x)))$, on préfère tester $\log(\text{rand}()) < b (H(y) - H(x))$. Qui sera moins sujet à provoquer des erreurs d'approximation (de valeurs infinies).

4.4 Algorithme du recuit simulé

On se place toujours dans un espace d'états E fini et $H : E \rightarrow \mathbb{R}$ une fonction. On souhaite déterminer :

$$\begin{cases} \max_E H \\ \arg\max_E H \end{cases}$$

La méthode du recuit simulé permet de déterminer un estimateur de ces quantités pour un coût relativement faible par rapport à une exploration complète de l'espace d'états. Elle est basée sur la remarque suivante.

Lemme Soit E^* l'ensemble définit par :

$$E^* := \{e \in E \mid H(e) = \max_E H\} \quad (4.4.1)$$

On a $\lim_{\beta \rightarrow \infty} \mu_\beta \sim \mathcal{U}(E^*)$.

Démonstration Soit $\delta > 0$ tel que $H(e) < \max H - \delta$ pour tout $e \notin E^*$. On a donc :

$$\begin{aligned} \sum_{e \in E^*} e^{\beta H(e)} &\leq z_\beta = \sum_{e \in E} e^{\beta H(e)} = \sum_{e \in E^*} e^{\beta H(e)} + \sum_{e \in E/E^*} e^{\beta H(e)} \\ &\iff |E^*| e^{\beta \max H} \leq z_\beta \leq |E^*| e^{\beta \max H} + |E| e^{\beta(\max H - \delta)} \end{aligned}$$

Donc pour $e \in E^*$, on a :

$$\mu_\beta(e) = \frac{e^{\beta \max H}}{z_\beta}$$

$$\iff \frac{e^{\beta \max H}}{e^{\beta \max H} |E^*| + |E| e^{\beta(\max H - \delta)}} \leq \mu_\beta(e) \leq \frac{e^{\beta \max H}}{e^{\beta \max H} |E^*|}$$

donc $\frac{1}{|E^*| + |E| e^{-\beta \delta}} \leq \mu_\beta(e) \leq \frac{1}{|E^*|}$. D'où $\lim_{\beta \rightarrow \infty} \mu_\beta(e) = 1/|E^*|$.

De même pour $e \in E/E^*$, on a :

$$\mu_\beta(e) \leq \frac{e^{\beta(\max H - \delta)}}{|E^*| e^{\beta \max H}} \leq \frac{e^{-\beta \delta}}{|E^*|} \xrightarrow{\beta \rightarrow \infty} 0$$

d'où $\mu_\beta(e) \xrightarrow{\beta \rightarrow \infty} 1/|E^*| \mathbb{1}_{e \in E^*}$.

L'algorithme du recuit simulé consiste donc à utiliser à chaque étape l'algorithme de Metropolis-Hasting pour simuler μ_β avec $\beta_n \xrightarrow{n \rightarrow \infty} \infty$ assez lentement.

Proposition (Algorithme du recuit simulé) On se donne **etape** Q , et une suite $(\beta_n)_{n \geq 0}$. On définit ensuite :

```

1
2 def recuitSimule(x,n):
3     y = etapeQ(x)
4     if rand() < exp(beta[n] (H(y) - H(x))) * (Q(y,x) / Q(x,y)):
5         return y
6     return x
7
```

La différence fondamentale de cet algorithme est que la probabilité de transition dépend maintenant de n .

Théorème (Convergence de l'algorithme du recuit simulé) . On note $(z_n)_{n \geq 0}$ la chaîne de Markov inhomogène (la chaîne dépend de l'état précédent et de la position dans le temps n) dans le temps de recuit simulé. Il existe $c_* > 0$ tel que pour tout $c < c_*$, en posant $\beta_n := c \log n$, alors :

$$\lim_{n \rightarrow \infty} z_n \in E^* \quad \text{p.s} \quad (4.4.2)$$

Ici, la suite $(\beta_n)_{n \geq 0}$ est la suite des températures inverses que nous utilisons pour simuler le refroidissement du système lors de l'exécution de l'algorithme. Ce théorème nous assure donc qu'il existe une courbe de refroidissement optimale qui permet la convergence de l'algorithme.

Remarque Ce théorème nous assure que si l'on refroidit petit à petit le système (i.e $n \rightarrow \infty$), alors la chaîne tend à s'arrêter sur un maximum de H . Dans cette situation, $\hat{M}_n = \max_{k \leq n} H(z_k)$ et $\hat{E}_n = \operatorname{argmax}_{z \in \{z_k | k \in \mathbb{N}\}} H(z)$ sont des estimateurs fortement consistants de $\max H$ et $\operatorname{argmax} H$.

En pratique ces estimateurs convergent très lentement pour un tel choix de β . On préférera donc faire tendre β vers ∞ plus rapidement et simuler plusieurs chaînes partant de points différents de l'espace d'état.

Cet algorithme est très performant en plusieurs dimensions puisque l'algorithme ne dépend pas de la nature des données sur lesquelles sont calculées H . En revanche, il est peu performant pour des fonctions H trop périodiques.

Chapitre 5

Algorithme de Robbins Monro

Contents

5.1	Cadre Général	90
5.2	Convergence de l'algorithme	91
5.2.1	Théorème de Robbins-Sigmund	91
5.2.2	Convergence de l'algorithme de Robbins-Monro	93

5.1 Cadre Général

Dans ce chapitre, nous allons présenter un algorithme permettant de déterminer les solutions de l'équation :

$$h(\theta) = 0$$

pour une fonction h lorsqu'on peut écrire $h(\theta) = \mathbb{E}(g(\theta, Y))$. Par exemple, chercher θ tel que :

$$\begin{aligned} & \int_{\mathbb{R}_+} (x - \theta) e^{-x^3} dx = 0 \\ \iff & \int_{\mathbb{R}_+} x e^{-x^3} dx - \theta \int_{\mathbb{R}_+} e^{-x^3} dx = 0 \\ \iff & \theta = \frac{\int_{\mathbb{R}_+} x e^{-x^3} dx}{\int_{\mathbb{R}_+} e^{-x^3} dx} \end{aligned}$$

Proposition (Algorithme) On se fixe une suite $(\gamma_n)_{n \in \mathbb{N}} \in (\mathbb{R}_+^*)^{\mathbb{N}}$ telle que $\sum_{n \geq 0} \gamma_n = \infty$ mais $\sum_{n \geq 0} \gamma_n^2 < \infty$. On définit par récurrence :

$$\boxed{\theta_{n+1} = \theta_n - \gamma_n g(\theta_n, Y_{n+1})} \quad (5.1.1)$$

où les $(Y_n)_{n \in \mathbb{N}}$ sont des variables aléatoires iid de même loi que Y . Sous de bonnes conditions, $(\theta_n)_{n \in \mathbb{N}}$ converge vers une solution θ de $h(\theta) = 0$.

Exemple Supposons que $g(\theta, Y) = \theta - Y$ et $\gamma_n = \frac{1}{n+1}$. La solution de $h(\theta) = 0$ est exactement $\theta = \mathbb{E}(Y)$. On remarque que :

$$\begin{aligned} \theta_{n+1} &= \theta_n - \frac{1}{n+1} (\theta_n - Y_{n+1}) \\ &= \frac{n}{n+1} \theta_n + \frac{1}{n+1} Y_{n+1} \end{aligned}$$

Donc $(n+1)\theta_{n+1} - n\theta_n = Y_{n+1}$, d'où $n\theta_n = Y_1 + \dots + Y_n$. Soit $\theta_n = \frac{1}{n} \sum_{i=1}^n Y_i$. On a bien $\lim_{n \rightarrow \infty} \theta_n = \mathbb{E}(Y)$ d'après la loi des grands nombres.

Dans le cas déterministe, on suppose ggg connue et calculable, ce qui permet d'utiliser des méthodes comme Newton. Cependant, en pratique, ggg est souvent inconnue (définie par une espérance) ou bruitée (observations aléatoires). L'algorithme de Robbins-Monro résout ce problème en remplaçant $g(\theta)$ par une estimation $H(\theta, Y)$, tout en garantissant la convergence vers la solution de $h(\theta) = 0$ sous des conditions sur les pas (α_n) .

5.2 Convergence de l'algorithme

Toute cette section est consacrée à la démonstration de la convergence de l'algorithme. Pour cela, nous allons introduire un théorème très important dans l'étude des algorithmes stochastiques : le théorème de Robbins-Sigmund.

5.2.1 Théorème de Robbins-Sigmund

Nous allons étudier ce théorème sous la forme suivante :

Théorème (Robbins-Sigmund) . Soient quatre suites $(z_n)_{n \in \mathbb{N}}, (\alpha_n)_{n \in \mathbb{N}}, (\beta_n)_{n \in \mathbb{N}}$ et $(\gamma_n)_{n \in \mathbb{N}}$ adaptées pour une filtration $(\mathcal{F}_n)_{n \in \mathbb{N}}$. Supposons qu'elles vérifient :

- i) $\sum_{n \geq 0} \alpha_n = \infty$ p.s
- ii) $\sum_{n \geq 0} \beta_n < \infty$ p.s
- iii) $\sum_{n \geq 0} \gamma_n < \infty$ p.s
- iv) $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq (1 - \alpha_n + \beta_n)z_n + \gamma_n$

Alors :

$$\lim_{n \rightarrow \infty} z_n = 0 \text{ ps} \quad (5.2.1)$$

Dans notre situation, nous pouvons considérer que $\beta_n = 0$ pour tout $n \in \mathbb{N}$. En effet, nous cherchons à modifier l'hypothèse $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq (1 - \alpha_n + \beta_n)z_n + \gamma_n$ en :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq (1 - \alpha_n)z_n + \gamma_n$$

Lemme Réduisons le théorème au cas $\beta_n = 0$.

Démonstration Supposons donc que :

- i $\sum_{n \geq 0} \alpha_n = \infty$ p.s
- ii $\sum_{n \geq 0} \beta_n < \infty$ p.s
- iii $\sum_{n \geq 0} \gamma_n < \infty$ p.s
- iv $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq (1 - \alpha_n)z_n + \gamma_n$

Montrons que si $\beta_n = 0$, alors $z_n \xrightarrow[n \rightarrow \infty]{} 0$ aussi. Pour cela, posons, pour tout $n \in \mathbb{N}$:

$$Y_n = \frac{z_n}{\prod_{j=1}^n (1 + \beta_j)}$$

On a les égalités suivantes :

$$\begin{aligned} \mathbb{E} \left(\frac{z_{n+1}}{\prod_{j=1}^{n+1} (1 + \beta_j)} \mid \mathcal{F}_n \right) &= \frac{\mathbb{E}(z_{n+1} \mid \mathcal{F}_n)}{\prod_{j=1}^{n+1} (1 + \beta_j)} \\ &\leq \frac{(1 - \alpha_n + \beta_n) z_n}{\prod_{j=1}^{n+1} (1 + \beta_j)} + \frac{\gamma_n}{\prod_{j=1}^{n+1} (1 + \beta_j)} \\ &= \frac{1 - \alpha_n + \beta_n}{1 + \beta_{n+1}} \frac{z_n}{\prod_{j=1}^n (1 + \beta_j)} + \frac{\gamma_n}{\prod_{j=1}^{n+1} (1 + \beta_j)} \end{aligned}$$

Or, pour n grand, on a :

$$\frac{1 - \alpha_n + \beta_n}{1 + \beta_n} \sim_{\infty} 1 - \alpha_n$$

D'où :

$$\mathbb{E}(Y_{n+1} \mid \mathcal{F}_n) \leq (1 - \alpha'_n) Y_n + \gamma'_n$$

Avec :

$$Y_n = \frac{z_n}{\prod_{j=1}^n (1 + \beta_j)}, \quad \alpha'_n = \frac{\alpha_n}{1 + \beta_n} \quad \text{et} \quad \gamma'_n = \frac{\gamma_n}{\prod_{j=1}^n (1 + \beta_j)}$$

On souhaite maintenant appliquer le théorème de Robbins-Sigmund à la suite $(Y_n)_{n \geq 0}$. Pour cela, vérifions les hypothèses du théorème :

i Pour α'_n , on a que $\alpha'_n = \frac{\alpha_n}{1 + \beta_n}$ donc $\alpha'_n \sim_{\infty} \alpha_n$. Par comparaison :

$$\sum_{n \geq 0} \alpha_n < \infty \text{ p.s} \implies \sum_{n \geq 0} \alpha'_n < \infty \text{ p.s}$$

ii Pour γ'_n , on a :

$$\sum_{n \geq 0} \gamma'_n = \sum_{n \geq 0} \frac{\gamma_n}{\prod_{j=1}^n (1 + \beta_j)} < \sum_{n \geq 0} \gamma_n < \infty$$

On peut donc appliquer le théorème d'où :

$$\lim_{n \rightarrow \infty} Y_n = 0 \text{ p.s}$$

De plus, par construction de Y_n , on a :

$$\lim_{n \rightarrow \infty} z_n = \lim_{n \rightarrow \infty} Y_n \underbrace{\prod_{j=1}^n (1 + \beta_j)}_{< \infty} \implies \lim_{n \rightarrow \infty} z_n = 0$$

Le théorème est donc valable avec ce nouveau jeu d'hypothèses. □

Nous pouvons désormais passer à la démonstration du théorème de Robbins-Sigmund.

Démonstration La démonstration de ce résultat est basée sur la construction d'une surmartingale et de l'utilisation du théorème de convergence des martingales pour conclure. Nous commençons donc avec les hypothèses suivantes :

- i) $\sum_{n \geq 0} \alpha_n = \infty \text{ p.s}$
- ii) $\sum_{n \geq 0} \beta_n < \infty \text{ p.s}$
- iii) $\sum_{n \geq 0} \gamma_n < \infty \text{ p.s}$
- iv) $\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq (1 - \alpha_n) z_n + \gamma_n$

Montrons que $z_n \xrightarrow[n \rightarrow \infty]{} 0$.

1. *Construction de la sur-martingale.* Posons :

$$\forall n \in \mathbb{N}, \quad S_n = z_n + \sum_{k=0}^{n-1} \alpha_k z_k - \sum_{k=0}^{n-1} \gamma_k$$

Montrons que $(S_n)_{n \geq 0}$ est une sur-martingale.

$$\begin{aligned} \mathbb{E}(S_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}(z_{n+1} \mid \mathcal{F}_n) + \sum_{k=0}^n \mathbb{E}(\alpha_k z_k \mid \mathcal{F}_n) - \sum_{k=0}^n \mathbb{E}(\gamma_k \mid \mathcal{F}_n) \\ &= \mathbb{E}(z_{n+1} \mid \mathcal{F}_n) + \sum_{k=0}^n \alpha_k z_k - \sum_{k=0}^n \gamma_k \\ &\leq (1 - \alpha_n) z_n + \gamma_n + \sum_{k=0}^n \alpha_k z_k - \sum_{k=0}^n \gamma_k \\ &= \sum_{k=0}^{n-1} \alpha_k z_k - \sum_{k=0}^{n-1} \gamma_k + z_n = S_n \end{aligned}$$

Par définition, $(S_n)_{n \geq 0}$ est bien une sur-martingale pour la filtration $(\mathcal{F}_n)_{n \geq 0}$.

2. *Convergence de la sur-martingale.* Soit τ_c le temps d'arrêt suivant :

$$\tau_c = \inf \left\{ n \in \mathbb{N} \mid \sum_{k=0}^n \gamma_k \geq 1 \right\}$$

Soit la martingale arrêtée par τ_C $(S_{n \wedge \tau_C})_{n \geq 0}$, elle est bornée inférieurement par $-C$ car :

$$S_{n \wedge \tau_C} \geq - \sum_{k=0}^{\tau_C - 1} \gamma_k \geq -C$$

Par théorème, elle converge donc presque sûrement vers une limite finie W_C . Puisque $\sum_{k \geq 1} \gamma_k < \infty$ p.s, il existe $C > \sum_{k \geq 1} \gamma_k$ tel que $\tau_c = \infty$ p.s. Donc sur l'évènement $\{\tau_c = \infty\}$, on a $S_n = S_{n \wedge \tau_C}$ pour tout n , donc :

$$\lim_{n \rightarrow \infty} S_n = W_c \text{ p.s}$$

Or $S_n = z_n + \sum_{k=0}^{n-1} \alpha_k z_k - \sum_{k=0}^{n-1} \gamma_k$, et comme $\sum_{k \geq 1} \gamma_k$ converge, la convergence de S_n implique celle de :

$$z_n + \sum_{k=0}^{n-1} \alpha_k z_k$$

mais $\sum_{k \geq 1} \alpha_k = \infty$ donc nécessairement $\sum_{k \geq 1} \alpha_k z_k < \infty$. Finalement :

$$z_n \xrightarrow[n \rightarrow \infty]{} 0 \text{ p.s}$$

□

5.2.2 Convergence de l'algorithme de Robbins-Monro

Maintenant que nous avons tous les outils nécessaires, nous pouvons passer à l'étude de la convergence de cet algorithme. Pour rappel, on souhaite déterminer les solutions de l'équation $h(\theta) = 0$ où h peut s'écrire sous la forme :

$$h(\theta) = \mathbb{E}[H(Y, \theta)],$$

où Y est une variable aléatoire quelconque que l'on sait simuler. L'algorithme de Robbins-Monro approche la solution θ^* par les itérés suivants :

$$\forall n \geq 0, \quad \theta_{n+1} = \theta_n - \gamma_n H(Y_{n+1}, \theta_n),$$

où $(Y_n)_{n \geq 0}$ est une suite de variables i.i.d. de même loi que Y .

Le théorème suivant nous assure sa convergence sous de bonnes hypothèses :

Théorème (Conditions suffisantes de convergence) . Avec les mêmes notations que précédemment :

- i) (Coercivité) Il existe θ^* tel que $h(\theta)(\theta - \theta^*) \geq \mu \|\theta - \theta^*\|^2$ pour un certain $\mu > 0$,
- ii) (Croissance de h) Il existe $A, B > 0$ tels que $\|h(\theta)\|^2 \leq A + B\|\theta\|^2$,
- iii) (Contrôle du bruit) $\mathbb{E}(\|h(\theta) - H(\theta, Y)\|^2) \leq K$ pour un certain $K > 0$.

Alors, si $\sum_{n \geq 0} \gamma_n = \infty$ et $\sum_{n \geq 0} \gamma_n^2 < \infty$, on a :

$$\lim_{n \rightarrow \infty} \theta_n = \theta^* \quad \text{p.s.}$$

Démonstration La démonstration de cette convergence utilise la construction récursive de l'algorithme ainsi que le théorème de Robbins-Siegmund. Soit $\mathcal{F}_n = \sigma(\theta_0, \dots, \theta_n, Y_0, \dots, Y_n)$ la filtration associée au n -ième itéré de l'algorithme. L'idée est de séparer l'incrément de l'algorithme en deux parties :

$$\Delta M_n = h(\theta_n) - H(\theta_n, Y_{n+1}),$$

appelée incrément de martingale. On peut donc réécrire l'algorithme comme suit :

$$\theta_{n+1} = \theta_n - \gamma_n h(\theta_n) + \gamma_n \Delta M_n.$$

Montrons que $(M_n)_{n \geq 0}$ est une martingale :

$$\begin{aligned} \mathbb{E}(\Delta M_n \mid \mathcal{F}_n) &= \mathbb{E}(h(\theta_n) - H(\theta_n, Y_{n+1}) \mid \mathcal{F}_n) \\ &= h(\theta_n) - \mathbb{E}(H(\theta_n, Y_{n+1}) \mid \mathcal{F}_n) \\ &= h(\theta_n) - h(\theta_n) = 0. \end{aligned}$$

Pour étudier la convergence, nous considérons la distance quadratique des itérés :

$$z_n = \|\theta_n - \theta^*\|^2.$$

ainsi que l'égalité suivante :

$$\|a + b + c\|^2 = \|a\|^2 + \|b\|^2 + \|c\|^2 + 2\langle a, b \rangle + 2\langle a, c \rangle + 2\langle b, c \rangle.$$

On a donc :

$$\begin{aligned} \mathbb{E}(z_{n+1} \mid \mathcal{F}_n) &= \mathbb{E}(\|\theta_{n+1} - \theta^*\|^2 \mid \mathcal{F}_n) \\ &= \mathbb{E}(\|\theta_n - \gamma_n h(\theta_n) + \gamma_n \Delta M_n - \theta^*\|^2 \mid \mathcal{F}_n) \\ &= \|\theta_n - \theta^*\|^2 + \gamma_n^2 \|h(\theta_n)\|^2 + \gamma_n^2 \mathbb{E}(\|\Delta M_n\|^2 \mid \mathcal{F}_n) \\ &\quad - 2\gamma_n \langle \theta_n - \theta^*, h(\theta_n) \rangle + 2\gamma_n \langle \theta_n - \theta^*, \Delta M_n \rangle \\ &\quad - 2\gamma_n^2 \langle h(\theta_n), \Delta M_n \rangle. \end{aligned}$$

Comme $\mathbb{E}(\Delta M_n \mid \mathcal{F}_n) = 0$, les termes croisés $\langle \theta_n - \theta^*, \Delta M_n \rangle$ et $\langle h(\theta_n), \Delta M_n \rangle$ disparaissent en espérance conditionnelle. On obtient donc :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq z_n + \gamma_n^2 \|h(\theta_n)\|^2 + \gamma_n^2 \mathbb{E}(\|\Delta M_n\|^2 \mid \mathcal{F}_n) - 2\gamma_n \langle \theta_n - \theta^*, h(\theta_n) \rangle.$$

En utilisant les hypothèses du théorème :

$$\begin{aligned}\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) &\leq z_n + \gamma_n^2(A + B\|\theta_n\|^2) + \gamma_n^2 K - 2\gamma_n \mu \|\theta_n - \theta^*\|^2 \\ &\leq z_n(1 - 2\gamma_n \mu) + \gamma_n^2(A + K + B\|\theta_n\|^2).\end{aligned}$$

Comme $\|\theta_n\|^2 \leq 2\|\theta_n - \theta^*\|^2 + 2\|\theta^*\|^2$, on peut écrire :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq z_n(1 - 2\gamma_n \mu + 2B\gamma_n^2) + \gamma_n^2(A + K + 2B\|\theta^*\|^2).$$

En posant $\alpha_n = 2\gamma_n \mu - 2B\gamma_n^2$ et $\beta_n = \gamma_n^2(A + K + 2B\|\theta^*\|^2)$, on obtient :

$$\mathbb{E}(z_{n+1} \mid \mathcal{F}_n) \leq (1 - \alpha_n)z_n + \beta_n.$$

Sous les hypothèses $\sum \gamma_n = \infty$ et $\sum \gamma_n^2 < \infty$, on a $\sum \alpha_n = \infty$ et $\sum \beta_n < \infty$. On peut donc appliquer le théorème de Robbins-Siegmund pour conclure que :

$$\lim_{n \rightarrow \infty} z_n = 0 \quad \text{p.s.},$$

c'est-à-dire $\lim_{n \rightarrow \infty} \theta_n = \theta^*$ p.s. □

Statistiques

Chapitre 1

Théorie de l'Information

Contents

1.1	Cadre	97
1.1.1	Modèle	97
1.1.2	Les variables aléatoires d'un modèle	98
1.1.3	Domination et Densités	99
1.1.4	Hypothèses de régularité	99
1.1.5	Vraisemblance	100
1.1.6	Maximum de Vraisemblance	101
1.2	Informations Qualitatives	102
1.2.1	Exhaustivité	102
1.2.2	Minimalité et complétude	103
1.2.3	Statistiques Libres	103
1.3	Information de Kullback-Leibler	104
1.4	Informations Quantitatives	104
1.4.1	Vecteur Score	104
1.4.2	Modèles exponentiels	105
1.4.3	Information de Fisher	106
1.4.4	Propriétés de l'information de Fisher	107

1.1 Cadre

Dans cette section, nous posons le cadre de l'étude des statistiques ainsi que l'ensemble des notations et objets utilisés.

1.1.1 Modèle

L'étude des statistiques se résume à l'étude de la loi de variables aléatoires ou de suites de variables aléatoires. Nous devons donc définir un cadre dans lequel ces différents objets vivrons. On considère ainsi des variables aléatoires X définies telles que :

$$X : \begin{cases} (\Omega, \mathcal{T}, \mathbb{P}) \longrightarrow (\mathcal{X}, \mathcal{A}, \mathbb{P}^X) \\ \omega \longmapsto X(\omega) \end{cases} \quad (1.1.1)$$

Dans notre contexte, nous n'avons accès qu'à des *réalisations* $X(\omega)$ de la variable aléatoire X et nous souhaitons déterminer sa loi $\mathbb{P}^X$. Pour cela, nous faisons une hypothèse appelée *hypothèse*

paramétrique forte qui suppose que la loi de X ne dépend que d'un paramètre $\theta \in \Theta$, où $\Theta \subset \mathbb{R}^d$. Ainsi, la recherche de la loi $\mathbb{P}^X$ se réduit à la détermination d'un paramètre θ_* tel que $\mathbb{P}^X = \mathbb{P}_{\theta_*}$.

Définition (Modèle Paramétrique) . On appellera *modèle paramétrique* l'espace probabilisé $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ tel que $\mathcal{P}_\Theta = \{\mathbb{P}_\theta \mid \theta \in \Theta\}$ où Θ est un ouvert convexe de $\mathbb{R}^d$.

Or cet espace ne nous permet de considérer que de simples variables aléatoires. Dans un cadre plus général, nous souhaiterions étudier des *échantillons* $(X_1, \dots, X_n)$ i.i.d d'une variable aléatoire X . Nous définissons pour cela le *modèle d'échantillonnage*.

Définition (Modèle d'échantillonnage) . Soit $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ un modèle paramétrique. Un *échantillon* $(X_1(\omega), \dots, X_n(\omega))$ de ce modèle est un n -uplet de réalisations indépendantes d'une variable aléatoire X de loi $\mathbb{P}_\theta \in \mathcal{P}_\Theta$. On note $\mathbb{P}^{\{X_1, \dots, X_n\}} = \mathbb{P}_{\theta_*}^{\otimes n}$ la loi de cet échantillon. Le modèle paramétrique de cet échantillon est donc :

$$(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^{\otimes n} = (\mathcal{X}^n, \mathcal{A}^n, \mathcal{P}_\Theta^{\otimes n})$$

On a donc :

$$(X_1, \dots, X_n) : \begin{cases} (\Omega, \mathcal{T}, \mathbb{P}) & \longrightarrow (\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^{\otimes n} \\ \omega & \longmapsto (X_1(\omega), \dots, X_n(\omega)) \end{cases}$$

Enfin, pour l'étude plus théorique de tels objets, il serait intéressant de pouvoir faire tendre vers l'infini la taille de ces échantillons $(X_1, \dots, X_n)$. On définit donc un modèle dit *asymptotique* :

$$(X_i)_{i \in \mathbb{N}^*} : \begin{cases} (\Omega, \mathcal{T}, \mathbb{P}) & \longrightarrow (\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^\infty \\ \omega & \longmapsto \{X_i(\omega) \mid i \in \mathbb{N}_*\} \end{cases}$$

1.1.2 Les variables aléatoires d'un modèle

L'objet principal que nous allons manipuler permet d'effectuer des opérations sur des échantillons d'une variable aléatoire. On les appelle les *statistiques*, notées S .

Définition (Statistique) . Une fonction mesurable S est une *statistique* si elle ne dépend pas de θ . On a donc :

$$S(X_1, \dots, X_n) : \begin{cases} (\mathcal{X}, \mathcal{A})^n & \longrightarrow (\mathcal{Y}, \mathcal{B}) \\ (X_1, \dots, X_n) & \longmapsto S(X_1, \dots, X_n) \end{cases}$$

On notera aussi $\mathbb{P}_\theta^S = (\mathbb{P}_\theta^{\otimes n})^S$ la mesure image de $\mathbb{P}_\theta^{\otimes n}$ par S .

Cette translation de nos observations $(X_1, \dots, X_n)$ implique la définition d'un *modèle image* nous permettant de manipuler ce nouvel objet $S(X_1, \dots, X_n)$.

Définition (Modèle Image) . Le modèle image de $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)^{\otimes n}$ par S_n est donc $(\mathcal{Y}, \mathcal{B}, \mathcal{P}_\Theta^{S_n})$, avec :

$$\mathcal{P}_\Theta^{S_n} := \{\mathbb{P}_\theta \circ S_n^{-1} \mid \theta \in \Theta\} \quad (1.1.2)$$

Pour rappel, l'objectif des statistiques est de déterminer le paramètre $\theta_* \in \Theta$. Pour cela, il faut que la donnée utilisée $((X_1, \dots, X_n), X, \text{ ou } S(X_1, \dots, X_n))$ contienne le plus d'information possible sur θ . Il serait donc préférable que la translation S ne réduise pas cette quantité. De telles statistiques seront donc dites *exhaustives*. Elles seront caractérisées par la loi conditionnelle de l'échantillon.

Définition (Loi conditionnelle d'une statistique) . Soit T une statistique. La loi conditionnelle de $(X_1, \dots, X_n)$ sachant $T(X_1, \dots, X_n)$ est définie par :

$$\mathbb{P}_\theta^{(X_1, \dots, X_n) | T}(A) = \mathbb{P}((X_1, \dots, X_n) \in A | T) = \mathbb{E}_\theta(\mathbb{1}_{(X_1, \dots, X_n) \in A} | T) \quad (1.1.3)$$

pour tout $A \in \mathcal{A}^n$.

1.1.3 Domination et Densités

Définition (Domination) . On dira qu'une mesure μ positive *domine* $\mathbb{P}_\theta$ sur $(\mathcal{X}, \mathcal{A})$ si :

$$\boxed{\forall A \in \mathcal{A}, \quad \mu(A) = 0 \implies \mathbb{P}_\theta(A) = 0} \quad (1.1.4)$$

On notera alors $\mathbb{P}_\theta \ll \mu$. Si c'est le cas pour tout $\theta \in \Theta$, on dira alors que *le modèle est dominé par μ* .

Plus formellement, $\mathbb{P}_\theta \ll \mu$ ssi

$$\forall \varepsilon > 0, \exists \delta > 0 \quad \text{tel que} \quad \forall A \in \mathcal{A}, \mu(A) < \delta \implies \mathbb{P}_\theta(A) < \varepsilon \quad (1.1.5)$$

Théorème (Radon-Nikodym) . Soit μ σ -finie. On a $\mathbb{P}_\theta \ll \mu$ ssi

$$\forall \theta \in \Theta, \exists f \in L_+^1(\mu) \quad \text{telle que} \quad \forall A \in \mathcal{A}, \quad \mathbb{P}_\theta(A) = \int_A f \, d\mu \quad (1.1.6)$$

La densité f de $\mathbb{P}_\theta$ est unique à égalité μ -presque partout. Elle est notée $f = d\mathbb{P}_\theta/d\mu$ comme dérivée de Radon-Nikodym.

Remarque (Cas discret) Le cas des lois discrètes peut paraître contre-intuitif, mais s'explique bien grâce à ce théorème. Soit $\mathcal{X}$ fini ou dénombrable et $\mathcal{A} = \mathcal{P}(\mathcal{X})$. Soit m_c la mesure de comptage sur $\mathcal{A}$. Par définition de l'intégrale de Lebesgue, pour toute fonction $f : \mathcal{X} \rightarrow \mathbb{R}$ et $A \in \mathcal{A}$, on a :

$$\int_A f \, d m_c = \sum_{x \in A} f(x) m_c(x) = \sum_{x \in A} f(x)$$

Soit $\mathcal{P}_\Theta$ une famille de lois sur $\mathcal{X}$ et $n = 1$. On a $\mathcal{P}_\theta \ll m_c$ et les densités $f = d\mathbb{P}_\theta/dm_c$ existent et vérifient :

$$\forall A = \{x\} \in \mathcal{A}, \quad \mathbb{P}_\theta(\{x\}) = \int_{\{x\}} f \, d m_c = f(x)$$

On a donc bien $f(x) = \mathbb{P}_\theta(X = x)$.

1.1.4 Hypothèses de régularité

Pour tous les raisonnements suivants sur notre modèle, nous devons définir des *hypothèses*. Celles-ci nous assureront certaines propriétés nécessaires à l'application des théorèmes suivants.

Définition (Modèle Identifiable) . S'il existe μ σ -finie telle que $\mathcal{P}_\Theta \ll \mu$ et $\mu(\{f_{\theta_0} \neq f_{\theta_1}\}) > 0$ pour tout $\theta_0 \neq \theta_1$, alors le modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ est dit *identifiable*.

Un modèle est identifiable si θ caractérise de façon unique une loi. Dans le cas contraire, un même valeur de θ pourrait correspondre à plusieurs lois initiales. L'estimation du paramètre n'aurait donc plus aucun sens.

Définition (Modèle Homogène) . Le modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ est dit *homogène* si $\mathcal{P}_\Theta \ll \mathbb{P}_\theta$ pour tout $\theta \in \Theta$.

L'homogénéité est une propriété fondamentale d'un modèle et elle intervient sur plusieurs aspects. Un modèle homogène garantit que tous les $\mathbb{P}_\theta$ ont les mêmes ensembles négligeables. D'autre part, l'homogénéité garantit que le support des variables aléatoires ne dépend pas de θ . Sans cela, les inversion de dérivées/intégrales (voir ci-dessous) ne seraient pas possibles.

Proposition Le modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ est homogène ssi, il existe μ σ -finie telle que :

- i) $\mathcal{P}_\Theta \ll \mu$
- ii) $\{f_\theta > 0\}$ μ -presque partout pour tout $\theta \in \Theta$.

En conséquence, pour tout $\theta \in \Theta$, $\mathcal{P}_\Theta \ll \mathbb{P}_\theta$. Donc les $\mathbb{P}_\theta$ ont tous les mêmes négligeables et la densité de $\mathbb{P}_{\theta_2}$ par rapport à $\mathbb{P}_{\theta_1}$ s'écrit :

$$\frac{d\mathbb{P}_{\theta_2}}{d\mathbb{P}_{\theta_1}} = \frac{f_{\theta_2}}{f_{\theta_1}} = \frac{d\mathbb{P}_{\theta_2}/d\mu}{d\mathbb{P}_{\theta_1}/d\mu}$$

En effet, on a pour tout $A \in \mathcal{A}$:

$$\begin{aligned} \mathbb{P}_{\theta_2}(A) &= \int_A \frac{f_{\theta_2}}{f_{\theta_1}} d\mathbb{P}_{\theta_1} = \int_A \frac{f_{\theta_2}}{f_{\theta_1}} f_{\theta_1} d\mu \\ &= \int_A f_{\theta_2} d\mu = \mathbb{P}_{\theta_2}(A). \end{aligned}$$

Définition (Hypothèses de régularité) . Définissons donc ces hypothèses :

(H_1) : Le modèle est **identifiable** et **dominé**.

$$i.e \quad \mathcal{P}_\Theta \ll \mu, \quad \text{et} \quad \forall \theta \in \Theta, f_\theta = \frac{d\mathbb{P}_\theta}{d\mu}, \quad \text{et} \quad \forall \theta_1 \neq \theta_2, \mu(\{f_{\theta_1} = f_{\theta_2}\}) = 0$$

(H_2) : Le modèle est **homogène**.

$$i.e \quad \forall \theta \in \Theta, \nabla f_\theta \in L^2(\mu)$$

(H_3) : Pour tout $A \in \mathcal{A}$, la fonction $\theta \in \Theta \mapsto \mathbb{P}_\theta(A)$, admet des dérivées partielles telles que :

$$\frac{\partial}{\partial \theta_k} \mathbb{P}_\theta(A) = \frac{\partial}{\partial \theta_k} \int_A f_\theta d\mu = \int_A \left(\frac{\partial}{\partial \theta_k} f_\theta \right) d\mu$$

(H_4) : Pour tout $A \in \mathcal{A}$, la fonction $\theta \in \Theta \mapsto \mathbb{P}_\theta(A)$, admet des dérivées partielles d'ordre 2 telles que :

$$\frac{\partial^2}{\partial \theta_j \partial \theta_k} \mathbb{P}_\theta(A) = \frac{\partial^2}{\partial \theta_j \partial \theta_k} \int_A f_\theta d\mu = \int_A \left(\frac{\partial^2}{\partial \theta_j \partial \theta_k} f_\theta \right) d\mu$$

1.1.5 Vraisemblance

Soit un échantillon $X = (X_1, \dots, X_n)$ d'un modèle vérifiant $H_{0,1}$.

Définition (Fonction de vraisemblance) . On appelle *fonction de vraisemblance* l'application L définie par :

$$L(\theta; x) = \prod_{i=1}^n f_{\theta}(x_i)$$

Pour un x observé, L est vue comme une fonction du paramètre θ . Il est crucial d'inclure les indicatrices de support $\mathbb{1}_{A(\theta)}(x_i)$ dans l'expression de f_{θ} .

Remarque (Double interprétation de la vraisemblance) La fonction $L(\theta; x)$ est un pivot entre deux mondes :

- *A priori* (Modélisation) : Pour θ fixé, $x \mapsto L(\theta; x)$ est la *densité* de l'échantillon. Elle décrit la distribution des observations possibles.
- *A posteriori* (Inférence) : Pour x observé et fixé, $\theta \mapsto L(\theta; x)$ est la *vraisemblance*. Elle mesure la *plausibilité* de chaque paramètre au vu des données.

Attention : La vraisemblance n'est pas une mesure de probabilité sur Θ . Elle permet de comparer deux paramètres entre eux via le rapport $\frac{L(\theta_1; x)}{L(\theta_0; x)}$, mais sa valeur absolue n'a pas de sens probabiliste intrinsèque.

Remarque (Log-vraisemblance) En pratique, on utilise la log-vraisemblance notée $\ell_n(\theta)$:

$$\ell_n(\theta) = \log L(\theta; x) = \sum_{i=1}^n \log f_{\theta}(x_i).$$

Elle facilite la recherche du Maximum de Vraisemblance (EMV) et le calcul du Score.

Théorème (Critère de factorisation de Fisher-Neyman) . T est une statistique exhaustive ssi il existe deux fonctions mesurables positives h et g_{θ} telles que :

$$L(\theta; x) = h(x) \cdot g_{\theta}(T(x))$$

Où h est indépendante de θ et g_{θ} ne dépend de x qu'à travers $T(x)$.

1.1.6 Maximum de Vraisemblance

Voyons à présent la vraisemblance comme fonction de θ . Elle traduit alors la plausibilité que θ soit le paramètre inconnu vis-à-vis de l'échantillon observé. On souhaiterait donc en déterminer le maximum pour trouver le θ "le plus probable". Pour cela, on utilise la méthode du *maximum de vraisemblance*.

Proposition Soit $X = (X_1, \dots, X_n)$ un n -échantillon d'une variable aléatoire de loi $\mathbb{P}_{\theta_*}$ et de densité f_{θ} . On souhaite déterminer le paramètre θ , pour cela, on considère la vraisemblance de l'échantillon définie par :

$$\forall \theta \in \Theta, \quad L(\theta, X_1, \dots, X_n) = \prod_{i=1}^n f_{\theta}(X_i)$$

dont on cherche le maximum. Or le produit n'est pas pratique à utiliser, notamment lors du calcul de la dérivée. On utilise donc la fonction log. Par sa croissance, la log-vraisemblance atteindra le même maximum de la vraisemblance. On calcule donc :

$$\ell_n(\theta) = \log \left(\prod_{i=1}^n f_{\theta}(X_i) \right) = \sum_{i=1}^n \log f_{\theta}(X_i).$$

Après quelques vérifications, on calculera sa dérivée à annuler pour trouver une valeur de θ candidate, noté $\hat{\theta}_n$, que l'on conservera après avoir confirmé la concavité de ℓ_n en étudiant le signe de sa dérivée seconde. Le $\hat{\theta}_n$ sera appelé un *estimateur* et sera une fonction de $(X_1, \dots, X_n)$.

Définition (Estimateur) . Dans un modèle $(\mathcal{X}, \mathcal{A}, \mathbb{P}_\theta)$ dominé par μ , un *estimateur du maximum de vraisemblance* est une statistique $\hat{\theta}$ telle que :

$$L(\hat{\theta}, X_1, \dots, X_n) = \max_{\theta \in \Theta} L(\theta, X_1, \dots, X_n) \quad (1.1.7)$$

pour n'importe quel échantillon $(X_1, \dots, X_n)$ de loi $\mathbb{P}_{\theta_*}$.

1.2 Informations Qualitatives

1.2.1 Exhaustivité

Une statistique T est exhaustive si elle "conserve" toute l'information utile sur θ . Mathématiquement, cela signifie qu'une fois T connue, la loi de l'échantillon ne dépend plus de θ .

Définition (Statistique Exhaustive) . T est exhaustive pour θ si la loi conditionnelle de $(X_1, \dots, X_n)$ sachant T est indépendante de θ .

$$\forall \theta \in \Theta, \quad \mathcal{L}_\theta(X_1, \dots, X_n \mid T) = \mathcal{L}(X_1, \dots, X_n \mid T)$$

Remarque En pratique, on utilise presque exclusivement le théorème de factorisation de Fisher-Neyman pour prouver l'exhaustivité.

Proposition T est une *statistique exhaustive* ssi pour toute statistique S réelle intégrable, on a :

$$\mathbb{E}_\theta(S \mid \sigma(T)) = \varphi(T) \quad (1.2.1)$$

(i.e si cela ne dépend pas de θ .)

Exemple (Statistique exhaustive) Soit $(X_1, \dots, X_n)$ un échantillon d'une variable aléatoire $X \sim \mathcal{B}(1, \theta)$. Posons $S_n = \sum_{i=1}^n X_i \sim \mathcal{B}(n, \theta)$. On a ici, $\mathcal{X} = \{0, 1\}^n$. On cherche à estimer θ . Montrons que S est une statistique exhaustive. Soit $k \in \llbracket 1, n \rrbracket$, on :

$$\begin{aligned} & \mathbb{P}_\theta((X_1, \dots, X_n) = (x_1, \dots, x_n) \mid S_n = k) \\ &= \frac{\mathbb{P}_\theta((X_1, \dots, X_n) = (x_1, \dots, x_n) \cap S_n = k)}{\mathbb{P}_\theta(S_n = k)} \\ &= \mathbb{1}_{\{\sum_{i=1}^n x_i = k\}} \times \frac{\mathbb{P}_\theta((X_1, \dots, X_n) = (x_1, \dots, x_n))}{\mathbb{P}_\theta(S_n = k)} \\ &= \mathbb{1}_{\{\sum_{i=1}^n x_i = k\}} \times \prod_{i=1}^n \frac{\mathbb{P}_\theta(X_i = x_i)}{\binom{n}{k} \theta^k (1-\theta)^{n-k}} \\ &= \frac{1}{\binom{n}{k}} \end{aligned}$$

qui est indépendant de θ donc S_n est bien une statistique exhaustive en θ .

1.2.2 Minimalité et complétude

On souhaite définir des statistiques dites minimales, contenant uniquement l'information nécessaire sur θ à partir de l'échantillon. Pour cela, on pose :

$$\mathcal{A}^* := \bigcap_{T \text{ exhaustive}} \sigma(T)$$

appelée la *tribu exhaustive minimale*. Elle contient donc toute l'information apportée par $(X_1, \dots, X_n)$ sur θ .

Définition (Statistique exhaustive minimale) . On dira d'une statistique exhaustive S^* qu'elle est *minimale* si $\sigma(S^*) = \mathcal{A}^*$.

Proposition (Caractérisation de $\mathcal{A}$) Sous H_1 , pour $\theta_0 \in \Theta$ fixé, on a :

$$\mathcal{A} = \sigma \left\{ \frac{L_\theta}{L_{\theta_0}}(X_1, \dots, X_n) \mid \theta \in \Theta \right\}$$

Proposition (Statistiques exhaustives minimales) Soit S une statistique exhaustive telle que :

$$S(x_1, \dots, x_n) = S(y_1, \dots, y_n) \iff \frac{L_\theta(x_1, \dots, x_n)}{L_\theta(y_1, \dots, y_n)} \text{ indépendants de } \theta \quad (1.2.2)$$

alors S est *minimale*.

Exemple (Loi exponentielle) Soit la densité de la loi exponentielle $f_\theta(x) = \theta e^{-\theta x}$. On cherche à déterminer S^* . Posons $S_n = \sum_{i=1}^n X_i$. On a :

$$L_\theta(X_1, \dots, X_n) = \prod_{i=1}^n f_\theta(X_i) = \theta^n e^{-\theta S_n}$$

Pour tout $\theta_0 \neq \theta_1$, on a :

$$\frac{L_{\theta_1}}{L_{\theta_2}}(X_1, \dots, X_n) = \exp((\theta_0 - \theta_1)S_n + n(\log \theta_1 - \log \theta_0))$$

Donc S est fonction mesurable du rapport $L_{\theta_1}/L_{\theta_2}$, elle est donc minimale.

Définition (Statistique Complète (ou Totale)) . Une statistique T est dite *complète* si pour toute fonction mesurable h telle que $\mathbb{E}_\theta[h(T)]$ existe :

$$\forall \theta \in \Theta, \quad \mathbb{E}_\theta[h(T)] = 0 \implies h(T) = 0 \quad \mathbb{P}_\theta\text{-p.s.}$$

Théorème (Lien entre Complétude et Minimalité) . Si une statistique T est *exhaustive* et *complète*, alors elle est *exhaustive minimale*.

Remarque Dans le cas des *modèles exponentiels* (voir plus loin), la statistique naturelle $T(X)$ est toujours exhaustive et complète (si l'espace des paramètres Θ est d'intérieur non vide). C'est le moyen le plus rapide en exercice pour obtenir une minimale.

1.2.3 Statistiques Libres

Définition (Statistique Libre) . Une statistique S est dite *libre* si sa loi ne dépend pas de θ .

$$\forall \theta, \theta' \in \Theta, \quad \mathbb{P}_\theta^S = \mathbb{P}_{\theta'}^S$$

Cette notion est fondamentale pour isoler le "bruit" du "signal" (θ). Le lien avec l'exhaustivité est donné par le théorème suivant, très utile pour montrer l'indépendance de deux variables ou pour prouver qu'une statistique n'est pas exhaustive.

Théorème (Théorème de Basu) . Si T est une statistique *exhaustive complète* et S est une statistique *libre*, alors T et S sont *indépendantes* sous toute loi $\mathbb{P}_\theta$.

Exemple Dans un modèle gaussien $\mathcal{N}(\mu, \sigma^2)$ avec σ^2 connu, la moyenne empirique $\bar{X}_n$ est exhaustive complète. La variance empirique S_n^2 (ou toute valeur centrée $X_i - \bar{X}_n$) est libre. Par le théorème de Basu, $\bar{X}_n$ et S_n^2 sont indépendantes.

1.3 Information de Kullback-Leibler

Définition (Information de Kullback-Leibler) . L'information de *Kullback-Leibler* (ou entropie relative) entre deux lois du modèle est définie par :

$$K(\theta_0 | \theta_1) = \mathbb{E}_{\theta_0} \left[\log \frac{f_{\theta_0}(X)}{f_{\theta_1}(X)} \right] = \int_{\mathcal{X}} f_{\theta_0} \log \left(\frac{f_{\theta_0}}{f_{\theta_1}} \right) d\mu \quad (1.3.1)$$

On parle de *divergence* ou de *contraste* plutôt que de distance, car cette application n'est pas symétrique ($K(\theta_0|\theta_1) \neq K(\theta_1|\theta_0)$) et ne respecte pas l'inégalité triangulaire.

Proposition (Propriétés fondamentales) Soit $\mathcal{P}_\Theta$ un modèle dominé.

- *Indépendance* : $K(\theta_0 | \theta_1)$ ne dépend pas du choix de la mesure dominante μ .
- *Positivité (Inégalité de Gibbs)* : $K(\theta_0 | \theta_1) \geq 0$ (découle de l'inégalité de Jensen).
- *Séparation* : $K(\theta_0 | \theta_1) = 0 \iff \mathbb{P}_{\theta_0} = \mathbb{P}_{\theta_1}$.
- *Lien Identifiabilité* : Si le modèle est identifiable, $K(\theta_0 | \theta_1) = 0 \iff \theta_0 = \theta_1$.

1.4 Informations Quantitatives

1.4.1 Vecteur Score

Durant toute cette section, nous serons amenés à calculer le gradient de log-vraisemblance. Pour simplifier les calculs et étudier leurs propriétés, nous allons les définir sous la forme de *vecteurs scores*.

Définition (Vecteur Score) . Soit $(H_{1,2})$, le vecteur aléatoire score d'un échantillon X est défini par le gradient de la log-vraisemblance de l'échantillon. On a donc :

$$S_\theta(X) = \nabla_\theta \log f_\theta(X) = \frac{1}{f_\theta(X)} \nabla f_\theta(X). \quad (1.4.1)$$

C'est une *variable aléatoire*.

Propriété (Espérance) . Sous $(H_{1,2,3})$, les vecteurs scores donc centrés.

$$i.e \quad (H_{1,2,3}) \implies \forall \theta \in \Theta, \mathbb{E}_\theta(S_\theta(X)) = 0_p$$

Démonstration Supposons $(H_{1,2,3})$, on a alors que $\int_{\mathcal{X}} f_\theta d\mu = 1$ car c'est une densité. Donc par $(H_{1,2,3})$, on a :

$$\begin{aligned} & \int_{\mathcal{X}} \frac{\partial}{\partial \theta_k} f_\theta d\mu = 0 \\ \iff & \int_{\mathcal{X}} \frac{1}{f_\theta(x)} \frac{\partial}{\partial \theta_k} f_\theta(x) \times \underbrace{f_\theta(x) d\mu(x)}_{d\mathbb{P}_\theta(x)} = 0 \\ \iff & \int_{\mathcal{X}} \frac{\partial}{\partial \theta_k} \log f_\theta d\mathbb{P}_\theta = 0 \\ \iff & \mathbb{E}_\theta \left(\frac{\partial}{\partial \theta_k} \log f_\theta \right) = 0 \end{aligned}$$

1.4.2 Modèles exponentiels

Voyons à présent un modèle régulier très utilisé : le *modèle exponentiel*.

Définition (Modèle Exponentiel) . Un modèle $(\mathcal{X}, \mathcal{A}, \mathcal{P}_\Theta)$ sera dit *exponentiel* si :

$$\boxed{\forall \theta \in \Theta, \quad f_\theta(x) = k_\theta h(x) \exp(\langle T(x), \gamma(\theta) \rangle) .}$$

Avec $\gamma(\Theta) \subset \mathbb{R}^q$, $T : \mathcal{X} \longrightarrow \mathbb{R}^q$ et $h : \mathcal{X} \longrightarrow \mathbb{R}$ la fonction de changement de mesure. Il sera qualifié de *plein rang* si $p = q$. On dira aussi qu'il est *canonique* si $p = q$ et γ est l'identité. On appellera enfin $T(X)$ la *statistique privilégiée du modèle*. Par soucis de clarté, on notera souvent $\kappa_\theta = \log k_\theta$.

Théorème (Régularité) . Un modèle exponentiel canonique de plein rang vérifie $(H_{1,2,3,4})$ et $k_\theta \in \mathcal{C}^\infty$.

Proposition Les modèles canoniques sont très pratiques à manipuler. On a :

$$f_\theta(x) = k_\theta h(x) \exp(\langle T(x), \theta \rangle)$$

Après calcul, on obtient que $S_\theta(X) = \nabla_\theta \log k_\theta + T(X) \in \mathbb{R}^p$. Or, on sait que $\mathbb{E}_\theta(S_\theta(X)) = 0$, donc :

$$\mathbb{E}_\theta(T(X)) = -\nabla_\theta \kappa_\theta.$$

Propriété (Cas général) . Dans un cadre plus général, un modèle exponentiel de plein rang admet des scores de la forme :

$$S_\theta(X) = J_\gamma^t \times T(X) + \nabla_\theta \kappa_\theta$$

avec $J_\gamma = \left(\left(\frac{\partial \gamma_i}{\partial \theta_j}(\theta) \right)_{i,j} \right)$ ma matrice Jacobienne $p \times p$ de $\gamma : \Theta \longrightarrow \mathbb{R}^p$. Si J_γ est définie positive, alors :

$$\mathbb{E}_\theta(T(X)) = - (J_\gamma^{-1})^t \times \nabla_\theta \kappa_\theta$$

Propriété (Statistique privilégiée) . Dans le cas d'un modèle exponentiel, la statistique T est *exhaustive, complète et minimale*.

1.4.3 Information de Fisher

Passons à la définition principale de cette partie : l'information de Fisher. Elle quantifie l'information contenue sur θ dans l'acte d'observer X .

Définition (Information de Fisher) . Sous $(H_{1,2})$, la *quantité d'information de Fisher* en θ apportée par l'acte d'observer une réalisation $X(\omega)$ sous $\mathbb{P}_\theta$ est :

$$\mathcal{I}_\theta(X) = \mathbb{E}_\theta(S_\theta(X) \times S_\theta^t(X)) \quad (1.4.2)$$

Si (H_4) est vérifiée, alors $\mathcal{I}_\theta(X) = \mathbb{V}_\theta(S_\theta(X))$, la matrice de covariance du vecteur score.

Propriété (Information de Fisher) .

- Sous $(H_{1,2})$, $\mathcal{I}_\theta(X)$ est intrinsèque au modèle (i.e elle ne dépend plus de la mesure dominante μ).
- Sous $(H_{1,2,3,4})$, on a :

$$\mathcal{I}_\theta(X) = -\mathbb{E}_\theta(J_{S_\theta(X)}) = -\mathbb{E}_\theta \left(\left(\frac{\partial^2 \log f_\theta(X)}{\partial \theta_i \partial \theta_j} \right)_{i,j} \right)$$

avec $J_{S_\theta(X)}$ la Jacobienne du score.

Ces propriétés seront donc privilégiées pour le calcul pratique de l'information de Fisher.

Propriété (Cas du modèle exponentiel canonique) . Dans le cas d'un modèle exponentiel canonique, on a :

$$\mathcal{I}_\theta(X) = \mathbb{V}_\theta(T(X)) = -H_{\kappa_\theta}$$

où H_{κ_θ} est la hessienne de $\kappa_\theta = \log k_\theta$. Donc si $p = 1$, alors $\mathcal{I}_\theta(X) = -\frac{\partial^2}{\partial \theta^2} \log k_\theta$.

Proposition Comme dit précédemment, les modèles exponentiels de plein rang sont très utiles. Ils permettent de calculer simplement l'information de Fisher à partir des fonctions définissant la densité. On a donc :

$$\begin{aligned} \mathcal{I}_\theta(X) & \stackrel{\text{def}}{=} J_\gamma^t \times \mathbb{V}_\theta(T(X)) \times J_\gamma(\theta) \\ & \stackrel{\text{prop}}{=} -H_{\kappa_\theta} - H_\gamma (J_\gamma^{-1})^t \nabla_\theta \kappa_\theta \end{aligned}$$

Si $p = 1$,

$$\mathcal{I}_\theta(X) = (\gamma'(\theta))^2 \times V_\theta(T(X)) = \frac{-k'_\theta}{k_\theta} \left(\frac{k''_\theta}{k'_\theta} + \frac{\gamma''_\theta}{\gamma'_\theta} - \frac{k'_\theta}{k_\theta} \right)$$

Exemple Soit le modèle $X \sim \mathcal{N}(\theta_1, \theta_2^2)$, avec $\begin{cases} \theta_1 = \mathbb{E}_\theta(X) \\ \theta_2 = \sqrt{\mathbb{V}_\theta(X)} \end{cases}$. On a donc :

$$f_\theta(x) = \frac{1}{\sqrt{2\pi}} \exp(\langle T(x), \gamma(\theta) \rangle + \kappa_\theta)$$

où $p = q = 2$ et :

$$T(x) = (x, x^2), \quad \kappa_\theta = -\frac{\theta_1^2}{2\theta_2^2} - \log \theta_2, \quad \gamma(\theta) = \left(\frac{\theta_1}{\theta_2}, -\frac{1}{2\theta_2^2} \right)$$

On peut donc ensuite appliquer les formules :

$$J_\gamma = \begin{pmatrix} \frac{1}{\theta_2^2} & -\frac{2\theta_1}{\theta_2^2} \\ 0 & \frac{1}{\theta_2^2} \end{pmatrix} \quad \mathbb{V}_\theta(T(X)) = \begin{pmatrix} \theta_2^2 & \text{Cov}(X, X^2) \\ \text{Cov}(X, X^2) & \mathbb{V}_\theta(X^2) \end{pmatrix}$$

On a donc :

$$\begin{aligned} \mathcal{I}_\theta(X) &= Y_\gamma^t \times V_\theta(T(X)) \times J_\gamma \\ &= -\mathbb{E}_\theta \left(\frac{\partial^2}{\partial \theta_i \partial \theta_j} \left(-\frac{(X - \theta_1)^2}{2\theta_2^2} - \log \theta_2 \right) \right)_{i,j} = \begin{pmatrix} \frac{1}{\theta_2^2} & 0 \\ 0 & \frac{2}{\theta_2^2} \end{pmatrix} \end{aligned}$$

1.4.4 Propriétés de l'information de Fisher

L'information de Fisher possède des propriétés de structure fondamentales qui découlent de la linéarité du score et de la géométrie du modèle.

1. **Additivité** : Soient S et T deux statistiques $\mathcal{P}_\Theta$ -indépendantes. Si les modèles associés vérifient $(H_{1,2,3})$, l'information jointe est la somme des informations :

$$\mathcal{I}_\theta(S, T) = \mathcal{I}_\theta(S) + \mathcal{I}_\theta(T)$$

Conséquence : Dans le modèle d'échantillonnage i.i.d. $\mathcal{P}_\Theta^{\otimes n}$, on a :

$$\boxed{\mathcal{I}_\theta(X_1, \dots, X_n) = n\mathcal{I}_\theta(X_1)}$$

2. **Décroissance par transfert (Information image)** : Soit S une statistique. Si le modèle image vérifie $(H_{1,2,3})$, le score image est $S_\theta(S) = \mathbb{E}_\theta[S_\theta(X) \mid S]$. Par l'inégalité de Jensen (ou la formule de décomposition de la variance), on a :

$$\mathcal{I}_\theta(S) = \mathbb{V}_\theta(\mathbb{E}_\theta[S_\theta(X) \mid S]) \preceq \mathbb{V}_\theta(S_\theta(X)) = \mathcal{I}_\theta(X)$$

Ainsi, $\mathcal{I}_\theta(X) - \mathcal{I}_\theta(S)$ est une matrice semi-définie positive. On perd de l'information en condensant les données, sauf si la statistique est exhaustive.

3. **Conservation par exhaustivité** : Sous $(H_{1,2,3})$, une statistique S est exhaustive si et seulement s'il n'y a pas de perte d'information par transfert :

$$\boxed{S \text{ est exhaustive} \iff \mathcal{I}_\theta(S(X_1, \dots, X_n)) = n\mathcal{I}_\theta(X_1)}$$

4. **Lien avec le contraste de Kullback** : Sous $(H_{1,2,3,4})$, pour tout $\theta_0 \in \Theta$ fixé, l'information de Fisher est la matrice Hessienne du contraste de Kullback-Leibler (courbure locale de la divergence) :

$$\mathcal{I}_{\theta_0}(X) = \text{Hess}_\theta (K(\theta_0 \mid \theta)) \Big|_{\theta=\theta_0} = \left(\left(\frac{\partial^2 K(\theta_0 \mid \theta)}{\partial \theta_i \partial \theta_j} \right)_{i,j} \right)_{\theta=\theta_0}$$

Chapitre 2

Estimation et Risque

Contents

2.1	Risque	108
2.2	Estimateurs	109
2.2.1	Définition	109
2.2.2	Comparaisons	109
2.2.3	Comparaison par le risque	109
2.2.4	Optimalité	111
2.2.5	Biais	111
2.3	Risque Quadratique	112
2.3.1	Cas $q = 1$	112
2.3.2	Cas $q > 1$	113
2.3.3	Amélioration de Rao-Blackwell	113
2.3.4	Optimalité de Lehman-Scheffe	114
2.4	Bornes inférieures de variance et Efficacité	114
2.4.1	Inégalité de Fréchet-Darmois-Cramér-Rao (FDCR)	115
2.4.2	La réciproque de Koopman (Cas d'égalité)	115

En statistiques paramétriques, on cherche à déterminer la valeur d'un paramètre θ^* inconnu. Nous posons ici une autre démarche que celle des estimateurs vue précédemment.

2.1 Risque

Définition (Règle de décision) . On appelle *règle de décision* δ une statistique à valeurs dans un espace $(\mathbb{D}, \mathcal{D})$ telle que :

$$\delta : (\mathcal{X}, \mathcal{A})^{\oplus n} \longrightarrow (\mathbb{D}, \mathcal{D}).$$

Remarque Une règle de décision est donc une fonction mesurable et indépendante d'un paramètre θ puis c'est une statistique.

Proposition Il existe différents types de règles de décisions :

- **Estimation** : On choisit un paramètre θ par un estimateur $\delta(X_1, \dots, X_n)$ à partir d'informations contenues dans un échantillon. On a donc $\mathbb{D} = \Theta$ et on peut définir *l'erreur quadratique moyenne* :

$$R_Q(\delta, \theta) = \mathbb{E}_\theta(\|\delta - \theta\|_p^2)$$

sous $\mathbb{P}_\theta$. Ici, on aura $\mathbb{D} = \mathcal{P}(\Theta)$.

- **Localisation** : Ici, on choisit plutôt une région pouvant contenir θ . On veut donc une notion de *confiance* telle que :

$$\forall \theta \in \Theta, \quad \mathbb{P}_\theta(\theta \in \delta(X_1, \dots, X_n)) \geq 1 - \alpha.$$

- **Test** : On a ici $\mathbb{D} = \{0, 1\}$ ou $\mathbb{D} = [0, 1]$. On cherche à vérifier si une affirmation sur θ est vraie plutôt qu'une autre. Elles seront définies par deux types d'hypothèses : *l'hypothèse nulle* $\mathcal{H}_0$ et *l'hypothèse alternative* $\mathcal{H}_1$. On souhaite donc que :

- $\mathbb{P}(\delta(X_1, \dots, X_n) = 0)$ soit proche de 1 si $\theta \in \Theta_0$.
- $\mathbb{P}(\delta(X_1, \dots, X_n) = 1)$ soit proche de 1 si $\theta \notin \Theta_0$.

2.2 Estimateurs

2.2.1 Définition

On cherche ici à estimer une transformation $g(\theta)$ d'un paramètre θ connu. Pour cela, on dispose de :

$$g : \begin{cases} \Theta \subseteq \mathbb{R}^p \longrightarrow \mathbb{R}^q \\ \theta \longmapsto g(\theta) \end{cases}$$

Définition (Estimateur) . On appelle *estimateur* de $g(\theta)$ toute fonction mesurable :

$$\delta : (\mathcal{X}, \mathcal{A})^{\oplus n} \longrightarrow (g(\Theta), \mathcal{D})$$

qui ne dépend pas de θ . On note Δ l'ensemble des estimateurs de θ .

Remarque Si $\hat{\theta}_n$ est un "bon" estimateur de θ , alors $g(\hat{\theta}_n)$ n'est pas nécessairement un aussi bon estimateur pour $g(\theta)$. La composition doit s'étudier au cas par cas.

2.2.2 Comparaisons

On souhaite qu'un estimateur se comporte comme une boîte noire pointant toujours sur θ peu importe sa valeur.

Définition (Estimateur consistant) . Une suite d'estimateurs $\delta_n \in \Delta^{\mathbb{N}}$ sera dite *consistante* si :

$$\forall \theta \in \Theta, \quad \lim_{n \rightarrow \infty} \delta_n(X_1, \dots, X_n) = g(\theta) \quad \mathbb{P}_\theta\text{-p.s.}$$

Définition (Estimateur Asymptotiquement Gaussien) . Une suite d'estimateurs $\delta_n \in \Delta^{\mathbb{N}}$ sera dite *asymptotiquement gaussienne* si pour tout $\theta \in \Theta$, il existe une matrice V_θ semi-définie positive telle que sous P_θ^∞ , on ait :

$$\sqrt{n}(\delta_n(X_1, \dots, X_n) - g(\theta)) \xrightarrow[n \rightarrow \infty]{\mathcal{L}} \mathcal{N}(0_p, V_\theta).$$

2.2.3 Comparaison par le risque

Un autre façon de comparer deux estimateurs est d'introduire une notion de *perte* qui quantifie le degré d'erreur d'estimation de $g(\theta)$. Pour cela, on définit les fonctions de perte ou *loss* en anglais.

Définition (Fonction de perte) . Une fonction de perte ℓ est une fonction mesurable positive :

$$\ell : \Theta \times g(\Theta) \longrightarrow (\mathbb{R}_+, \mathcal{B}_{\mathbb{R}})$$

telle que :

$$\boxed{\forall (\theta, d) \in \Theta \times g(\Theta), \quad \ell(\theta, d) = 0 \iff d = g(\theta)}.$$

Comme dit précédemment, on souhaite que nos estimateurs se comportent comme des boîtes noires. Il nous faut cela minimiser la perte moyenne d'une décision pour toutes les valeurs de $g(\theta)$ possibles. Cette notion sera donc définie comme le *risque*.

Définition (Risque) . On définit la *fonction de risque* associée à une décision $\delta \in \Delta$, comme l'application :

$$\theta \longmapsto R(\theta, \delta) = \mathbb{E}_{\theta}(\ell(\theta, \delta)).$$

Ainsi, un estimateur δ sera dit *préférable* à un autre $\delta' \in \Delta$, ssi :

$$\forall \theta \in \Theta, \quad R(\theta, \delta') \leq R(\theta, \delta).$$

On notera alors $\delta \succeq \delta'$.

Remarque Se pose alors un problème fondamental dans la recherche de l'estimateur optimal. Admettons que vous ayez un ami complotiste qui croit les extraterrestres sont parmi nous. Dans tous les mondes qu'il pourrait exister, la plupart ne sont pas dirigés par une secte martienne. Or, il suffit qu'un seul monde le soit pour que vous ne soyez pas meilleur que votre ami pour une valeur spécifique de θ .

C'est exactement le problème que posent les estimateurs constants. On va donc chercher à les exclure en créant des sous-classes $\Delta_i \subset \Delta$.

Démonstration Supposons qu'il existe $\delta^* \in \Delta$ un estimateur parfait tel que :

$$\forall \theta \in \Theta, \quad \forall \delta \in \Delta, \quad R(\theta, \delta^*) \leq R(\theta, \delta).$$

alors $\delta = \delta_{\theta_0}$ qui répond θ_0 presque-partout quelque soit l'échantillon proposé (un estimateur constant). D'où :

$$R(\theta, \delta_{\theta_0}) \begin{cases} = 0 & \text{si } \theta = \theta_0 \\ > 0 & \text{si } \theta \neq \theta_0. \end{cases}$$

Donc nécessairement $\forall \theta \in \Theta, R(\theta, \delta^*) = 0$ pour faire mieux que chacun des estimateurs constants.

Définition (Estimateurs admissibles) . On définit l'ensemble des estimateurs *admissibles*, noté Δ_0 , comme l'ensemble des estimateurs "meilleurs en moyenne". Plus formellement,

$$\Delta_0 = \{\delta \in \Delta \mid \nexists \delta' \in \Delta, \delta' \succeq \delta\}.$$

Proposition Supposons que Δ_0 est stable par combinaison convexe et que ℓ est strictement convexe en la décision δ . Alors si δd_0 est admissible dans Δ_0 et si δ_1 admet la même fonction de risque, alors $\delta_1 = \delta_0$ $\mathbb{P}_{\theta}$ -p.p pour tout $\theta \in \Theta$.

Exemple Soit Δ_0 la classe des estimateurs sans biais moyen :

$$\Delta_0 := \{\delta \in \Delta \mid \mathbb{E}_{\theta}(\delta(X_1, \dots, X_n)) = g(\theta)\}.$$

Soit $\delta_0, \delta_1 \in \Delta_0$, alors :

$$\forall \alpha \in [0, 1], \quad \alpha \delta_0 + (1 - \alpha) \delta_1 \in \Delta_0$$

par linéarité de $\mathbb{E}_\theta(\cdot)$. Donc Δ_0 est bien stable et :

$$\delta \mapsto \ell(\theta, \delta) = \|g(\theta) - \delta\|_2^2$$

est convexe. Donc dans la classe des estimateurs sans biais, s'il existe un estimateur de plus petit risque, alors il est unique.

2.2.4 Optimalité

On cherche donc à déterminer des estimateurs optimaux (i.e minimisant la fonction de risque).

Définition (Estimateur Optimal) . On dira que δ_{opt} est ℓ -optimal dans Δ_0 si :

$$\forall \theta \in \Theta, \quad R(\theta, \delta_{opt}) = \min_{\delta \in \Delta_0} R(\theta, \delta).$$

Définition (Estimateur Asymptotiquement Optimal) . On dit qu'une suite $\delta_n(X_1, \dots, X_n)$ est asymptotiquement ℓ -optimale relativement à une suite Δ_n de classes d'estimateurs si :

$$\forall \theta \in \Theta, \quad \lim_{n \rightarrow \infty} \frac{R(\theta, \delta_n(X_1, \dots, X_n))}{\inf_{\delta \in \Delta_n} R(\theta, \delta)} = 1.$$

2.2.5 Biais

La notion de biais permet de quantifier la façon dont un estimateur δ approche $g(\theta)$ en moyenne. Pour cela, il existe différents types de biais.

Définition (Biais Moyen) . On dira qu'un estimateur $\delta \in \Delta$ est *sans biais moyen* pour $g(\theta)$ si :

$$\mathbb{E}_\theta(\delta(X_1, \dots, X_n)) = g(\theta).$$

On peut ainsi définir la *fonction de biais* b d'un estimateur :

$$b : \theta \in \Theta \mapsto \mathbb{E}_\theta(\delta) - g(\theta).$$

De même que pour l'optimalité, il est souhaitable que le biais d'un estimateur diminue lorsque la taille de l'échantillon augmente. On parle alors de *biais moyen asymptotique*.

Définition (Biais moyen asymptotique) . Un estimateur δ est dit *sans biais moyen asymptotique* si :

$$\forall \theta \in \Theta, \quad \lim_{n \rightarrow \infty} b(\delta_n) = 0.$$

Ce qui revient à $\mathbb{E}_\theta(\delta_n) \xrightarrow{n \rightarrow \infty} g(\theta)$.

Remarque Un estimateur centré n'est pas nécessairement sans biais moyen. La notion de centrage exprime seulement le fait que $\mathbb{E}_\theta(\delta(X_1, \dots, X_n)) = 0_q$ pour tout $\theta \in \Theta$. Or, si $g(\theta) \neq 0_q$ pour tout θ , alors l'estimateur est biaisé.

Définition (Biais Général) . Un estimateur δ est dit *sans biais général* pour une fonction de perte ℓ si :

$$\forall \theta, \theta' \in \Theta, \quad R(\theta, \delta) = \mathbb{E}_\theta(\ell(\theta, \delta)) \leq \mathbb{E}_\theta(\ell(\theta', \delta)).$$

On notera Δ_{sb} la classe des estimateurs sans biais.

Définition (Biais Médian) . Si $q = 1$, un estimateur $\delta \in \Delta$ est *sans biais médian* si :

$$\forall \theta \in \Theta, \quad \begin{cases} \mathbb{P}_\theta(\delta(X_1, \dots, X_n) > g(\theta)) \leq 1/2 \\ \mathbb{P}_\theta(\delta(X_1, \dots, X_n) < g(\theta)) \leq 1/2 \end{cases}.$$

La notion de biais médian signifie qu'un estimateur a autant de chances de surestimer $g(\theta)$ que de le sous-estimer. Elle traduit de façon plus fine la dispersion possible de l'estimateur autour de $g(\theta)$.

2.3 Risque Quadratique

La notion de biais vue précédemment traduit le fait qu'un estimateur soit en moyenne "précis" ou pas pour estimer $g(\theta)$, mais elle ne donne aucune notion sur la dispersion d'un estimateur autour du paramètre. Or, il serait parfois préférable de travailler avec un estimateur moins "précis" (biaisé) mais moins dispersé.

2.3.1 Cas $q = 1$

On cherche donc une mesure permettant d'additionner la notion de biais et de variance d'un estimateur. En effet, un estimateur sans biais peut être complètement divergent pour n grand (non consistant) et inversement.

Au lieu d'utiliser une métrique différente, on pourrait adapter celles déjà introduites telles que la *perte* et le *risque moyen*. Une bonne fonction de perte pourrait être la *perte quadratique* :

$$\ell_Q(\theta, \delta) = (\delta - g(\theta))^2$$

possédant de bonnes propriétés de convexité et de différentiabilité. De plus elle pénalise de façon égale les erreurs négatives ou positives tout en pénalisant fortement les grosses erreurs.

Définition (Risque Quadratique Moyen) . On définit le *risque quadratique moyen* R_Q d'un estimateur δ comme le risque de cet estimateur appliqué à la fonction de perte quadratique. D'où :

$$\forall \theta \in \Theta, \quad R_Q(\theta, \delta) = \mathbb{E}_\theta(\ell_Q(\theta, \delta)) = \mathbb{E}_\theta((\delta - g(\theta))^2).$$

Propriété (Décomposition) . Comme souhaité, le risque quadratique moyen vérifiera :

$$\forall \delta \in \Delta, \forall \theta \in \Theta, \quad \boxed{R_Q(\theta, \delta) = V_\theta(\delta) + b(\theta, \delta)^2}.$$

Démonstration Soient $\delta \in \Delta$ et $\theta \in \Theta$, on a :

$$\begin{aligned} R_Q(\theta, \delta) &= \mathbb{E}_\theta((\delta - g(\theta))^2) \\ &= \mathbb{E}_\theta((\delta - \mathbb{E}_\theta(\delta) + \mathbb{E}_\theta(\delta) - g(\theta))^2) \\ &= \mathbb{E}_\theta((\delta - \mathbb{E}_\theta(\delta))^2) + (\mathbb{E}_\theta(\delta) - g(\theta))^2 + \underbrace{2\mathbb{E}_\theta((\delta - \mathbb{E}_\theta(\delta))(\mathbb{E}_\theta(\delta) - g(\theta)))}_{\text{d'esp. nulle}} \\ &= V_\theta(\delta) + b(\theta, \delta). \end{aligned}$$

Proposition Ainsi, sur Δ_{sb} , on aura $\delta \succeq \delta'$ si $R_Q(\theta, \delta) \leq R_Q(\theta, \delta')$ pour tout $\theta \in \Theta$. Ce qui équivaut donc à $V_\theta(\delta) \leq V_\theta(\delta')$. Dire que δ_{opt} est ℓ_Q -optimal signifie que δ_{opt} est *sans biais de variance minimale* pour tout $\theta \in \Theta$.

2.3.2 Cas $q > 1$

De même que précédemment, on cherche à quantifier la pertience d'un estimateur en termes de dispersion et de précision par rapport au paramètre. On souhaite ici à estimer plusieurs paramètres $\theta = (\theta_1, \dots, \theta_n)$. On notera alors un estimateur $\delta(X_1, \dots, X_n) \in \mathbb{R}^q$ simplement δ de coordonnées $\delta^t = (\delta_1, \dots, \delta_q)$. De même, on aura :

$$\mathbb{E}_\theta(\delta) = (\mathbb{E}_\theta(\delta_1), \dots, \mathbb{E}_\theta(\delta_q))$$

$$\mathbb{V}_\theta(\delta) = \mathbb{E}_\theta [(\delta - \mathbb{E}_\theta(\delta))(\delta - \mathbb{E}_\theta(\delta))^t] = \left((\text{cov}_\theta(\delta_i, \delta_j))_{i,j} \right).$$

Définition (Perte, Risque, Comparaison - Cas Multivarié) . Pour toute matrice M semi-définie positive, on définit la perte ℓ d'un estimateur par rapport à un vecteur paramètre θ comme l'application telle que :

$$\ell_M(\theta, \delta) = (\delta - g(\theta))^t M (\delta - g(\theta)).$$

Aussi le *risque quadratique associé* à M sera :

$$\mathbb{R}_M(\theta, \delta) = \mathbb{E}_\theta(\ell_M(\theta, \delta)).$$

Si $M = Id$, on retrouve alors le risque quadratique moyen. On dira ainsi que $\delta \succeq_Q \delta'$ (i.e il est Q -préférable à δ') si :

$$R_M(\theta, \delta) \leq R_M(\theta, \delta') \quad \forall M.$$

Théorème (Caractérisation) . On dira $\delta \succeq_Q \delta'$ ssi la matrice de $\mathcal{M}_q(\mathbb{R})$:

$$\mathbb{E}_\theta((\delta' - g(\theta))(\delta' - g(\theta))^t) - \mathbb{E}_\theta((\delta - g(\theta))(\delta - g(\theta))^t)$$

est semi-définie positive.

2.3.3 Amélioration de Rao-Blackwell

Théorème (Rao-Blackwell) . Soit $\delta \in L^2(\mathcal{P}_\Theta)$ sans biais (moyen) pour $g(\theta)$ et S une statistique exhaustive en θ . Alors, l'estimateur δ dit *amélioré par S* , tel que $\delta_S = \mathbb{E}_\theta(\delta \mid S)$ est Q -préférable à δ .

Donc pour réduire une variance dans L^2 sans perte d'information, il suffit de conditionner une statistique exhaustive.

Démonstration δ_S est un estimateur, car S est exhaustive dont $\mathbb{E}_\theta(\delta \mid S)$ ne dépend pas de θ . D'autre part, δ_S est sans biais, puisque :

$$\mathbb{E}_\theta(\mathbb{E}_\theta(\delta \mid S)) = \mathbb{E}_\theta(\delta) = g(\theta).$$

Enfin, δ_S est de moindre variance, comme $\mathbb{E}_\theta(\delta \mid S)$ est la projection dans $L^2(\mathbb{P}_\theta)$.

Remarque Dans le cas $q = 1$, pour $\Psi(x) = x^2$ convexe, on a :

$$\begin{aligned} \mathbb{E}_\theta((\delta - g(\theta))^2 \mid S) &\geq [\mathbb{E}_\theta(\delta - g(\theta)) \mid S]^2 \\ &= [\delta_S - g(\theta)]^2 \end{aligned}$$

par l'inégalité de Jensen conditionnelle. Donc en prenant $\mathbb{E}_\theta(\cdot)$ à gauche et à droite, on a :

$$\begin{aligned}\mathbb{E}_\theta((\delta - g(\theta))^2) &\geq \mathbb{E}_\theta(\delta_S - g(\theta))^2 \\ R_Q(\theta, \delta) &\geq R_Q(\theta, \delta_S).\end{aligned}$$

Donc conditionner par une statistique exhaustive améliore toujours la fonction de risque.

2.3.4 Optimalité de Lehman-Scheffe

Théorème (Lehman-Scheffe) . Si δ est sans biais pour $g(\theta)$ et S exhaustive complète, alors $\delta_S = \mathbb{E}_\theta(\delta | S)$ est l'unique estimateur sans biais de variance minimale (i.e l'optimal δ_Q^{opt}).

Démonstration Montrons que cet estimateur ne peut plus être amélioré. Soit $\delta' \in \Delta_{sb}$, posons $\delta'_S = \mathbb{E}_\theta(\delta' | S)$. On a :

$$\mathbb{E}_\theta(\delta') = \mathbb{E}_\theta(\delta) = g(\theta)$$

Donc, pour tout $\theta \in \Theta$:

$$\mathbb{E}_\theta(\underbrace{\delta'_S - \delta_S}_{=\varphi(\delta)}) = \mathbb{E}_\theta(\delta - \delta') = 0.$$

Avec $\mathbb{E}_\theta(\varphi(S)) = 0$ pour tout θ . Or S est complète donc $\varphi(S) = 0$. D'où $\delta'_S = \delta_S$.

Proposition Pour trouver le δ_Q^{opt} , il faut donc :

- Trouver S complète : on cherche $\varphi(S)$ telle que $\mathbb{E}_\theta(\varphi(S)) = g(\theta)$ et la solution est bien l'estimateur sans biais de variance minimale.
- On cherche $\delta \in \Delta_{sb}$ simple.
- Puis, on conditionne $\mathbb{E}_\theta(\delta | S)$. Si S est complète, alors c'est gagné.

2.4 Bornes inférieures de variance et Efficacité

Nous avons précédemment défini Δ_{sb} comme la classe des estimateurs sans biais. Par la suite, nous avons introduit le risque quadratique pour quantifier la pertinence d'un estimateur en fonction de son biais et de sa variance.

Bien que nous sachions construire des estimateurs sans biais, l'enjeu est désormais d'étudier leur variance afin de réduire leur dispersion autour du paramètre cible. Une question fondamentale se pose alors : au sein de Δ_{sb} , existe-t-il un estimateur dont la variance est minimale de manière uniforme ? Si oui, comment peut-on le caractériser ?

Pour répondre à cette question, nous nous plaçons dans le cadre des modèles réguliers $(H_{1,2,3,4})$. L'utilisation de l'information de Fisher comme mesure de précision nous impose toutefois d'ajouter des conditions sur la finitude de l'information (H_5) et sur la régularité de l'estimateur δ (H_6) pour permettre l'intervention intégrale-dérivée :

(H_5) : L'information de Fisher est finie et non nulle :

$$\forall \theta \in \Theta, \quad \mathcal{I}_\theta(X) \in]0, +\infty[$$

(H_6) : δ est de carré intégrable et permet la dérivation sous le signe somme :

$$\delta \in L^2(\mathcal{P}_\Theta), \forall \theta \in \Theta, \quad \mathbb{E}_\theta(\delta(X)) = g(\theta) \quad \text{et} \quad \frac{\partial f_\theta}{\partial \theta} \delta \in L^1(\mu)$$

Tels que pour tout $A \in \mathcal{A}$ et $\theta \in \Theta$:

$$\frac{\partial}{\partial \theta} \left(\int_A \delta f_\theta d\mu \right) = \int_A \frac{\partial f_\theta}{\partial \theta} \delta d\mu$$

2.4.1 Inégalité de Fréchet-Darmois-Cramér-Rao (FDCR)

Théorème (Borne de Cramér-Rao) . Sous les hypothèses de régularité usuelles ($H_{1...5}$) et pour un estimateur δ_n sans biais de $g(\theta)$, on a :

$$\mathbb{V}_\theta(\delta_n) \geq \frac{(g'(\theta))^2}{n\mathcal{I}_\theta(X)} \quad (2.4.1)$$

Plus l'information de Fisher est grande, plus la variance minimale possible est petite (le modèle est "précis"). Il convient alors de définir proprement les estimateurs atteignant cette borne.

Définition (Estimateur Efficace) . Un estimateur δ_n est dit *efficace* s'il est sans biais et qu'il *atteint la borne* de Cramér-Rao :

$$\mathbb{V}_\theta(\delta_n) = \frac{(g'(\theta))^2}{n\mathcal{I}_\theta(X)}$$

2.4.2 La réciproque de Koopman (Cas d'égalité)

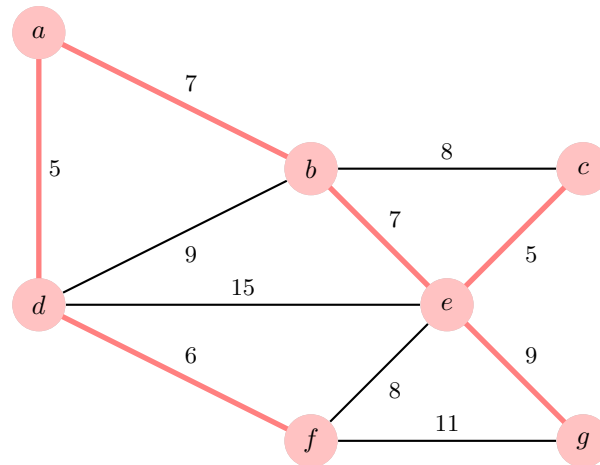
Propriété (Lien avec les modèles exponentiels) . La borne de Cramér-Rao est atteinte (cas d'égalité) ssi le modèle est exponentiel de la forme :

$$f_\theta(x) = h(x)k_\theta \exp(\gamma(\theta)\delta(x))$$

Dans ce cas, $\delta(X)$ est l'unique estimateur efficace pour son espérance $g(\theta) = -\frac{\kappa'_\theta}{\gamma'(\theta)}$.

Exemple (Application TD) Pour une loi de Poisson $\mathcal{P}(\theta)$, on montre que $\bar{X}_n$ est efficace car le modèle est exponentiel. Pour une loi Uniforme $\mathcal{U}[0, \theta]$, la borne ne s'applique pas (support dépendant de θ) : l'estimateur peut avoir une variance *plus petite* que la borne de Fisher.

Algorithmique



Chapitre 1

Flot maximal dans un graphe

Contents

1.1	Rappels sur les graphes	117
1.2	Cycle et flot	119
1.3	Flot et coupure	120
1.3.1	Flot maximal dans un réseau	120
1.3.2	Théorème de Coupure minimale	121
1.4	Flot maximal : principe de Ford-Fulkerson	122
1.4.1	Recherche de Flot Maximal	122
1.4.2	Recherche de flot maximal de coût minimal	123

L'objectif de ce chapitre est d'être capable d'organiser efficacement un flot d'objets dans un réseau. Par exemple, on souhaite amener le maximum de camions d'un point A à un point B en passant par certaines routes. De même, on souhaiterait savoir comment router efficacement le plus de paquets dans un réseau informatique.

Tout ces problèmes se généralisent en ce que l'on appelle la *recherche de flot maximal dans un graphe orienté*. Nous allons ici définir proprement ce genre de problème, donner des conditions d'existence de flot maximal et proposer des algorithmes pour les construire.

1.1 Rappels sur les graphes

Pour représenter une structure de réseau (routier, informatique, etc...), on utilise les graphes. Ce cours fait donc suite à celui sur la théorie des graphes enseigné en licence. Effectuons quelques rappels.

Définition (Graphe Orienté) . On appelle graphe orienté un couple d'ensembles finis $G = (X, E)$ où $X = \{1, \dots, n\}$ représente les sommets du graphe et $E = \{(x_i, y_j), \dots\}$, $x_i, y_j \in X$ l'ensemble ordonné des arrêtes du graphe. Pour un graphe orienté, les notions de graphe simple et multigraphes sont les mêmes que pour le cas non orienté.

Exemple (Graphes et leur représentation gravarphique) Soient G_1 et G_2 deux graphes, en voici une représentation :

Définition (Voisinage) . Le voisinage d'un sommet x de G est l'ensemble des sommets

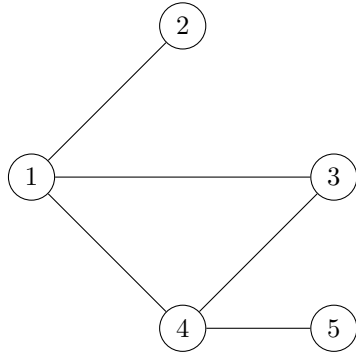


FIGURE 1.1 – Graphe non orienté

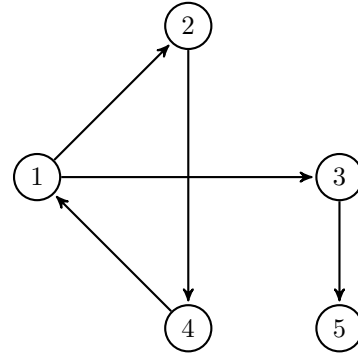


FIGURE 1.2 – Graphe orienté

y de G tels qu'il existe une arête entre x et y dans G . On le note $V(x)$.

Remarque Si x est dans le voisinage de y , on dira que x est adjacent à y et inversement.

Définition (Degré) . Le degré d'un sommet x de G est le cardinal du voisinage x . On le note $d(x)$. Un sommet de voisinage nul est dit *isolé* et un sommet de voisinage égal à 1 est dit *pendant*.

Définition (Voisinage entrant et sortant) . Soit G un graphe orienté et x un sommet de G . On définit deux types de voisinages :

- *Voisinage entrant* : noté $V^-(X)$ est l'ensemble des prédécesseurs de x .
- *Voisinage sortant* : noté $V^+(X)$ est l'ensemble des successeurs de x .

On définira comme précédemment le degré sortant et le degré entrant d'un sommet x .

Définition (Chaîne) . On appelle chaîne de $G = (X, E)$ de longueur n toute suite alternée de sommets et d'arêtes de G telle que :

$$c = (x_0, a_1, x_1, \dots, a_n, x_n), \text{ telle que } \forall i \in \llbracket 1, n \rrbracket, a_i = (x_{i-1}, x_i)$$

Ici, n représente le nombre d'arêtes de la chaîne. Dans le cas d'un graphe simple, on notera les chaînes de la façon suivante :

$$c = (x_0, \dots, x_n) \quad \text{ou} \quad c = x_0 - x_1 - \dots - x_n$$

Dans le cas des graphes orientés, on notera une chaîne simplement avec les arêtes dont elle est constituée :

$$c = (u_1, \dots, u_n), \forall i \in \llbracket 1, n \rrbracket u_i \in E$$

Définition (Accessibilité) . Soit $G = (X, E)$ un graphe. On a :

- Soient x et y deux sommets de G . On dit que y est accessible à partir de x s'il existe une chaîne joignant x et y dans G .
- G est dit *connexe* ssi $\forall x \in X, \forall y \in X, y$ est accessible à partir de x .

- L'accessibilité est une relation d'équivalence entre les sommets. Ses classes d'équivalences sont les composantes connexes de G .

Définition (Chaîne Simple) . Une chaîne est *simple* si elle ne passe pas deux fois par la même *arrête* et elle est dite *élémentaire* si elle ne passe pas deux fois par le même *sommet*. On remarquera facilement qu'une chaîne élémentaire est simple.

Définition (Cycle) . Un cycle de G est une chaîne simple dont le départ et l'arrivée sont le même sommet. Un cycle est donc de la forme :

$$c = x_0 - x_1 - \dots - x_{n-1} - x_0$$

Théorème (Propriétés des cycles et chaînes) . Soit G un graphe à $n \in \mathbb{N}$ sommets.

- Toute chaîne élémentaire a une longueur inférieure à $n - 1$
- Toute cycle élémentaire a une longueur inférieure à n
- De toute chaîne, on peut en extraire une chaîne élémentaire.

1.2 Cycle et flot

Définissons maintenant la notion de flot dans un cas général.

Définition (Vecteur de chaîne) . Soit $c = (u_1, \dots, u_n)$ une *chaîne élémentaire* dans un graphe orienté $G = (X, E)$ tel que $|E| = n$ et d'arcs u_i . On appelle *vecteur de chaîne* associé à c l'application φ associée à c telle que :

$$\varphi : \begin{cases} E \longrightarrow \mathbb{Z} \\ u_i \longmapsto \begin{cases} 1 & \text{si } u_i \text{ est dans le sens de } c \\ -1 & \text{si } u_i \text{ n'est pas dans le sens de } c \\ 0 & \text{si } u_i \text{ n'est pas dans } c \end{cases} \end{cases}$$

On la note en général $\varphi = (\varphi(u_1), \varphi(u_2), \dots, \varphi(u_n)) \in \mathbb{Z}^n$.

Venons en maintenant à la définition phare de ce chapitre :

Définition (Flot) . Soit $G = (X, E, c)$ un graphe orienté. On appelle *flot* dans G tout vecteur de cycle φ de G qui vérifie la loi de Kirchoff :

$$\forall x \in X, \quad \sum_{u_i \in \omega^-(\{x\})} \varphi(u_i) = \sum_{u_i \in \omega^+(\{x\})} \varphi(u_i)$$

Et on note $\varphi^i = \varphi(u_i)$ le *flot* de l'arrête u_i .

Remarque Un flot est donc un chaîne élémentaire d'un graphe orienté dont, pour chaque noeud, la somme des flots sortants est égale à la somme des flots entrants.

Propriété (Flots) . Soit $G = (X, E, c)$ un graphe tel que $|E| = n$, alors :

- Le vecteur nul de $\mathbb{Z}^n$ est un flot de G .
- Toute combinaison linéaire de flots de G est aussi un flot de G .
- Tout vecteur cyclique de G est un flot de G .

Exemple Soit G le graphe suivant : Le vecteur $\varphi_1 = (2, -2, 1, 3, 4, -1, 1)$ est un flot de G . Le

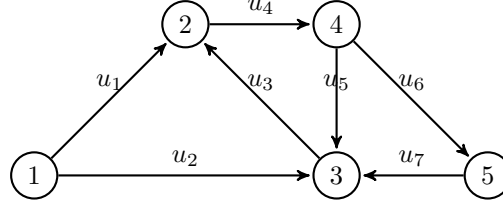


FIGURE 1.3 – flot dans un graphe

vecteur $\varphi_2 = (0, 0, 0, 0, 1, -1, 1)$ est aussi un flot. Soit $v = 2\varphi_1 + 3\varphi_2$, c'est aussi un flot de G .

Proposition (Loi de Kirchoff généralisée) Soit $G = (X, E, c)$ et $A \subset E$ non vide. Un chaîne φ est un flot de G ssi :

$$\sum_{u \in \omega^-(A)} \varphi(u) = \sum_{u \in \omega^+(A)} \varphi(u)$$

1.3 Flot et coupure

1.3.1 Flot maximal dans un réseau

Dans cette section, nous revenons à notre problème de départ en définissant la notion de graphe de transport. Graphes dans lesquels nous chercherons à construire des flots maximaux.

Définition (Réseau de Transport) . Un *réseau de transport* est un graphe orienté pondéré $R = (X, E, c)$ qui contient :

- un sommet sans prédécesseur appelé *source* (noté s).
- un sommet sans fils appelé *puits* (noté t).
- c , une application, qui à chaque arc, lui associe sa capacité notée c :

$$c : \begin{cases} E \longrightarrow \mathbb{R} \cup \{\infty\} \\ u = (x, y) \longmapsto c(u) \end{cases}$$

- un arc fictif de t à s doté d'une capacité infinie, appelé *arc retour*.

Définition (Flot compatible) . Un *flot compatible* sur un réseau $R = (X, E, c)$ tel que $|E| = n$ est un vecteur $\varphi \in \mathbb{Z}^n$ qui vérifie :

- **(Conservation de Kirchhoff)** pour tout $x \in X \setminus \{s, t\}$ (hors source s et puits t), on a :

$$\sum_{u \in \omega^+(x)} \varphi(u) = \sum_{u \in \omega^-(x)} \varphi(u)$$

où $\omega^+(x)$ (resp. $\omega^-(x)$) est l'ensemble des arcs sortants (resp. entrants) de x .

- **(Compatibilité avec les capacités) :**

$$\forall u \in E, \quad 0 \leq \varphi(u) \leq c(u).$$

La *valeur* du flot, notée $v(\varphi)$, est définie par :

$$v(\varphi) := \sum_{u \in \omega^+(s)} \varphi(u) - \sum_{u \in \omega^-(s)} \varphi(u),$$

c'est-à-dire le flux net sortant de la source (équivalent au flux net entrant dans le puits). Enfin, on dit qu'un arc u est *saturé* si $\varphi(u) = c(u)$.

Exemple Voici un exemple de réseau de transport :

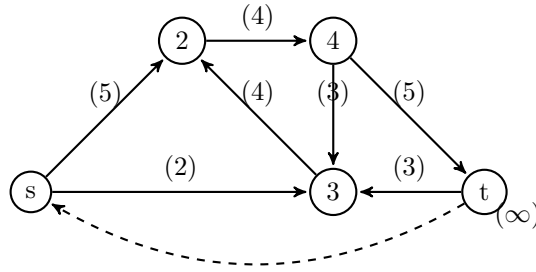


FIGURE 1.4 – Réseau de Transport

Définition (Flot maximal) . Un *flot maximal* d'un réseau est un flot compatible qui maximise la valeur $v(\varphi)$ parmi tous les flots compatibles du réseau.

1.3.2 Théorème de Coupure minimale

Pour construire un flot maximal dans un réseau, nous avons besoin d'un peu de théorie.

Définition (Coupure et cocycle) . Soit $R = (X, E, c)$ un réseau, où X est l'ensemble des sommets, E l'ensemble des arêtes et $c : E \rightarrow \mathbb{R}^+$ la fonction de capacité.

Une *coupure* ct séparant s de t est un couple $(A, X \setminus A)$ tel que :

$$A \subset E, \quad s \in A, \quad t \notin A.$$

L'ensemble des arcs de la coupure, noté $\text{arcs}(ct)$ est l'ensemble :

$$\text{arcs}(ct) = \omega^+(A) := \{u = (i, j) \in E \mid i \in A, j \in X \setminus A\}.$$

Qui correspond à l'ensemble des arcs qui, si on les enlève, coupe tout les chemins entre s et t .

Définition (Capacité d'une coupure) . La *capacité* d'une coupure ct est la somme des

capacités de ses arêtes :

$$\text{capa}(ct) := \sum_{u \in \text{arcs}(ct)} c(u).$$

Théorème (Flot et coupure minimale) . Pour tout flot compatible φ et pour toute coupure ct d'un réseau R , on a :

$$v(\varphi) \leq \text{capa}(ct)$$

1.4 Flot maximal : principe de Ford-Fulkerson

L'objectif de l'algorithme de Ford-Fulkerson est de construire de manière itérative un flot maximal. Pour cela, on part du flot nul dans le réseau. Et petit-à petit, on cherche des chemins non saturés de la source s vers le puit t pour lesquels on peut augmenter leur flot. Une fois que l'on n'est plus capable de trouver de chemin, alors l'algorithme se termine. La définition ci-dessous détaille le processus de construction d'un tel chemin.

1.4.1 Recherche de Flot Maximal

Définition (Principe de Ford-Fulkerson) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . L'algorithme de construction d'un chemin non saturé est :

1. **Initialisation** : On marque s
2. **Itération** : Pour tout sommet marqué $x, y \in X$ peut être marqué si :
 - y n'est pas encore marqué
 - $\begin{cases} \exists u = (x, y) \in E, \varphi(u) < c(u) & \text{(marquage direct)} \\ \exists u = (y, x) \in E, \varphi(u) < c(u) & \text{(marquage indirect)} \end{cases}$

L'algorithme se termine si on arrive à marquer le puit t . Dans ce cas, la chaîne produite est alors appelée *chaîne augmentante*, notée ch . On note ch^+ les sommets de ch marqués directement et ch^- les sommets marqués indirectement.

Propriété (Calcul d'un nouveau flot) . Soit $R = (X, E, c)$ un réseau et φ un flow sur R . Soit ch une chaîne augmentante de R . Il est alors possible d'augmenter la valeur du flot φ de $k \in \mathbb{R}$ en ajoutant à chaque arcs le flot définit par :

$$k = \min \left(\min_{u \in ch^+} \{c(u) - \varphi(u)\}, \min_{u \in ch^-} \{\varphi(u)\} \right)$$

Le nouveau flot est obtenu en augmentant de k le flow des arcs de ch^+ et en diminuant le flot de k des arcs de ch^- .

Théorème (Coupure Minimale - Flot Maximal) . Soit $R = (X, E, c)$ un réseau. Soit F l'ensemble des flots possibles sur R et K l'ensemble des coupures possibles de R . Alors la valeur du flot maximal est égale à la capacité de la coupure minimale.

$$\text{i.e. } \max_{\varphi \in F} \{v(\varphi)\} = \min_{ct \in K} \{\text{capa}(ct)\}$$

Ce théorème nous permet donc de trouver directement la valeur du flot maximal que l'on peut faire passer dans notre réseau sans avoir à en construire un.

Remarque La dernière itération de l'algorithme de Ford-Fulkerson nous donne une coupure minimale. Les sommets marqués correspondent aux sommets de l'ensemble A de la coupure $ct = (AX/A)$.

1.4.2 Recherche de flot maximal de coût minimal

Rajoutons maintenant une contrainte dans la recherche de notre flot maximal. On souhaite trouver un flot maximal dans un réseau pour lequel le coût du flot sera le plus faible. On cherche donc à faire passer le plus de marchandise dans notre réseau par les routes qui "coûtent le moins cher".

Nous devons donc définir un nouveau type de réseau intégrant cette notion de coût.

Définition (Réseau à coût) . Soit $R = (X, E, c)$ un réseau de noeuds E , d'arcs E et de fonction de capacité c . On appelle *réseau à coût* le réseau R auquel on a rajouté une fonction de coût γ , qui à chaque arête lui associe un coût :

$$\gamma : \begin{cases} E \longrightarrow \mathbb{R} \\ (x, y) \longmapsto \gamma(x, y) \end{cases}$$

Exemple On représentera donc un réseau à coût comme un réseau normal mais en rajoutant une valeur numérique sur chaque arête correspondant à son coût :

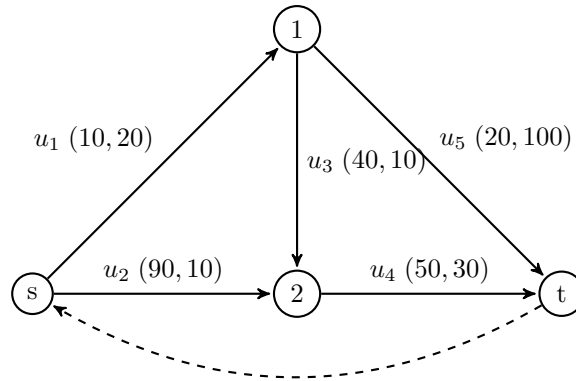


FIGURE 1.5 – Réseau de transport à coût

Remarque Chercher un flot maximal de coût minimal dans R revient donc à construire un flot via l'algorithme de Ford-Fulkerson en choisissant une chaîne augmentante de coût minimal à chaque itération.

Pour trouver cette chaîne augmentante de coût minimal, nous devons définir le graphe résiduel d'un réseau.

Définition (Graphe Résiduel) . Soit $R = (X, E, c, \gamma)$ un réseau et φ un flot sur R , on définit le *graphe résiduel* $G_R(\varphi) = (X, U_\varphi, r_\varphi)$ associé à R et φ comme le graphe tel que pour tout arc $(x, y) \in E$, on ait :

- un *arc direct* $(x, y) \in U_\varphi$ si $\varphi(x, y) < c(x, y)$ avec $r_\varphi = c(x, y) - \varphi(x, y)$
- un *arc indirect* $(y, x) \in U_\varphi$ si $\varphi(y, x) > 0$ avec $r_\varphi = \varphi(y, x)$.

Chapitre 2

Programmation Linéaire

Contents

2.1	Programmation Linéaire	124
2.1.1	Généralités	124
2.1.2	Solution Géométrique	127
2.1.3	Algorithme du Simplexe	128
2.1.4	Dualité	131
2.2	Programmation Linéaire en Nombres Entiers	133
2.2.1	Définition et relaxation	133
2.2.2	Algorithme "Branch and Bound"	135

Intéressons nous maintenant aux problèmes de programmation linéaire. Même si le nom peut prêter à confusion, il s'agit en réalité de problèmes d'optimisation d'une *fonction objectif* en respectant certaines *contraintes*. Le caractère linéaire de tels problèmes vient du fait que nous ne traiterons que des fonctions objectifs et des contraintes linéaires.

Ces problèmes de programmation linéaires peuvent être résolus en temps polynomial, notamment grâce à l'algorithme du simple. Les problème de flots maximaux, vus précédemment, peuvent s'exprimer comme des problèmes de programmation linéaire.

2.1 Programmation Linéaire

2.1.1 Généralités

Définition (Programme linéaire) . Un *programme linéaire* ou *problème de programmation linéaire* est un problème d'optimisation de $n \in \mathbb{N}$ variables et $m \in \mathbb{N}$ contraintes de la forme :

$$\begin{aligned} \text{minimize} \quad & z = \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^n a_{ij} x_j \preceq b_i \quad \forall i \in \llbracket 1, m \rrbracket, \\ & x \succeq 0 \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

où $\preceq \in \{\leq, \geq, =\}$. Un programme linéaire est donc un simple problème d'optimisation sur un polyèdre convexe. Dans le cas défini plus haut, on dit que le programme est sous forme générale. Différentes formes d'un même problème existent.

Exemple Voyons dès à présent un exemple de programme linéaire.

Un barman possède 30 litres de jus d'orange, 24 litres de pamplemousse et 18 litres de jus de framboise. Il peut préparer 2 types de cocktails :

- *Le "sunrise", vendu 80 euros le baril de 10 litres composé de 5 litres de jus d'orange, de 2 litres de jus de pamplemousse, d'un litre de jus de framboise et de 2 litres d'eau.*
- *Le "swing", vendu 60 euros le baril et composé de 3 litres de jus de pamplemousse, 3 litres de jus d'orange, 3 litres de jus de framboise et d'un litre d'eau.*

Il souhaite préparer la plus importante quantité des deux cocktails pour maximiser son profit.

Pour résoudre ce problème, nous pouvons construire un programme linéaire de deux variables x et y représentant chacune le nombre de litres d'un des deux cocktails à produire. La fonction objectif, décrivant le prix de revente de la marchandise est donc :

$$z = 80x + 60y$$

Et nous avons les contraintes suivantes :

$$\begin{aligned} 5x + 3y &\leq 30 && \text{(stock de jus d'orange)} \\ 2x + 3y &\leq 24 && \text{(stock de jus de pamplemousse)} \\ x + 3y &\leq 18 && \text{(stock de jus de framboise)} \\ x, z &\geq 0 \end{aligned}$$

On peut donc formuler le problème suivant :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

Proposition Comme dit dans la définition d'un programme linéaire, il peut exister différentes formes d'un même problème en fonction de la méthode de résolution utilisée. Le caractère linéaire de ces programmes nous permet de passer facilement d'une forme à une autre. Nous définissons la forme canonique d'un programme linéaire comme suit :

$$\begin{aligned} \text{maximize} \quad & z = \sum_{j=1}^n c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^n a_{ij} x_j = b_i, \quad \forall i \in \llbracket 1, m \rrbracket, \\ & x_j \geq 0, \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

Nous pouvons ensuite passer à la formulation dite « standard » où les contraintes d'ordre sont remplacées par des contraintes d'égalités. Il faut donc rajouter un certain nombre de variables à notre programme.

FIGURE 2.2 – Formulation standard d'un programme linéaire de maximisation

$$\begin{aligned}
&\text{maximize} && z = \sum_{j=1}^n c_j x_j \\
&\text{subject to} && \sum_{j=1}^n a_{ij} x_j \preceq b_i, \quad \forall i \in \llbracket 1, m \rrbracket, \\
&&& x_j \succeq 0, \quad \forall j \in \llbracket 1, n \rrbracket
\end{aligned}$$

FIGURE 2.1 – Formulation canonique d'un programme linéaire

$$\begin{aligned}
&\text{maximize} && z = CX \\
&\text{subject to} && AX = B, \\
&&& X \geq 0_{\mathbb{R}^n}
\end{aligned}$$

FIGURE 2.3 – Formulation matricielle d'un programme linéaire de maximisation

Enfin, nous avons la forme matricielle, qui nous permettra d'utiliser des algorithmes itératifs pour résoudre ce genre de problèmes :
où $C \in \mathbb{R}^n$, $A \in \mathcal{M}_{m,n}(\mathbb{R})$ et $B \in \mathbb{R}^m$.

Exemple Pour le problème de notre barman, nous pouvons ainsi représenter les différentes formes de notre problème :

$$\begin{aligned}
&\text{maximize} && z = 80x + 60y \\
&\text{subject to} && 5x + 3y \leq 30, \\
&&& 2x + 3y \leq 24, \\
&&& x + 3y \leq 18, \\
&&& x, y \geq 0
\end{aligned}$$

FIGURE 2.4 – Forme canonique du problème du barman

$$\begin{aligned}
&\text{maximize} && z = 80x + 60y \\
&\text{subject to} && 5x + 3y + e_1 = 30, \\
&&& 2x + 3y + e_2 = 24, \\
&&& x + 3y + e_3 = 18, \\
&&& x, y, e_1, e_2, e_3 \geq 0
\end{aligned}$$

FIGURE 2.5 – Forme standard du problème du barman

Et enfin :

$$\begin{aligned}
&\text{maximize} && z = \underbrace{(80, 60, 0, 0, 0)}_C \times \underbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}_X \\
&\text{subject to} && \underbrace{\begin{pmatrix} 5 & 3 & 1 & 0 & 0 \\ 2 & 3 & 0 & 1 & 0 \\ 1 & 3 & 0 & 0 & 1 \end{pmatrix}}_A \times \underbrace{\begin{pmatrix} x \\ y \\ e_1 \\ e_2 \\ e_3 \end{pmatrix}}_X = \underbrace{\begin{pmatrix} 30 \\ 24 \\ 18 \end{pmatrix}}_B
\end{aligned}$$

Maintenant que nous avons défini formellement un programme linéaire, passons à l'étude de ses solutions.

Définition (Solutions d'un programme linéaire) . Soit P un programme linéaire sous forme matricielle standard. Le vecteur $X = (x_1, \dots, x_n)$ est :

- une solution de P si $A^t X = B$
- une solution possible de P si $A^t X = B$ et $X \geq 0_{\mathbb{R}^n}$
- une solution optimale de P si c'est une solution possible et qu'elle maximise z parmi l'ensemble des solutions possibles.

Proposition Comme énoncé en introduction, on peut représenter un problème de maximisation de flot dans un réseau par un programme linéaire. Soit $R = (X, E, c)$ un réseau tel que $|E| = n$, avec $s, t \in X$ la source et le puits. On note $c(x, y)$ la capacité de l'arête entre x et y dans le réseau. On cherche une application

$$\varphi : E \longrightarrow \mathbb{R}, \quad (x, y) \longmapsto f(x, y)$$

qui maximise le flot entre la source s et le puits t .

Plus formellement, si l'on ordonne les arêtes E sous la forme $E = \{e_1, \dots, e_n\}$ et que l'on note $\varphi(E) = (f_1, \dots, f_n) \in \mathbb{R}^n$, alors on cherche à maximiser

$$z = \sum_{(x,y) \in v^-(s)} \varphi(x, y)$$

sous les contraintes :

$$0 \leq f(x, y) \leq c(x, y), \quad \forall (x, y) \in E$$

ainsi que les contraintes de conservation du flot pour tout sommet $v \in X \setminus \{s, t\}$:

$$\sum_{(u,v) \in v^-(v)} f(u, v) = \sum_{(v,w) \in v^+(v)} f(v, w).$$

2.1.2 Solution Géométrique

La recherche d'une solution d'un programme linéaire revient à trouver une droite de la forme $z = \sum_{i=1}^n x_i c_i$ pour laquelle z est maximal tout en respectant les contraintes du programme. En petite dimension (2 variables ou moins) il est possible de tracer ce problème et de le résoudre géométriquement. Détaillons cela autour d'un exemple.

Exemple (Solution Géométrique) Reprenons le problème de notre barman énoncé comme suit :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

On peut tracer les courbes des contraintes pour identifier la zone contenant les solutions possibles :

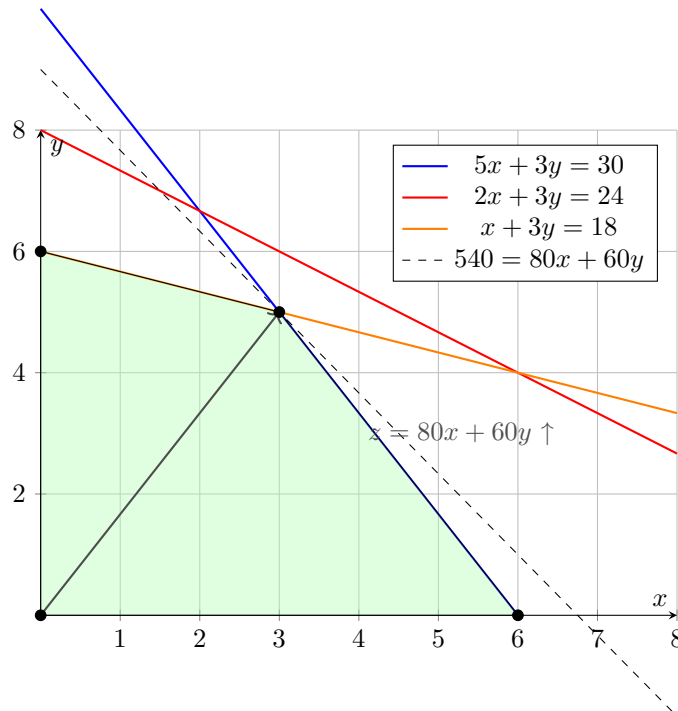


FIGURE 2.6 – Espace des solutions réalisables du programme linéaire
Le barman peut donc produire 80 litres de "sunrise" et 3 litres de "swing" pour un profit total de 540 euros.

Remarque En pratique, il sera très rare de pouvoir résoudre un tel problème graphiquement car le nombre de variable sera bien supérieur à 2.

2.1.3 Algorithme du Simplexe

Comme dit précédemment, le caractère linéaire des contraintes d'un programme linéaire nous permettent d'en déduire que l'espace des solutions possible est un polyèdre. D'où le théorème suivant.

Théorème (Solutions Possibles) . L'ensemble S des solutions possibles d'un programme linéaire est convexe et est un polyèdre non borné ou un polytope. Si S est non vide et borné, alors il existe au moins un vecteur de S pour lequel le maximum de la fonction objectif est atteint.

Remarque Ce théorème nous permet de distinguer plusieurs cas pour l'ensemble des solutions :

- Le programme linéaire a au moins une solution :
 - soit elle est unique
 - soit il en existe une infinité (i.e le maximum est atteint en plusieurs vecteurs) on parle alors de cas dégénéré.
- Le programme linéaire n'a pas de solution, lorsque les contraintes ne sont pas compatibles, l'ensemble des solutions est donc vide.
- Le programme linéaire n'a pas d'optimum (i.e l'ensemble des solutions n'est pas borné).

Proposition (Résolution d'un programme linéaire par le simplexe) Nous avons précédemment résolu nos programmes linéaires graphiquement. Nous allons voir que nous pouvons

aussi les résoudre *algébriquement*, en utilisant l'*algorithme du simplexe*. Soit le programme suivant :

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y \leq 300, \\ & 5x \leq 310, \\ & 2x + 4y \leq 200, \\ & 3y \leq 100, \\ & x, y \geq 0 \end{aligned}$$

On passe ce programme sous *forme standard* en introduisant des variables d'écart $e_1, e_2, e_3, e_4 \geq 0$:

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y + e_1 = 300, \\ & 5x + e_2 = 310, \\ & 2x + 4y + e_3 = 200, \\ & 3y + e_4 = 100, \\ & x, y, e_1, e_2, e_3, e_4 \geq 0 \end{aligned}$$

Le vecteur $(x, y, e_1, e_2, e_3, e_4) = (0, 0, 300, 310, 200, 100)$ est une *solution de base réalisable* : toutes les contraintes sont vérifiées et les variables sont positives. L'idée du simplexe est de :

1. Choisir une *base initiale* (ici, les variables d'écart e_i).
2. Exprimer les variables hors base (x, y) en fonction des variables de base.
3. *Augmenter la valeur de la fonction objectif* z en remplaçant une variable de base par une variable hors base qui améliore z .
4. Répéter jusqu'à ce qu'aucune amélioration ne soit possible : la solution est alors optimale.

Formellement, le simplexe cherche à écrire le système sous la forme :

$$\begin{cases} z = C + \sum_i c_i x_i \\ x_B = b - Ax_N \end{cases} \quad \text{où } x_B \geq 0, x_N \geq 0$$

À chaque itération :

- une variable hors base entre dans la base (celle qui augmente le plus z),
- une variable de base quitte la base (celle qui devient nulle en premier).

Lorsque tous les coefficients $c_i \leq 0$, aucune amélioration n'est possible et la solution actuelle est *optimale* : la valeur optimale est alors $z = C$.

Exemple (Résolution algébrique du programme par le simplexe) Considérons le programme linéaire suivant :

$$\begin{aligned} \text{maximize} \quad & z = x + 2y \\ \text{subject to} \quad & 4x + 2y \leq 300, \\ & 5x \leq 310, \\ & 2x + 4y \leq 200, \\ & 3y \leq 100, \\ & x, y \geq 0 \end{aligned}$$

1. **Mise sous forme standard :** On introduit une variable d'écart pour chaque contrainte :

$$\begin{cases} 4x + 2y + e_1 = 300 \\ 5x + e_2 = 310 \\ 2x + 4y + e_3 = 200 \\ 3y + e_4 = 100 \end{cases} \quad \text{avec } e_i \geq 0.$$

La fonction objectif s'écrit :

$$z = x + 2y.$$

Une solution de base réalisable évidente est obtenue en prenant $x = y = 0$, ce qui donne :

$$(e_1, e_2, e_3, e_4) = (300, 310, 200, 100), \quad z = 0.$$

C'est notre solution de base initiale.

2. **Recherche d'une direction d'amélioration :** Comme $z = x + 2y$, augmenter y est plus rentable que x (le coefficient 2 est supérieur à 1). On va donc chercher à **augmenter** y tant que c'est possible sans violer les contraintes. On part du système :

$$\begin{cases} e_1 = 300 - 2y - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4y \\ e_4 = 100 - 3y \end{cases}$$

Comme on garde $x = 0$ au départ, ces équations deviennent :

$$\begin{cases} e_1 = 300 - 2y \\ e_2 = 310 \\ e_3 = 200 - 4y \\ e_4 = 100 - 3y \end{cases}$$

Pour rester faisable, il faut $e_i \geq 0$. On obtient donc :

$$y \leq 150, \quad y \leq 50, \quad y \leq 25, \quad y \leq 33,3.$$

La contrainte la plus restrictive est $y \leq 25$ (provenant de $e_3 = 200 - 4y$). On choisit donc $y = 25$ et $e_3 = 0$. Cela définit une nouvelle base : $\{e_1, e_2, e_4, y\}$.

3. **Amélioration suivante :** Pour cette valeur $y = 25$, les variables d'écart valent :

$$e_1 = 300 - 2(25) = 250, \quad e_2 = 310, \quad e_3 = 0, \quad e_4 = 100 - 3(25) = 25.$$

La fonction objectif vaut alors :

$$z = x + 2y = 2 \times 25 = 50.$$

On peut encore améliorer z en augmentant x , tant que toutes les contraintes restent valides. En remplaçant $y = 25$, on obtient :

$$\begin{cases} e_1 = 300 - 2(25) - 4x = 250 - 4x \\ e_2 = 310 - 5x \\ e_3 = 200 - 2x - 4(25) = 100 - 2x \\ e_4 = 100 - 3(25) = 25 \end{cases}$$

Les contraintes donnent :

$$x \leq 62,5, \quad x \leq 62, \quad x \leq 50.$$

Le plus restrictif est $x \leq 50$ (provenant de $e_3 \geq 0$). On prend donc $x = 50$ et $e_3 = 0$.

4. **Calcul de la solution optimale** : On obtient :

$$\begin{cases} x = 50, & y = 25, \\ e_1 = 300 - 4(50) - 2(25) = 100, \\ e_2 = 310 - 5(50) = 60, \\ e_3 = 0, \\ e_4 = 100 - 3(25) = 25. \end{cases}$$

La fonction objectif vaut :

$$z = x + 2y = 50 + 2 \times 25 = 100.$$

Si on essaye d'augmenter y ou x davantage, on viole immédiatement une contrainte. La solution est donc **optimale**.

5. **Conclusion** : Le programme atteint son maximum en :

$$x^* = 50, \quad y^* = 25, \quad z_{\max} = 100.$$

Cette résolution illustre la logique du simplexe : on part d'une solution réalisable triviale, puis on améliore la fonction objectif pas à pas en déplaçant la base vers les sommets adjacents du polytope faisable, jusqu'à atteindre un sommet où toute augmentation violerait une contrainte.

2.1.4 Dualité

Définition (Programme Primal) . Le *problème primal* (ou primaire) est la forme standard d'un programme linéaire :

$$\begin{aligned} &\text{maximize} && z = {}^t c x \\ &\text{subject to} && A x \leq b, \\ &&& x \geq 0 \end{aligned}$$

où :

- $x \in \mathbb{R}^n$ est le vecteur des variables de décision ;
- $c \in \mathbb{R}^n$ est le vecteur des coefficients de la fonction objectif ;
- $A \in \mathbb{R}^{m \times n}$ est la matrice des coefficients des contraintes ;
- $b \in \mathbb{R}^m$ est le vecteur des bornes du système d'inégalités.

Définition (Problème Dual) . À tout problème primal est associé un *problème dual* défini par :

$$\begin{aligned} &\text{minimize} && w = {}^t b y \\ &\text{subject to} && {}^t A y \geq c, \\ &&& y \geq 0 \end{aligned}$$

où :

- $y \in \mathbb{R}^m$ est le vecteur des variables duales ;

- chaque contrainte du primal correspond à une variable du dual ;
- chaque variable du primal correspond à une contrainte du dual.

Remarque (Interprétation) Le problème *primal* cherche à *maximiser un profit* ou une valeur objective en respectant des contraintes de ressources. Le problème *dual* cherche à *minimiser le coût* de ces ressources tout en garantissant que la valeur de chaque ressource couvre la contribution de chaque variable du primal. Autrement dit :

Le primal cherche ce qu'on peut obtenir au maximum avec des ressources limitées, tandis que le dual cherche le coût minimal que doivent avoir ces ressources pour que le système reste équilibré.

Exemple En reprenant l'exemple de notre barman, le programme primal cherche à maximiser le profit du barman en fixant le prix des barrils c_j de cocktails j qu'il produit en utilisant au plus b_i litres de jus i . Les variables x_j représentent alors le nombre de barrils de cocktail j . Ici, le programme dual cherche à minimiser le profit total obtenu grâce aux stocks b_i de chaque jus i si le barman vend chaque cocktail j pour un prix minimal de c_j . Les variables y_i représentent donc le profit que le barman réalise par litre de jus i .

Voici les expressions mathématiques de ces deux programmes :

$$\begin{aligned} \text{maximize} \quad & z = 80x + 60y \\ \text{subject to} \quad & 5x + 3y \leq 30, \\ & 2x + 3y \leq 24, \\ & x + 3y \leq 18, \\ & x, y \geq 0 \end{aligned}$$

FIGURE 2.7 – Programme primal du barman

$$\begin{aligned} \text{minimize} \quad & w = 20y_1 + 24y_2 + 18y_3 \\ \text{subject to} \quad & 5y_1 + 2y_2 + y_3 \geq 80, \\ & 3y_1 + 3y_2 + 3y_3 \geq 60, \\ & y_1, y_2, y_3 \geq 0 \end{aligned}$$

FIGURE 2.8 – Programme dual du barman

La solution optimale du primal est $(x_1, x_2) = (3, 5)$ pour un profit maximal de 540. En revanche la solution du dual est $(y_1, y_2, y_3) = (15, 0, 5)$ pour un profit de 540 aussi.

Il existe un lien direct entre les solutions du dual et celles du primal. Elles sont décrites par les théorèmes de dualité.

Théorème (Dualité Faible) . La solution optimale du programme primal est inférieure ou égale à celle du dual.

Théorème (Dualité Forte) . Nous avons les deux implications suivantes :

1. Si l'un des deux programmes (primal ou dual) admet une solution optimale finie, alors l'autre aussi. De plus, la valeur optimale pour des deux programmes est la même.
2. Si l'ensemble des solutions de l'un des deux programmes est non borné, alors celui de l'autre est vide.

Remarque Le théorème de coupe minimale/flot maximal dans un réseau est l'équivalent du théorème de dualité forte ici. En effet, le problème de coupe minimale dans un réseau est l'exact dual du problème de flot maximal dans le même réseau.

Exemple (Utilité de la dualité) Considérons le programme linéaire suivant :

$$\begin{aligned} \text{maximize} \quad & x + 3x + 6y + z \\ \text{subject to} \quad & w + x + 5y + 6z \leq 20, \\ & w + 2x + y + 3z \leq 30, \\ & x, y, w, z \geq 0 \end{aligned}$$

Ce programme ne peut pas être résolu graphiquement du fait du trop grand nombre de variables. En passant au dual, nous aurons un programme linéaire à deux variables et 4 contraintes que l'on pourra résoudre graphiquement. Attention, la résolution du dual nous donnera seulement une idée de la valeur de la solution maximale.

Remarque Ainsi, passer au problème dual peut être avantageux dans plusieurs cas :

- Le dual peut être plus simple à résoudre (moins de variables ou moins de contraintes).
- Il peut exister une solution optimale évidente pour le dual mais pas pour le primal.
- La valeur de n'importe quelle solution du dual nous donne une borne supérieure pour la solution du primal.
- La solution optimale du dual nous donne des informations sur la solution optimale du primal.

2.2 Programmation Linéaire en Nombres Entiers

Dans cette section, nous allons traiter un nouveau type de problèmes : les *programmes linéaires en nombres entiers*. Vous l'aurez compris, il s'agit maintenant de considérer des programmes linéaires à variables entières. Nous allons donc une méthode pour les résoudre ainsi que quelques techniques de modélisation. Commençons tout d'abord par un exemple introductif.

Exemple Un commerçant souhaite remplir son sac à dos d'au plus n objets de poids et valeurs respectifs $p_i \in \mathbb{N}$ et $v_i \in \mathbb{N}$. Or le sac du commerçant ne peut contenir qu'un poids maximal $C \in \mathbb{N}$. Le commerçant veut choisir les objets à prendre de manière à maximiser la valeur emportée. On modélisera donc le problème de la façon suivante :

$$\begin{aligned} \text{maximize} \quad & V = \sum_{i=1}^n v_i \times x_i \\ \text{subject to} \quad & \sum_{i=1}^n p_i x_i \leq C, \\ & \forall i \in \llbracket 1, n \rrbracket, x_i \in \mathbb{N} \end{aligned}$$

Ici le vecteur $(x_1, \dots, x_n) \in \{0, 1\}^n$ représentera les objets que l'on prend. En effet, $x_i = 0$ si le commerçant ne prend pas l'objet i et 1 sinon.

Nous avons donc affaire à un programme linéaire. Or sa résolution pourrait non conduire à une solution comportant des valeurs $x_i \notin \{0, 1\}^n$ alors que l'on considère que l'on ne peut pas prendre une partie d'un objet (ex : 3/4 de l'objet i). Nous devons donc chercher une nouvelle méthode pour résoudre ce genre de programmes.

2.2.1 Définition et relaxation

Définition (Programme linéaire en nombres entiers) . Un *programme linéaire en nombres entiers* est un programme linéaire comme défini précédemment où un certain nombre de variables sont à valeurs entières. Plus formellement, la forme standard d'un tel programme est de la forme :

$$\begin{aligned} & \text{maximize} && \sum_{j=1}^n c_j x_j \\ & \text{subject to} && \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \in \llbracket 1, m \rrbracket, \\ & && x_j \geq 0 \quad \forall j \in \llbracket 1, n \rrbracket, \\ & && x_j \in \mathbb{N} \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

où les constantes c_j, a_{ij} et b_i sont entières.

Définition (Relaxation linéaire) . Soit un programme linéaire en nombres entiers (PLNE) :

$$\begin{aligned} & \text{maximize} && \sum_{j=1}^n c_j x_j \\ & \text{subject to} && \sum_{j=1}^n a_{ij} x_j = b_i \quad \forall i \in \llbracket 1, m \rrbracket, \\ & && x_j \in \mathbb{N} \quad \forall j \in \llbracket 1, n \rrbracket \end{aligned}$$

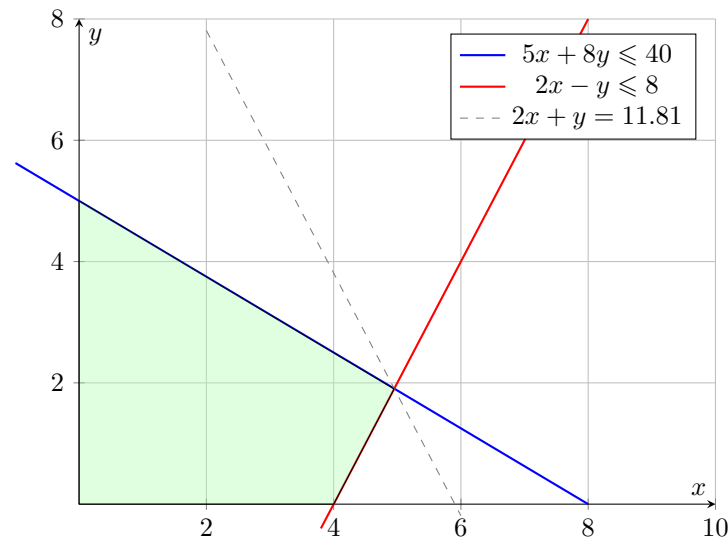
La *relaxation linéaire* de ce programme consiste à supprimer les contraintes de la forme :

$$x_j \in \mathbb{N} \quad \forall j \in \llbracket 1, n \rrbracket$$

On obtient ainsi un programme linéaire classique, plus facile à résoudre.

Exemple Considérons le programme linéaire suivant que nous allons résoudre graphiquement :

$$\begin{aligned} & \text{maximize} && z = 2x + 3y \\ & \text{subject to} && 5x + 8y \leq 40, \\ & && 2x - y \leq 8, \\ & && x, y \geq 0 \end{aligned}$$



Graphiquement, nous trouvons une solution optimale de valeur 11.81.

2.2.2 Algorithme "Branch and Bound"

Théorème (Lien entre PLNE et relaxation) . Soit un programme linéaire en nombres entiers (PLNE) et sa relaxation linéaire correspondante. Notons z_{int}^* la valeur optimale du PLNE et z_{relax}^* celle de la relaxation. Alors :

$$z_{\text{relax}}^* \geq z_{\text{int}}^* \quad (\text{en cas de maximisation}), \quad z_{\text{relax}}^* \leq z_{\text{int}}^* \quad (\text{en cas de minimisation}).$$

Autrement dit, la solution optimale de la relaxation fournit une *borne supérieure* (ou *borne inférieure*) pour le PLNE initial.

La méthode *Branch and Bound* est une technique de résolution des programmes linéaires en nombres entiers (PLNE) qui repose sur l'utilisation des relaxations linéaires pour obtenir des bornes sur la solution optimale. Cet algorithme construit un arbre de recherche binaire dont la racine est le problème initial II. L'algorithme s'arrête lorsque tous les nœuds feuilles de l'arbre sont *fathomés* (c'est-à-dire qu'ils n'ont plus besoin d'être explorés).

Définition (Nœud fathomé) . Un nœud est dit *fathomé* si au moins une des conditions suivantes est vérifiée :

1. La relaxation linéaire est *infaisable*, c'est-à-dire qu'aucune solution réelle ne satisfait les contraintes.
2. La valeur optimale de la relaxation linéaire est inférieure ou égale à la meilleure solution entière connue.
3. La solution optimale de la relaxation linéaire est entière, et est donc également solution optimale du programme entier.

Algorithm 1 Branch and Bound pour un programme linéaire en nombres entiers

- 1: Résoudre la relaxation linéaire du problème Π .
- 2: **if** le nœud est fathomé **then**
- 3: arrêter l'exploration de ce nœud.
- 4: **else**
- 5: Identifier une variable x_i fractionnaire dans la solution x^* .
- 6: Créer deux sous-problèmes en ajoutant respectivement les contraintes :

$$x_i \leq \lfloor x_i^* \rfloor \quad \text{et} \quad x_i \geq \lceil x_i^* \rceil$$

- 7: Appeler **Branch and Bound** récursivement sur ces sous-problèmes.
 - 8: **end if**
-

Remarque L'algorithme explore donc l'arbre de recherche en combinant deux techniques :

- Le *branching*, qui divise le problème en sous-problèmes plus simples.
- Le *bounding*, qui utilise la relaxation linéaire pour élaguer les sous-problèmes ne pouvant pas améliorer la meilleure solution entière connue.

Chapitre 3

Méta-heuristiques, CSP et SAT

Chapitre 4

Théorie de la complexité

Contents

4.1	Généralités	138
4.2	Problèmes de décisions, classes P, NP et NP-complet	139
4.2.1	Complexité temporelle	139
4.2.2	Classe P	139
4.2.3	Vérificateurs et classe NP	140

Dans ce chapitre, nous nous intéressons aux problèmes d'optimisation combinatoire "difficiles" pour lesquels nous n'avons pas de méthode explicite pour déterminer une solution. Nous essaierons de "trier" ces problèmes en classes de complexité et nous verrons que certains problèmes sont étrangement similaires les uns des autres.

4.1 Généralités

Un problème d'optimisation combinatoire est défini par plusieurs éléments. On définit ainsi $\mathcal{X}$ un ensemble de *variables*, *d'éléments* ou *d'objets*. Il représente ainsi l'ensemble des paramètres. La résolution de ce problème vise à trouver un *groupement* ou des affectations de $\mathcal{X}$ dont on $\mathcal{E}$ l'ensemble des solutions possibles.

Notre problème peut aussi être défini par un certain nombre de *contraintes* d'où l'existence d'un ensemble $\mathcal{S}$ regroupant toutes les solutions *faisables* (i.e respectant les contraintes).

Remarque On a donc en général $\mathcal{S} \subset \mathcal{E}$. $\mathcal{E}$, énumérer $\mathcal{S}$ peut être impossible d'où la difficulté du problème.

Enfin, nous pouvons aussi définir une fonction de coût $v : \mathcal{E} \rightarrow \mathbb{R}$ définissant une valeur ou un coût pour chaque affectation de $\mathcal{X}$ permettant ainsi d'ordonner les solutions et d'en définir une *optimale*.

Définition (Instance) . Un *instance* I d'un problème d'optimisation combinatoire est donc un cas particulier de ces problèmes dans un contexte donné. Elle est caractérisée par un ensemble d'objets/de variables $\mathcal{X}$ et des paramètres.

Définition (Problème d'optimisation combinatoire) . Ainsi, un *problème d'optimisation combinatoire* noté Π est défini par :

- un ensemble d'instances $\mathcal{I}$.

- pour chaque $I \in \mathcal{I}$, un ensemble de solutions faisables $\mathcal{S}(I)$.
- une fonction de coût/valeur.

Remarque La différence fondamentale entre une instance et un problème est qu'une instance s'applique dans un contexte donné et a une (ou plusieurs) solution(s) bien spécifique(s). Un problème est plutôt la représentation théorique de ce genre de problèmes.

Pour un problème donné Π il est intéressant de connaître sa classe de complexité pour savoir comment le résoudre et si c'est possible.

Les classes de complexité nous permettront donc de réunir ensemble des problèmes d'optimisation combinatoire ayant la même complexité temporelle (dans le pire des cas).

4.2 Problèmes de décisions, classes P, NP et NP-complet

4.2.1 Complexité temporelle

Définition (Problème de décision) . Un *problème de décision* est un problème dans lequel on souhaite répondre par oui ou par non pour une instance d'un problème donné.

On dit qu'une instance d'un problème est *positive* si la réponse au problème de décision est "oui".

Exemple Le problème des cliques dans un graphe est un problème de décision. En effet, on souhaite répondre par oui ou non à la question "Un graphe G contient-il un sous-graphe complet d'ordre $k \in \mathbb{N}$?

Pour différencier la difficulté de tels problèmes, nous nous intéressons à leur *complexité temporelle*.

Définition (Complexité temporelle) . Pour un algorithme donné A , on définit sa *complexité temporelle* notée $Time_A$ où $Time_A(x)$ est le nombre d'instructions exécutées par A pour une entrée x .

Proposition On souhaiterait ainsi définir la complexité d'un problème Π par la complexité temporelle de l'algorithme le plus efficace pour résoudre Π . Or, il est impossible de dire qu'un algorithme est optimal et ne peut être amélioré. On parlera donc de complexité *asymptotique*.

Définition (Ordre de complexité) . Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ une application. Soit Π un problème. On dit que la complexité temporelle de Π est d'ordre $O(f)$ ssi il existe un algorithme A résolvant Π et $n_0, c \in \mathbb{R}$ tels que :

$$\forall x, \quad |x| > n_0 \implies Time_A(x) \leq cf(|x|) \quad (4.2.1)$$

4.2.2 Classe P

Comme dit précédemment, nous souhaitons regrouper des problèmes de même ordre de complexité temporelle. Nous définissons donc le concept de classe.

Définition (Classes de complexité) . La classe de complexité $TIME[f]$ est l'ensemble des problèmes de complexité temporelle $O(f)$. On a donc :

$$TIME[n] \subset TIME[n \log n] \subset TIME[n^2] \subset TIME[e^n] \subset TIME[2^n]$$

Définition (Classe P) . On définit ainsi la classe P comme l'ensemble des classes de problèmes de complexité polynomiale. D'où :

$$P = \bigcup_{k \geq 0} \text{TIME}[n^k] \quad (4.2.2)$$

La classe P est indépendante du modèle d'ordinateur considéré (machine de Turing ou à RAM). Elle est aussi indépendante du langage utilisé pour l'algorithme ainsi que des structures de données utilisées pour stocker les variables du problème.

Remarque Un problème est donc dit polynomial si il peut être résolu par un algorithme de complexité temporelle polynomiale. Or pour de très grandes valeurs de k , cela revient à résoudre un problème de complexité exponentielle. En pratique, nous pourrions donc résoudre des problèmes de complexité $O(n^2)$, $O(n^3)$ ou $O(n^4)$.

Rappel (k-SAT) Soit $k \in \mathbb{N}$. Un problème de k -SAT est un problème de *satisfaisabilité booléenne* portant sur des formules booléennes en forme normale conjonctive (CNF).

Une formule est composée :

- de *variables booléennes* $x_1, \dots, x_n$;
- de *littéraux*, qui sont soit une variable soit sa négation : $l \in \{x_i, \neg x_i\}$;
- de *clauses*, qui sont des disjonctions de k littéraux :

$$C = l_1 \vee l_2 \vee \dots \vee l_k;$$

- et d'une *formule* CNF, qui est une conjonction de clauses :

$$\varphi = \bigwedge_{j=1}^m C_j.$$

Le problème de k -SAT consiste à déterminer s'il existe une assignation de vérité des variables qui rende la formule φ vraie. Si une telle assignation existe, on dit que la formule est *satisfaisable*.

Théorème (SAT) . $2\text{-SAT} \in P$.

Définition (Réduction polynomiale) . On dit qu'il existe une *réduction polynomiale* entre deux problèmes Π_1 et Π_2 si il existe une fonction f calculable en temps polynomial si, x est une instance positive de Π_1 ssi $f(x)$ est une instance positive de Π_2 . On note alors $\Pi_1 \leq \Pi_2$.

Théorème (Réduction) . $2\text{-COLORATION} \leq 2\text{-SAT}$.

4.2.3 Vérificateurs et classe NP

Apprentissage Automatique

Chapitre 1

Fondements, Réseaux Denses et Optimisation

Contents

1.1	Généralités	142
1.1.1	Un peu d'histoire...	142
1.1.2	Définition et terminologie	143
1.1.3	L'apprentissage	144
1.2	Du Perceptron au MLP	144
1.2.1	Concept	144
1.2.2	Apprentissage	145
1.2.3	Perceptron Multi-Couches (MLP)	145
1.3	Fonctions d'Activation et Classification Multiclasse	146
1.3.1	Fonctions d'activation usuelles	146
1.3.2	Classification multiclasse et fonction Softmax	147
1.4	Fonctions de Perte (<i>Loss Functions</i>)	147
1.4.1	Entropie croisée binaire (BCE)	147
1.5	Rétropropagation du Gradient et Dérivation Automatique	147
1.5.1	Rétropropagation sur un neurone formel	148
1.5.2	Rétropropagation dans un MLP (<i>Backpropagation</i>)	148
1.5.3	Graphes de calcul et implémentation PyTorch	148
1.5.4	Optimisation par descente de gradient avec inertie (Momentum)	149

1.1 Généralités

1.1.1 Un peu d'histoire...

Nous sommes en été 1955 aux États-Unis. La naissance de l'informatique pousse les mathématiciens John McCarthy, Marvin Minsky, Nathaniel Rochester et l'ingénieur Claude Shannon à proposer un projet de recherche sur l'intelligence artificielle au Dartmouth college "A proposal for the Dartmouth summer research project on artificial intelligence". Ils obtiennent ainsi 7500 \$ de la fonction Rockefeller pour organiser un atelier de travail sur les machines pensantes durant deux mois. Cependant, l'intérêt pour ce domaine de recherche est encore très faible parmi la

communauté scientifique. Les faibles performances des premières machines constituent aussi un important frein au développement de prototypes.

En 1957, Frank Rosenblatt présente le premier réseau de neurones tel que nous les connaissons aujourd'hui appelés *Perceptron*. En 1986, la notion de Perceptron multicouches est explorée et en 1998 Yann LeCun et Yoshua Bengio développe le premier réseau de neurones à convolution à la base de la vision par ordinateur.

La victoire de DeepBlue sur Garry Kasparov aux échecs en 1997 démontre la puissance et le potentiel de ces nouvelles technologies. L'affluence de données disponibles ces 30 dernières années et l'évolution des capacités de calcul des ordinateurs a ouvert la porte à des systèmes de plus en plus performants. Enfin, 2015 marque l'apogée des modèles d'IA dits "classiques" avec la victoire d'AlphaGo sur les champions d'Europe et du monde.

1.1.2 Définition et terminologie

Selon Wikipédia, l'intelligence artificielle se définit comme suit :

"L'intelligence artificielle (IA) est l'ensemble des systèmes informatiques capables d'effectuer des tâches typiquement associées à l'intelligence, telles que l'apprentissage, le raisonnement, la résolution de problèmes, la perception ou la prise de décision."

Le but de l'IA est donc de reproduire numériquement le mode de pensée humaine. Ces concepts ont été, et sont toujours, à la base des réseaux de neurones actuels puisqu'ils sont constitués de neurones organisés en couches permettant de traiter l'information. C'est donc une discipline très vaste dans lequel il est facile de confondre les notions.

L'*apprentissage automatique* en est un sous-domaine qui consiste en la création d'algorithmes capables d'apprendre à partir de données précédemment collectées. Il faut donc distinguer plusieurs phases dans le processus de développement d'un tel modèle :

1. La collecte des données d'entraînement.
2. L'entraînement du modèle.
3. La phase de test des performances du modèle.
4. La mise sur le marché pour une utilisation publique.

Nous traiterons ici les phases 1 et 2 de ce processus. Les autres appartiennent à des domaines différents de l'IA. Pour le développement d'un tel modèle, nous devons donc avoir accès à des **observations** constituées d'exemples ou de données collectés. Ces données sont constituées de deux éléments : les **attributs**, appelés *features*, constituent des représentations numériques des observations et les **étiquettes**, ou *labels*, qui associent à chaque élément une catégorie ou une valeur. L'enjeu est donc d'entraîner un modèle à distinguer les différents attributs pour chaque observation et d'être capable de répondre correctement à la question : "Étant donné une observation, à quelle catégorie appartient-elle?". Pour cela, le jeu de données initial, appelée *dataset*, sera séparé en deux, l'un pour l'entraînement, et l'autre pour le test.

Plus formellement, pour un ensemble $E = \{(x_i, y_i) \mid i \in \llbracket 1, N \rrbracket\}$ de données constitué d'observation X et de données Y , on cherche à apprendre une fonction f telle que :

$$f : \begin{cases} X \longrightarrow Y \\ x_i \longmapsto y_j = f(x_i) \end{cases}$$

et l'on souhaite que $i = j$ pour le plus de cas possibles. C'est-à-dire que notre modèle ne se trompe pas dans la prédiction y_i de la classe de x_i . En fonction de la nature de l'ensemble Y , on distinguera plusieurs types de problèmes. Si Y est un ensemble fini, on parlera de *problème de classification*, si $Y = \{\text{vrai}, \text{faux}\}$, on parlera de *prédiction* et si $Y = \mathbb{R}$, on parlera de régression.

1.1.3 L'apprentissage

L'apprentissage constitue le cœur de ce que nous allons voir ici. Cette phase permet de définir "les bonnes réponses" au modèle. On distingue ainsi différents types d'apprentissages :

- **L'apprentissage supervisé** : on dispose ici de données étiquetées et le modèle doit apprendre à distinguer les classes des observations.
- **L'apprentissage non supervisé** : ici le modèle n'a accès qu'aux observations et doit déterminer lui-même leur classes.
- **L'apprentissage par renforcement** : le modèle est ici très différent, il se représente sous la forme d'un agent devant apprendre une série d'actions en fonction de la configuration de son environnement. Pour cela, à chaque essai, on lui attribue une récompense positive ou négative.

Nous nous intéressons ici à l'apprentissage supervisé.

1.2 Du Perceptron au MLP

Le *Perceptron* peut être vu comme le premier réseau de neurones moderne, bien que sa conception date de 1956. À cette époque, Frank Rosenblatt pense à une "solution à tout" basée sur une représentation simplifiée du cerveau humain. L'idée est de recréer un ensemble de neurones connectés les uns aux autres capables d'"apprendre".

1.2.1 Concept

On se pose donc un problème à deux classes (c_- , c_+) avec m exemples :

$$\mathcal{D} = \{(x_i, y_i) \in \mathbb{R}^p \times \{-1, 1\} \mid j \in \llbracket 1, m \rrbracket\}$$

Pour chaque exemple, x_i représente l'observation et y_i la classe correspondante. On veut développer un modèle capable de déterminer la classe d'une observation brute. On effectue donc une hypothèse forte qui est que les deux classes c_- et c_+ sont séparables par un hyperplan de $\mathbb{R}^p$. Le problème revient donc à construire une famille $(w_1, \dots, w_p)$ de *poids* qui définiront cet hyperplan. Formellement, on veut "apprendre" une fonction y telle que :

$$\forall i \in \llbracket 1, m \rrbracket, \quad y(x) = \begin{cases} 1 & \text{si } \sum_{j=1}^p w_j x_j > T \\ -1 & \text{sinon} \end{cases}.$$

On appellera cette fonction y , notée $\hat{y}$ la *prédiction*. Ici notre réseau de neurones a déjà pris forme. Il se représente encore plutôt sous la forme d'un neurone :

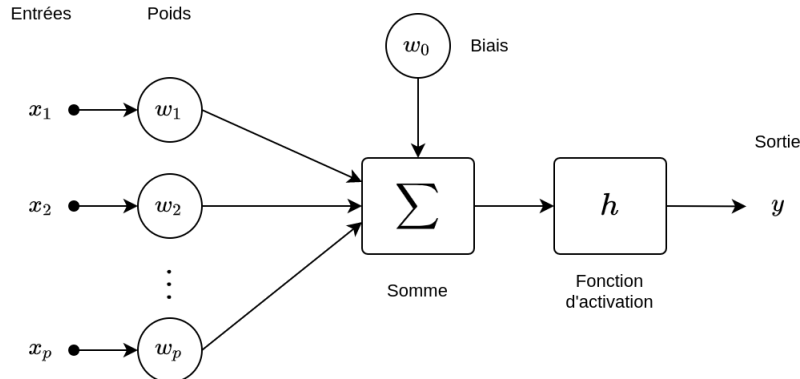


FIGURE 1.1 – Schéma formel d'un neurone

On utilisera aussi une *fonction d'activation* représentée par le seuil T . Plus tard, nous utiliserons des fonctions plus complexes pour donner une valeur de "probabilité" à la sortie d'un neurone.

1.2.2 Apprentissage

Une fois la structure posée, nous devons définir un algorithme pour apprendre les poids $(w_1, \dots, w_p)$ dans le but d'avoir une séparation optimale des deux classes dans $\mathbb{R}^p$. L'idée est donc de faire passer des valeurs x dans le perceptron et d'actualiser les poids w_j si la prédiction $\hat{y}$ est différente de la classe y . Une règle simple d'actualisation est de modifier w_j en fonction de la valeur x_j de l'entrée qu'il pondère. On aura donc la règle suivante :

$$w = \begin{cases} w + x & \text{si } y = 1 \text{ et } \hat{y} = -1 \\ w - x & \text{si } y = -1 \text{ et } \hat{y} = 1 \end{cases} \iff w = w + yx \text{ si } x \text{ est mal classé.}$$

Remarque (Variantes) Notre définition de l'actualisation des poids les modifie à chaque itération lors de l'apprentissage. Cela permet une convergence rapide de l'apprentissage, mais peut conduire à une instabilité si le nombre de poids à traiter est trop grand. Une solution est donc d'effectuer un *apprentissage par batch*. Cela consiste à faire passer successivement plusieurs vecteurs x , on calcule ensuite l'erreur moyenne commise et on met à jour les poids en fonction de cette erreur moyenne. Un autre possibilité est d'utiliser un *learning rate* ν qui bride l'apprentissage. On utilise alors la règle suivante pour actualiser w :

$$w = \eta(y - \hat{y})x.$$

Les perceptrons permettent donc de simuler des fonctions booléennes comme illustré ci-dessous :

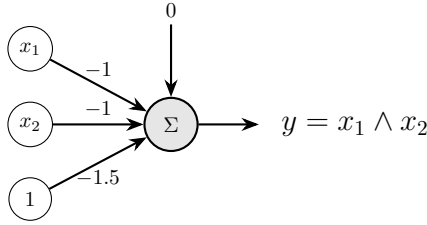


FIGURE 1.2 – Fonction AND avec un perceptron

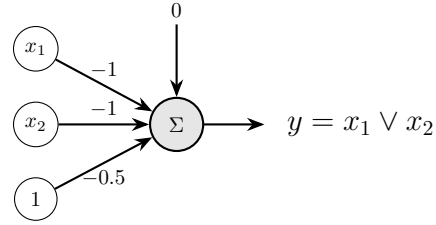


FIGURE 1.3 – Fonction OR avec un perceptron

Après réflexion, on se rend compte qu'il est impossible de simuler la fonction XOR avec un seul neurone. Ce sera donc l'objectif des *MLP (Multi Layer Perceptron)* composés de plusieurs couches de neurones interconnectés.

1.2.3 Perceptron Multi-Couches (MLP)

Comme énoncé précédemment, le perceptron simple de Rosenblatt présente une limitation fondamentale mise en lumière par Marvin Minsky et Seymour Papert en 1969 : l'impossibilité géométrique de séparer des classes qui ne sont pas linéairement séparables dans l'espace d'entrée $\mathbb{R}^d$, à l'instar de la fonction logique XOR (ou exclusif).

Pour contourner ce problème, l'approche naturelle consiste à empiler plusieurs couches de neurones formels. Un *Perceptron Multi-Couches* (Multi-Layer Perceptron, noté MLP) est composé :

1. D'une couche d'entrée (vecteur $x \in \mathbb{R}^d$) ;
2. D'une suite de $L \geq 1$ **couches cachées** (hidden layers), qui apprennent des représentations hiérarchiques distribuées ;
3. D'une couche de sortie produisant la prédiction $\hat{y}$.

Définition (Formalisation de la passe avant d'un MLP) . Soit un réseau à L couches cachées. Notons $h^{(0)} = x \in \mathbb{R}^{d_0}$ le vecteur d'entrée. Pour chaque couche $k \in \llbracket 1, L \rrbracket$ de dimension d_k , le vecteur d'activation est régi par les relations de récurrence :

$$a^{(k)} = W^{(k)}h^{(k-1)} + b^{(k)} \in \mathbb{R}^{d_k} \quad (\text{pré-activation}) \quad (1.2.1)$$

$$h^{(k)} = g^{(k)}(a^{(k)}) \in \mathbb{R}^{d_k} \quad (\text{activation}) \quad (1.2.2)$$

où $W^{(k)} \in \mathbb{R}^{d_k \times d_{k-1}}$ désigne la matrice de poids de la couche k , $b^{(k)} \in \mathbb{R}^{d_k}$ le vecteur de biais associé, et $g^{(k)} : \mathbb{R} \rightarrow \mathbb{R}$ une fonction d'activation non linéaire appliquée composante par composante. Pour la couche de sortie $k = L + 1$, on applique une fonction terminale o :

$$\hat{y} = h^{(L+1)} = o\left(W^{(L+1)}h^{(L)} + b^{(L+1)}\right). \quad (1.2.3)$$

Remarque (Interprétation géométrique des frontières de décision) Chaque neurone d'une première couche cachée partitionne l'espace $\mathbb{R}^d$ par un demi-espace délimité par un hyperplan affine. Les neurones de la deuxième couche cachée effectuent une intersection booléenne (opérateur ET) de ces demi-espaces, formant ainsi des polyèdres convexes. Enfin, les couches suivantes peuvent réaliser l'union (opérateur OU) de régions convexes disjointes, rendant le réseau théoriquement capable d'approximer n'importe quelle frontière de décision fermée ou ouverte arbitrairement complexe.

1.3 Fonctions d'Activation et Classification Multiclasse

1.3.1 Fonctions d'activation usuelles

Le choix d'une fonction d'activation non linéaire g est indispensable : sans elle, la composition successive d'opérations affines $W^{(L)}(\dots(W^{(1)}x + b^{(1)})\dots) + b^{(L)}$ resterait strictement équivalente à une unique transformation affine globale $\tilde{W}x + \tilde{b}$, ruinant tout l'intérêt de la profondeur.

Définition (Fonctions d'activation courantes) . On définit ainsi différentes fonctions d'activations communément utilisées en sortie d'un neurone. Pour tout réel $z \in \mathbb{R}$ on a :

- **Sigmoïde (logistique)** :

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1.3.1)$$

Elle projette $\mathbb{R}$ sur $]0, 1[$ et vérifie l'identité remarquable $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

- **Tangente hyperbolique** :

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in]-1, 1[, \quad (1.3.2)$$

centrée en zéro, de dérivée $\tanh'(z) = 1 - \tanh^2(z)$.

- **Rectified Linear Unit (ReLU)** :

$$g(z) = \max(0, z). \quad (1.3.3)$$

Sa dérivée vaut 1 pour $z > 0$ et 0 pour $z < 0$. Elle évite la saturation des gradients pour des entrées positives.

- **Leaky ReLU** : pour $\alpha \in]0, 1[$ (couramment $\alpha = 0,01$), $g(z) = \max(\alpha z, z)$. Elle évite le problème des "neurones morts" lorsque $z < 0$.

- **Softplus :**

$$g(z) = \log(1 + e^z), \quad (1.3.4)$$

approximation analytique différentiable de la ReLU.

1.3.2 Classification multiclass et fonction Softmax

Considérons un problème de classification à $K > 2$ classes mutuellement exclusives. La couche finale produit un vecteur de scores réels non normalisés $a = W^{(L+1)}h^{(L)} + b^{(L+1)} \in \mathbb{R}^K$, appelés *logits*.

Définition (Opérateur Softmax) . L'opérateur Softmax normalise un vecteur de logits $a = (a_1, \dots, a_K)^\top$ en une distribution discrète de probabilités $p = (p_1, \dots, p_K)^\top \in [0, 1]^K$ définie par :

$$\forall c \in \llbracket 1, K \rrbracket, \quad p_c = \text{Softmax}(a)_c = \frac{e^{a_c}}{\sum_{j=1}^K e^{a_j}}. \quad (1.3.5)$$

On a immédiatement $\forall c, p_c > 0$ et $\sum_{c=1}^K p_c = 1$. La classe prédite est $\hat{y} = \arg \max_c p_c$.

1.4 Fonctions de Perte (*Loss Functions*)

En apprentissage supervisé, puisque nous connaissons les *labels* associés à chaque prédiction, nous sommes capables d'évaluer le modèle en temps réel sur chacune de ses prédictions sur les données d'entraînement. Ainsi, l'évaluation de la discordance entre la cible réelle y et la sortie estimée $\hat{y}$ s'effectue au moyen d'une fonction de coût différentiable $\mathcal{L}(y, \hat{y})$ que l'on appellera *fonction de perte* ou *loss*.

1.4.1 Entropie croisée binaire (BCE)

Remarque (Limites de la MSE) Pour une classification binaire avec $y \in \{0, 1\}$ et une prédiction de probabilité $\hat{y} = \sigma(z) \in]0, 1[$, l'erreur quadratique usuelle MSE s'avère sous-optimale du fait de zones de saturation où le gradient s'annule pour des erreurs maximales. On introduit la perte d'entropie croisée binaire.

Définition (Binary Cross-Entropy) . Pour $y \in \{0, 1\}$ et $\hat{y} \in]0, 1[$:

$$\mathcal{L}_{\text{BCE}}(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y}). \quad (1.4.1)$$

Proposition (Stabilité numérique de la BCE) En injectant l'activation logistique $\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$ directement dans la perte, on évite les dépassements de capacité flottante (*overflow*) via l'expression analytique exacte :

$$\mathcal{L}_{\text{BCE}}(y, \sigma(z)) = \max(z, 0) - z \cdot y + \log(1 + e^{-|z|}). \quad (1.4.2)$$

1.5 Rétropropagation du Gradient et Dérivation Automatique

Comme vu au début de ce chapitre, l'apprentissage se fait par une recherche des "meilleurs" poids possibles pour le modèle dans l'objectif de minimiser l'erreur commise. Maintenant que

nous avons introduit la fonction de perte *loss*, nous cherchons donc à faire une minimisation de cette fonction *loss*.

La principale problématique dans ce processus est le calcul de son *gradient*. En effet, pour trouver des candidats minimiseurs, la condition d'optimalité d'ordre 1 nous demande d'annuler ce gradient. Nous devons donc être capable de calculer les dérivées partielles de la *loss* en fonction des poids du modèle. On utilise pour cela l'algorithme de *rétropropagation* à l'aide de *graphes computationnels*.

1.5.1 Rétropropagation sur un neurone formel

Considérons un unique neurone formel doté d'une entrée $x \in \mathbb{R}^d$, d'une pré-activation $a = w^\top x + b$, d'une activation $\hat{y} = \sigma(a)$, et d'une perte quadratique $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$. Par application directe de la règle de dérivation en chaîne (*Chain Rule*) :

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{d\hat{y}}{da} \cdot \frac{\partial a}{\partial w_i} = -(y - \hat{y}) \cdot \hat{y}(1 - \hat{y}) \cdot x_i. \quad (1.5.1)$$

On retrouve le terme d'erreur $(y - \hat{y})$, pondéré par la pente de la sigmoïde $\hat{y}(1 - \hat{y})$ et la projection d'entrée x_i . Si l'on remplace la perte quadratique par la $\mathcal{L}_{\text{BCE}}$, le terme $\hat{y}(1 - \hat{y})$ se simplifie élégamment au dénominateur :

$$\frac{\partial \mathcal{L}_{\text{BCE}}}{\partial a} = \hat{y} - y \implies \frac{\partial \mathcal{L}_{\text{BCE}}}{\partial w} = (\hat{y} - y)x. \quad (1.5.2)$$

1.5.2 Rétropropagation dans un MLP (*Backpropagation*)

Dans un graphe multicouches, les erreurs locales sont propagées de l'aval vers l'amont :

Définition (Rétropropagation multicouche) . Posons $\delta^{(k)} = \frac{\partial \mathcal{L}}{\partial a^{(k)}}$ le vecteur de sensibilité de la couche k .

$$\text{Couche de sortie } (L + 1) : \quad \delta^{(L+1)} = \nabla_{a^{(L+1)}} \mathcal{L} \quad (1.5.3)$$

$$\text{Couches cachées } (k = L, \dots, 1) : \quad \delta^{(k)} = \left((W^{(k+1)})^\top \delta^{(k+1)} \right) \odot g'^{(k)} \left(a^{(k)} \right) \quad (1.5.4)$$

où $\odot$ désigne le produit d'Hadamard (terme à terme). Les gradients s'en déduisent immédiatement :

$$\frac{\partial \mathcal{L}}{\partial W^{(k)}} = \delta^{(k)} \left(h^{(k-1)} \right)^\top \in \mathbb{R}^{d_k \times d_{k-1}}, \quad \frac{\partial \mathcal{L}}{\partial b^{(k)}} = \delta^{(k)} \in \mathbb{R}^{d_k}. \quad (1.5.5)$$

1.5.3 Graphes de calcul et implémentation PyTorch

En pratique, le calcul symbolique manuel des dérivées est remplacé par l'auto-différenciation en mode inverse (*reverse-mode automatic differentiation*) matérialisée par un graphe acyclique orienté (DAG).

```

1 import torch
2 import torch.nn as nn
3
4 class MultiLayerPerceptron(nn.Module):
5     def __init__(self, in_features: int, hidden_dim: int, out_classes: int):
6         super(MultiLayerPerceptron, self).__init__()
7         self.network = nn.Sequential(
8             nn.Linear(in_features, hidden_dim),
9             nn.ReLU(),
10            nn.Linear(hidden_dim, hidden_dim),

```

```

11         nn.ReLU(),
12         nn.Linear(hidden_dim, out_classes)
13     )
14
15     def forward(self, x: torch.Tensor) -> torch.Tensor:
16         # Retourne les logits bruts (non normalisés)
17         return self.network(x)
18
19 # Instanciation et passe avant/arrière
20 model = MultilayerPerceptron(in_features=784, hidden_dim=128, out_classes=10)
21 criterion = nn.CrossEntropyLoss() # Integre log-softmax et NLL
22 optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
23
24 x_dummy = torch.randn(32, 784) # Batch de 32 observations
25 y_target = torch.randint(0, 10, (32,))
26
27 optimizer.zero_grad()
28 logits = model(x_dummy)
29 loss = criterion(logits, y_target)
30 loss.backward() # Calcul automatique des gradients via Autograd
31 optimizer.step()

```

Listing 1.1 – Implémentation d'un MLP complet sous PyTorch

1.5.4 Optimisation par descente de gradient avec inertie (Momentum)

La descente de gradient stochastique standard (SGD) présente de fortes oscillations transverses dans les vallées mal conditionnées (matrice hessienne à fort conditionnement). La méthode avec moment régularise la trajectoire en accumulant une moyenne exponentielle glissante des vitesses passées :

$$v_t = \beta v_{t-1} + \alpha \nabla_W \mathcal{L}_t \quad \text{avec } \beta \in [0, 1[\quad (1.5.6)$$

$$W_{t+1} = W_t - v_t \quad (1.5.7)$$

où $\alpha > 0$ représente le pas d'apprentissage (*learning rate*).

Chapitre 2

Réseaux Convolutifs

Contents

2.1	L'Opérateur de Convolution en Vision Artificielle	150
2.1.1	Formulation mathématique	151
2.1.2	Dimensions, Stride, Padding et Dilatation	151
2.2	Sous-Échantillonnage, Champ Réceptif et Calcul	151
2.2.1	Couches de Pooling	152
2.2.2	Notion de champ réceptif (<i>Receptive Field</i>)	152
2.2.3	Algorithmique Bas Niveau : L'Approche Im2Col et GEMM	152
2.3	Techniques de Normalisation et Réseaux Résiduels	152
2.3.1	Normalisation par Lots (<i>Batch Normalization</i>)	152
2.3.2	Réseaux Résiduels (ResNet)	153
2.3.3	Implémentation d'un bloc résiduel sous PyTorch	153

Dans le cadre du traitement d'images numériques, l'utilisation de couches entièrement connectées (MLP) devient vite impraticable :

1. **Explosion du nombre de paramètres** : pour une image couleur haute définition de taille $1000 \times 1000 \times 3$, un premier neurone connecté à chaque pixel requiert 3×10^6 poids, induisant un coût mémoire prohibitif et un sur-apprentissage critique.
2. **Perte de corrélation spatiale** : vectoriser une image 2D/3D détruit la proximité géométrique locale entre pixels adjacents.
3. **Absence d'invariance par translation** : si un motif d'intérêt (un œil, une roue) se déplace dans l'image, un MLP doit réapprendre ce motif sur chacun des emplacements distincts.

La solution réside dans l'exploitation de la structure spatiale via deux principes directeurs : la *connectivité locale* (chaque neurone n'observe qu'un sous-voisinage) et le *partage des paramètres* (weight sharing).

2.1 L'Opérateur de Convolution en Vision Artificielle

L'objectif est donc d'être capable d'identifier des schémas caractéristiques dans une image, indépendamment de sa position dans celle-ci. Pour cela, on utilisera un opérateur de *convolution*. L'idée est de balayer la surface de l'image par une matrice plus petite appelée *noyau*. À chaque déplacement, on effectuera le produit de la partie de l'image couverte par le noyau. L'image ainsi obtenue sera plus petite et contiendra la même information mais sous forme condensée.

En effectuant cette étape de façon successive, on sera capable de réduire la taille de l'image d'origine à celle d'un vecteur qui pourra être facilement interprété par un MLP.

2.1.1 Formulation mathématique

En traitement du signal numérique, l'opération discrète bidimensionnelle appliquée entre une entrée matricielle I et un noyau (*kernel*) $K \in \mathbb{R}^{k_h \times k_w}$ est définie usuellement sous la forme d'une corrélation croisée :

$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I(i+m, j+n) K(m, n). \quad (2.1.1)$$

Dans une couche de convolution standard d'un CNN, l'entrée est un tenseur à trois dimensions $\mathcal{X} \in \mathbb{R}^{C_{\text{in}} \times H_{\text{in}} \times W_{\text{in}}}$ où C_{in} est le nombre de canaux (ex : Rouge, Vert, Bleu). La couche est paramétrée par C_{out} filtres tridimensionnels $W_k \in \mathbb{R}^{C_{\text{in}} \times k_h \times k_w}$ et un vecteur de biais $b \in \mathbb{R}^{C_{\text{out}}}$. La k -ième carte de caractéristiques de sortie s'exprime par :

$$S_k = b_k + \sum_{c=1}^{C_{\text{in}}} \mathcal{X}_c * W_{k,c}. \quad (2.1.2)$$

2.1.2 Dimensions, Stride, Padding et Dilatation

Propriété (Régression des dimensions spatiales) . Soit une entrée de taille $(H_{\text{in}}, W_{\text{in}})$, convoluée avec un noyau de taille (K_h, K_w) , un rembourrage (*padding*) P , un pas (*stride*) S , et un taux de dilatation (*dilation*) D . La taille de la carte de sortie $(H_{\text{out}}, W_{\text{out}})$ vérifie :

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P_h - D_h(K_h - 1) - 1}{S_h} + 1 \right\rfloor, \quad (2.1.3)$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P_w - D_w(K_w - 1) - 1}{S_w} + 1 \right\rfloor. \quad (2.1.4)$$

Proposition (Nombre de paramètres d'une couche convolutive) Pour une couche convolutive associant C_{in} canaux d'entrée à C_{out} canaux de sortie au travers de noyaux $K_h \times K_w$, le nombre de paramètres scalaires libres vaut :

$$N_{\text{params}} = C_{\text{out}} \times (C_{\text{in}} \times K_h \times K_w + 1) \quad (2.1.5)$$

où le terme $+1$ comptabilise le biais scalaire associé à chaque carte de sortie. Ce nombre est strictement indépendant de la résolution spatiale $(H_{\text{in}}, W_{\text{in}})$ de l'image.

Remarque Les couches de convolutions servent donc à *extraire* des schémas (*pattern*) et les caractéristiques (*features*) de l'image et de les compresser en vecteurs. Ensuite, un MLP est chargé d'*interpréter* ces *features* en prédiction (voir chapitre précédent). L'utilisation des convolutions permet de réduire drastiquement le nombre de paramètres et de couches à utiliser pour l'extraction de ces *features*.

2.2 Sous-Échantillonnage, Champ Réceptif et Calcul

L'opération de convolution est très simple à mettre en place algorithmiquement. En fonction de l'opération effectuée avec son résultat et de la dimension de son noyau, on peut observer plusieurs propriétés remarquables. Elles permettent ensuite de définir de nouvelles couches de convolutions ayant différents cas d'usages.

2.2.1 Couches de Pooling

Les couches de regroupement (*pooling*) réduisent la résolution spatiale des tenseurs intermédiaires pour accroître l'invariance aux translations locales et diminuer la charge computationnelle :

- **Max-Pooling** : extrait la valeur maximale sur une fenêtre $p_h \times p_w$. Il conserve le signal d'activation dominant tout en rejetant les variations de fond.
- **Average-Pooling** : calcule la moyenne arithmétique sur la fenêtre, agissant comme un filtre passe-bas régularisant.

2.2.2 Notion de champ réceptif (*Receptive Field*)

Le champ réceptif d'un neurone correspond à la zone d'extension dans l'image d'entrée d'origine qui influence directement son activation.

Proposition (Croissance du champ réceptif) Pour une succession de couches convolutives à noyaux de taille k_l et de pas unitaire, le champ réceptif cumulé R_L à la couche L vérifie :

$$R_L = R_{L-1} + (k_L - 1) = 1 + \sum_{l=1}^L (k_l - 1). \quad (2.2.1)$$

Ainsi, l'empilement de deux convolutions successives de taille 3×3 possède un champ réceptif équivalent de 5×5 , tout en utilisant $2 \times (3^2) = 18$ poids au lieu de $5^2 = 25$ pour un noyau direct, tout en introduisant deux non-linéarités intermédiaires au lieu d'une.

2.2.3 Algorithmique Bas Niveau : L'Approche Im2Col et GEMM

Les unités graphiques de calcul (GPU) n'exécutent pas les convolutions spatiales par boucles imbriquées, mais projettent l'opération sous forme de multiplication matricielle dense hautement vectorisée (*General Matrix Multiply*, GEMM) :

1. **L'algorithme Im2Col** déplie chaque bloc spatial réceptif $C_{in} \times K_h \times K_w$ présent dans le tenseur d'entrée en un vecteur colonne, formant une vaste matrice :

$$X_{col} \in \mathbb{R}^{(C_{in} \cdot K_h \cdot K_w) \times (H_{out} \cdot W_{out})}. \quad (2.2.2)$$

2. Les poids des filtres sont aplatis en lignes pour construire une matrice :

$$W_{row} \in \mathbb{R}^{C_{out} \times (C_{in} \cdot K_h \cdot K_w)}. \quad (2.2.3)$$

3. Le produit matriciel dense calcule l'ensemble des réponses en une passe :

$$Y_{flat} = W_{row} \cdot X_{col} \in \mathbb{R}^{C_{out} \times (H_{out} \cdot W_{out})}. \quad (2.2.4)$$

4. La matrice résultante est réorganisée (*reshapée*) en dimension $(C_{out}, H_{out}, W_{out})$.

2.3 Techniques de Normalisation et Réseaux Résiduels

2.3.1 Normalisation par Lots (*Batch Normalization*)

Pour stabiliser la covariance interne des couches profondes (*Internal Covariate Shift*), on introduit la couche de Batch Normalization.

Définition (Batch Normalization) . Pour un mini-lot d’activations scalaires $\mathcal{B} = \{x_1, \dots, x_m\}$ correspondant à un canal donné :

$$\mu_{\mathcal{B}} = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_{\mathcal{B}}^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad (2.3.1)$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad (2.3.2)$$

$$y_i = \gamma \hat{x}_i + \beta \quad \text{avec } \gamma, \beta \text{ paramètres différentiables appris.} \quad (2.3.3)$$

2.3.2 Réseaux Résiduels (ResNet)

Au-delà d’une vingtaine de couches, l’entraînement d’un réseau standard subit une dégradation critique due à l’évanescence des gradients. He et al. (2015) introduisent les connexions résiduelles directes (*skip connections*) :

$$\mathcal{H}(x) = \mathcal{F}(x, \{W_i\}) + x \quad (2.3.4)$$

où $\mathcal{F}$ représente la transformation non linéaire de quelques couches convolutives. Si la solution optimale est l’identité, le réseau n’a besoin que d’annuler les poids $\mathcal{F}(x) \rightarrow 0$, ce qui est infiniment plus aisé à apprendre. Lors de la rétropropagation, le gradient transite sans atténuation multiplicative via la dérivation directe :

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \left(\frac{\partial \mathcal{F}}{\partial x} + I \right). \quad (2.3.5)$$

2.3.3 Implémentation d’un bloc résiduel sous PyTorch

```

1 import torch
2 import torch.nn as nn
3
4 class ResidualBlock(nn.Module):
5     def __init__(self, channels: int):
6         super(ResidualBlock, self).__init__()
7         self.conv_block = nn.Sequential(
8             nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1,
9             bias=False),
10            nn.BatchNorm2d(channels),
11            nn.ReLU(inplace=True),
12            nn.Conv2d(channels, channels, kernel_size=3, stride=1, padding=1,
13            bias=False),
14            nn.BatchNorm2d(channels)
15        )
16        self.relu = nn.ReLU(inplace=True)
17
18    def forward(self, x: torch.Tensor) -> torch.Tensor:
19        identity = x
20        out = self.conv_block(x)
21        out += identity # Connexion résiduelle directe
22        out = self.relu(out)
23        return out

```

Listing 2.1 – Bloc résiduel classique sous PyTorch

Chapitre 3

Réseaux Récurrents

Contents

3.1	Modélisation Séquentielle et Réseaux Récurrents (RNN)	154
3.1.1	Architecture et Dépliage Temporel	154
3.2	Rétropropagation à travers le Temps (BPTT) et Pathologies	155
3.2.1	Calcul du gradient BPTT	155
3.2.2	Évanescence et explosion du gradient	155
3.3	Cellules à Portes de Contrôle : LSTM et GRU	155
3.3.1	Long Short-Term Memory (LSTM)	155
3.3.2	Gated Recurrent Unit (GRU)	156
3.4	Modèles Encodeur-Décodeur et Mécanismes d'Attention	156
3.4.1	Le paradigme Sequence-to-Sequence (Seq2Seq)	156
3.4.2	Mécanisme d'attention d'alignement dynamique	156
3.4.3	Implémentation PyTorch d'un module Seq2Seq récurrent	156

3.1 Modélisation Séquentielle et Réseaux Récurrents (RNN)

Contrairement aux images dont les dimensions peuvent être ajustées à des formats fixes, les signaux temporels, acoustiques ou textuels se présentent sous la forme de séquences de longueur variable :

$$X = (x^{(1)}, x^{(2)}, \dots, x^{(T)}), \quad x^{(t)} \in \mathbb{R}^d. \quad (3.1.1)$$

3.1.1 Architecture et Dépliage Temporel

Un réseau récurrent traite séquentiellement chaque élément en maintenant un vecteur d'état interne $h^{(t)} \in \mathbb{R}^m$, constituant la mémoire synthétique du passé.

Définition (Équations du RNN simple d'Elman) . Pour chaque pas temporel $t \in \llbracket 1, T \rrbracket$:

$$h^{(t)} = \tanh(W h^{(t-1)} + U x^{(t)} + b_h) \quad (3.1.2)$$

$$\hat{y}^{(t)} = \text{Softmax}(V h^{(t)} + b_y) \quad (3.1.3)$$

où $U \in \mathbb{R}^{m \times d}$ est la matrice de projection des entrées, $W \in \mathbb{R}^{m \times m}$ la matrice de récurrence

état-à-état, et $V \in \mathbb{R}^{K \times m}$ la matrice de décodage vers l'espace de sortie. Les paramètres $\{U, W, V, b_h, b_y\}$ sont **partagés** à travers tous les pas de temps.

3.2 Rétropropagation à travers le Temps (*BPTT*) et Pathologies

3.2.1 Calcul du gradient BPTT

L'erreur globale commise sur la séquence est la somme des coûts temporels $E = \sum_{t=1}^T E_t$. Pour dériver le gradient par rapport à la matrice de récurrence W , on déroule la dépendance temporelle :

$$\frac{\partial E}{\partial W} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial E_t}{\partial h^{(t)}} \cdot \left(\prod_{j=k+1}^t \frac{\partial h^{(j)}}{\partial h^{(j-1)}} \right) \cdot \frac{\partial h^{(k)}}{\partial W}. \quad (3.2.1)$$

3.2.2 Évanescence et explosion du gradient

Le produit matriciel cumulé $\prod_{j=k+1}^t \frac{\partial h^{(j)}}{\partial h^{(j-1)}}$ gouverne la propagation temporelle du signal d'erreur. Si la matrice jacobienne présente des valeurs singulières strictement supérieures à 1, la norme du gradient diverge exponentiellement vers l'infini pour de grandes séquences : c'est l'**explosion du gradient**. On y remédie par la troncature heuristique (*Gradient Clipping*) :

$$g \leftarrow g \cdot \min \left(1, \frac{\text{seuil}}{\|g\|_2} \right). \quad (3.2.2)$$

Inversement, si les valeurs propres sont inférieures à 1, le gradient s'évanouit exponentiellement vers zéro au fur et à mesure que $t - k$ croît (**dissipation du gradient**), interdisant l'apprentissage de corrélations à long terme.

3.3 Cellules à Portes de Contrôle : LSTM et GRU

3.3.1 Long Short-Term Memory (LSTM)

Conçue par Hochreiter et Schmidhuber (1997), l'architecture LSTM résout l'évanescence du gradient via un état cellulaire interne C_t mis à jour par des opérations additives protégées par des portes logiques différentiables.

Définition (Formulation complète d'un bloc LSTM) . Soit x_t l'entrée au temps t et h_{t-1} l'état caché antérieur. Les portes sont régies par :

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{porte d'oubli / Forget Gate}) \quad (3.3.1)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{porte d'entrée / Input Gate}) \quad (3.3.2)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (\text{état candidat}) \quad (3.3.3)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{mise à jour cellulaire additive}) \quad (3.3.4)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{porte de sortie / Output Gate}) \quad (3.3.5)$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{état caché final}). \quad (3.3.6)$$

Grâce à la relation linéaire $C_t = f_t \odot C_{t-1} + \dots$, si la porte d'oubli est activée à saturation ($f_t \approx 1$), le gradient d'erreur se propage à travers l'axe temporel sans aucune atténuation multiplicative exponentielle.

3.3.2 Gated Recurrent Unit (GRU)

Proposé par Cho et al. (2014), le bloc GRU fusionne l'état cellulaire et l'état caché tout en réduisant le nombre de portes à deux :

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (\text{porte de mise à jour / Update Gate}) \quad (3.3.7)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (\text{porte de réinitialisation / Reset Gate}) \quad (3.3.8)$$

$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t] + b) \quad (3.3.9)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (3.3.10)$$

3.4 Modèles Encodeur-Décodeur et Mécanismes d'Attention

3.4.1 Le paradigme Sequence-to-Sequence (Seq2Seq)

Dans une tâche de traduction ou de résumé, la longueur de la séquence générée est distincte de la séquence source. L'encodeur compresse l'intégralité de la séquence d'entrée en un vecteur contexte unique $c = h^{(T_{in})}$. Le décodeur génère les sorties mot par mot en conditionnant chaque tirage sur ce vecteur. Ce goulot d'étranglement unique entraîne une forte détérioration des performances lorsque la phrase dépasse une vingtaine de mots.

3.4.2 Mécanisme d'attention d'alignement dynamique

L'attention (Bahdanau et al., 2015 ; Luong et al., 2015) lève cette contrainte en permettant au décodeur de consulter l'ensemble des états intermédiaires de l'encodeur $(s_1, \dots, s_S)$ à chaque pas de génération t .

Définition (Calcul du vecteur de contexte par attention) . Pour un état courant du décodeur h_t et un état source s_k :

$$\text{Score d'alignement : } e_{t,k} = \begin{cases} v_a^\top \tanh(W_a h_t + U_a s_k) & (\text{Attention additive de Bahdanau}) \\ h_t^\top W_a s_k & (\text{Attention générale de Luong}) \\ h_t^\top s_k & (\text{Attention par produit scalaire direct}) \end{cases} \quad (3.4.1)$$

$$\text{Poids d'attention : } \alpha_{t,k} = \frac{\exp(e_{t,k})}{\sum_{j=1}^S \exp(e_{t,j})} \in [0, 1] \quad (3.4.2)$$

$$\text{Vecteur de contexte : } c_t = \sum_{k=1}^S \alpha_{t,k} s_k. \quad (3.4.3)$$

3.4.3 Implémentation PyTorch d'un module Seq2Seq récurrent

```
1 import torch
2 import torch.nn as nn
3
4 class Seq2SeqRNN(nn.Module):
5     def __init__(self, vocab_size: int, embed_dim: int, hidden_dim: int):
6         super(Seq2SeqRNN, self).__init__()
7         self.embedding = nn.Embedding(vocab_size, embed_dim)
8         self.encoder = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
9                                 bidirectional=True)
```

```
9         # Decodeur prenant en compte la bidirectionnalite (hidden_dim * 2)
10         self.decoder = nn.LSTM(embed_dim, hidden_dim * 2, batch_first=True)
11         self.fc_out = nn.Linear(hidden_dim * 2, vocab_size)
12
13     def forward(self, src: torch.Tensor, trg: torch.Tensor) -> torch.Tensor:
14         # Encodage
15         embedded_src = self.embedding(src)
16         enc_outputs, (h_n, c_n) = self.encoder(embedded_src)
17
18         # Concatener les etats terminaux des deux directions de l'encodeur
19         h_dec = torch.cat((h_n[0:1], h_n[1:2]), dim=2)
20         c_dec = torch.cat((c_n[0:1], c_n[1:2]), dim=2)
21
22         # Decodage autoregressif
23         embedded_trg = self.embedding(trg)
24         dec_outputs, _ = self.decoder(embedded_trg, (h_dec, c_dec))
25         logits = self.fc_out(dec_outputs)
26         return logits
```

Listing 3.1 – Encodeur LSTM et Décodeur avec projection sous PyTorch

Chapitre 4

Modèles de Markov Caché

Contents

4.1	Modèles de Langage et Hypothèses Markoviennes	158
4.1.1	Modélisation séquentielle par règle de la chaîne	158
4.1.2	Du Sac de Mots au Modèle de Markov d'ordre m	159
4.1.3	Modèles à classes de Brown	160
4.2	Formalisme des Modèles de Markov Cachés (HMM)	160
4.2.1	Définition mathématique	160
4.2.2	Les trois problèmes fondamentaux de Rabiner (1989)	161
4.3	Résolution Algorithmique des Trois Problèmes de Rabiner	161
4.3.1	Problème 1 : Évaluation et Algorithme Forward-Backward	161
4.3.2	Problème 2 : Décodage et Inférence	163
4.3.3	Stabilité Numérique et Domaine Logarithmique	164
4.3.4	Problème 3 : Apprentissage des Paramètres	165

Ce chapitre traite des *Modèles de Markov Cachés* (Hidden Markov Models, HMM). Historiquement développés pour le traitement automatique de la parole (*Automatic Speech Recognition*, Rabiner, 1989) et le traitement du langage naturel (étiquetage morpho-syntaxique *POS tagging*, modèles de langage, alignement bilingue), les HMM offrent un cadre probabiliste rigoureux pour les séquences discrètes. Bien que les modèles de fondation neuronaux (Transformers) dominent aujourd'hui ces applications, les algorithmes sous-jacents aux HMM (Forward-Backward, Viterbi, Baum-Welch/EM) constituent les bases fondamentales des modèles graphiques probabilistes et de la programmation dynamique.

4.1 Modèles de Langage et Hypothèses Markoviennes

Dans ce cours, on introduit les HMM en tant que modèles de langage. On comment ici par formaliser mathématiquement ces modèles en utilisant les chaînes de Markov.

4.1.1 Modélisation séquentielle par règle de la chaîne

Soit $\mathcal{V}$ un vocabulaire fini de symboles (mots ou *tokens*). On cherche à définir une loi de probabilité conjointe sur des séquences de longueur arbitraire bordées par deux balises déterministes start et stop :

$$O = (\text{start}, O_1, O_2, \dots, O_n, \text{stop}) \in \mathcal{V}^*. \quad (4.1.1)$$

Par application exacte de la règle de décomposition en chaîne des probabilités conditionnelles, toute distribution sur une séquence s'écrit de manière causale (contexte gauche) :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \text{start}, O_1, \dots, O_{i-1}) \quad (4.1.2)$$

en posant conventionnellement $O_{n+1} = \text{stop}$.

Remarque (Intuition des dés de Noah Smith et explosion combinatoire) On peut interpréter ce processus génératif à l'aide d'une collection de dés dont les faces représentent les mots de $\mathcal{V}$. À chaque étape, l'historique complet $h = (\text{start}, O_1, \dots, O_{i-1})$ détermine le dé à lancer, dont la distribution sur les faces fournit le mot suivant O_i . Bien que d'une expressivité totale, ce modèle présente deux écueils rédhibitoires :

1. **Explosion paramétrique** : pour un historique de longueur au plus n et un vocabulaire de taille $V = |\mathcal{V}|$, le nombre d'historiques possibles vaut $\sum_{k=0}^{n-1} V^k = \frac{V^n - 1}{V - 1}$. Pour $V = 10^5$ et $n = 10$, cela exigerait de stocker les paramètres d'environ 10^{50} distributions catégorielles, ce qui est matériellement impossible.
2. **Défaut de généralisation** : toute séquence contenant un préfixe non observé lors de l'apprentissage se voit affecter une probabilité nulle.

4.1.2 Du Sac de Mots au Modèle de Markov d'ordre m

Pour rendre l'estimation calculable, on réduit l'historique conditionnel via des hypothèses d'indépendance conditionnelle.

Définition (Modèle Sac de Mots / Unigramme) . L'hypothèse la plus forte pose l'indépendance mutuelle totale de chaque observation vis-à-vis du contexte passé :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^n \mathbb{P}(O_i). \quad (4.1.3)$$

Ce modèle ne requiert que $V - 1$ paramètres libres. En revanche, il détruit toute syntaxe : la probabilité d'une séquence est invariante par permutation de ses termes.

Le compromis canonique consiste à borner la mémoire du processus aux m derniers symboles émis.

Définition (Chaîne de Markov d'ordre m / n -grammes) . Un modèle de Markov stationnaire d'ordre $m \geq 1$ pose que la loi conditionnelle du token courant ne dépend que des m tokens immédiatement antérieurs :

$$\mathbb{P}(O_i \mid \text{start}, O_1, \dots, O_{i-1}) = \mathbb{P}(O_i \mid O_{i-m}, \dots, O_{i-1}). \quad (4.1.4)$$

La probabilité conjointe de la séquence s'écrit alors :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid O_{i-m}^{i-1}) \quad (4.1.5)$$

où $O_{i-m}^{i-1} = (O_{i-m}, \dots, O_{i-1})$.

Remarque (Taxonomie et complexité des n -grammes) Dans la terminologie du traitement automatique du langage :

- $m = 0$ correspond au modèle *unigramme* ($V - 1$ paramètres) ;
- $m = 1$ correspond au modèle *bigramme* ($V(V - 1)$ paramètres de transition) ;
- $m = 2$ correspond au modèle *trigramme* ($V^2(V - 1)$ paramètres).

Au-delà de $m = 3$ ou 4 , la matrice des comptes devient extrêmement creuse (*sparsity*), rendant indispensables des techniques de lissage statistique (Good-Turing, Kneser-Ney).

4.1.3 Modèles à classes de Brown

Pour injecter de la régularité syntaxique sans subir l'explosion combinatoire des n -grammes lexicaux, Brown et al. (1992) partitionnent le vocabulaire $\mathcal{V}$ en $C \ll V$ classes syntaxiques disjointes via une application déterministe $\text{cl} : \mathcal{V} \rightarrow \llbracket 1, C \rrbracket$ obtenue par partitionnement statistique :

Définition (Modèle de séquence fondé sur des classes) . La génération d'une observation O_i est scindée en deux lois : une transition d'état à état entre classes, suivie de l'émission du mot conditionnellement à sa propre classe :

$$\mathbb{P}(\text{start}, O_1, \dots, O_n, \text{stop}) = \prod_{i=1}^{n+1} \mathbb{P}(O_i \mid \text{cl}(O_i)) \cdot \mathbb{P}(\text{cl}(O_i) \mid \text{cl}(O_{i-1})). \quad (4.1.6)$$

En notant $A \in \mathbb{R}^{C \times C}$ la matrice de transition inter-classes et $B \in \mathbb{R}^{C \times V}$ la matrice d'émission des mots sachant la classe, ce modèle ne requiert que $C^2 + C \times V$ paramètres.

Cependant, ce modèle suppose que la fonction de projection cl est déterministe et connue à l'avance. Or, dans la langue naturelle, un mot est fondamentalement polysémique : le mot « *flies* » peut être un verbe ou un nom selon le contexte. L'état syntaxique sous-jacent est donc par essence *non observé* : c'est l'acte de naissance des *Modèles de Markov Cachés*.

4.2 Formalisme des Modèles de Markov Cachés (HMM)

4.2.1 Définition mathématique

Un modèle de Markov caché de premier ordre modélise un processus bivarié $(S_t, O_t)_{t \geq 1}$ où la suite des états (S_t) forme une chaîne de Markov non observable (latente), et (O_t) est la suite des observations visibles émises à chaque instant conditionnellement à l'état courant.

Définition (Paramètres canoniques d'un HMM) . Un HMM à états discrets est formellement caractérisé par le triplet $\lambda = (\pi, A, B)$:

1. Un ensemble fini d'états cachés $\mathcal{S} = \{s_1, \dots, s_N\}$.
2. Un alphabet fini d'observations $\mathcal{V} = \{v_1, \dots, v_M\}$.
3. La distribution initiale des états $\pi = (\pi_i)_{i=1}^N$ avec $\pi_i = \mathbb{P}(S_1 = s_i)$.
4. La matrice de transition $A = (a_{ij}) \in \mathbb{R}^{N \times N}$ régie par l'hypothèse markovienne :

$$a_{ij} = \mathbb{P}(S_{t+1} = s_j \mid S_t = s_i), \quad \sum_{j=1}^N a_{ij} = 1. \quad (4.2.1)$$

5. La matrice d'émission $B = (b_j(k)) \in \mathbb{R}^{N \times M}$ vérifiant l'hypothèse d'indépendance

conditionnelle :

$$b_j(k) = \mathbb{P}(O_t = v_k \mid S_t = s_j), \quad \sum_{k=1}^M b_j(k) = 1. \quad (4.2.2)$$

Pour une séquence d'états $S = (S_1, \dots, S_T)$ et d'observations $O = (O_1, \dots, O_T)$, la loi conjointe se factorise immédiatement grâce aux indépendances conditionnelles du graphe :

$$\mathbb{P}(O, S \mid \lambda) = \pi_{S_1} b_{S_1}(O_1) \prod_{t=2}^T a_{S_{t-1}, S_t} b_{S_t}(O_t). \quad (4.2.3)$$

4.2.2 Les trois problèmes fondamentaux de Rabiner (1989)

L'utilisation des modèles HMM induit trois problèmes fondamentaux appelés des *problèmes de Rabiner*, en l'honneur de la personne qui les a formalisés la première fois en 1989.

Proposition (Problèmes de Rabiner - 1989) Les 3 problèmes présentés par Rabiner sont :

1. **L'Évaluation** : À partir d'une séquence d'observation donnée $O = (O_1, \dots, O_m)$ comment calculer la probabilité qu'elle soit générée par le modèle ? L'approche naïve serait de dénombrer toutes les séquences possibles de probabilité non nulle, mais c'est un problème de complexité exponentielle par rapport à la taille de la séquence. En pratique, on utilisera l'algorithme de programmation dynamique *Forward*.
2. **Inférence/Décodage** : À partir d'une observation $O = (O_1, \dots, O_n)$ quelle est la séquence d'états cachés $S = (S_1, \dots, S_n)$ qui aurait généré ces observations la plus probable ? On pourrait aussi calculer des fréquences sur toutes les séquences d'états possibles pouvant générer cette observation mais c'est aussi de complexité exponentielle. On utilisera l'algorithme de *Viterbi*.
3. **L'Apprentissage** : À partir d'un jeu de données d'observations, comment estimer les matrices de transition A et B pour construire le modèle ? On distingue ici deux cas :
 - **Apprentissage Supervisé** : On dispose ici des séquences d'observations et les états cachés correspondant : il suffit alors de calculer des fréquences dessus pour estimer les deux matrices.
 - **Apprentissage Non-Supervisé** : On ne dispose que des séquences d'observations. On utilisera pour cela l'algorithme *Expectation-Maximization (EM)* développé par Dempster en 1977. Cet algorithme se base sur le calcul d'un maximum de vraisemblance sur les états possibles.
On peut aussi citer des approches par clustering sur les observations pour définir des états cachés ou des k -means.

4.3 Résolution Algorithmique des Trois Problèmes de Rabiner

4.3.1 Problème 1 : Évaluation et Algorithme Forward-Backward

Soit un modèle HMM fixé $\lambda = (\pi, A, B)$ et une séquence d'observations $O = (O_1, O_2, \dots, O_T) \in \mathcal{V}^T$. Le problème d'évaluation consiste à déterminer la vraisemblance marginale $\mathbb{P}(O \mid \lambda)$. Commençons par étudier la complexité de l'approche naïve pour se convaincre de l'utilisation d'algorithmes plus performants.

Proposition (Approche naïve) Par la loi des probabilités totales, la vraisemblance marginale s'obtient en sommant la loi conjointe sur l'ensemble $\mathcal{S}^T$ des trajectoires d'états latents possibles :

$$\mathbb{P}(O \mid \lambda) = \sum_{S \in \mathcal{S}^T} \mathbb{P}(O, S \mid \lambda) = \sum_{s_{i_1}=1}^N \cdots \sum_{s_{i_T}=1}^N \pi_{s_{i_1}} b_{s_{i_1}}(O_1) \prod_{t=2}^T a_{s_{i_{t-1}}, s_{i_t}} b_{s_{i_t}}(O_t). \quad (4.3.1)$$

L'espace $\mathcal{S}^T$ possède un cardinal de N^T . Chaque terme nécessitant $2T - 1$ multiplications, la complexité temporelle brute est en $\mathcal{O}(T \cdot N^T)$. Ce calcul devient physiquement irréalisable dès que T dépasse quelques dizaines d'instantes (pour $N = 10$ et $T = 50$, $N^T = 10^{50}$).

On préférera donc l'algorithme Forward ou Backward définis ci-dessous. L'algorithme Forward résout ce problème par *programmation dynamique* en factorisant les calculs sur un treillis d'états (*trellis*).

Proposition (Algorithme Forward) On introduit la variable intermédiaire $\alpha_t(i)$, définie comme la probabilité conjointe du préfixe d'observations jusqu'à l'instant t et de l'état latent au temps t :

$$\alpha_t(i) = \mathbb{P}(O_1, \dots, O_t, S_t = s_i \mid \lambda), \quad \forall i \in \llbracket 1, N \rrbracket, t \in \llbracket 1, T \rrbracket. \quad (4.3.2)$$

Algorithm 2 Algorithme Forward

1. **Initialisation** ($t = 1$) :

$$\alpha_1(i) = \pi_i b_i(O_1), \quad \forall i \in \llbracket 1, N \rrbracket. \quad (4.3.3)$$

2. **Induction / Récurrence** ($t = 1, \dots, T - 1$) :

$$\alpha_{t+1}(j) = \left[\sum_{i=1}^N \alpha_t(i) a_{ij} \right] b_j(O_{t+1}), \quad \forall j \in \llbracket 1, N \rrbracket. \quad (4.3.4)$$

3. **Terminaison** :

$$\mathbb{P}(O \mid \lambda) = \sum_{i=1}^N \alpha_T(i). \quad (4.3.5)$$

Remarque (Complexité) Le treillis compte $N \times T$ nœuds. À chaque transition temporelle $t \rightarrow t+1$, le calcul de $\alpha_{t+1}(j)$ agrège les contributions des N états antérieurs via N multiplications et additions. La complexité temporelle chute ainsi à $\mathcal{O}(N^2T)$, avec une complexité spatiale en mémoire de $\mathcal{O}(NT)$ (ou $\mathcal{O}(N)$ si l'on ne conserve que le vecteur de l'instant précédent).

Proposition (Algorithme Backward) De manière duale, on définit la variable Backward $\beta_t(i)$ comme la probabilité conditionnelle du suffixe d'observations allant de $t + 1$ à T , conditionnellement à l'état latent présent $S_t = s_i$:

$$\beta_t(i) = \mathbb{P}(O_{t+1}, \dots, O_T \mid S_t = s_i, \lambda), \quad \forall i \in \llbracket 1, N \rrbracket, t \in \llbracket 1, T \rrbracket. \quad (4.3.6)$$

Algorithm 3 Algorithme Backward

- 1.
- Initialisation**
- (
- $t = T$
-) :

$$\beta_T(i) = 1, \quad \forall i \in \llbracket 1, N \rrbracket. \quad (4.3.7)$$

- 2.
- Induction régressive**
- (
- $t = T - 1, \dots, 1$
-) :

$$\beta_t(i) = \sum_{j=1}^N a_{ij} b_j(O_{t+1}) \beta_{t+1}(j), \quad \forall i \in \llbracket 1, N \rrbracket. \quad (4.3.8)$$

- 3.
- Terminaison**
- :

$$\mathbb{P}(O \mid \lambda) = \sum_{i=1}^N \pi_i b_i(O_1) \beta_1(i). \quad (4.3.9)$$

4.3.2 Problème 2 : Décodage et Inférence

Étant donnée une séquence observée O , le décodage consiste à inférer la séquence d'états latents sous-jacente. Selon l'objectif visé, deux formulations distinctes sont envisagées. Le critère le plus classique recherche la trajectoire complète $S^* = (S_1^*, \dots, S_T^*)$ qui maximise la loi conjointe (et donc la loi a posteriori globale $\mathbb{P}(S \mid O, \lambda)$) :

$$S^* = \arg \max_{S \in \mathcal{S}^T} \mathbb{P}(S \mid O, \lambda) = \arg \max_{S \in \mathcal{S}^T} \mathbb{P}(O, S \mid \lambda). \quad (4.3.10)$$

On introduit la variable de Viterbi $\delta_t(i)$, représentant la probabilité maximale d'une séquence partielle d'états se terminant en s_i au temps t , ainsi qu'une matrice de pointeurs $\psi_t(i)$ (*backpointers*) stockant l'antécédent optimal :

$$\delta_t(i) \triangleq \max_{S_1, \dots, S_{t-1}} \mathbb{P}(O_1, \dots, O_t, S_1, \dots, S_{t-1}, S_t = s_i \mid \lambda). \quad (4.3.11)$$

Algorithm 4 Algorithme de Viterbi

- 1.
- Initialisation**
- (
- $t = 1$
-) :

$$\delta_1(i) = \pi_i b_i(O_1), \quad \forall i \in \llbracket 1, N \rrbracket, \quad \psi_1(i) = 0. \quad (4.3.12)$$

- 2.
- Récurrence**
- (
- $t = 2, \dots, T$
-) :

$$\delta_t(j) = \left[\max_{1 \leq i \leq N} (\delta_{t-1}(i) a_{ij}) \right] b_j(O_t), \quad \forall j \in \llbracket 1, N \rrbracket, \quad (4.3.13)$$

$$\psi_t(j) = \arg \max_{1 \leq i \leq N} (\delta_{t-1}(i) a_{ij}), \quad \forall j \in \llbracket 1, N \rrbracket. \quad (4.3.14)$$

- 3.
- Terminaison**
- :

$$P^* = \max_{1 \leq i \leq N} \delta_T(i), \quad (4.3.15)$$

$$S_T^* = \arg \max_{1 \leq i \leq N} \delta_T(i). \quad (4.3.16)$$

- 4.
- Reconstruction du chemin**
- (
- Backtracking*
-) (
- $t = T - 1, \dots, 1$
-) :

$$S_t^* = \psi_{t+1}(S_{t+1}^*). \quad (4.3.17)$$

Proposition (*Posterior Decoding*) Une alternative consiste à déterminer, pour chaque pas de temps t pris isolément, l'état qui maximise la probabilité a posteriori conditionnelle $\mathbb{P}(S_t = s_i \mid O, \lambda)$:

$$S_*^t = \arg \max_{1 \leq i \leq N} \mathbb{P}(S_t = s_i \mid O, \lambda). \quad (4.3.18)$$

Grâce aux indépendances conditionnelles du modèle, la probabilité conjointe de l'état $S_t = s_i$ et de l'ensemble de la séquence d'observations O se factorise par le produit des variables Forward et Backward :

$$\mathbb{P}(S_t = s_i, O \mid \lambda) = \alpha_t(i) \beta_t(i). \quad (4.3.19)$$

Par la formule de Bayes :

$$\mathbb{P}(S_t = s_i \mid O, \lambda) = \frac{\mathbb{P}(S_t = s_i, O \mid \lambda)}{\mathbb{P}(O \mid \lambda)} = \frac{\alpha_t(i) \beta_t(i)}{\sum_{j=1}^N \alpha_t(j) \beta_t(j)}. \quad (4.3.20)$$

Le décodage a posteriori revient donc à évaluer :

$$S_*^t = \arg \max_{1 \leq i \leq N} (\alpha_t(i) \beta_t(i)). \quad (4.3.21)$$

Remarque (Comparaison Viterbi vs Décodage a posteriori) • **Viterbi** garantit la validité structurelle globale du chemin prédit : aucune transition de probabilité nulle ($a_{ij} = 0$) ne peut être empruntée.

- **Le décodage a posteriori** minimise le taux d'erreur moyen par position temporelle. Cependant, en maximisant chaque état indépendamment de ses voisins, il peut produire deux états consécutifs S_*^t et S_*^{t+1} pour lesquels la transition est impossible ($a_{S_*^t, S_*^{t+1}} = 0$). Dans ce cas, la probabilité globale de la séquence d'états résultante est strictement nulle.

Les algorithmes Forward et Viterbi partagent la même topologie de programmation dynamique sur le treillis. Ils correspondent à une instanciation formelle du même algorithme abstrait sur deux *semi-anneaux* distincts $(K, \oplus, \otimes, \bar{0}, \bar{1})$:

Caractéristique	Semi-anneau des Réels (Forward)	Semi-anneau Tropical (Viterbi)
Support K	$\mathbb{R}_+$	$\mathbb{R}_+$
Élément nul $\bar{0}$	0	0
Élément unité $\bar{1}$	1	1
Addition $\oplus$	+ (somme des alternatives)	max (sélection du meilleur chemin)
Multiplication $\otimes$	$\times$ (accumulation causale)	$\times$ (accumulation causale)

TABLE 4.1 – Correspondance algébrique entre Forward et Viterbi via les semi-anneaux.

4.3.3 Stabilité Numérique et Domaine Logarithmique

Sur de longues séquences, la multiplication itérative de probabilités inférieures à 1 entraîne rapidement un dépassement de capacité par le bas (*arithmetic underflow*) sur les flottants standard. Pour éviter l'écrasement à zéro, on manipule les logarithmes des probabilités. Pour deux log-probabilités $u = \ln p$ et $v = \ln q$:

- La multiplication devient une addition : $\ln(p \times q) = u + v$.
- Dans l'algorithme de Viterbi, la monotonie du logarithme commute avec le maximum :

$$\ln \delta_t(j) = \ln b_j(O_t) + \max_{1 \leq i \leq N} [\ln \delta_{t-1}(i) + \ln a_{ij}]. \quad (4.3.22)$$

Pour l'algorithme Forward, l'addition de probabilités dans le domaine log nécessite le calcul de $\ln(p + q) = \ln(e^u + e^v)$. De manière générale, pour un vecteur $l = (l_1, \dots, l_n) \in \mathbb{R}^n$, on définit l'opérateur *logsumexp* :

$$\text{LSE}(l) \triangleq \ln \left(\sum_{i=1}^n e^{l_i} \right). \quad (4.3.23)$$

Si certains composants de l sont très grands ($l_i \gg 0$), le calcul brut de e^{l_i} provoque un dépassement par le haut (*overflow*).

Proposition (Factorisation stable de Log-Sum-Exp) Pour tout scalaire $a \in \mathbb{R}$:

$$\text{LSE}(l) = a + \text{LSE}(l - a) = a + \ln \left(\sum_{i=1}^n e^{l_i - a} \right). \quad (4.3.24)$$

Démonstration En factorisant par e^a dans la somme :

$$\begin{aligned} \text{LSE}(l) &= \ln \left(\sum_{i=1}^n e^{l_i} \right) = \ln \left(\sum_{i=1}^n e^a \cdot e^{l_i - a} \right) \\ &= \ln \left(e^a \sum_{i=1}^n e^{l_i - a} \right) = \ln(e^a) + \ln \left(\sum_{i=1}^n e^{l_i - a} \right) \\ &= a + \ln \left(\sum_{i=1}^n e^{l_i - a} \right). \end{aligned} \quad (4.3.25)$$

En pratique, on pose $a = \max_{1 \leq i \leq n} l_i$. Ainsi, pour tout i , $l_i - a \leq 0$, ce qui borne $e^{l_i - a} \in]0, 1]$ et neutralise tout risque d'overflow.

4.3.4 Problème 3 : Apprentissage des Paramètres

L'objectif de l'apprentissage est d'estimer les paramètres du modèle $\lambda = (\pi, A, B)$ à partir d'un ensemble de séquences d'entraînement.

Proposition (Cas Supervisé) Lorsque les séquences d'observations et les séquences d'états associées sont intégralement observées (par exemple en étiquetage POS annoté manuellement), l'estimation des paramètres par *Maximum de Vraisemblance* (MLE) se réduit à des ratios de fréquences empiriques :

$$\hat{\pi}_i = \frac{\text{Count}(S_1 = s_i)}{\sum_{j=1}^N \text{Count}(S_1 = s_j)}, \quad (4.3.26)$$

$$\hat{a}_{ij} = \frac{\text{Count}(S_{t-1} = s_i, S_t = s_j)}{\sum_{k=1}^N \text{Count}(S_{t-1} = s_i, S_t = s_k)}, \quad (4.3.27)$$

$$\hat{b}_j(k) = \frac{\text{Count}(S_t = s_j, O_t = v_k)}{\sum_{m=1}^M \text{Count}(S_t = s_j, O_t = v_m)}. \quad (4.3.28)$$

Proposition (Cas Non-Supervisé : Algorithme EM) Lorsque les états latents ne sont pas observés, l'optimisation directe de la log-vraisemblance marginale $\ln \mathbb{P}(O \mid \lambda) = \ln \sum_S \mathbb{P}(O, S \mid \lambda)$ est non convexe. On applique l'algorithme d'*Espérance-Maximisation* (EM), qui prend le nom d'algorithme de *Baum-Welch* dans le cadre des HMM.

L'algorithme itère deux étapes jusqu'à convergence :

1. **Étape E (Espérance)** : À l'aide des paramètres courants $\lambda^{(k)}$ et via les algorithmes Forward et Backward, on calcule les distributions a posteriori sur les variables latentes :

- Probabilité d'être dans l'état s_i au temps t :

$$\gamma_t(i) = \mathbb{P}(S_t = s_i \mid O, \lambda^{(k)}) = \frac{\alpha_t(i) \beta_t(i)}{\sum_{j=1}^N \alpha_t(j) \beta_t(j)}. \quad (4.3.29)$$

- Probabilité de transiter de s_i à s_j entre les instants t et $t+1$:

$$\xi_t(i, j) = \mathbb{P}(S_t = s_i, S_{t+1} = s_j \mid O, \lambda^{(k)}) = \frac{\alpha_t(i) a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)}{\mathbb{P}(O \mid \lambda^{(k)})}. \quad (4.3.30)$$

2. **Étape M (Maximisation)** : On met à jour les paramètres $\lambda^{(k+1)}$ en maximisant l'espérance de la log-vraisemblance des données complètes :

$$\pi_i^{(k+1)} = \gamma_1(i), \quad (4.3.31)$$

$$a_{ij}^{(k+1)} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \quad (4.3.32)$$

$$b_j(m)^{(k+1)} = \frac{\sum_{t=1}^T \gamma_t(j) \cdot \mathbb{I}(O_t = v_m)}{\sum_{t=1}^T \gamma_t(j)}. \quad (4.3.33)$$

L'algorithme garantit la croissance monotone de la vraisemblance marginale $\mathbb{P}(O \mid \lambda)$ à chaque itération et converge vers un extremum local du paysage d'optimisation.

Mathématiques du Machine Learning

Chapitre 1

Introduction à l'Apprentissage Supervisé

Contents

1.1	Des données et des prédictions	168
1.2	Formalisation mathématique	169
1.2.1	Fonction de Perte	169
1.2.2	Les Risques	169
1.3	Risque et prédicteur de Bayes	170
1.3.1	Forme explicite selon la tâche	171
1.4	Paradigmes classiques d'apprentissage	171
1.4.1	Minimisation du Risque Empirique (ERM)	171
1.4.2	Méthodes probabilistes et Maximum de Vraisemblance	172
1.4.3	Méthodes par moyennage local (k -plus proches voisins)	173
1.4.4	Méthodes génératives	173
1.4.5	Théorème No Free Lunch	174

L'objectif de ce cours est de formaliser le processus d'apprentissage d'un modèle dans le cas d'un apprentissage supervisé. On commencera par définir ce qu'est ce type d'apprentissage puis ce que l'on entend par "apprendre". Différentes métriques seront introduites pour le quantifier.

1.1 Des données et des prédictions

L'ensemble du processus d'apprentissage s'appuie sur les concepts de *données* et de *prédictions*.

Définition (Observations) . Formellement, on définira les *données* d'un problème de machine learning par un ensemble de couples $(x, y) \in \mathcal{X} \times \mathcal{Y}$ que l'on appellera *données d'entraînement* ou *observations*. On notera cet ensemble D qui contiendra N couples.

Ces entrées et sorties attendues peuvent être de différents types en fonction du problème considéré. Les entrées peuvent être des images, des vidéos, du son ou un texte. On les modélisera en général par des vecteurs de $\mathbb{R}^d$ lorsque c'est possible. On appellera donc $d \in \mathbb{N}$ la dimension des données à l'entrée du modèle.

Les sorties peuvent être de deux types différents en apprentissage supervisé :

- Un ensemble d'entiers $\{1, \dots, k\}$. On parlera alors de problème de *classification*.

- Un vecteur de $\mathbb{R}^{d'}$. On parlera alors d'un problème de *régression*.

L'objectif est d'être capable, à partir d'une nouvelle donnée $x \in \mathcal{X}$ d'inférer/prédire le y qui lui correspond le mieux. Plus formellement, on cherche à *apprendre* une fonction f à partir des données d'entraînement telle que :

$$y \simeq f(x) \quad (1.1.1)$$

1.2 Formalisation mathématique

Pour apprendre une telle fonction, nous devons admettre différentes hypothèses sur le jeu de données de départ D . La première est que ce jeu de données suit une certaine loi de probabilité $\mathbb{P}$. La seconde est que chaque échantillon $(x_i, y_i), i \in \llbracket 1, N \rrbracket$ est indépendant des autres (D est un ensemble de réalisation i.i.d de la loi $\mathbb{P}$).

1.2.1 Fonction de Perte

Pour effectuer cet apprentissage, nous devons être capable de savoir à quel point notre modèle se trompe sur une prédiction donnée par rapport à la vraie valeur issue du jeu de données. Nous définissons pour cela une *fonction de perte*.

Définition (Fonction de perte) . La *fonction de perte*, notée l , calcule l'erreur induite par la prédiction de $z \in \mathcal{Y}$ alors que la valeur attendue était $y \in \mathcal{Y}$ pour une entrée $x \in \mathcal{X}$. Elle vérifie donc :

$$l : \mathcal{Y} \times \mathcal{Y} \longrightarrow \mathbb{R}. \quad (1.2.1)$$

Exemple Dans le cas d'une classification binaire (i.e $\mathcal{Y} = \{0, 1\}$), on peut simplement utiliser la *Zero/One Loss* définie par $l(y, z) = \mathbb{1}_{z \neq y}$.

Dans le cas de la régression, on utilisera plutôt les L^1 Loss ou L^2 Loss définies par :

$$l(y, z) = \|y - z\|^2, \quad \text{et} \quad l(y, z) = \|y - z\|$$

1.2.2 Les Risques

Ces différentes fonctions de pertes, aussi bien définies soient-elles, ne nous donnent d'un aperçu des erreurs du modèle en temps réel. Nous avons aussi besoin d'avoir une idée plus fine des performances du modèle vis-à-vis de la distribution de probabilité $\mathbb{P}$ du jeu de données D . On définit pour cela les *risques*.

Définition (Risque Moyen) . On définit le *risque moyen*, noté $\mathcal{R}(f)$, d'une fonction f sur l'espace $\mathcal{X} \times \mathcal{Y}$ par l'espérance de la perte du ce même jeu de données.

$$\mathcal{R}(f) := \int_{\mathcal{X} \times \mathcal{Y}} l(y, f(x)) dp(x, y) dx dy = \mathbb{E}(l(y, f(x))) \quad (1.2.2)$$

où p est le densité de la probabilité $\mathbb{P}$.

Exemple Dans le cas de la classification binaire, le risque moyen se simplifie facilement :

$$\begin{aligned} \mathcal{R}(f) &= \int_{\mathcal{X} \times \mathcal{Y}} \mathbb{1}_{y \neq f(x)} d\mathbb{P}(x, y) \\ &= 0 \times \mathbb{P}(f(x) = y) + 1 \times \mathbb{P}(f(x) \neq y) \\ &= \mathbb{P}(f(x) \neq y) \end{aligned}$$

À noter que ce risque est calculé sur l'espace entier des données possibles. En pratique, si celui-ci est infini, il est incalculable. On préférera donc utiliser le *risque empirique*.

Définition (Risque Empirique) . Le risque empirique, noté $\hat{\mathcal{R}}(f)$ est donné par la moyenne empirique des pertes sur un jeu de données D .

$$\hat{\mathcal{R}}(f) := \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i)). \quad (1.2.3)$$

Proposition (Convergence) Grâce aux hypothèses précédentes, on peut invoquer des théorèmes de convergence sur les suites de variables aléatoires pour caractériser la limite de ce risque empirique.

Ainsi, si $f \in L^1$ alors, d'après la *Loi Forte des Grands Nombres*, on a que :

$$\hat{\mathcal{R}}(f) \xrightarrow[n \rightarrow \infty]{\text{p.s.}} \mathcal{R}(f). \quad (1.2.4)$$

Si, de plus $f \in L^2$, alors on a une idée de la vitesse de convergence de ce risque empirique :

$$\sqrt{n}(\hat{\mathcal{R}}(f) - \mathcal{R}(f)) \xrightarrow[n \rightarrow \infty]{\text{Loi}} \mathcal{N}(0, \mathbb{V}(l(y, f(x)))). \quad (1.2.5)$$

Remarque (Indépendance) En pratique, pour toute fonction apprise f , on aura $\hat{\mathcal{R}}(f) \simeq \mathcal{R}(f)$. Or, lorsque l'on calcule f sur des données empiriques (ex : jeu de données), on casse l'indépendance de la suite de variables aléatoires et donc le TCL. Formellement, en prenant l'estimateur :

$$\frac{1}{n} \sum_{i=1}^n l(x_i, \hat{f}(x_i)),$$

on remarque que le $\hat{f}$ est calculé à partir des $(x_1, y_1), \dots, (x_n, y_n)$. On peut solutionner ce problème en séparant le jeu de données en deux : *entraînement* et *test* (et éventuellement *validation*). Puisque le modèle ne voit pas les données de test et de validation, la fonction apprise ne dépend pas de ces données, ce qui permet de conserver l'indépendance des variables. On peut ensuite utiliser le TCL pour estimer le risque $\mathcal{R}$ à partir du risque empirique $\hat{\mathcal{R}}$.

1.3 Risque et prédicteur de Bayes

L'objectif théorique de l'apprentissage statistique est de trouver une fonction f qui minimise le risque moyen $\mathcal{R}(f)$ parmi l'ensemble $\mathcal{F}_{\text{all}}$ de toutes les fonctions mesurables de $\mathcal{X}$ dans $\mathcal{Y}$.

Définition (Prédicteur et Risque de Bayes) . On appelle *prédicteur de Bayes*, noté f^* , la fonction optimale (si elle existe) qui minimise le risque moyen sur l'espace de toutes les fonctions mesurables :

$$f^* \in \arg \min_{f \in \mathcal{F}_{\text{all}}} \mathcal{R}(f) \quad (1.3.1)$$

Le risque associé, noté $\mathcal{R}^* = \mathcal{R}(f^*)$, est appelé le *risque de Bayes* (ou erreur irréductible). Pour toute fonction mesurable f , on a trivialement :

$$\mathcal{R}(f) \geq \mathcal{R}^* \quad (1.3.2)$$

Pour calculer explicitement f^* , on utilise la loi des espérances itérées (conditionnement par rapport à X) :

$$\mathcal{R}(f) = \mathbb{E}_{(X,Y)} [l(Y, f(X))] = \mathbb{E}_X [\mathbb{E}_{Y|X} [l(Y, f(X)) | X]] \quad (1.3.3)$$

Minimiser $\mathcal{R}(f)$ revient donc à minimiser l'espérance conditionnelle point par point pour chaque $x \in \mathcal{X}$:

$$f^*(x) \in \arg \min_{z \in \mathcal{Y}} \mathbb{E}_{Y|X=x} [l(Y, z) \mid X = x] \quad (1.3.4)$$

1.3.1 Forme explicite selon la tâche

Proposition (Cas de la régression quadratique (L^2)) Pour la perte quadratique $l(y, z) = (y - z)^2$, le prédicteur de Bayes est l'espérance conditionnelle de Y sachant X :

$$f^*(x) = \mathbb{E}[Y \mid X = x] \quad (1.3.5)$$

Démonstration Fixons $x \in \mathcal{X}$. On cherche $z \in \mathbb{R}$ minimisant $\phi(z) = \mathbb{E}[(Y - z)^2 \mid X = x]$.

$$\phi(z) = \mathbb{E}[Y^2 \mid X = x] - 2z\mathbb{E}[Y \mid X = x] + z^2$$

ϕ est une fonction convexe et dérivable en z . En annulant la dérivée :

$$\phi'(z) = -2\mathbb{E}[Y \mid X = x] + 2z = 0 \iff z = \mathbb{E}[Y \mid X = x]$$

Proposition (Cas de la classification binaire (perte 0/1)) Pour $\mathcal{Y} = \{0, 1\}$ et la perte 0/1 $l(y, z) = \mathbb{1}_{y \neq z}$, le prédicteur de Bayes attribue la classe la plus probable a posteriori :

$$f^*(x) = \mathbb{1}_{\mathbb{P}(Y=1|X=x) \geq 1/2} \quad (1.3.6)$$

Démonstration Pour un $x \in \mathcal{X}$ donné et pour $z \in \{0, 1\}$:

$$\begin{aligned} \mathbb{E}[\mathbb{1}_{Y \neq z} \mid X = x] &= \mathbb{P}(Y \neq z \mid X = x) \\ &= 1 - \mathbb{P}(Y = z \mid X = x) \end{aligned}$$

Minimiser cette quantité revient à maximiser $\mathbb{P}(Y = z \mid X = x)$, ce qui donne la règle du maximum a posteriori (MAP).

Pour résumer, dans le cas d'une *classification*, le prédicteur de Bayes associe la classe la plus probable selon la distribution de probabilité simulée. Dans le cas d'une *régression*, ce prédicteur retournera la moyenne de la distribution de probabilité générée sur les y_i .

1.4 Paradigmes classiques d'apprentissage

Le prédicteur de Bayes f^* étant inconnu car il dépend de la distribution $\mathbb{P}$, différentes stratégies existent pour l'approcher à partir du jeu de données $S = ((x_1, y_1), \dots, (x_n, y_n))$.

1.4.1 Minimisation du Risque Empirique (ERM)

Définition (Estimateur ERM) . On restreint la recherche à une classe paramétrique de fonctions $\mathcal{F}_\Theta = \{f_\theta : \mathcal{X} \rightarrow \mathcal{Y}, \theta \in \Theta\}$. L'estimateur par *minimisation du risque empirique* (ERM) est défini par :

$$\hat{\theta}_n \in \arg \min_{\theta \in \Theta} \hat{\mathcal{R}}_S(f_\theta) = \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n l(y_i, f_\theta(x_i)) \quad (1.4.1)$$

Exemple (Régression linéaire et polynomiale) Pour $f_\theta(x) = \theta^\top \varphi(x)$ (où φ est un morphisme d'extraction de caractéristiques / *feature map*) et la perte quadratique L^2 , le problème s'écrit :

$$\hat{\theta}_n \in \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n (y_i - \theta^\top \varphi(x_i))^2 \quad (1.4.2)$$

Si $\varphi(x) = (1, x, x^2, \dots, x^k)^\top$, on retrouve la régression polynomiale de degré k .

Proposition (Décomposition de l'erreur d'excès de risque) Pour tout estimateur $\hat{\theta}$, l'excès de risque par rapport au risque de Bayes se décompose canoniquement en :

$$\mathcal{R}(f_{\hat{\theta}}) - \mathcal{R}^* = \underbrace{\left(\mathcal{R}(f_{\hat{\theta}}) - \inf_{\theta \in \Theta} \mathcal{R}(f_\theta) \right)}_{\text{Erreur d'estimation}} + \underbrace{\left(\inf_{\theta \in \Theta} \mathcal{R}(f_\theta) - \mathcal{R}^* \right)}_{\text{Erreur d'approximation}} \quad (1.4.3)$$

- **Erreur d'approximation** : Déterministe et indépendante de n , elle mesure la perte due au choix de la classe restreinte Θ (diminue si la richesse de Θ augmente).
- **Erreur d'estimation** : Aléatoire (dépend de S), elle mesure le coût lié au nombre fini d'échantillons n (décroît avec n , croît avec la richesse/complexité de Θ).

Remarque (Contrôle de l'erreur d'estimation) En notant $\theta^* \in \arg \min_{\theta \in \Theta} \mathcal{R}(f_\theta)$, on majore l'erreur d'estimation par :

$$\mathcal{R}(f_{\hat{\theta}}) - \mathcal{R}(f_{\theta^*}) \leq 2 \sup_{\theta \in \Theta} |\hat{\mathcal{R}}_S(f_\theta) - \mathcal{R}(f_\theta)| + \sup_{\theta \in \Theta} (\hat{\mathcal{R}}_S(f_{\hat{\theta}}) - \hat{\mathcal{R}}_S(f_\theta)) \quad (1.4.4)$$

Le premier terme est une déviation uniforme (objet central de la théorie de l'apprentissage statistique), et le second représente l'erreur d'optimisation numérique (nulle si le minimum empirique est calculé exactement).

La décomposition du risque comme présentée ci-dessus peut être observée sur l'apprentissage du modèle. On parlera alors de sur-apprentissage ou de sous-apprentissage en fonction de la complexité du modèle.

- **Sous-apprentissage (*Underfitting*)** : Modèle trop simple (faible complexité de Θ), dominé par l'erreur d'approximation.
- **Sur-apprentissage (*Overfitting*)** : Modèle trop expressif (par exemple interpolation polynomiale de Lagrange de degré $n-1$), l'erreur empirique est quasi-nulle mais l'erreur d'estimation explose sur les nouvelles données.

Pour contrôler ce phénomène, on introduit une pénalisation de complexité $\Omega(\theta)$:

$$\hat{\theta}_{\text{reg}} \in \arg \min_{\theta \in \Theta} \left\{ \hat{\mathcal{R}}_S(f_\theta) + \lambda \Omega(\theta) \right\} \quad (1.4.5)$$

où $\lambda > 0$ est un hyperparamètre calibré sur le jeu de validation (exemples : Ridge avec $\Omega(\theta) = \|\theta\|_2^2$, Lasso avec $\Omega(\theta) = \|\theta\|_1$).

1.4.2 Méthodes probabilistes et Maximum de Vraisemblance

L'idée consiste à postuler une famille paramétrique de densités conditionnelles $\{p_\theta(y | x) : \theta \in \Theta\}$ et à choisir le paramètre qui maximise la log-vraisemblance sur l'échantillon i.i.d :

$$\hat{\theta}_{\text{MLE}} \in \arg \max_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \ln p_\theta(x_i, y_i) \quad (1.4.6)$$

Exemple (Régression Logistique) Pour des labels $y \in \{-1, 1\}$, en posant la fonction sigmoïde $\sigma(z) = \frac{1}{1+e^{-z}}$, le modèle postule :

$$p_\theta(y | x) = \sigma(y\theta^\top x) = \frac{1}{1 + e^{-y\theta^\top x}} \quad (1.4.7)$$

Maximiser la vraisemblance revient exactement à un problème d'ERM :

$$\hat{\theta} \in \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ln \left(1 + e^{-y_i \theta^\top x_i} \right) \quad (1.4.8)$$

avec la perte logistique convexe $l(y, z) = \ln(1 + e^{-yz})$.

Remarque (Pont entre Maximum de Vraisemblance et ERM) En posant $l(y, \hat{y}) = -\ln p_\theta(y | x)$, toute maximisation de vraisemblance s'interprète comme une minimisation de risque empirique :

- Bruit gaussien $\varepsilon \sim \mathcal{N}(0, \sigma^2) \implies$ Perte quadratique (moindres carrés).
- Loi de Bernoulli $\implies$ Perte logistique.

En classification, on utilise ces fonctions de perte convexes (surrogates) car minimiser directement la perte 0/1 est un problème NP-difficile et non dérivable (gradient nul presque partout).

1.4.3 Méthodes par moyennage local (k -plus proches voisins)

Ces méthodes non paramétriques exploitent une hypothèse de régularité locale : des points proches dans $\mathcal{X}$ partagent des distributions conditionnelles $\mathbb{P}_{y|x}$ similaires.

Définition (k -Nearest Neighbours – k -NN) . Pour une entrée x' , on considère l'ensemble $\mathcal{V}_k(x')$ de ses k plus proches voisins dans le jeu d'entraînement :

- **En classification** : Règle du vote majoritaire :

$$\hat{f}(x') = \arg \max_{c \in \mathcal{Y}} \sum_{i \in \mathcal{V}_k(x')} \mathbb{1}_{y_i=c} \quad (1.4.9)$$

- **En régression** : Moyenne empirique locale :

$$\hat{f}(x') = \frac{1}{k} \sum_{i \in \mathcal{V}_k(x')} y_i \quad (1.4.10)$$

Remarque (Compromis biais-variance pour k -NN) Pour k -NN, l'axe de complexité est inversé :

- k petit (ex. $k = 1$) $\implies$ Modèle très flexible, risque de sur-apprentissage (faible approximation, forte estimation).
- k grand $\implies$ Modèle rigide/lissé, risque de sous-apprentissage (forte approximation, faible estimation).

1.4.4 Méthodes génératives

Au lieu de modéliser directement $f(x)$ ou $\mathbb{P}(Y | X)$, les modèles génératifs estiment la loi conjointe en modélisant :

1. La loi a priori des classes $p(y)$.
2. Les densités conditionnelles de chaque classe $p(x | y)$.

On en déduit ensuite la loi a posteriori par la règle de Bayes :

$$p(y | x) = \frac{p(x | y)p(y)}{\sum_c p(x | y = c)p(y = c)} \propto p(x | y)p(y) \quad (1.4.11)$$

La règle de décision optimale est donnée par le maximum a posteriori (MAP) : $\hat{f}(x') = \arg \max_{\hat{y}} p(x' | \hat{y})p(\hat{y})$.

1.4.5 Théorème No Free Lunch

Il n'existe pas d'algorithme d'apprentissage universel optimal sur toutes les distributions possibles sans hypothèse préalable sur les données.

Théorème (No Free Lunch (Devroye, Györfi, Lugosi)) . Soit le problème de classification binaire avec la perte 0/1 et $\mathcal{X}$ un ensemble infini. Soit $\mathcal{P}$ l'ensemble de toutes les lois de probabilité sur $\mathcal{X} \times \{0, 1\}$. Pour tout entier n et tout algorithme d'apprentissage $\mathcal{A}$, on a :

$$\sup_{\mathbb{P} \in \mathcal{P}} \{\mathbb{E}_{S \sim \mathbb{P}^{\otimes n}} [\mathcal{R}_{\mathbb{P}}(\mathcal{A}(S))] - \mathcal{R}_{\mathbb{P}}^*\} \geq \frac{1}{2} \quad (1.4.12)$$

Ce résultat implique que pour n'importe quel algorithme fixé, il existe une distribution sous-jacente défavorable pour laquelle l'algorithme ne fait pas mieux qu'un choix aléatoire (même si le risque de Bayes $\mathcal{R}^* = 0$). Il est donc mathématiquement indispensable de poser des hypothèses régularité, marge, séparabilité ou compacité pour garantir des résultats d'apprentissage.

Chapitre 2

Régression Linéaire et Régression Ridge

Contents

2.1	Moindres Carrés	175
2.2	Analyse statistique – design fixe	176
2.3	Régression Ridge – design fixe	178

On pose ici le contexte d'une *régression linéaire*. On cherche donc à apprendre une fonction pour décrire un jeu de données $(x_1, y_1), \dots, (x_n, y_n) \in \mathcal{X} \times \mathcal{Y}$. Dans le cas du modèle linéaire, on fera l'hypothèse forte suivante :

$$(H) : f_\theta = \varphi(x)^t \theta, \quad \theta \in \mathbb{R}^d. \quad (2.0.1)$$

Pour construire cette fonction, on utilisera l'estimateur du risque empirique (ERM) défini par :

$$\hat{R}(\theta) \in \arg \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n (y_i - f_\theta(x_i))^2. \quad (2.0.2)$$

Exemple La forme de l'application φ caractérisera la *feature map* (i.e l'ensemble des fonctions possibles pour interpoler le nuage de points).

- si $\varphi(x) = x$, alors on aura une estimation linéaire en x ,
- si $\varphi(x) = \begin{pmatrix} x \\ 1 \end{pmatrix}$ alors l'estimation sera *affine* en x ,
- si $\varphi(x) \in \mathbb{R}^d[x]$, alors l'estimation sera *polynômiale* en x . On pourra aller jusqu'au degré d pour utiliser des polynômes de Lagrange.

Notation On notera $y = \begin{pmatrix} y_1 \\ \dots \\ y_n \end{pmatrix}$ le vecteur des étiquettes du jeu de données et $\Phi = \begin{pmatrix} \varphi(x_1)^t \\ \dots \\ \varphi(x_n)^t \end{pmatrix} \in$

$\mathbb{R}^{n \times d}$ la matrice contenant toutes les prédictions données par la *feature map*. On aura donc : $\hat{R}(\theta) = \frac{1}{n} \|y - \Phi\theta\|_2^2$.

2.1 Moindres Carrés

Nous sommes en présence d'un problème de régression. Grâce aux notations matricielles précédemment posées, nous sommes capable de calculer une solution explicite à la main.

On cherche à calculer explicitement $\hat{\theta} \in \arg \min_{\theta \in \mathbb{R}^d} \hat{R}(\theta)$. Pour cela, on utilise la condition nécessaire d'optimalité d'ordre 1 : $\nabla_{\theta} \hat{R}(\hat{\theta}) = 0$. Commençons donc par calculer le gradient :

$$\hat{R}(\theta) = \frac{1}{n} \|y - \Phi\theta\|_2^2 \quad (2.1.1)$$

$$\nabla_{\theta} \hat{R}(\theta) = \frac{2}{n} \Phi^{\top} (\Phi\theta - y) \quad (2.1.2)$$

En résolvant l'équation $\nabla_{\theta} \hat{R}(\hat{\theta}) = 0$, on obtient les *équations normales* :

$$(*) : \Phi^{\top} \Phi \hat{\theta} = \Phi^{\top} y \quad (2.1.3)$$

En supposant que Φ est de rang plein ($rg(\Phi) = d$, soit des colonnes linéairement indépendantes), la matrice de Gram $\Phi^{\top} \Phi \in \mathbb{R}^{d \times d}$ est inversible, ce qui donne :

$$(*) \iff \boxed{\hat{\theta} = (\Phi^{\top} \Phi)^{-1} \Phi^{\top} y} \quad (2.1.4)$$

La fonction $\hat{R}$ étant convexe, ce point stationnaire est un minimum global. On calcule ainsi explicitement l'estimateur des moindres carrés (OLS), qui fournit le prédicteur linéaire optimal sur l'échantillon d'entraînement.

Proposition Géométriquement, les prédictions $\hat{y} = \Phi \hat{\theta}$ correspondent à la projection orthogonale du vecteur d'observations $y \in \mathbb{R}^n$ sur le sous-espace vectoriel $\text{Im}(\Phi) \subset \mathbb{R}^n$ engendré par les colonnes de Φ :

$$\hat{\theta} \in \arg \min_{\theta \in \mathbb{R}^d} \frac{1}{n} \|y - \Phi\theta\|_2^2 \iff \hat{y} \in \arg \min_{z \in \text{Im}(\Phi)} \frac{1}{n} \|y - z\|_2^2 \quad (2.1.5)$$

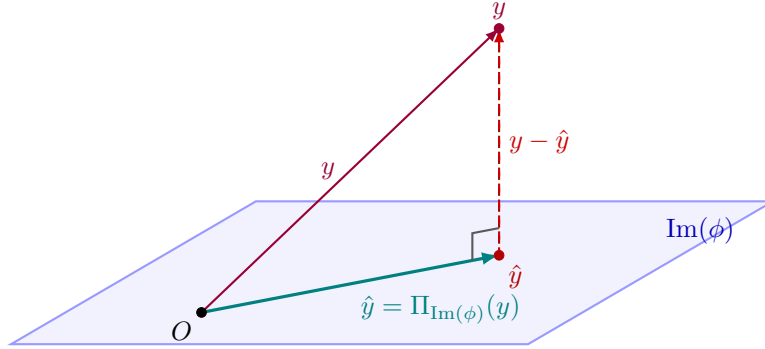


FIGURE 2.1 – Projection orthogonale du vecteur d'observations y sur le sous-espace affine $\text{Im}(\phi)$ et décomposition de l'erreur résiduelle $y - \hat{y} \perp \text{Im}(\phi)$.

2.2 Analyse statistique – design fixe

Dans cette section, on se place sous l'hypothèse du *design fixe* : les points $x_i \in \mathcal{X}$ (et donc la matrice $\Phi \in \mathbb{R}^{n \times d}$) sont déterministes et fixés. Seules les étiquettes y_i sont aléatoires.

Définition (Modèle à design fixe) . Le modèle probabiliste d'observation s'écrit :

1. $y = \Phi\theta_* + \varepsilon$, avec $\theta_* \in \mathbb{R}^d$ le paramètre vrai inconnu.
2. Le vecteur de bruit $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)^\top \in \mathbb{R}^n$ vérifie $\mathbb{E}(\varepsilon) = 0$ et $\text{Var}(\varepsilon) = \mathbb{E}(\varepsilon\varepsilon^\top) = \sigma^2 I_n$.

Le risque moyen sous design fixe est défini par :

$$R(\theta) = \frac{1}{n} \mathbb{E}_y \left[\|y - \Phi\theta\|_2^2 \right]. \quad (2.2.1)$$

On introduit la matrice de covariance empirique $\hat{\Sigma} := \frac{1}{n} \Phi^\top \Phi \in \mathbb{R}^{d \times d}$, associée à la semi-norme $\|u\|_{\hat{\Sigma}}^2 := u^\top \hat{\Sigma} u = \frac{1}{n} \|\Phi u\|_2^2$.

Proposition (Décomposition Biais-Variance) Pour tout vecteur fixe $\theta \in \mathbb{R}^d$:

$$\begin{aligned} R(\theta) &= \frac{1}{n} \mathbb{E}_\varepsilon \left[\|\Phi(\theta_* - \theta) + \varepsilon\|_2^2 \right] \\ &= \frac{1}{n} \|\Phi(\theta_* - \theta)\|_2^2 + \frac{1}{n} \mathbb{E} \left[\|\varepsilon\|_2^2 \right] + \frac{2}{n} (\theta_* - \theta)^\top \Phi^\top \mathbb{E}[\varepsilon] \\ &= \|\theta - \theta_*\|_{\hat{\Sigma}}^2 + \sigma^2. \end{aligned}$$

Le risque de Bayes est atteint en θ_* : $R^* = R(\theta_*) = \sigma^2$. Par conséquent, pour un estimateur aléatoire $\hat{\theta}$ dépendant de y (donc de ε) :

$$\mathbb{E}_{\hat{\theta}} \left[R(\hat{\theta}) \right] - R^* = \underbrace{\left\| \mathbb{E}(\hat{\theta}) - \theta_* \right\|_{\hat{\Sigma}}^2}_{\text{Biais}^2} + \underbrace{\mathbb{E}_{\hat{\theta}} \left[\left\| \hat{\theta} - \mathbb{E}(\hat{\theta}) \right\|_{\hat{\Sigma}}^2 \right]}_{\text{Variance}} \quad (2.2.2)$$

Théorème (Biais et Variance des Moindres Carrés) . Dans le cas de l'estimateur des moindres carrés $\hat{\theta} = (\Phi^\top \Phi)^{-1} \Phi^\top y$:

$$\text{Biais}^2 = 0 \quad \text{et} \quad \mathbb{E} \left[R(\hat{\theta}) \right] - R^* = \frac{\sigma^2 d}{n}. \quad (2.2.3)$$

Démonstration • *Calcul du biais* :

$$\begin{aligned} \mathbb{E}(\hat{\theta}) &= \mathbb{E} \left[(\Phi^\top \Phi)^{-1} \Phi^\top (\Phi\theta_* + \varepsilon) \right] \\ &= \theta_* + (\Phi^\top \Phi)^{-1} \Phi^\top \mathbb{E}[\varepsilon] = \theta_*. \end{aligned}$$

L'estimateur est sans biais, d'où $\text{Biais}^2 = 0$.

- *Calcul de la variance* : On exprime la variance sous forme de trace :

$$\begin{aligned} \text{Variance} &= \mathbb{E} \left[(\hat{\theta} - \theta_*)^\top \hat{\Sigma} (\hat{\theta} - \theta_*) \right] \\ &= \mathbb{E} \left[\text{tr} \left(\hat{\Sigma} (\hat{\theta} - \theta_*) (\hat{\theta} - \theta_*)^\top \right) \right] \\ &= \text{tr} \left(\hat{\Sigma} \mathbb{E} \left[(\hat{\theta} - \theta_*) (\hat{\theta} - \theta_*)^\top \right] \right). \end{aligned}$$

Or, $\hat{\theta} - \theta_* = (\Phi^\top \Phi)^{-1} \Phi^\top \varepsilon$, donc :

$$\begin{aligned} \mathbb{E} \left[(\hat{\theta} - \theta_*)(\hat{\theta} - \theta_*)^\top \right] &= (\Phi^\top \Phi)^{-1} \Phi^\top \mathbb{E}[\varepsilon \varepsilon^\top] \Phi (\Phi^\top \Phi)^{-1} \\ &= \sigma^2 (\Phi^\top \Phi)^{-1} \Phi^\top I_n \Phi (\Phi^\top \Phi)^{-1} \\ &= \sigma^2 (\Phi^\top \Phi)^{-1} = \frac{\sigma^2}{n} \hat{\Sigma}^{-1}. \end{aligned}$$

En réinjectant dans l'expression de la trace :

$$\text{Variance} = \text{tr} \left(\hat{\Sigma} \cdot \frac{\sigma^2}{n} \hat{\Sigma}^{-1} \right) = \frac{\sigma^2}{n} \text{tr}(I_d) = \frac{\sigma^2 d}{n}.$$

2.3 Régression Ridge – design fixe

Dans le modèle linéaire usuel, l'erreur de variance vaut $\frac{\sigma^2 d}{n}$. Lorsque les covariables sont fortement corrélées (multicolinéarité), la matrice $\hat{\Sigma}$ possède des valeurs propres très proches de 0, ce qui rend $(\Phi^\top \Phi)^{-1}$ instable et fait exploser la norme du paramètre appris.

La régression Ridge (pénalisation de Tikhonov / régularisation L^2) remédie à ce problème en contractant les coefficients vers zéro via une pénalisation quadratique. On accepte d'introduire un léger biais en échange d'une réduction drastique de la variance.

Définition (Estimateur Ridge) . Pour un paramètre de régularisation $\lambda > 0$, l'estimateur Ridge est défini par :

$$\hat{\theta}_\lambda \in \arg \min_{\theta \in \mathbb{R}^d} \frac{1}{n} \|y - \Phi \theta\|_2^2 + \lambda \|\theta\|_2^2. \quad (2.3.1)$$

En annulant le gradient, la solution explicite est unique et donnée par :

$$\hat{\theta}_\lambda = \left(\hat{\Sigma} + \lambda I_d \right)^{-1} \frac{1}{n} \Phi^\top y = \left(\Phi^\top \Phi + n \lambda I_d \right)^{-1} \Phi^\top y \quad (2.3.2)$$

où $\hat{\Sigma} = \frac{1}{n} \Phi^\top \Phi$. La matrice $\hat{\Sigma} + \lambda I_d$ est symétrique définie positive pour tout $\lambda > 0$, donc inversible même si $\Phi^\top \Phi$ ne l'est pas.

Proposition (Décomposition Biais-Variance de Ridge) Sous le modèle à design fixe $y = \Phi \theta_* + \varepsilon$:

- *Biais* : L'espérance de l'estimateur vaut $\mathbb{E}(\hat{\theta}_\lambda) = (\hat{\Sigma} + \lambda I_d)^{-1} \hat{\Sigma} \theta_*$. Le terme de biais s'écrit donc :

$$\text{Biais}^2 = \left\| \mathbb{E}(\hat{\theta}_\lambda) - \theta_* \right\|_{\hat{\Sigma}}^2 = \lambda^2 \theta_*^\top \left(\hat{\Sigma} + \lambda I_d \right)^{-1} \hat{\Sigma} \left(\hat{\Sigma} + \lambda I_d \right)^{-1} \theta_*. \quad (2.3.3)$$

- *Variance* : En utilisant $\text{Var}(\hat{\theta}_\lambda) = \frac{\sigma^2}{n} (\hat{\Sigma} + \lambda I_d)^{-1} \hat{\Sigma} (\hat{\Sigma} + \lambda I_d)^{-1}$, on obtient :

$$\text{Variance} = \frac{\sigma^2}{n} \text{tr} \left(\hat{\Sigma}^2 \left(\hat{\Sigma} + \lambda I_d \right)^{-2} \right). \quad (2.3.4)$$

Le risque d'excès s'écrit donc :

$$\mathbb{E} \left[R(\hat{\theta}_\lambda) \right] - R^* = \lambda^2 \theta_*^\top \left(\hat{\Sigma} + \lambda I_d \right)^{-2} \hat{\Sigma} \theta_* + \frac{\sigma^2}{n} \text{tr} \left(\hat{\Sigma}^2 \left(\hat{\Sigma} + \lambda I_d \right)^{-2} \right). \quad (2.3.5)$$

Remarque (Degrés de liberté effectifs) La quantité $df(\lambda) = \text{tr} \left(\hat{\Sigma}(\hat{\Sigma} + \lambda I_d)^{-1} \right) = \sum_{j=1}^d \frac{\mu_j}{\mu_j + \lambda}$ mesure les *degrés de liberté effectifs* du modèle. Pour $\lambda = 0$, on retrouve $df(0) = d$ et la variance des moindres carrés $\frac{\sigma^2 d}{n}$. Lorsque $\lambda \rightarrow \infty$, $df(\lambda) \rightarrow 0$.

Proposition (Contrôle spectral et borne d'erreur) Soit $\hat{\Sigma} = U \text{diag}(\mu_1, \dots, \mu_d) U^\top$ la décomposition spectrale de $\hat{\Sigma}$ (avec $\mu_j \geq 0$).

- Pour le biais : pour tout $\mu \geq 0$ et $\lambda > 0$, on a $\frac{\lambda^2 \mu}{(\mu + \lambda)^2} \leq \frac{\lambda}{4}$ (car $(\mu + \lambda)^2 \geq 4\lambda\mu$), d'où :

$$\text{Biais}^2 \leq \frac{\lambda}{4} \|\theta_*\|_2^2. \quad (2.3.6)$$

- Pour la variance : comme $\frac{\mu_j^2}{(\mu_j + \lambda)^2} \leq \frac{\mu_j}{\lambda}$, on a :

$$\text{Variance} \leq \frac{\sigma^2}{n\lambda} \text{tr}(\hat{\Sigma}). \quad (2.3.7)$$

En équilibrant les deux bornes avec le choix optimal $\lambda^* \propto \frac{\sigma}{\sqrt{n}}$, on garantit une vitesse de convergence de l'excès de risque en $\mathcal{O}(1/\sqrt{n})$, indépendante du plus petit spectre de $\hat{\Sigma}$.

Chapitre 3

Méthodes à Noyaux

Contents

3.1	De la similarité au Théorème du Représentant	180
3.1.1	Intuition géométrique et plongement	180
3.1.2	Le Théorème du Représentant	181
3.2	Noyaux définis positifs, Théorème de Bochner, et Théorème d’Aronszajn	182
3.2.1	Noyau SDP : Définition et propriétés	182
3.2.2	Construire des noyaux SDP : Théorème de Bochner	183
3.2.3	Théorème d’Aronszajn	184
3.3	Séparateurs à Vaste Marge (SVM)	186
3.3.1	Cas linéairement séparable : Maximisation de la marge	186
3.3.2	Cas non séparable : Variables d’écart et perte charnière	186
3.3.3	Dualité de Wolfe et formulation duale	187
3.3.4	Le Kernel SVM (SVM à noyau)	187

L’objectif des méthodes à noyaux est de transformer une mesure heuristique de similarité entre données en un algorithme d’apprentissage non linéaire. En s’appuyant implicitement sur un espace de Hilbert de dimension arbitraire (potentiellement infinie), on conserve toute la simplicité de l’optimisation linéaire tout en évitant d’avoir à calculer explicitement les plongements.

3.1 De la similarité au Théorème du Représentant

3.1.1 Intuition géométrique et plongement

La plupart des jeux de données ne sont pas linéairement séparables dans leur espace d’entrée d’origine $\mathcal{X}$. L’idée fondamentale consiste à envoyer les données dans un espace de caractéristiques de plus grande dimension (un espace de Hilbert $\mathcal{H}$) via une *feature map* :

$$\varphi : \mathcal{X} \longrightarrow \mathcal{H} \quad (3.1.1)$$

Dans $\mathcal{H}$, un hyperplan linéaire peut alors séparer des classes qui formaient des régions géométriques complexes ou intriquées dans $\mathcal{X}$. On formule alors le problème sous la forme d’une minimisation de risque empirique régularisé sur la totalité de l’espace hilbertien $\mathcal{H}$:

$$\hat{\theta}_S \in \arg \min_{\theta \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n l(y_i, \langle \varphi(x_i), \theta \rangle_{\mathcal{H}}) + \frac{\lambda}{2} \|\theta\|_{\mathcal{H}}^2 \quad (3.1.2)$$

où $\lambda > 0$ contrôle la régularisation.

Remarque (Obstruction numérique) Si $\mathcal{H}$ est de dimension infinie, on ne peut ni représenter le vecteur de paramètres $\theta \in \mathcal{H}$, ni évaluer explicitement $\varphi(x)$ par descente de gradient standard. Le théorème suivant résout ce paradoxe.

3.1.2 Le Théorème du Représentant

Théorème (Théorème du Représentant) . Soit $\mathcal{X}$ un ensemble, $\mathcal{H}$ un espace de Hilbert réel muni du produit scalaire $\langle \cdot, \cdot \rangle_{\mathcal{H}}$ et $\varphi : \mathcal{X} \rightarrow \mathcal{H}$. Soit un échantillon $S = (x_1, \dots, x_n)$ et le sous-espace engendré par les observations :

$$\mathcal{H}_D := \text{vect}\{\varphi(x_1), \dots, \varphi(x_n)\} \subset \mathcal{H} \quad (3.1.3)$$

Soit $\Psi : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}$ une fonction croissante par rapport à son second argument. En notant $u(\theta) = (\langle \varphi(x_1), \theta \rangle_{\mathcal{H}}, \dots, \langle \varphi(x_n), \theta \rangle_{\mathcal{H}})^\top \in \mathbb{R}^n$, on a :

$$\inf_{\theta \in \mathcal{H}} \Psi(u(\theta), \|\theta\|_{\mathcal{H}}^2) = \inf_{\theta \in \mathcal{H}_D} \Psi(u(\theta), \|\theta\|_{\mathcal{H}}^2) \quad (3.1.4)$$

Si l'infimum est atteint, il l'est en un point $\hat{\theta}_S \in \mathcal{H}_D$. De plus, si Ψ est *strictement croissante* par rapport à son second argument, tout minimiseur appartient nécessairement à $\mathcal{H}_D$.

Grâce à ce théorème, on cherche donc $\hat{\theta}_S \in \mathcal{H}_D$ de dimension finie. Puisque l'on en connaît une famille génératrice, on cherche donc :

$$\alpha = (\alpha_1, \dots, \alpha_n)^\top \in \mathbb{R}^n, \quad \text{tels que} \quad \hat{\theta}_S = \sum_{i=1}^n \alpha_i \varphi(x_i).$$

Ainsi, la dimension du problème d'optimisation ne dépend plus de celle de $\mathcal{H}$ mais seulement du nombre de d'échantillons n . Déterminons maintenant comment trouver ce $\hat{\alpha}_S$.

Proposition (Kernel Trick) Remplaçons θ par sa nouvelle expression dans le problème de départ 3.1.2 :

- La norme devient :

$$\begin{aligned} \|\theta\|_{\mathcal{H}}^2 &= \left\langle \sum_{i=1}^n \alpha_i \varphi(x_i), \sum_{j=1}^n \alpha_j \varphi(x_j) \right\rangle_{\mathcal{H}} = \sum_{i,j=1}^n \alpha_i \alpha_j \langle \varphi(x_i), \varphi(x_j) \rangle_{\mathcal{H}} \\ &= \sum_{i,j=1}^n \alpha_i \alpha_j K_{i,j} \quad \text{avec} \quad K = (\langle \varphi(x_i), \varphi(x_j) \rangle_{\mathcal{H}})_{i,j} \end{aligned}$$

- Les prédictions deviennent :

$$\langle \varphi(x_i), \theta \rangle_{\mathcal{H}} = \sum_{j=1}^n \alpha_j \langle \varphi(x_i), \varphi(x_j) \rangle_{\mathcal{H}} = (K\alpha)_i$$

On a donc le problème réécrit suivant :

$$\hat{\alpha}_S \in \arg \min_{\alpha \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^n l(y_i, (K\alpha)_i) + \frac{\lambda}{2} \alpha^\top K \alpha. \quad (3.1.5)$$

On remarque que l'on a plus besoin de calculer explicitement φ ni f : tout dépend de ce que l'on appellera le *noyau* K . Notons ici $K = (k(x_i, x_j))_{i,j}$ tel que $k(x, x_j) = \langle \varphi(x), \varphi(x_j) \rangle_{\mathcal{H}}$. Ainsi, pour l'entraînement nous n'avons besoin que de la matrice de Gram K de taille $n \times n$. Pour l'inférence sur un x donné, nous notons $k_x = (k(x, x_1), \dots, k(x, x_n)) \in \mathbb{R}^n$ tel que $f_{\hat{\theta}_S} = \hat{\alpha}_S^\top k_x$.

On peut maintenant remarquer que la *feature map* φ n'est jamais calculée et que les prédictions en x utilisent seulement les n valeurs $\langle \varphi(x), \varphi(x_i) \rangle_{\mathcal{H}}$. Ainsi, au lieu de définir un espace $\mathcal{H}$ dans lequel nous ne pouvons faire aucun calcul, on pourrait directement choisir k telle que $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ de la forme $k(x, x') = \langle \varphi(x), \varphi(x') \rangle$. C'est le sujet de la section suivante qui définit les *noyaux* et introduit la théorie nécessaire dessus.

3.2 Noyaux définis positifs, Théorème de Bochner, et Théorème d'Aronszajn

Dans la partie précédente, nous avons reformulé le problème de minimisation du risque empirique à l'aide d'un plongement dans un espace de Hilbert $\mathcal{H}$. Nous avons ensuite remarqué que tous les calculs dans cet espace dépendent en réalité d'une *fonction de similitude* encodée dans un *noyau* K .

Or, en pratique, on raisonne dans le sens inverse : à partir du jeu de données, on définit un noyau K et on souhaite savoir si l'on peut effectivement utiliser ce noyau pour notre minimisation. C'est le théorème d'Aronszajn qui nous l'assure. Mais avant de l'aborder, définissons formellement la notion de noyau ainsi que ses propriétés.

3.2.1 Noyau SDP : Définition et propriétés

Au lieu de définir $\mathcal{H}$ puis φ , on choisit directement une fonction $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ et on cherche à quelles conditions elle correspond à un produit scalaire hilbertien (i.e un noyau semi-défini positif).

Définition (Noyau semi-défini positif) . Une application $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ est appelée un *noyau semi-défini positif* (noyau SDP ou PSDK) si elle vérifie :

1. **Symétrie** : $\forall x, x' \in \mathcal{X}, \quad k(x, x') = k(x', x)$.
2. **Positivité** : $\forall n \geq 1, \forall (x_1, \dots, x_n) \in \mathcal{X}^n, \forall (\alpha_1, \dots, \alpha_n) \in \mathbb{R}^n :$

$$\sum_{i,j=1}^n \alpha_i \alpha_j k(x_i, x_j) \geq 0 \iff K \in \mathcal{S}_n^+(\mathbb{R}) \quad (3.2.1)$$

Propriété (Caractérisation par matrice de Gram) . La matrice de Gram $G = (\langle u_i, u_j \rangle)_{i,j}$ associée à toute famille finie $(u_1, \dots, u_n)$ est symétrique semi-définie positive.

Proposition (Tout produit scalaire définit un noyau SDP) Si $\mathcal{H}$ est un espace de Hilbert et $\varphi : \mathcal{X} \rightarrow \mathcal{H}$, alors $k(x, x') := \langle \varphi(x), \varphi(x') \rangle_{\mathcal{H}}$ est un noyau SDP.

Exemple Voici quelques exemples de noyaux SDP :

- Noyau *linéaire* : $k(x, x') = x^\top x'$ sur $\mathcal{X} = \mathbb{R}^d$.
- Noyau *polynomial* : $k(x, x') = (x^\top x')^q$ pour $q \in \mathbb{N}^*$.
- Noyau *exponentiel* : $k(x, x') = \exp(-a\|x - x'\|_2^2)$ pour $a > 0$.

Les noyaux SDP peuvent être construit par opérations. Le théorème suivant détaille les opérations possibles entre noyaux SDP.

Théorème (Opérations) . L'ensemble des noyaux semi-définis positifs sur un espace $\mathcal{X}$ est stable par :

- multiplication scalaire par un coefficient positif ou nul,
- somme finie,
- produits,
- limite ponctuelle quand elle existe,
- conjugaison par une fonction.

Grâce à ce théorème, nous pouvons montrer que beaucoup d'application k sont des noyaux. Voyons-en quelques unes.

Proposition Soit $q \geq 1$ un entier, alors $(x, x') \mapsto (x^\top x')^q$ et $(x, x') \mapsto (1 + x^\top x')^q$ sont des noyaux SDP.

Démonstration Pour le premier, $(x, x') \mapsto x^\top x'$ est un produit scalaire donc c'est un noyau. À la puissance q , c'est une multiplication finie. C'est donc un noyau par induction sur q . Pour le second, on identifie $1 + x^\top x' = k(x, x') + x^\top x'$ avec $k(x, x') = 1$. Puisque $\sum_{i,j} \alpha_i \alpha_j = (\sum_i \alpha_i)^2 \geq 0$, alors c'est un noyau. Par somme et d'après le résultat précédent, $(x, x') \mapsto (1 + x^\top x')^q$ est aussi un noyau SDP.

Proposition Soit k un noyau SDP, alors $\exp(k) : (x, x') \mapsto \exp(k(x, x'))$ est un noyau SDP.

Démonstration Si k est un noyau alors, $\forall x, x' \in \mathcal{X}$, on a :

$$\exp(k(x, x')) = \sum_{n=0}^{\infty} \frac{k(x, x')^n}{n!}.$$

Or, pour tout $n \geq 0$, $\frac{k(x, x')^n}{n!}$ est un noyau par produit fini et multiplication par un scalaire positif. Par somme, $\sum_{i=1}^n \frac{k(x, x')^i}{i!}$ en est un aussi. Enfin, par définition de l'exponentielle, $\exp a$ converge pour tout $a \in \mathbb{R}$, donc par limite de noyaux SDP, $\exp(k)$ est aussi un noyau SDP.

3.2.2 Construire des noyaux SDP : Théorème de Bochner

Montrer qu'un noyau est SDP est simple lorsqu'il dérive de produits scalaires combinés algébriquement. En revanche, si l'on cherche à mesurer la proximité géométrique locale entre deux observations, le calcul direct devient inextricable. Par exemple, pour le noyau gaussien $k(x, x') = \exp(-a\|x - x'\|_2^2)$, vérifier point par point que :

$$\forall n \geq 1, \quad \forall (x_1, \dots, x_n) \in (\mathbb{R}^d)^n, \quad \forall (\alpha_1, \dots, \alpha_n) \in \mathbb{R}^n, \quad \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \exp(-a\|x_i - x_j\|_2^2) \geq 0$$

est difficile sans outil analytique. Le théorème de Bochner permet de lever ce verrou en transposant la question de l'espace spatial à l'espace fréquentiel via la transformée de Fourier.

Théorème (Théorème de Bochner – Invariance par translation) . Soit $q : \mathbb{R}^d \rightarrow \mathbb{R}$ continue. Le noyau stationnaire $k(x, x') := q(x - x')$ est semi-défini positif si et seulement si q est la transformée de Fourier inverse d'une mesure borélienne positive finie ρ sur $\mathbb{R}^d$:

$$q(z) = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} e^{iz^\top \omega} d\rho(\omega). \quad (3.2.2)$$

Démonstration (Preuve de la condition suffisante) Supposons que q s'écrive sous forme d'intégrale de Fourier par rapport à une mesure positive finie ρ . Pour toute famille $(x_1, \dots, x_n)$ et tout vecteur $\alpha \in \mathbb{R}^n$:

$$\begin{aligned}
 \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j q(x_i - x_j) &= \sum_{i,j=1}^n \alpha_i \alpha_j \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} e^{i(x_i - x_j)^\top \omega} d\rho(\omega) \\
 &= \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \sum_{i=1}^n \alpha_i e^{ix_i^\top \omega} \sum_{j=1}^n \alpha_j e^{-ix_j^\top \omega} d\rho(\omega) \\
 &= \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \left(\sum_{i=1}^n \alpha_i e^{ix_i^\top \omega} \right) \overline{\left(\sum_{j=1}^n \alpha_j e^{ix_j^\top \omega} \right)} d\rho(\omega) \\
 &= \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \underbrace{\left| \sum_{i=1}^n \alpha_i e^{ix_i^\top \omega} \right|^2}_{\geq 0} \underbrace{d\rho(\omega)}_{\geq 0} \geq 0.
 \end{aligned}$$

La fonction q étant paire dès lors que la mesure est symétrique ($\rho(-\omega) = \rho(\omega)$), le noyau est bien symétrique et semi-défini positif.

Exemple (Cas du noyau Gaussien / RBF) Pour $a > 0$ et $z \in \mathbb{R}^d$, la fonction gaussienne $q(z) = \exp(-a\|z\|_2^2)$ admet pour transformée de Fourier inverse :

$$e^{-a\|z\|_2^2} = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} e^{iz^\top \omega} \left(\frac{\pi}{a}\right)^{d/2} e^{-\frac{\|\omega\|_2^2}{4a}} d\omega.$$

La mesure $d\rho(\omega) = (\pi/a)^{d/2} e^{-\|\omega\|_2^2/(4a)} d\omega$ est une mesure gaussienne positive, finie et invariante par $\omega \mapsto -\omega$. Le théorème de Bochner garantit immédiatement que le noyau gaussien est semi-défini positif.

3.2.3 Théorème d'Aronszajn

Reprenons le raisonnement de ce chapitre. Le théorème du représentant nous a montré que pour notre problème d'optimisation, si on a un espace de Hilbert $\mathcal{H}$ et une application φ , il nous suffit seulement de calculer des produits scalaires $k(x, x') = \langle \varphi(x), \varphi(x') \rangle_{\mathcal{H}}$.

Ensuite, nous avons développé une théorie sur les noyaux SDP pour les construire facilement. Le théorème de Bochner nous permet d'en construire autant que l'on souhaite à partir de mesures positives finies.

Or, nous ne sommes pas assurés que pour un tel noyau construit spécifiquement pour nos données, il existe un espace $\mathcal{H}$ et un plongement φ qui convienne. C'est ici que le théorème d'Aronszajn intervient : il nous assure que tout noyau SDP est le produit scalaire d'un espace unique de fonctions appelé RKHS.

Théorème (Théorème d'Aronszajn) . Soit $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ un noyau symétrique semi-défini positif. Alors, il existe un unique espace de Hilbert $\mathcal{H}_k$ constitué de fonctions de $\mathcal{X}$ dans $\mathbb{R}$, et une application canonique $\varphi_k : \mathcal{X} \rightarrow \mathcal{H}_k$, tels que :

1. Pour tout $x \in \mathcal{X}$, la fonction $k_x := k(x, \cdot)$ appartient à $\mathcal{H}_k$.
2. **Propriété de reproduction** : pour toute fonction $f \in \mathcal{H}_k$ et tout $x \in \mathcal{X}$,

$$f(x) = \langle f, k_x \rangle_{\mathcal{H}_k}. \quad (3.2.3)$$

3. Pour tous $x, x' \in \mathcal{X}$:

$$k(x, x') = \langle k_x, k_{x'} \rangle_{\mathcal{H}_k} = \langle \varphi_k(x), \varphi_k(x') \rangle_{\mathcal{H}_k} \quad \text{en posant } \varphi_k(x) = k_x. \quad (3.2.4)$$

L'espace $\mathcal{H}_k$ est appelé l'*espace de Hilbert à noyau reproduisant* (RKHS) associé à k .

Démonstration (Esquisse de preuve) La construction s'effectue en trois étapes principales :

1. **Espace pré-hilbertien** : on pose $\mathcal{H}_0 = \text{vect}\{k_x \mid x \in \mathcal{X}\}$. Pour deux combinaisons linéaires $f = \sum_{i=1}^m a_i k_{x_i}$ et $g = \sum_{j=1}^{m'} b_j k_{z_j}$, on munit $\mathcal{H}_0$ de la forme bilinéaire :

$$\langle f, g \rangle_{\mathcal{H}_0} := \sum_{i=1}^m \sum_{j=1}^{m'} a_i b_j k(x_i, z_j).$$

La symétrie et la positivité de k assurent que cette forme est symétrique et positive. De plus, par définition, $\langle f, k_x \rangle_{\mathcal{H}_0} = \sum_{i=1}^m a_i k(x_i, x) = f(x)$.

2. **Caractère défini** : d'après l'inégalité de Cauchy-Schwarz appliquée à cette forme semi-définie positive, pour tout $x \in \mathcal{X}$:

$$|f(x)| = |\langle f, k_x \rangle_{\mathcal{H}_0}| \leq \|f\|_{\mathcal{H}_0} \sqrt{k(x, x)}.$$

Ainsi, $\|f\|_{\mathcal{H}_0} = 0 \implies \forall x \in \mathcal{X}, f(x) = 0$, donc f est identiquement nulle. L'espace $\mathcal{H}_0$ est donc un espace pré-hilbertien séparé.

3. **Complétion fonctionnelle** : on complète $\mathcal{H}_0$ en un espace de Hilbert $\mathcal{H}_k$. Grâce à la majoration ponctuelle issue de Cauchy-Schwarz, toute suite de Cauchy converge ponctuellement vers une fonction bien définie sur $\mathcal{X}$. Les éléments complétés restent donc de véritables fonctions, et la propriété de reproduction s'étend par continuité.

Nous avons donc l'équivalence : les noyaux semi-définis positifs sont exactement des produits scalaires de *feature maps*. Choisir un noyau k ou choisir un espace $(\mathcal{H}, \varphi)$ est donc la même chose, sauf que l'on sait explicitement calculer k . On peut donc complètement oublier $\mathcal{H}$ et φ dans le processus d'apprentissage.

Proposition (La recette) L'algorithme complet d'apprentissage supervisé à noyau se résume donc en 4 étapes :

- *Choix de k* : On définit un noyau k mesurant la "distance" de nos données en fonction de leur forme.
- *Matrice de Gram* : On calcule l'ensemble des paires $K_{i,j} = k(x_i, x_j)$ pour construire la matrice de Gram $K \in \mathbb{R}^{n \times n}$.
- *Optimisation* : On résout un problème d'optimisation pour trouver $\hat{\alpha}_S \in \mathbb{R}^n$:

$$\hat{\alpha}_S \in \arg \min_{\alpha \in \mathbb{R}^n} \frac{1}{n} \sum_{i=1}^n l(y_i, (K\alpha)_i) + \frac{\lambda}{2} \alpha^\top K \alpha. \quad (3.2.5)$$

- *Inférence* : Une fois trouvé, on put prédire le label de tout nouveau x par :

$$\hat{y} = \sum_{j=1}^n \hat{\alpha}_j k(x, x_j). \quad (3.2.6)$$

3.3 Séparateurs à Vaste Marge (SVM)

L'algorithme des Séparateurs à Vaste Marge (*Support Vector Machines*) illustre parfaitement la puissance des méthodes à noyaux. On commence par formuler le problème géométrique dans le cas linéaire élémentaire, avant d'en dériver la version duale et d'y injecter le *Kernel Trick*.

3.3.1 Cas linéairement séparable : Maximisation de la marge

Soit un échantillon d'apprentissage $S = ((x_1, y_1), \dots, (x_n, y_n)) \in (\mathbb{R}^d \times \{-1, 1\})^n$. On suppose dans un premier temps les données linéairement séparables : il existe un hyperplan passant par l'origine $H(\theta) = \{z \in \mathbb{R}^d \mid \theta^\top z = 0\}$ tel que pour tout $i \in \llbracket 1, n \rrbracket$, $y_i(\theta^\top x_i) > 0$.

Proposition (Distance à l'hyperplan) Pour tout $\theta \neq 0$, la distance orthogonale d'un point $x \in \mathbb{R}^d$ à l'hyperplan $H(\theta)$ est donnée par :

$$\text{dist}(x, H(\theta)) = \frac{|\theta^\top x|}{\|\theta\|_2}. \quad (3.3.1)$$

Sous l'hypothèse de séparabilité, $y_i(\theta^\top x_i) = |\theta^\top x_i|$. L'objectif consiste à trouver l'hyperplan qui maximise la distance minimale aux observations d'entraînement (la marge géométrique) :

$$\max_{\theta \neq 0} \min_{i \in \llbracket 1, n \rrbracket} \frac{y_i(\theta^\top x_i)}{\|\theta\|_2}. \quad (3.3.2)$$

Le problème étant invariant par changement d'échelle $\theta \mapsto c\theta$ ($c > 0$), on impose la normalisation canonique $\min_i y_i(\theta^\top x_i) = 1$. Maximiser $1/\|\theta\|_2$ sous cette contrainte équivaut à minimiser $\frac{1}{2}\|\theta\|_2^2$. En relâchant l'égalité en une inégalité (qui sera saturée à l'optimum), on obtient le programme quadratique convexe :

$$\min_{\theta \in \mathbb{R}^d} \frac{1}{2} \|\theta\|_2^2 \quad \text{s.c.} \quad \forall i \in \llbracket 1, n \rrbracket, \quad y_i(\theta^\top x_i) \geq 1. \quad (3.3.3)$$

3.3.2 Cas non séparable : Variables d'écart et perte charnière

En pratique, les données se chevauchent ou comportent du bruit, rendant la séparation stricte impossible. Pour autoriser des erreurs tout en les contrôlant, on introduit des variables d'écart (*slack variables*) $\xi_i \geq 0$ pénalisées par un coefficient $C > 0$:

$$\min_{\theta \in \mathbb{R}^d, \xi \in \mathbb{R}^n} \frac{1}{2} \|\theta\|_2^2 + C \sum_{i=1}^n \xi_i \quad \text{s.c.} \quad \forall i \in \llbracket 1, n \rrbracket, \quad y_i(\theta^\top x_i) \geq 1 - \xi_i \quad \text{et} \quad \xi_i \geq 0. \quad (3.3.4)$$

Ce problème admet toujours des solutions admissibles dès que les ξ_i sont choisis suffisamment grands.

Proposition (Pont avec la minimisation du risque empirique) À θ fixé, l'objectif est strictement croissant en chaque variable ξ_i . À l'optimum, ξ_i atteint donc sa plus petite borne autorisée par les contraintes, soit :

$$\xi_i = \max(0, 1 - y_i(\theta^\top x_i)) = (1 - y_i(\theta^\top x_i))_+. \quad (3.3.5)$$

En substituant cette expression, en divisant l'objectif par $nC > 0$ et en posant $\lambda := \frac{1}{nC}$, le problème géométrique coïncide rigoureusement avec un problème d'ERM régularisé pour la perte charnière (*hinge loss*) :

$$\min_{\theta \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n (1 - y_i(\theta^\top x_i))_+ + \frac{\lambda}{2} \|\theta\|_2^2. \quad (3.3.6)$$

3.3.3 Dualité de Wolfe et formulation duale

Le problème primal étant convexe avec des contraintes affines, les conditions de qualification de Slater sont vérifiées (par exemple pour $\theta = 0$ et $\xi_i = 2$), garantissant la dualité forte.

On introduit les multiplicateurs de Lagrange $\alpha_i \geq 0$ (pour les contraintes de marge) et $\beta_i \geq 0$ (pour la positivité de ξ_i) :

$$\mathcal{L}(\theta, \xi, \alpha, \beta) = \frac{1}{2} \|\theta\|_2^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i (y_i \theta^\top x_i + \xi_i - 1) - \sum_{i=1}^n \beta_i \xi_i. \quad (3.3.7)$$

Proposition Minimisons le Lagrangien :

- **En ξ :** Le Lagrangien s'écrit $\sum_{i=1}^n (C - \alpha_i - \beta_i) \xi_i + \dots$. L'infimum par rapport à $\xi_i \geq 0$ vaut $-\infty$ sauf si $C - \alpha_i - \beta_i = 0$. Cette condition permet d'éliminer $\beta_i \geq 0$, ce qui équivaut à la contrainte de boîte :

$$0 \leq \alpha_i \leq C \quad \forall i \in \llbracket 1, n \rrbracket. \quad (3.3.8)$$

- **En θ :** La condition d'annulation du gradient $\nabla_\theta \mathcal{L} = \theta - \sum_{i=1}^n \alpha_i y_i x_i = 0$ impose :

$$\theta^* = \sum_{i=1}^n \alpha_i y_i x_i. \quad (3.3.9)$$

En réinjectant ces relations dans $\mathcal{L}$, le problème dual de Wolfe s'écrit sous forme d'un programme quadratique concave :

$$\max_{\alpha \in \mathbb{R}^n} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^\top x_j) \quad \text{s.c.} \quad \forall i \in \llbracket 1, n \rrbracket, \quad 0 \leq \alpha_i \leq C. \quad (3.3.10)$$

Remarque Les données d'entrée x_i n'interviennent dans ce problème dual que par leurs produits scalaires $x_i^\top x_j$.

3.3.4 Le Kernel SVM (SVM à noyau)

Soit k un noyau semi-défini positif sur $\mathcal{X}$. En vertu du théorème d'Aronszajn, k définit implicitement un RKHS $\mathcal{H}_k$ et une application canonique φ_k tels que $k(x, x') = \langle \varphi_k(x), \varphi_k(x') \rangle_{\mathcal{H}_k}$. On formule le problème primal directement dans $\mathcal{H}_k$:

$$\min_{\theta \in \mathcal{H}_k} \frac{1}{n} \sum_{i=1}^n (1 - y_i \langle \theta, \varphi_k(x_i) \rangle_{\mathcal{H}_k})_+ + \frac{\lambda}{2} \|\theta\|_{\mathcal{H}_k}^2. \quad (3.3.11)$$

Grâce au *Kernel Trick*, on substitue directement chaque produit scalaire par l'évaluation du noyau $k(x_i, x_j) = K_{i,j}$ dans la formulation duale. En posant le changement de variable $\mu_i = \lambda \alpha_i$ (avec $\lambda = \frac{1}{nC}$, ce qui donne $0 \leq \mu_i \leq \frac{1}{n}$), le problème dual s'écrit sous forme matricielle :

$$\boxed{\max_{\mu \in \mathbb{R}^n} \sum_{i=1}^n \mu_i - \frac{1}{2\lambda} \sum_{i=1}^n \sum_{j=1}^n y_i y_j \mu_i \mu_j k(x_i, x_j) \quad \text{s.c.} \quad \forall i \in \llbracket 1, n \rrbracket, \quad 0 \leq \mu_i \leq \frac{1}{n}.} \quad (3.3.12)$$

Il s'agit d'un problème d'optimisation quadratique à n variables, dont la résolution numérique s'effectue uniquement à l'aide de la matrice de Gram $K \in \mathcal{S}_n^+(\mathbb{R})$.

Proposition (Règle de décision et vecteurs de support) D'après l'expression de θ^* et la propriété de reproduction dans le RKHS, la fonction de décision optimale pour classer un nouveau point $x \in \mathcal{X}$ est :

$$f(x) = \text{sign} \left(\sum_{i=1}^n \hat{\alpha}_i y_i k(x_i, x) \right). \quad (3.3.13)$$

D'après les conditions de complémentarité de Karush-Kuhn-Tucker (KKT), les coefficients $\hat{\alpha}_i$ sont strictement nuls pour tous les points situés en dehors de la marge. La décision ne dépend donc que des points vérifiant $\hat{\alpha}_i > 0$, appelés *vecteurs de support* :

$$f(x) = \text{sign} \left(\sum_{i \in \text{Support}} \hat{\alpha}_i y_i k(x_i, x) \right). \quad (3.3.14)$$

Ce résultat garantit la parcimonie du modèle et rend l'évaluation en phase d'inférence très rapide.

Chapitre 4

Théorie de l'Apprentissage et Minimisation du Risque Empirique

Dans les chapitres précédents, nous avons présenté différentes méthodes d'apprentissage basées sur la minimisation du risque empirique (ERM, *Empirical Risk Minimization*). Or la seule analyse de la convergence de ces méthodes que nous ayons faite est la décomposition biais-variance du risque ainsi que la décomposition de son erreur en deux : l'erreur *d'approximation* et l'erreur *d'estimation*.

Dans la seconde partie de ce chapitre, nous nous attardons justement sur l'erreur faite sur le risque lors de l'apprentissage. Pour cela, nous développons une nouvelle théorie basée sur la complexité de l'espace de recherche $\mathcal{F}$.

Mais avant, nous développons le concept de *convexification du risque* dans la première partie. En effet, pour certaines fonction de pertes (notamment la *0-1 loss*) nous ne disposons pas de "bonnes" propriétés analytiques pour utiliser des algorithmes d'optimisation. Nous présentons ainsi une méthode permettant d'y remédier.

4.1 Convexification du risque et substituts convexes

4.1.1 Reparamétrisation et identité de marge

En classification binaire, on cherche à prédire une étiquette parmi deux classes $\mathcal{Y} = \{-1, 1\}$ via une fonction $f : \mathcal{X} \rightarrow \mathcal{Y}$. La mesure naturelle d'erreur est la perte 0-1 (*0-1 loss*) définie par :

$$l(y, \hat{y}) = \mathbb{1}_{y \neq \hat{y}}. \quad (4.1.1)$$

La minimisation exacte du risque empirique sous la perte 0-1 sur une classe d'hypothèses générale est un problème combinatoire NP-difficile. De plus, le risque empirique associé est une fonction constante par morceaux de ses paramètres : son gradient est nul presque partout, ce qui rend inapplicables les méthodes d'optimisation par descente de gradient.

Pour lever ce verrou, on reparamétrise le classifieur discret à l'aide d'une fonction à valeurs réelles.

Définition (Prédicteur par fonction signe randomisée) . Soit $g : \mathcal{X} \rightarrow \mathbb{R}$ une fonction réelle mesurable. On définit le classifieur binaire $f : \mathcal{X} \rightarrow \{-1, 1\}$ par $f(x) = \text{sgn}(g(x))$ avec

la convention :

$$\operatorname{sgn}(u) = \begin{cases} 1 & \text{si } u > 0, \\ -1 & \text{si } u < 0, \\ U \sim \operatorname{Unif}(\{-1, 1\}) & \text{si } u = 0, \end{cases} \quad (4.1.2)$$

où U est une variable aléatoire auxiliaire indépendante du couple d'observations $(X, Y) \sim \mathbb{P}$.

Cette reparamétrisation permet de découpler la *décision brute* (donnée par $\operatorname{sgn}(g(x))$) de la *confiance / distance à la frontière* (quantifiée par l'amplitude $|g(x)|$). On introduit alors la *marge fonctionnelle* $u = Yg(X)$:

- $u > 0$: la prédiction est correcte ($\operatorname{sgn}(g(X)) = Y$).
- $u < 0$: la prédiction est erronée ($\operatorname{sgn}(g(X)) \neq Y$).
- $u = 0$: le point se situe exactement sur la frontière de décision.

Proposition (Identité de marge pour la perte 0-1) Le risque théorique de classification binaire associé à $f = \operatorname{sgn} \circ g$ s'exprime comme l'espérance d'une fonction scalaire de la marge fonctionnelle $Yg(X)$:

$$R(f) = \mathbb{E}_{(X,Y) \sim \mathbb{P}} [\Phi_{0-1}(Yg(X))], \quad (4.1.3)$$

avec la fonction de saut :

$$\Phi_{0-1}(u) = \mathbb{1}_{u < 0} + \frac{1}{2} \mathbb{1}_{u=0}. \quad (4.1.4)$$

Démonstration En explicitant l'espérance conjointe sur (X, Y) et sur la variable de randomisation U :

$$\begin{aligned} R(f) &= \mathbb{E}_{(X,Y),U} [\mathbb{1}_{f(X) \neq Y}] \\ &= \mathbb{E} [\mathbb{1}_{g(X) \neq 0} \mathbb{1}_{f(X) \neq Y}] + \mathbb{E} [\mathbb{1}_{g(X)=0} \mathbb{1}_{f(X) \neq Y}] \\ &= \mathbb{P}(Yg(X) < 0) + \mathbb{E} [\mathbb{1}_{g(X)=0} \mathbb{1}_{U \neq Y}]. \end{aligned}$$

Comme U est indépendante de (X, Y) et uniformément distribuée sur $\{-1, 1\}$, $\mathbb{P}(U \neq Y \mid g(X) = 0) = \frac{1}{2}$. D'où :

$$R(f) = \mathbb{P}(Yg(X) < 0) + \frac{1}{2} \mathbb{P}(g(X) = 0) = \mathbb{E}_{(X,Y) \sim \mathbb{P}} [\Phi_{0-1}(Yg(X))].$$

4.1.2 Risque de substitution et théorème de calibration

Dans le cas général, lorsque l'on fait face à une fonction de coût l non continue ou non convexe, on définit une fonction convexe Φ et une fonction g pour définir un risque de substitution ou *surrogate risk* $R_\Phi(g)$.

Définition (Risque de substitution – Surrogate Risk) . Soit $\Phi : \mathbb{R} \rightarrow [0, \infty[$ une fonction convexe. Le risque de substitution associé à Φ pour une fonction $g : \mathcal{X} \rightarrow \mathbb{R}$ est défini par :

$$R_\Phi(g) := \mathbb{E}_{(X,Y) \sim \mathbb{P}} [\Phi(Yg(X))]. \quad (4.1.5)$$

Exemples classiques :

- Perte charnière (Hinge) : $\Phi(u) = (1 - u)_+$,
- Perte quadratique : $\Phi(u) = (1 - u)^2$,
- Perte logistique : $\Phi(u) = \ln(1 + e^{-u})$.

Or, nous ne sommes pas assurés qu'optimiser sur $R_\Phi(g)$ nous donne une solution optimale $\hat{f}$ pour notre risque initial $R(f)$. Le théorème de calibration ci-dessous nous donne une estimation de l'erreur commise.

Théorème (Calibration – Zhang, Bartlett-Jordan-McAuliffe) . Soit $\Phi : \mathbb{R} \rightarrow [0, \infty[$ une fonction convexe, dérivable en 0 avec $\Phi'(0) < 0$. Il existe une fonction convexe, non décroissante et continue $\psi : [0, 1] \rightarrow [0, \infty[$ vérifiant $\psi(0) = 0$ telle que pour toute fonction mesurable $g : \mathcal{X} \rightarrow \mathbb{R}$:

$$\psi(R(\text{sgn} \circ g) - R^*) \leq R_\Phi(g) - \inf_{g'} R_\Phi(g'). \quad (4.1.6)$$

En particulier :

- Pour la perte charnière : $\psi(t) = t$,
- Pour la perte quadratique : $\psi(t) = t^2$,
- Pour la perte logistique : $\psi(t) \geq \frac{t^2}{2}$.

4.2 Décomposition du Risque et Classes Finies

4.2.1 Décomposition canonique de l'excès de risque

Dans les chapitres précédents, nous avons décomposé le risque empirique en une erreur *d'approximation* (liée au choix de l'espace de fonction à optimiser $\mathcal{F}$) et une erreur *d'estimation* (liée à l'échantillon restreint S de taille n que l'on considère, purement statistique) :

$$R(\hat{f}_S) - R^* = \underbrace{\left(R(\hat{f}_S) - \inf_{f \in \mathcal{F}} R(f) \right)}_{\text{estimation}} + \underbrace{\left(\inf_{f \in \mathcal{F}} R(f) + R^* \right)}_{\text{approximation}}, \quad (4.2.1)$$

avec $\hat{f}_S$ le minimiseur du risque empirique sur $\mathcal{F}$. Soit $f^\dagger \in \mathcal{F}$ le meilleur prédicteur théorique possible dans $\mathcal{F}$. On a donc :

$$R(\hat{f}_S) - R^* = R(\hat{f}_S) - R(f^\dagger) + R(f^\dagger) - R^*. \quad (4.2.2)$$

Or ici, $\hat{f}_S$ est calculé sur un échantillon fixé S , on ne peut donc pas appliquer la LGN sur la taille de l'échantillon pour obtenir une convergence p.s vers $f^\dagger$. Pour séparer ce qui dépend des données et ce qui dépend de l'estimation dans ce risque, on introduit les risques empiriques $\hat{R}_S(\hat{f}_S)$ et $\hat{R}_S(f^\dagger)$:

$$R(\hat{f}_S) - R(f^\dagger) = \left[R(\hat{f}_S) - \hat{R}_S(\hat{f}_S) \right] + \left[\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger) \right] + \left[\hat{R}_S(f^\dagger) - R(f^\dagger) \right]. \quad (4.2.3)$$

En regardant chaque élément :

- $R(\hat{f}_S) - \hat{R}_S(\hat{f}_S)$ correspond à l'erreur commise sur le calcul du risque de l'estimateur restreint à l'échantillon S . Comme $\hat{f}_S \in \mathcal{F}$, on peut le majorer brutalement par le pire estimateur de la classe $\mathcal{F}$:

$$R(\hat{f}_S) - \hat{R}_S(\hat{f}_S) \leq \sup_{f \in \mathcal{F}} \left| R(f) - \hat{R}_S(f) \right|. \quad (4.2.4)$$

- $\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger)$ correspond à l'erreur empirique du risque entre l'optimum calculé et le meilleur théorique,
- $\hat{R}_S(f^\dagger) - R(f^\dagger)$ correspond à l'erreur commise sur le calcul du risque sur l'échantillon S du meilleur estimateur théorique $f^\dagger$. De la même façon que pour le premier, on peut le majorer par le pire estimateur de $\mathcal{F}$:

$$\hat{R}_S(\hat{f}_S) - R(f^\dagger) \leq \sup_{f \in \mathcal{F}} \left| \hat{R}_S(f) - R(f) \right|. \quad (4.2.5)$$

Le terme du milieu $\hat{R}_S(\hat{f}_S) - \hat{R}_S(f^\dagger)$ correspond à l'erreur d'optimisation. Cette erreur est simplement due à la performance de notre algorithme ou à la puissance de notre unité de calcul. On peut donc la majorer arbitrairement par une constante ε_{opt} (qui dépend par exemple de la puissance de notre GPU, du temps de calcul, etc...).

Proposition (Décomposition canonique de l'excès de risque) Soit $\mathcal{F}$ une classe d'hypothèses, $\hat{f}_S \in \arg \min_{f \in \mathcal{F}} \hat{R}_S(f)$ le minimiseur empirique et $f^\dagger \in \arg \min_{f \in \mathcal{F}} R(f)$ le meilleur estimateur possible dans $\mathcal{F}$. L'erreur d'estimation se majore par :

$$R(\hat{f}_S) - R(f^\dagger) \leq 2 \sup_{f \in \mathcal{F}} |\hat{R}_S(f) - R(f)| + \varepsilon_{opt} \quad (4.2.6)$$

où $\varepsilon_{opt} = \hat{R}_S(\hat{f}_S) - \inf_{f \in \mathcal{F}} \hat{R}_S(f)$ représente l'erreur d'optimisation numérique (nulle si la minimisation est exacte).

On souhaite maintenant estimer (ou borner) l'erreur maximale que l'on peut faire dans $\mathcal{F}$ en estimant le risque $\hat{R}_S$ sur l'échantillon S : $\sup_{f \in \mathcal{F}} |\hat{R}_S(f) - R(f)|$. On va pour cela distinguer deux cas : si $\mathcal{F}$ est une classe finie ou non.

4.2.2 Majoration de l'erreur du risque – cas fini

On suppose ici que $\mathcal{F}$ dispose d'un nombre fini d'éléments $|\mathcal{F}| = M < \infty$ et que les pertes sont bornées par l_∞ . Pour majorer cette quantité, on utilise l'inégalité de Hoeffding.

Théorème (Inégalité de Hoeffding) . Soit $(X_k)_{1 \leq k \leq n}$ une suite de variables aléatoires réelles *indépendantes* telles que pour deux suites $(a_k)_{1 \leq k \leq n}$ et $(b_k)_{1 \leq k \leq n}$, on ait :

$$\forall k \in \llbracket 1, n \rrbracket, \quad \mathbb{P}(a_k \leq X_k \leq b_k) = 1. \quad (4.2.7)$$

Alors, pour tout $t \geq 0$, on peut minorer la concentration de $S_n = X_1 + \dots + X_n$ autour de sa moyenne en probabilité :

$$\mathbb{P}(|S_n - \mathbb{E}(S_n)| \geq t) \leq 2 \exp \left(- \frac{2t^2}{\sum_{i=1}^n (b_i - a_i)^2} \right). \quad (4.2.8)$$

Proposition Or ici, pour toute fonction $f \in \mathcal{F}$ fixée indépendante de l'échantillon S , la perte sur chaque observation $Z_i = l(y_i, f(x_i))$ est une variable aléatoire réelle telle que :

1. Les $(Z_i)_{1 \leq i \leq n}$ sont i.i.d.
2. Pour tout $i \in \llbracket 1, n \rrbracket$, on a $Z_i \in [0, l_\infty]$.
3. La moyenne empirique est : $\hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n Z_i$.
4. L'espérance est : $\mathbb{E}[Z_i] = R(f)$.

On peut donc appliquer l'inégalité de Hoeffding sur la moyenne empirique $\hat{R}_S(f)$ avec $a_k = 0$ et $b_k = l_\infty/n$ pour tout $k \in \llbracket 1, n \rrbracket$. D'où :

$$\forall t \geq 0, \quad \mathbb{P}(|R(f) - \hat{R}_S(f)| \geq t) \leq 2 \exp \left(- \frac{2nt^2}{l_\infty^2} \right). \quad (4.2.9)$$

Sachant contrôler chaque estimateur de la classe $\mathcal{F}$ indépendamment, on souhaite maintenant les contrôler tous¹. Pour contrôler simultanément tous les estimateurs de la classe, on applique

1. "Un anneau unique pour les contrôler tous".

l'inégalité de Boole (*Union Bound*)² sur les $M = |\mathcal{F}|$ éléments :

$$\begin{aligned} \mathbb{P} \left(\sup_{f \in \mathcal{F}} |\hat{R}_S(f) - R(f)| \geq t \right) &\leq \sum_{f \in \mathcal{F}} \mathbb{P} \left(|\hat{R}_S(f) - R(f)| \geq t \right) \\ &\leq 2|\mathcal{F}| \exp \left(-\frac{2nt^2}{l_\infty^2} \right). \end{aligned}$$

En posant cette probabilité d'erreur égale à $\delta \in]0, 1[$, on obtient le seuil d'écart $t = l_\infty \sqrt{\frac{\ln(2|\mathcal{F}|/\delta)}{2n}}$.

Théorème (Borne uniforme pour les classes finies) . Sous les hypothèses $l(y, f(x)) \in [0, l_\infty]$ et $|\mathcal{F}| < \infty$, pour tout $\delta \in]0, 1[$, avec probabilité au moins $1 - \delta$ sur le tirage de $S \sim \mathbb{P}^{\otimes n}$:

$$\sup_{f \in \mathcal{F}} |\hat{R}_S(f) - R(f)| \leq l_\infty \sqrt{\frac{\ln(2|\mathcal{F}|/\delta)}{2n}}. \quad (4.2.10)$$

Par conséquent, pour le minimiseur empirique exact $\hat{f}_S$, on a avec probabilité au moins $1 - \delta$:

$$R(\hat{f}_S) \leq \inf_{f \in \mathcal{F}} R(f) + 2l_\infty \sqrt{\frac{\ln(2|\mathcal{F}|/\delta)}{2n}} + \varepsilon_{\text{opt}}. \quad (4.2.11)$$

Remarque Le résultat est immédiat ici. Si les hypothèses sont vérifiées, nous sommes capables de borner le risque. En revanche, si $|\mathcal{F}| = \infty$, les bornes de celui-ci explosent. Nous devons donc décrire une nouvelle théorie pour traiter ce cas.

4.2.3 Majoration de l'erreur du risque – cas infini

Analysons le résultat précédent : on calcule la probabilité que $\sup_{f \in \mathcal{F}} |\hat{R}_S(f) - R(f)| \geq t$ par une somme finie d'unions d'événements sur $\mathcal{F}$. C'est donc de là que la borne explose si $\mathcal{F}$ est infini.

La solution est donc de traiter la classe entière de fonctions $\mathcal{F}$ en considérant une application g définie sur l'échantillon S qui lui associe l'erreur la plus importante possible sur la classe de fonctions $\mathcal{F}$:

$$g(S) := \sup_{f \in \mathcal{F}} \left(\hat{R}_S(f) - R(f) \right). \quad (4.2.12)$$

C'est donc maintenant l'échantillon qui définit la variation d'erreur. On souhaite toujours la borner : on introduit donc la *méthode des différences bornées*.

Définition (Différences Bornées) . Soit $f : Z_1 \times \cdots \times Z_n \rightarrow \mathbb{R}$ une application. On dit que f vérifie la *condition des différences bornées* pour les constantes $(c_1, \dots, c_n) \in \mathbb{R}^n$ si pour tout $i \in \llbracket 1, n \rrbracket$ on a :

$$\sup_{z_1, \dots, z_i, \dots, z_n, z'_i} |f(z_1, \dots, z_i, \dots, z_n) - f(z_1, \dots, z'_i, \dots, z_n)| \leq c_i. \quad (4.2.13)$$

Autrement dit, si l'on change la i ème entrée z_i de f en z'_i , sa sortie ne bouge pas plus de c_i , et cela pour tout i .

2. Cette inégalité découle des propriétés des mesures. Elle affirme que pour une famille au plus dénombrable de partie mesurables $A_1, A_2, A_3, \dots$, pour toute mesure μ , on a : $\mu \left(\bigcup_n A_n \right) \leq \sum_n \mu(A_n)$.

Proposition (Différences bornées sur g) Appliquons les différences bornées sur l'application g . Puisque pour tout $f \in \mathcal{F}$, le risque $R(f)$ ne dépend pas des données S , on a :

$$\hat{R}_S(f) - \hat{R}_{S'}(f) = \frac{1}{n} (l(z_i, f) - l(z'_i, f)). \quad (4.2.14)$$

Or, par hypothèse, la perte directe est contenue dans $[0, l_\infty]$. Nous devons passer au sup pour conclure sur g .

Lemme (Passage au sup) Soient $u, v : \mathcal{F} \rightarrow \mathbb{R}$ deux applications. Pour tout $f \in \mathcal{F}$, on a l'implication suivante :

$$|u(f) - v(f)| \leq c \longrightarrow \left| \sup_{f \in \mathcal{F}} u(f) - \sup_{f \in \mathcal{F}} v(f) \right| \leq c. \quad (4.2.15)$$

En appliquant le lemme précédent avec $c = \frac{l_\infty}{n}$, on en déduit que pour tout $i \in \llbracket 1, n \rrbracket$:

$$|g(S) - g(S')| = \left| \sup_{f \in \mathcal{F}} (\hat{R}_S(f) - R(f)) - \sup_{f \in \mathcal{F}} (\hat{R}_{S'}(f) - R(f)) \right| \leq \frac{l_\infty}{n}. \quad (4.2.16)$$

La fonction g vérifie donc la propriété des différences bornées avec les constantes coordonnées :

$$c_i = \frac{l_\infty}{n} \quad \text{pour tout } i \in \llbracket 1, n \rrbracket. \quad (4.2.17)$$

Théorème (Inégalité de McDiarmid) . Soient $Z_1, \dots, Z_n$ des variables aléatoires indépendantes à valeurs dans un espace mesurable $\mathcal{Z}$. Soit $\phi : \mathcal{Z}^n \rightarrow \mathbb{R}$ une fonction vérifiant l'hypothèse des différences bornées pour des constantes $(c_1, \dots, c_n) \in \mathbb{R}_+^n$. Alors, pour tout $t \geq 0$:

$$\mathbb{P}(\phi(S) - \mathbb{E}_S[\phi(S)] \geq t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2}\right), \quad (4.2.18)$$

$$\mathbb{P}(\mathbb{E}_S[\phi(S)] - \phi(S) \geq t) \leq \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2}\right), \quad (4.2.19)$$

$$\mathbb{P}(|\phi(S) - \mathbb{E}_S[\phi(S)]| \geq t) \leq 2 \exp\left(-\frac{2t^2}{\sum_{i=1}^n c_i^2}\right). \quad (4.2.20)$$

On décompose la déviation uniforme bilatérale en deux déviations unilatérales :

$$\sup_{f \in \mathcal{F}} |\hat{R}_S(f) - R(f)| \leq \sup_{f \in \mathcal{F}} (\hat{R}_S(f) - R(f)) + \sup_{f \in \mathcal{F}} (R(f) - \hat{R}_S(f)). \quad (4.2.21)$$

Puisque chacune de ces deux applications admet les constantes de variations coordonnées $c_i = \frac{l_\infty}{n}$, la somme des carrés s'évalue explicitement par :

$$\sum_{i=1}^n c_i^2 = \sum_{i=1}^n \left(\frac{l_\infty}{n}\right)^2 = \frac{l_\infty^2}{n}. \quad (4.2.22)$$

L'application de la forme unilatérale de McDiarmid fournit alors le contrôle de concentration suivant :

Proposition (Concentration de la déviation uniforme unilatérale) Sous l'hypothèse d'une perte bornée $l(y, f(x)) \in [0, l_\infty]$, pour tout $\delta \in]0, 1[$, chacune des inégalités suivantes est vérifiée avec probabilité au moins $1 - \delta$ sur le tirage de $S \sim \mathbb{P}^{\otimes n}$:

$$\sup_{f \in \mathcal{F}} (\hat{R}_S(f) - R(f)) \leq \mathbb{E}_S \left[\sup_{f \in \mathcal{F}} (\hat{R}_S(f) - R(f)) \right] + l_\infty \sqrt{\frac{\ln(1/\delta)}{2n}}, \quad (4.2.23)$$

$$\sup_{f \in \mathcal{F}} (R(f) - \hat{R}_S(f)) \leq \mathbb{E}_S \left[\sup_{f \in \mathcal{F}} (R(f) - \hat{R}_S(f)) \right] + l_\infty \sqrt{\frac{\ln(1/\delta)}{2n}}. \quad (4.2.24)$$

Remarque Ce résultat marque une avancée théorique majeure : la déviation uniforme se concentre à une vitesse en $\mathcal{O}(1/\sqrt{n})$ autour de son espérance, *sans aucune hypothèse de finitude ou de structure* sur la classe $\mathcal{F}$.

Toutefois, le problème n'est pas encore résolu : la déviation se concentre autour d'une espérance théorique inconnue $\mathbb{E}_S \left[\sup_{f \in \mathcal{F}} (\hat{R}_S(f) - R(f)) \right]$. Si la classe $\mathcal{F}$ est arbitrairement riche (par exemple l'ensemble de toutes les fonctions mesurables), cette espérance vaut identiquement 1 pour tout n . L'objectif de la section suivante consiste donc à borner cette espérance résiduelle à l'aide de la *symétrisation* et de la *complexité de Rademacher*.

4.3 Complexité de Rademacher

Notation Pour alléger les calculs, on note $z = (x, y)$ et $z_i = (x_i, y_i)$ tirés selon la loi $\mathbb{P}$. Pour tout prédicteur $f \in \mathcal{F}$ et toute fonction de perte $l : \mathcal{Y} \times \mathcal{X} \rightarrow \mathbb{R}$, on associe la fonction de perte induite $h_f : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$ définie par $h_f(z) = l(y, f(x))$. On étudie ainsi la classe de fonctions :

$$\mathcal{H} = l \circ \mathcal{F} = \{z \mapsto l(y, f(x)) \mid f \in \mathcal{F}\}. \quad (4.3.1)$$

Le risque empirique devient $\hat{R}_S(f) = \frac{1}{n} \sum_{i=1}^n h(z_i)$ et le risque théorique $R(f) = \mathbb{E}_z[h(z)]$.

La section précédente a établi que la déviation uniforme se concentre autour de son espérance théorique :

$$\mathbb{E}_S \left[\sup_{f \in \mathcal{F}} (R(f) - \hat{R}_S(f)) \right] = \mathbb{E}_S \left[\sup_{h \in \mathcal{H}} \left(\mathbb{E}_z[h(z)] - \frac{1}{n} \sum_{i=1}^n h(z_i) \right) \right]. \quad (4.3.2)$$

Le processus de *symétrisation* permet de majorer cette espérance par une quantité calculable à partir de perturbations aléatoires de l'échantillon.

4.3.1 Le procédé de symétrisation

Proposition (Symétrisation par échantillon fantôme) Soit $S' = (z'_1, \dots, z'_n) \sim \mathbb{P}^{\otimes n}$ un second échantillon (dit *échantillon fantôme*) indépendant de S et de même loi. Par linéarité de l'espérance :

$$\mathbb{E}_z[h(z)] = \mathbb{E}_{S'} \left[\frac{1}{n} \sum_{i=1}^n h(z'_i) \right]. \quad (4.3.3)$$

Le premier échantillon S étant indépendant de S' , on regroupe les termes :

$$\mathbb{E}_z[h(z)] - \frac{1}{n} \sum_{i=1}^n h(z_i) = \mathbb{E}_{S'} \left[\frac{1}{n} \sum_{i=1}^n (h(z'_i) - h(z_i)) \right]. \quad (4.3.4)$$

Par convexité de la fonction supremum (inégalité de Jensen, $\sup \mathbb{E} \leq \mathbb{E} \sup$), puis en intégrant par rapport à S :

$$\mathbb{E}_S \left[\sup_{h \in \mathcal{H}} \left(\mathbb{E}_z[h(z)] - \frac{1}{n} \sum_{i=1}^n h(z_i) \right) \right] \leq \mathbb{E}_{S, S'} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n (h(z'_i) - h(z_i)) \right]. \quad (4.3.5)$$

Proposition (Injection des variables de Rademacher) Posons $d_i = h(z'_i) - h(z_i)$. Comme z_i et z'_i sont i.i.d., $d_i \stackrel{\mathcal{L}}{=} -d_i$. Soient $\varepsilon_1, \dots, \varepsilon_n$ des variables de Rademacher indépendantes de S, S' telles que $\mathbb{P}(\varepsilon_i = 1) = \mathbb{P}(\varepsilon_i = -1) = \frac{1}{2}$. Alors $(d_1, \dots, d_n) \stackrel{\mathcal{L}}{=} (\varepsilon_1 d_1, \dots, \varepsilon_n d_n)$ pour tout $h \in \mathcal{H}$ conjointement :

$$\begin{aligned} \mathbb{E}_{S, S'} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n d_i \right] &= \mathbb{E}_{S, S', \varepsilon} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \varepsilon_i (h(z'_i) - h(z_i)) \right] \\ &\leq \mathbb{E}_{S', \varepsilon} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \varepsilon_i h(z'_i) \right] + \mathbb{E}_{S, \varepsilon} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n (-\varepsilon_i) h(z_i) \right]. \end{aligned}$$

Comme $-\varepsilon_i \stackrel{\mathcal{L}}{=} \varepsilon_i$, les deux termes sont égaux et l'échantillon fantôme s'élimine :

$$\mathbb{E}_S \left[\sup_{h \in \mathcal{H}} \left(\mathbb{E}_z[h(z)] - \frac{1}{n} \sum_{i=1}^n h(z_i) \right) \right] \leq 2 \mathbb{E}_{S, \varepsilon} \left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \varepsilon_i h(z_i) \right]. \quad (4.3.6)$$

Informatique Graphique

Chapitre 1

Modélisation Géométrique

Contents

1.1	Cadre mathématique et prérequis géométriques	198
1.1.1	Espaces affines et arithmétique des points	199
1.1.2	Combinaisons affines et barycentres	199
1.1.3	Courbes paramétrées	200
1.2	Représentations polynomiales	200
1.2.1	Interpolation vs Approximation	200
1.2.2	Courbes de Bézier et Polynômes de Bernstein	201
1.2.3	B-Splines et Vecteur Nodal	203

L'informatique graphique est une discipline définissant différents modèles et algorithmes pour modéliser numériquement des scènes et des objets. Pour cela, elle se base sur des notions de Mathématiques Appliquées, Géométrie Différentielle et d'Optimisation. L'objectif est de proposer à des graphistes, des dessinateurs, ou des ingénieurs des algorithmes et logiciels capables de modéliser et visualiser tout types de formes, d'un personnage d'animation à une pièce à usiner. Historiquement, l'informatique graphique se regroupe sur trois sous-disciplines transverses. La première se concentre sur *l'animation* : on cherche ici à développer des modèles pour animer des pièces déjà créées. Par exemple on peut chercher à définir des rotations à appliquer à des objets pour décrire le mouvement d'un personnage. La seconde est le *rendu en temps réel*. On cherche ici à donner de la texture à une animation en temps réel pour donner à un utilisateur une vision acceptable de sa scène. Par exemple, on peut s'intéresser au rendu temps réel dans un jeu vidéo où l'utilisateur doit avoir sous les yeux des images qui rentrent dans la charte graphique de son jeu tout en gardant une fluidité suffisante d'affichage à l'écran. Enfin, le *rendu physiquement réaliste* cherche à rendre une image ou une scène la plus réaliste possible. Ici, on cherchera à simuler au plus près les jeux de lumière sur la matière pour donner une vision la plus réaliste possible à l'utilisateur.

L'objectif de ce cours est de proposer des méthodes mathématiques et algorithmiques permettant la création et la modélisation d'environnements en 3D et le traitement d'une géométrie à partir d'une capture.

1.1 Cadre mathématique et prérequis géométriques

La modélisation géométrique vise à représenter et manipuler des formes numériques par des outils d'analyse et d'algèbre linéaire. Contrairement à une description discrète (maillages polygonaux), on privilégie ici une modélisation paramétrique continue à haut niveau.

Pour construire cette modélisation paramétrique, nous nous basons sur des *points de contrôles*. Ces points seront définis par l'utilisateur et serviront de base pour générer une *courbe approximante* de la forme qu'ils décrivent.

1.1.1 Espaces affines et arithmétique des points

En synthèse d'images et en modélisation, il est donc fondamental de distinguer les notions d'*emplacements* (points) et de *déplacements* (vecteurs). Pour les premiers, nous utilisons la notions d'*espace affine* pour les définir formellement.

Définition (Espace affine) . Soit E un $\mathbb{R}$ -espace vectoriel. Un *espace affine* attaché à E est un ensemble non vide $\mathcal{E}$ muni d'une action libre et transitive du groupe additif $(E, +)$ sur $\mathcal{E}$:

$$\begin{aligned}\Phi : \mathcal{E} \times E &\longrightarrow \mathcal{E} \\ (A, \vec{v}) &\longmapsto A + \vec{v}\end{aligned}\tag{1.1.1}$$

qui vérifie :

1. $\forall A \in \mathcal{E}, A + \vec{0} = A$;
2. $\forall A \in \mathcal{E}, \forall \vec{u}, \vec{v} \in E, (A + \vec{u}) + \vec{v} = A + (\vec{u} + \vec{v})$;
3. $\forall A, B \in \mathcal{E}$, il existe un unique vecteur noté $\vec{AB} \in E$ tel que $B = A + \vec{AB}$.

La dimension de $\mathcal{E}$ est par définition la dimension de E . Dans la suite, on considérera $\mathcal{E} \simeq \mathbb{R}^2$ ou $\mathbb{R}^3$.

On définira donc l'espace des points comme un *espace affine* dans lequel nous ne pourrons effectuer que ces opérations. À noter que les opérations sur les points ainsi que leur définition sont *indépendantes* du repère utilisé.

Remarque (Opérations licites) L'espace affine n'est pas muni d'une addition interne naturelle sur ses éléments. Ainsi :

- La somme de deux points $A + B$ n'a *aucun sens géométrique* intrinsèque (elle dépendrait de l'origine choisie).
- La différence de deux points $B - A$ est un vecteur bien défini ($\vec{AB} \in E$).
- La translation d'un point par un vecteur $A + \vec{v}$ est un point bien défini de $\mathcal{E}$.

1.1.2 Combinaisons affines et barycentres

Pour interpoler ou moyenner des positions de points de contrôle, nous avons besoin d'une opération linéaire adaptée à l'espace affine.

Définition (Combinaison affine et Barycentre) . Soit $(P_0, \dots, P_n) \in \mathcal{E}^{n+1}$ une famille finie de points et $(\alpha_0, \dots, \alpha_n) \in \mathbb{R}^{n+1}$ une famille de scalaires.

L'expression formelle $\sum_{i=0}^n \alpha_i P_i$ n'est indépendante de tout repère d'origine $O \in \mathcal{E}$ si et seulement si la somme des poids vérifie la *condition de partition de l'unité* :

$$\sum_{i=0}^n \alpha_i = 1\tag{1.1.2}$$

Dans ce cas, le point $G \in \mathcal{E}$ défini de manière unique par :

$$\sum_{i=0}^n \alpha_i G \vec{P}_i = \vec{0} \iff G = O + \sum_{i=0}^n \alpha_i O \vec{P}_i \quad (\forall O \in \mathcal{E})\tag{1.1.3}$$

est appelé le *barycentre* du système pondéré $\{(P_i, \alpha_i)\}_{i=0}^n$, noté par abus $G = \sum_{i=0}^n \alpha_i P_i$.

Définition (Enveloppe convexe) . Une combinaison affine est dite *convexe* si de plus tous les coefficients sont positifs ou nuls :

$$\forall i \in \llbracket 0, n \rrbracket, \alpha_i \geq 0 \quad \text{avec} \quad \sum_{i=0}^n \alpha_i = 1 \quad (1.1.4)$$

L'*enveloppe convexe* d'un ensemble de points $\mathcal{P} = \{P_0, \dots, P_n\}$, notée $\text{Conv}(\mathcal{P})$, est l'ensemble de toutes leurs combinaisons convexes :

$$\text{Conv}(\mathcal{P}) := \left\{ \sum_{i=0}^n \alpha_i P_i \mid \alpha_i \geq 0, \sum_{i=0}^n \alpha_i = 1 \right\} \quad (1.1.5)$$

En topologie affine, $\text{Conv}(\mathcal{P})$ correspond au plus petit ensemble convexe fermé contenant $\mathcal{P}$. C'est une propriété stabilisatrice clé : toute courbe construite par mélange convexe reste confinée dans cette zone sans divergence numérique.

1.1.3 Courbes paramétrées

Une fois nos *points de contrôles* définis, nous allons définir une courbe approximante de ces points. Nous utilisons pour cela les *courbes paramétriques*.

Définition (Courbe paramétrée) . Une *courbe paramétrée* C dans $\mathcal{E}$ de classe $\mathcal{C}^k$ est la donnée d'une application $f : I \subset \mathbb{R} \longrightarrow \mathcal{E}$ de classe $\mathcal{C}^k$. On posera $I = [u_{\min}, u_{\max}]$ un intervalle fermé de $\mathbb{R}$. On posera donc :

$$C = f(I) = \{f(u) \mid u \in I\} \subset \mathcal{E}. \quad (1.1.6)$$

1.2 Représentations polynomiales

L'objectif est de modéliser des courbes ou des surfaces par des représentations paramétriques continues. Contrairement aux maillages discrets, cette approche dite de *haut niveau* offre une empreinte mémoire réduite et garantit un tracé parfaitement lisse.

Pour manipuler ces formes, on introduit une famille de *points de contrôle* $(P_0, \dots, P_n)$. Ces points servent d'interface intuitive à l'utilisateur pour contraindre la géométrie globale. Idéalement, la courbe générée doit satisfaire trois propriétés géométriques fondamentales :

1. **Diminution de la variation** : la courbe ne doit pas osciller davantage que son polygone de contrôle.
2. **Confinement convexe** : la courbe doit être entièrement contenue dans l'enveloppe convexe formée par ses points de contrôle.
3. **Invariance par symétrie** : le tracé ne doit pas dépendre du sens de parcours des indices des points.

1.2.1 Interpolation vs Approximation

La première approche, naturelle, consiste à chercher une courbe qui *interpole* strictement tous les points de contrôle. Cependant, l'interpolation polynomiale globale (comme celle de Lagrange) devient rapidement instable à mesure que le nombre de points augmente. Comme l'illustre la

– Ajouter figure –

FIGURE 1.1

Figure 1.1 avec le phénomène de Runge, le polynôme produit de fortes oscillations parasites aux extrémités et s'échappe largement de l'enveloppe convexe des données.

Face à ces limites, on privilégie l'approche par *approximation*. Plutôt que d'imposer le passage par chaque point intermédiaire, on utilise les points de contrôle comme des "attracteurs". On garantit ainsi le lissage, la stabilité numérique et le respect strict des contraintes géométriques énoncées ci-dessus.

1.2.2 Courbes de Bézier et Polynômes de Bernstein

Une première approche d'approximation incontournable est celle des courbes de Bézier. Celles-ci reposent sur une pondération barycentrique via les *polynômes de Bernstein*.

Définition (Polynômes de Bernstein) . Pour tout entier $n \geq 0$, on définit la famille des $n + 1$ *polynômes de Bernstein* de degré n sur $[0, 1]$ par :

$$B_i^n(u) = \binom{n}{i} u^i (1-u)^{n-i}, \quad \forall i \in \llbracket 0, n \rrbracket. \quad (1.2.1)$$

Propriété (Propriétés des polynômes de Bernstein) . Pour tout $u \in [0, 1]$, la famille $\{B_i^n(u)\}_{i=0}^n$ satisfait les propriétés fondamentales suivantes :

1. *Partition de l'unité* :

$$\sum_{i=0}^n B_i^n(u) = 1 \quad (1.2.2)$$

2. *Positivité* :

$$\forall i \in \llbracket 0, n \rrbracket, \quad B_i^n(u) \geq 0 \quad (1.2.3)$$

3. *Symétrie* :

$$\forall i \in \llbracket 0, n \rrbracket, \quad B_i^n(u) = B_{n-i}^n(1-u) \quad (1.2.4)$$

4. *Valeurs aux bornes* :

$$B_i^n(0) = \delta_{i,0} \quad \text{et} \quad B_i^n(1) = \delta_{i,n} \quad (1.2.5)$$

où $\delta_{i,j}$ désigne le symbole de Kronecker.

Remarque (Origine algébrique via la formule du binôme) Ces polynômes s'obtiennent naturellement en développant l'identité scalaire $1^n = 1$ pour tout $u \in [0, 1]$:

$$1 = (u + (1-u))^n = \sum_{i=0}^n \binom{n}{i} u^i (1-u)^{n-i} = \sum_{i=0}^n B_i^n(u) \quad (1.2.6)$$

Ce développement démontre immédiatement la partition de l'unité ainsi que la positivité sur $[0, 1]$ (car $u \geq 0$ et $1-u \geq 0$).

On utilisera donc les polynômes de Bernstein comme premier approximant de nos points de contrôle. Pour chaque point de contrôle, on définira un polynôme de Bernstein associé. La somme de tous les polynômes pondérés par les points définira une *courbe paramétrique* dans l'enveloppe convexe de ces derniers.

Définition (Courbe de Bézier) . Soit $\mathcal{E}$ un espace affine de dimension 2 ou 3 et $(P_0, \dots, P_n) \in \mathcal{E}^{n+1}$ une famille ordonnée de $n + 1$ points de contrôle. La courbe de Bézier associée est l'application paramétrique définie par l'application $p : [0, 1] \rightarrow \mathcal{E}$:

$$p(u) = \sum_{i=0}^n B_i^n(u) P_i, \quad \forall u \in [0, 1]. \quad (1.2.7)$$

Puisque $\sum_{i=0}^n B_i^n(u) = 1$ et $B_i^n(u) \geq 0$, le point $p(u)$ est une combinaison convexe bien définie dans $\mathcal{E}$ pour tout $u \in [0, 1]$. Pour $m = n + 1$ points de contrôle, on définira le degré de la courbe par $d = n = m - 1$.

Propriété (Courbes de Bézier) . Une courbe de Bézier B_i^n définie sur un ensemble de points de contrôle $(P_0, \dots, P_n)$ possède différentes propriétés :

- Elle interpole son premier et son dernier point P_0 et P_n .
- Elle est *tangente* au premier segment du polygone de contrôle en P_0 et au dernier segment en P_n .
- Elle approxime tous les autres points $\forall u \in]0, 1[$.

Exemple (Tracé d'une approximation de Bézier) Soient quatre points de contrôle (P_0, P_1, P_2, P_3) placés dans un repère quelconque. On souhaite construire la courbe de Bézier cubique associée (Figure 1.2). Le procédé se décompose en trois étapes :

1. Définition du polygone de contrôle formé par les quatre sommets (Figure 1.2a).
2. Évaluation de la base des polynômes de Bernstein de degré 3 correspondants (Figure 1.2b).
3. Combinaison barycentrique $p(u) = \sum_{i=0}^3 B_i^3(u) P_i$ formant la trajectoire lissée (Figure 1.2c).

On observe bien l'interpolation stricte en P_0 et P_3 , la tangence aux premier et dernier segments, ainsi que le confinement rigoureux dans l'enveloppe convexe.

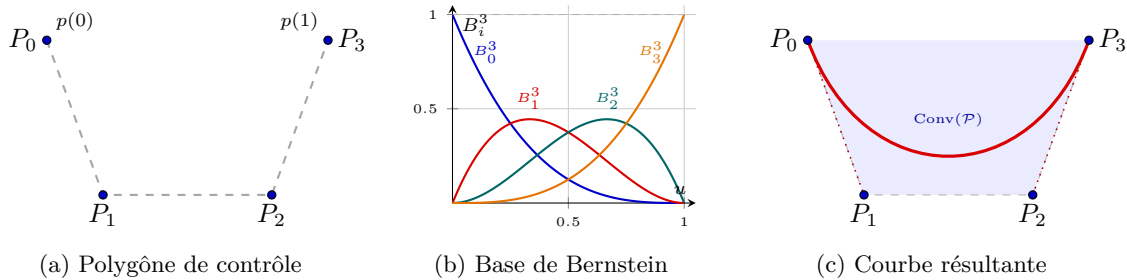


FIGURE 1.2 – Construction d'une approximation de Bézier cubique : du polygone de contrôle au tracé final borné par l'enveloppe convexe.

Remarque En fonction du nombre de points de contrôle, la courbe obtenue aura différentes propriétés :

- Segment Linéaire : 2 points de contrôle,
- Parabole quardatique : 3 points de contrôle,
- Parabole cubique : 4 points de contrôle.

Après plusieurs tracés, on identifie rapidement plusieurs limitations à l'utilisation des courbes de Bézier pour tracer des formes plus complexes. La première est un phénomène d'écrasement

lorsque l'on dépasse 5 à 6 points de contrôles. En effet, par définition, la famille des polynômes de Bernstein à utiliser doit sommer à 1, ils doivent donc mécaniquement s'applatisir pour respecter cette contrainte (visible dans la Figure 1.3a). Cela produit donc une courbe paramétrique plus aplatie (voir Figure 1.3).

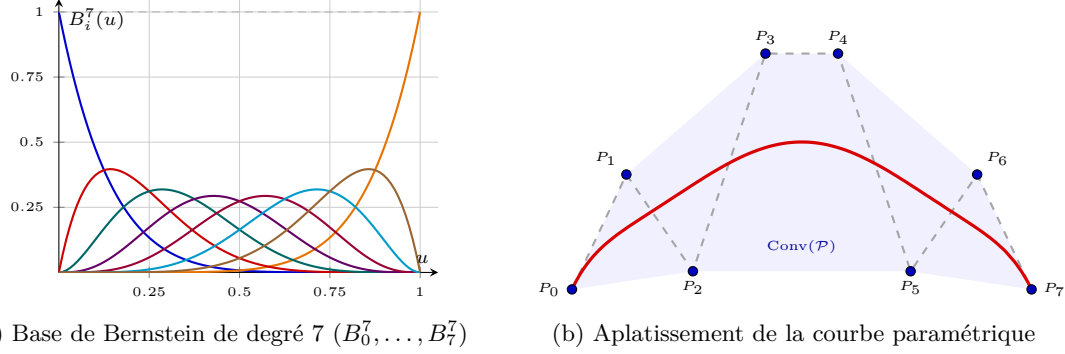


FIGURE 1.3 – Phénomène d'aplatissement pour un degré élevé ($n = 7$) : l'écrasement des polynômes intermédiaires limite la portée locale des points de contrôle centraux.

1.2.3 B-Splines et Vecteur Nodal

Les B-Splines permettent de pallier le principal défaut des courbes de Bézier (l'absence de contrôle local et l'aplatissement) en découplant le degré polynomial du nombre de points de contrôle.

Définition (Vecteur nodal et fonctions de base de Cox-de Boor) . Soit $k \in \mathbb{N}^*$ l'ordre de la B-Spline (correspondant à un degré polynomial $p = k - 1$). Soit $U = (u_0, u_1, \dots, u_m)$ une suite finie non décroissante de réels appelée *vecteur nodal*. Pour tout $i \in \llbracket 0, m - k \rrbracket$, les fonctions de base N_i^k sont définies par la récurrence de *Cox-de Boor* :

- **Initialisation (ordre 1, degré 0) :**

$$N_i^1(u) = \begin{cases} 1 & \text{si } u \in [u_i, u_{i+1}[\\ 0 & \text{sinon} \end{cases} \quad (1.2.8)$$

- **Hérédité (ordre $k \geq 2$, degré $k - 1$) :**

$$N_i^k(u) = \frac{u - u_i}{u_{i+k-1} - u_i} N_i^{k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} N_{i+1}^{k-1}(u) \quad (1.2.9)$$

Dans le cas où deux nœuds consécutifs sont confondus, on applique par convention $0/0 = 0$.

Définition (Courbe B-Spline) . Soit une famille de $n + 1$ points de contrôle $(P_0, \dots, P_n)$ et un ordre k . Le vecteur nodal associé doit nécessairement comporter $m + 1 = n + k + 1$ nœuds.

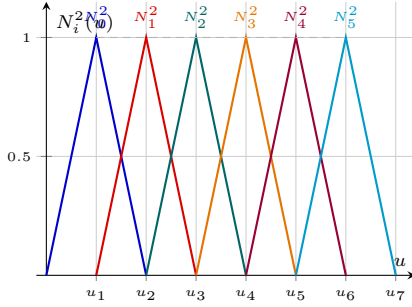
La *courbe B-Spline* associée est définie sur l'intervalle valide $[u_{k-1}, u_{n+1}]$ par :

$$\forall u \in [u_{k-1}, u_{n+1}], \quad P(u) = \sum_{i=0}^n N_i^k(u) P_i \quad (1.2.10)$$

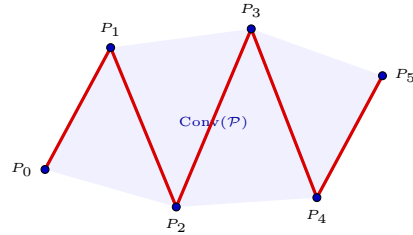
Sur cet intervalle, la partition de l'unité $\sum_{i=0}^n N_i^k(u) = 1$ est vérifiée, garantissant le confinement dans l'enveloppe convexe locale.

Exemple (Degré 1 — Ordre $k = 2$) Soient $n + 1 = 6$ points de contrôle $(P_0, \dots, P_5)$. On construit la B-Spline de degré 1 (ordre $k = 2$).

Le vecteur nodal uniforme nécessite $m + 1 = (n + 1) + k = 6 + 2 = 8$ nœuds : $U = (0, 1, 2, 3, 4, 5, 6, 7)$. Les fonctions de base $N_i^2(u)$ sont linéaires par morceaux (« dents de scie ») à support compact sur $[u_i, u_{i+2}]$ (Figure 1.4a). La courbe résultante $P(u) = \sum_{i=0}^5 N_i^2(u)P_i$ est définie sur $[u_1, u_6] = [1, 6]$ et réalise l'interpolation linéaire directe des sommets (Figure 1.4b).



(a) Fonctions de base $N_i^2(u)$ en dents de scie

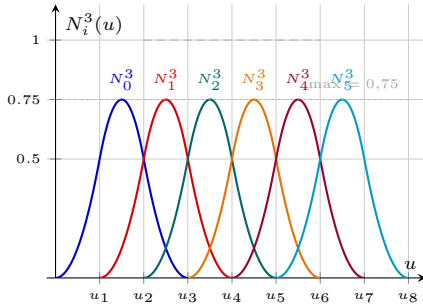


(b) Trajectoire linéaire par morceaux

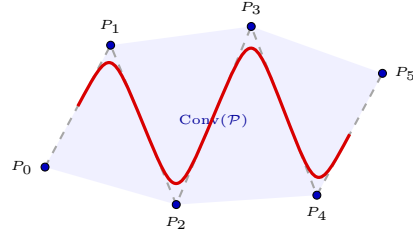
FIGURE 1.4 – B-Spline de degré 1 sur 6 points de contrôle : support compact à deux intervalles $[u_i, u_{i+2}]$ et interpolation linéaire continue C^0 épousant le profil en vague sans aplatissement.

Exemple (Degré 2 — Ordre $k = 3$) En conservant les 6 mêmes points de contrôle, on choisit cette fois un degré 2 (ordre $k = 3$).

Le vecteur nodal requiert désormais $m + 1 = 6 + 3 = 9$ nœuds : $U = (0, 1, 2, 3, 4, 5, 6, 7, 8)$. Les fonctions de base $N_i^3(u)$ sont quadratiques par morceaux de classe C^1 sur $[u_i, u_{i+3}]$ (Figure 1.5a). La courbe $P(u) = \sum_{i=0}^5 N_i^3(u)P_i$ est évaluée sur $[u_2, u_6] = [2, 6]$ et produit une trajectoire lissée sans angle vif (Figure 1.5b).



(a) Fonctions de base $N_i^3(u)$ de degré 2 (C^1)



(b) Courbe lissée C^1 par morceaux

FIGURE 1.5 – B-Spline uniforme de degré 2 sur 6 points de contrôle.

Planification

Chapitre 1

Formalisation de la Planification Classique

Contents

1.1	Le cadre classique	206
1.1.1	États et Actions	206
1.1.2	Plan d'action et problème de planification	208
1.1.3	Propriétés et hypothèses du cadre classique	208
1.2	Algorithmes et Langages de représentation	209
1.2.1	Typologie des approches algorithmiques	209
1.2.2	Formalisme STRIPS et résolution du problème du décor	210
1.2.3	Extension ADL – <i>Action Description Language</i> et PDDL	211

La planification est un outil essentiel dans beaucoup de domaines aujourd'hui et est utilisée par tout le monde : que ce soit pour prévoir des tâches à effectuer, suivre une recette de cuisine ou anticiper et prévoir l'évolution d'un programme industriel.

Toutes ces applications peuvent être formalisées et généralisées au sein de la théorie de la planification. Celle-ci cherche à synthétiser et résoudre de manière automatique un *problème de planification* par la production d'un *plan*. Ce dernier peut se voir comme une collection d'actions à effectuer pour modifier l'état courant de la tâche dans le but d'atteindre un *état cible* appelé *but*. L'exécution de ce plan est donc la réalisation de ces différentes étapes dans le monde pour lequel a été défini le problème.

Nous allons ici formaliser mathématiquement ces concepts pour l'instant abstrait et brièvement étudier la notion de complexité de résolution de tels problèmes.

1.1 Le cadre classique

Commençons donc par définir formellement ce qu'est un problème de planification et les différents objets utilisés pour le résoudre.

1.1.1 États et Actions

Définition (État – Fluent) . On appelle *état* un ensemble fini de formules atomiques sans symbole de variable pour un monde défini. Cet ensemble représente un état possible

du problème considéré.

Une formule atomique (suite finie d'éléments d'un alphabet ne contenant aucun symbole de variable) est appelée un *fluent*.

Pour passer d'un état à un autre, on définira des *actions*.

Définition (Opérateur – Action) . On appellera *opérateur* o un modèle d'action. Il est défini par deux éléments :

1. son nom,
2. un triplet d'ensembles $\langle \text{pr}, \text{add}, \text{de} \rangle$ finis de formules atomiques représentant respectivement les *préconditions*, les *ajouts* et les *suppression* de fluents opérés par l'opérateur sur l'état courant.

On appellera *action*, notée a , une instance d'un opérateur o (i.e toutes les variables de o sont instanciées à partir des variables/constantes du problème). On notera $\text{Pre}(o)$, $\text{Add}(o)$ et $\text{Del}(o)$ les ensembles pr , add et de de l'opérateur o (idem pour a).

Définition (Action Applicable) . Soit E un état et a une action d'un problème de planification. On dira que a est applicable à E si les préconditions de a sont vérifiées dans E , formellement, cela revient à dire que :

$$\text{Pre}(a) \subseteq E. \quad (1.1.1)$$

On définira alors l'état résultant E' de l'action de a sur E par :

$$E' := E \uparrow a = (E \setminus \text{Del}(a)) \cup \text{Add}(a). \quad (1.1.2)$$

Exemple Considérons la préparation d'un gâteau nécessitant de mélanger des ingrédients puis de les cuire. Définissons un opérateur permettant de cuire un ingrédient ou une préparation désignée par la variable $?x$:

$$\begin{aligned} \text{Cuire}(?x) = \langle & \text{Pre} = \{\text{melange}(?x), \text{cru}(?x), \text{four_chaud}\}, \\ & \text{Add} = \{\text{cuit}(?x)\}, \\ & \text{Del} = \{\text{cru}(?x)\} \rangle \end{aligned}$$

En instanciant la variable $?x$ avec la constante *pate*, on obtient l'action close :

$$\begin{aligned} a = \text{Cuire}(\text{pate}) = \langle & \text{Pre}(a) = \{\text{melange}(\text{pate}), \text{cru}(\text{pate}), \text{four_chaud}\}, \\ & \text{Add}(a) = \{\text{cuit}(\text{pate})\}, \\ & \text{Del}(a) = \{\text{cru}(\text{pate})\} \rangle \end{aligned}$$

Soit l'état courant :

$$E = \{\text{melange}(\text{pate}), \text{cru}(\text{pate}), \text{four_chaud}, \text{propre}(\text{moule})\}$$

Puisque $\text{Pre}(a) \subseteq E$, l'action a est applicable. L'état résultant $E' = E \uparrow a$ est :

$$\begin{aligned} E' &= (E \setminus \text{Del}(a)) \cup \text{Add}(a) \\ &= \{\text{melange}(\text{pate}), \text{four_chaud}, \text{propre}(\text{moule}), \text{cuit}(\text{pate})\} \end{aligned}$$

On note que le fait $\text{propre}(\text{moule})$ reste vrai dans E' sans qu'il ait été nécessaire de mentionner son invariance (résolution du problème du décor).

1.1.2 Plan d'action et problème de planification

La résolution d'un problème se fait par l'application séquentielle d'actions applicables sur un état initial E . On appellera cela, un *plan séquentiel*.

Définition (Plan Séquentiel – Application) . On appelle P un *plan séquentiel*, une séquence finie d'actions $\langle a_1, a_2, \dots, a_n \rangle$. L'application d'un plan P à un état E , notée EAP , est définie par récurrence sur la longueur de P :

- Si $E = \perp$, alors $\perp AP := \perp$.
- Pour le plan vide : $E\mathcal{A}\langle \rangle := E$.
- Pour $P = \langle a \rangle \cdot P'$ (où a est une action et P' une séquence d'actions) :

$$E\mathcal{A}(\langle a \rangle \cdot P') := \begin{cases} (E \uparrow a)AP' & \text{si } \text{Prec}(a) \subseteq E, \\ \perp & \text{sinon.} \end{cases} \quad (1.1.3)$$

On peut alors définir le concept de *problème de planification*, noté $\langle \mathcal{O}, I, B \rangle$ tel que :

- $\mathcal{O}$ désigne l'ensemble des opérateurs utilisables dans le domaine considéré. A désigne l'ensemble des actions que l'on peut instancier à partir des opérateurs de $\mathcal{O}$.
- I est l'état initial du problème composé d'un ensemble de fluents.
- B est le but du problème (un état), aussi composé de fluents.

Définition (Plan-solution – flexibilité) . Soit $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ un problème de planification. Un plan séquentiel P est un *plan-solution* (ou *plan correct*) si et seulement si :

$$IAP \neq \perp \quad \text{et} \quad B \subseteq (IAP) \quad (1.1.4)$$

La *flexibilité* d'un plan-solution désigne sa capacité à subir des modifications (modification de l'ordonnancement d'actions non contraintes, instanciation retardée de variables libres) sans perdre sa correction.

1.1.3 Propriétés et hypothèses du cadre classique

Enfin, la production d'un *plan-solution* sera réalisée par un algorithme de planification, noté Planif. On pourra définir plusieurs propriétés recherchées pour cet algorithme.

Propriété (Planificateur) . Soit un algorithme de planification Planif opérant sur un formalisme donné.

- **Sain (ou correct)** : Planif est sain si tout plan qu'il produit pour une instance $\mathcal{P}$ est un plan-solution.
- **Complet** : Planif est complet si, dès lors qu'il existe au moins un plan-solution pour $\mathcal{P}$, il en produit un en temps fini.
- **Systématique** : Planif est systématique si deux feuilles distinctes quelconques de son arbre de recherche correspondent à des plans-solutions distincts (absence de redondance dans l'exploration).
- **Optimal** : Planif est optimal relativement à un critère de coût donné (nombre d'actions, temps d'exécution, parallélisme) si tout plan-solution produit minimise ce critère.

Toute la complexité de la planification réside donc dans la recherche d'un tel algorithme.

Proposition (Hypothèses du cadre classique) Le cadre classique de la planification restreint l'environnement et l'espace d'actions selon six hypothèses :

1. **Fini** : L'ensemble des objets et des fluents est fini.
2. **Statique** : Le monde n'évolue que sous l'effet des actions de l'agent (pas d'événements exogènes).
3. **Observabilité totale** : L'état courant est parfaitement et entièrement connu à chaque étape.
4. **Déterministe** : L'application d'une action a sur un état E produit un unique état résultant prévisible.
5. **Discret** : Les transitions d'états s'effectuent par sauts discrets.
6. **Instantané** : Les actions n'ont pas de durée propre (durée nulle) et ne consomment pas de ressources continues.

La résolution d'un problème de planification par la recherche d'un plan-solution est assez difficile car elle nécessite de définir l'ensemble des objets pour modéliser le problème (états, fluents, actions, etc...). Il faut définir des actions pertinentes en fonction des états choisis et être capable de les décrire au mieux pour permettre un ordonnancement exécutable des plans d'actions générés. Le théorème ci-dessous donne un ordre d'idée de la complexité de cette recherche.

Théorème (Complexité en cadre classique) . Pour un problème de planification classique $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ représenté en logique propositionnelle / STRIPS :

- Le problème de décision de l'*existence d'un plan-solution* (PLANEXIST) est **PSPACE-complet**.
- Le problème de décision de l'*existence d'un plan-solution de longueur inférieure ou égale à $k \in \mathbb{N}$* (avec k encodé en unaire) est **NP-complet**.

1.2 Algorithmes et Langages de représentation

Cette section introduit les principaux paradigmes de résolution d'un problème de planification. On présentera les différentes méthodes canoniques avant d'introduire les langages de représentation (PDDL, STRIPS) et nous finirons par une introduction à GRAPHPLAN avant de le traiter plus en détail dans le chapitre suivant.

1.2.1 Typologie des approches algorithmiques

La résolution effective d'un problème de planification $\mathcal{P} = \langle \mathcal{O}, I, B \rangle$ se heurte à l'explosion combinatoire de l'espace de recherche. Les algorithmes du cadre classique se divisent en deux grandes familles fondamentales.

La première se concentre sur une recherche directe dans les espaces de recherche. Cette approche explore explicitement un graphe orienté où chaque transition correspond à une décision de planification. On distingue deux espaces fondamentaux :

- **Espace des états** : Les nœuds du graphe sont des états complets du monde (voir Figure 1.1). On peut alors trouver un plan-solution par *progression (chaînage avant)* en explorant l'espace depuis l'état initial I et en appliquant des actions applicables jusqu'à atteindre un état contenant B . Ou alors, par *régression (chaînage arrière)* : on part de la spécification du but B et on remonte vers l'état initial I par calcul de préconditions inverses.

- **Espace des plans partiels (POCL – *Partial Order Causal Link*)** : Les nœuds sont des plans incomplets (actions partiellement ordonnées, variables non totalement instanciées et liens causaux). La recherche consiste à éliminer les défauts du plan (préconditions non satisfaites ou conflits causaux) jusqu'à obtenir un plan valide.

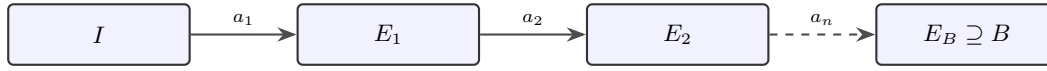


FIGURE 1.1 – Schéma d'un espace d'états d'un problème de planification. On représente les états dans un graphe orienté où chaque arrête traduit l'effet d'une action applicable entre deux états.

Plutôt que d'explorer l'espace d'états de manière séquentielle, les approches par *compilation de plans* (voir Figure 1.2) traduisent le problème de planification de taille ou d'horizon fixé k en une instance d'un problème cible bien caractérisé, bénéficiant ainsi de l'efficacité des solveurs génériques modernes.

- **SATPLAN / BLACKBOX (Compilation propositionnelle)** : Le problème de planification borné à k pas de temps est codé sous la forme d'une formule propositionnelle en forme normale conjonctive (CNF). Un solveur SAT (DPLL, CDCL) recherche une affectation satisfaisable dont le modèle est ensuite décodé en un plan-solution.
- **GP-CSP (Compilation en CSP)** : Les interactions entre fluents et actions sont modélisées sous la forme d'un problème de satisfaction de contraintes (CSP) où les variables représentent les supports d'actions et les contraintes modélisent les exclusions et préconditions.
- **GRAPHPLAN** : Approche intermédiaire qui commence par construire en temps polynomial un graphe compact alternant couches de fluents et d'actions, annoté d'exclusions mutuelles (*mutex*), avant d'y extraire une solution par retour arrière ou compilation.

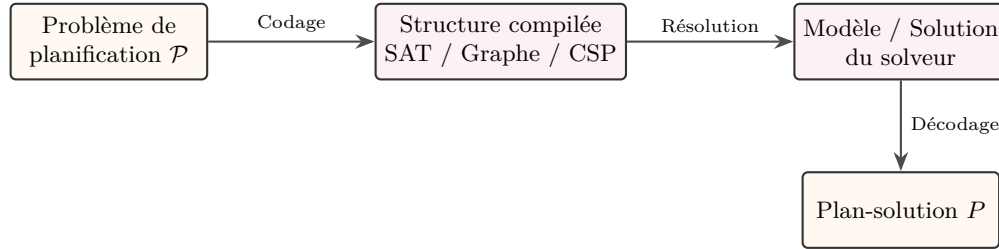


FIGURE 1.2 – Schéma représentant les étapes de construction d'un plan-solution à l'aide d'une méthode par compilation de plans.

La formalisation effective d'un univers de planification impose un compromis fondamental entre le pouvoir expressif du langage de modélisation et la complexité algorithmique du calcul des transitions d'états. L'évolution des formalismes historiques traduit la recherche de cet équilibre.

1.2.2 Formalisme STRIPS et résolution du problème du décor

Le formalisme STRIPS (*Stanford Research Institute Problem Solver*) constitue le noyau minimaliste de la planification classique.

Définition (Modélisation STRIPS) . Dans le formalisme STRIPS, le langage est restreint à la logique propositionnelle sans négation explicite :

- Un état $E \subseteq \mathcal{F}$ est une conjonction finie de formules atomiques closes (fluents). Les faits absents de E sont réputés faux en vertu de l'*hypothèse du monde clos*.
- Un opérateur o est défini par le triplet $\langle \text{Prec}(o), \text{Add}(o), \text{Del}(o) \rangle$, où chaque ensemble ne contient que des littéraux atomiques positifs.

Proposition (Résolution du problème du décor – *Frame Problem*) Soit une action a applicable sur un état E (i.e $\text{Prec}(a) \subseteq E$). STRIPS résout le problème du décor par le principe de persistance implicite :

$$\forall f \in E, \quad f \notin \text{Del}(a) \implies f \in (E \uparrow a) \quad (1.2.1)$$

Aucun axiome d'invariance n'est nécessaire : les fluents non mentionnés dans $\text{Del}(a)$ sont conservés par défaut après l'application de a sur l'état E .

Remarque (Verrous expressifs de STRIPS) La simplicité de STRIPS garantit un calcul de transition $E \uparrow a$ en temps linéaire, mais restreint la modélisation. En effet, il devient impossible de formuler des préconditions négatives ou disjonctives. L'impact d'une action sur un état est aussi statique et indépendant de faits périphériques. Enfin, il est impossible de quantifier sur des ensembles : modifier n entités impose de concevoir et planifier n actions unitaires distinctes.

1.2.3 Extension ADL – *Action Description Language* et PDDL

Introduit pour lever la rigidité de STRIPS, ADL enrichit la description des opérateurs en intégrant des fragments de la logique du premier ordre.

Définition (Caractéristiques du formalisme ADL) . Dans ADL, un opérateur est caractérisé par un couple $\langle \text{Prec}(o), \text{Eff}(o) \rangle$:

- **Préconditions du premier ordre** : $\text{Prec}(o)$ autorise les connecteurs booléens usuels ($\wedge, \vee, \neg$), les prédicats d'égalité ($=$) et les quantificateurs ($\forall, \exists$).
- **Effets conditionnels** : $\text{Eff}(o)$ accepte des clauses d'implication de la forme :

$$c \longrightarrow e \quad (1.2.2)$$

Le littéral e (positif ou négatif) n'est appliqué à l'état que si la condition c était satisfaite avant le déclenchement de l'action.

- **Effets universels** : Des expressions de type $\forall x (P(x) \rightarrow e(x))$ permettent de mettre à jour un ensemble arbitraire d'objets en une seule transition.

PDDL *Planning Domain Description Language* sépare formellement la physique du système de l'instance concrète à résoudre afin de rendre les solveurs indépendants des domaines d'application.

Définition (Architecture PDDL) . Une instance PDDL dissocie la modélisation en deux composantes :

1. **Fichier Domaine (domain.pddl)** : Spécifie les types d'objets, la liste des prédicats relationnels admis et les schémas d'actions paramétrés.
2. **Fichier Problème (problem.pddl)** : Déclare les constantes réelles (objets), l'état initial exhaustif I et la formule logique du but B .

Chapitre 2

Le Planificateur GRAPHLAN

Contents

2.1	Définitions et fonctionnement	212
2.1.1	Interactions entre actions et Indépendance	212
2.1.2	Exclusions mutuelles (<i>Mutex</i>)	213
2.2	Améliorations	214

2.1 Définitions et fonctionnement

L'algorithme de planification GRAPHLAN sépare le processus de planification en deux étapes différentes.

La première consiste en la construction d'un graphe de planification représentant les interdépendances entre les états au travers des actions possibles. Cette construction se fait avec une complexité polynômiale en temps et en espace par rapport à la taille du problème posé.

La seconde étape est la recherche d'un solution (s'il en existe) à partir d'un sous-graphe du précédent. Elle produit alors un plan-solution au problème initialement posé.

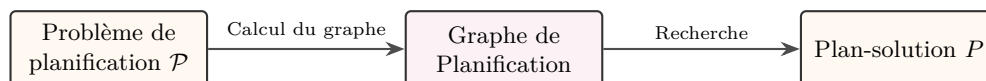


FIGURE 2.1 – Schéma représentant les étapes de l'algorithme GRAPHLAN pour la recherche d'un plan solution à partir d'un problème de planification énoncé dans le langage STRIPS.

Nous allons ici commencer par poser un formalisme supplémentaire pour traduire les relations d'inter-dépendances dans un tel graphe d'état.

2.1.1 Interactions entre actions et Indépendance

Pour de gros problèmes de planification, tels que des programmes de production industriels ou des projets collaboratifs, il est intéressant de pouvoir exécuter plusieurs actions simultanément dans le graphe de planification. On construit pour cela des ensemble d'*actions indépendantes* Q .

Définition (Interactions Positives/Négatives) . Soient deux actions closes a_1 et a_2 , on définit deux types d'interactions entre ces deux actions. On dira qu'elles sont en *coopération*

ou qu'elles interagissent *positivement* si elles produisent les mêmes effets utiles :

$$\text{Add}(a_1) \cap \text{Add}(a_2) = \emptyset \quad (2.1.1)$$

ou si l'une produit une condition requise par l'autre :

$$\text{Add}(a_1) \cap \text{Prec}(a_2) = \emptyset. \quad (2.1.2)$$

En revanche, on dira qu'elles sont en *conflit* si l'une supprime ce que l'autre ajoute (effet antagoniste) :

$$\text{Add}(a_1) \cap \text{Del}(a_2) \neq \emptyset \quad \text{ou} \quad \text{Add}(a_2) \cap \text{Del}(a_1) \neq \emptyset \quad (2.1.3)$$

ou si l'une supprime une condition nécessaire au déclenchement de l'autre (effet croisé) :

$$\text{Del}(a_1) \cap \text{Prec}(a_2) \neq \emptyset \quad \text{ou} \quad \text{Del}(a_2) \cap \text{Prec}(a_1) \neq \emptyset. \quad (2.1.4)$$

L'objectif sera de limiter au plus les effets conflits entre actions pour favoriser les interactions positives en vue de construire un plan.

Définition (Indépendance) . On dira que deux actions a_1 et a_2 sont *indépendantes*, noté $a_1 \# a_2$, si elles n'ont pas d'interactions négatives (conflits) :

$$\text{Del}(a_1) \cap (\text{Prec}(a_2) \cup \text{Add}(a_2)) = \emptyset \quad \text{et} \quad \text{Del}(a_2) \cap (\text{Prec}(a_1) \cup \text{Add}(a_1)) = \emptyset. \quad (2.1.5)$$

On dira que Q est un ensemble d'actions indépendantes si toutes ses actions sont deux à deux indépendantes.

Propriété (Indépendance) . Soient deux actions a_1 et a_2 applicables sur un état E . Si a_1 et a_2 sont indépendantes, alors leur ordre d'application sur E n'a pas d'importance :

$$(E \uparrow a_1) \uparrow a_2 = (E \uparrow a_2) \uparrow a_1 = E \uparrow \{a_1, a_2\}. \quad (2.1.6)$$

Proposition Soit Q un ensemble d'actions indépendantes et E un état. Alors Q est *applicable* sur E ssi :

$$\bigcup_{a_i \in Q} \text{Prec}(a_i) \subseteq E. \quad (2.1.7)$$

L'état résultant de l'application de tous les $a_i \in Q$ sur E sera caractérisé par :

$$E \uparrow Q = \left(E \setminus \bigcup_{a_i \in Q} \text{Del}(a_i) \right) \cup \left(\bigcup_{a_i \in Q} \text{Add}(a_i) \right). \quad (2.1.8)$$

On peut alors appliquer un ensemble d'actions indépendantes à un état donné en une fois. On définira alors des *plans d'ensembles d'actions indépendantes* $P = \langle Q_1, Q_2, \dots, Q_n \rangle$ où les Q_i sont des actions indépendantes. Leur application se fera donc successivement sur un état initial E comme illustré en Figure 2.2.

2.1.2 Exclusions mutuelles (*Mutex*)

Le graphe de planification sur-approximant l'ensemble des états atteignables, des relations d'exclusion mutuelle (*mutex*) sont introduites à chaque niveau pour contraindre la recherche et élaguer les combinaisons physiquement irréalisables. La relation est symétrique et binaire et est définie aussi bien sur les actions que sur les fluents.

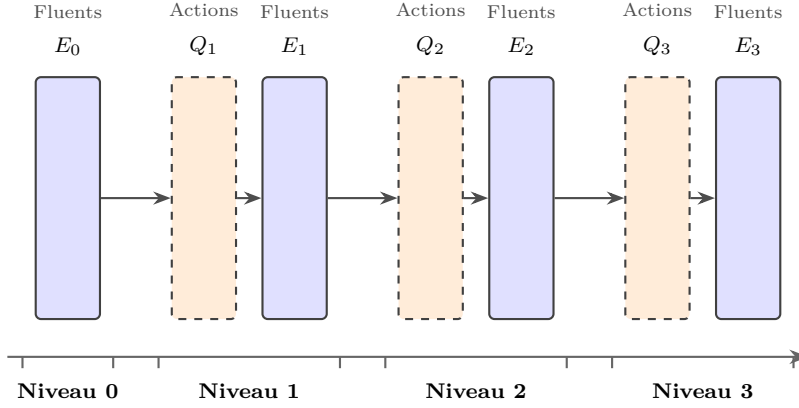


FIGURE 2.2 – Structure alternée par niveaux dans un plan d'ensembles d'actions. Chaque niveau $i \geq 1$ applique simultanément un ensemble d'actions indépendantes Q_i sur l'état courant pour produire le nouvel état de fluents E_i .

Définition (Exclusion mutuelle entre actions). Soient deux actions a_1, a_2 appartenant au même niveau d'actions i . Les actions a_1 et a_2 sont dites *mutuellement exclusives* au niveau i , noté $\text{Mutex}(a_1, a_2)$, si et seulement si l'une des deux conditions suivantes est vérifiée :

1. **Non-indépendance (conflit direct)** : Les actions a_1 et a_2 ne sont pas indépendantes ($a_1 \# a_2$), c'est-à-dire :

$$\text{Del}(a_1) \cap (\text{Prec}(a_2) \cup \text{Add}(a_2)) \neq \emptyset \quad \text{ou} \quad \text{Del}(a_2) \cap (\text{Prec}(a_1) \cup \text{Add}(a_1)) \neq \emptyset \quad (2.1.9)$$

2. **Besoins concurrents (conflit hérité)** : Il existe au moins une paire de préconditions mutuellement exclusives au niveau de fluents précédent F_{i-1} :

$$\exists (p, q) \in \text{Prec}(a_1) \times \text{Prec}(a_2) \quad \text{tel que} \quad \text{Mutex}(p, q) \text{ dans } F_{i-1} \quad (2.1.10)$$

Définition (Exclusion mutuelle entre fluents). Soient deux fluents $p, q \in F_i$ au niveau de fluents i . Les fluents p et q sont dits *mutuellement exclusifs* au niveau i , noté $\text{Mutex}(p, q)$, si et seulement si **toutes** les paires d'actions susceptibles de les produire simultanément à ce niveau sont en exclusion mutuelle :

$$\forall (a_1, a_2) \in A_i \times A_i, \quad (p \in \text{Add}(a_1) \wedge q \in \text{Add}(a_2)) \implies \text{Mutex}(a_1, a_2) \quad (2.1.11)$$

Autrement dit, p et q ne sont pas mutex dans F_i dès lors qu'il existe au moins une paire d'actions (a_1, a_2) non mutex à l'étape i telle que $p \in \text{Add}(a_1)$ et $q \in \text{Add}(a_2)$.

Remarque (Rôle des actions de persistance) Une action de maintien $N_f = \langle \{f\}, \{f\}, \emptyset \rangle$ est traitée comme une action usuelle. Par conséquent, si une action $a \in A_i$ supprime le fluent f ($f \in \text{Del}(a)$), elle entre directement en conflit de non-indépendance avec N_f car $\text{Del}(a) \cap \text{Add}(N_f) = \{f\} \neq \emptyset$. Cela garantit qu'un fluent f et sa négation deviennent automatiquement mutex au niveau suivant.

2.2 Améliorations

Traitement du signal

Chapitre 1

Signaux à temps discrets et Systèmes

Contents

1.1	Systèmes et signaux à temps discret	216
1.1.1	Généralités	216
1.1.2	Opérations élémentaires	218
1.1.3	Systèmes discrets	218
1.2	Convolution et corrélation	219
1.2.1	Convolution linéaire	219
1.2.2	Convolution circulaire et Zero-Padding	220
1.2.3	Corrélation à temps discret	220
1.2.4	Formulaire de sommes classiques utiles	221

1.1 Systèmes et signaux à temps discret

1.1.1 Généralités

Définition (Signal) . On définit un signal numérique comme une suite $(x_n)_{n \in \mathbb{Z}}$ à valeurs dans $\mathbb{C}$. En pratique, elle sera finie et sera stockée sous forme de vecteur :

$$x = [\dots, x(-1), x(0), x(1), x(2), \dots]^T$$

Exemple Voyons quelques exemples de signaux caractéristiques :

- **L'échantillon unitaire**, aussi appelé Dirac :

$$x(n) = \delta(n) = \begin{cases} 1 & \text{si } n = 0 \\ 0 & \text{sinon} \end{cases} \quad (1.1.1)$$

- **Fonction de Heaviside** :

$$x(n) = u(n) = \begin{cases} 1 & \text{si } n \geq 0 \\ 0 & \text{sinon} \end{cases} \quad (1.1.2)$$

- **Sinus/Cosinus réel :**

$$x(n) = A \cos(2\pi f_0 n) \quad (1.1.3)$$

Avec $A \in \mathbb{R}_+$ l'amplitude du signal.

- **Exponentiel réel :**

$$x(n) = a^n, \quad a \in \mathbb{R} \quad (1.1.4)$$

- **Exponentiel complexe :**

$$x(n) = c^n, \quad c \in \mathbb{C} \quad (1.1.5)$$

Si $c = e^{j\pi f_0}$ (i.e. c appartient au cercle unité), alors :

$$x(n) = e^{j\pi n f_0} = \cos(\pi n f_0) + j \sin(\pi n f_0)$$

- **Porte :**

$$x(n) = \begin{cases} 1 & \text{si } N \leq n \leq M \\ 0 & \text{sinon} \end{cases} \quad N, M \in \mathbb{Z} \quad (1.1.6)$$

Remarque On peut remarquer que l'échantillon unitaire peut s'écrire en fonction de la fonction de Heaviside sous la forme :

$$\forall n \in \mathbb{Z}, \quad \delta(n) = u(n) - u(n-1)$$

De même, on a :

$$u(n) = \sum_{k=-\infty}^n \delta(k)$$

Et pour le signal porte entre N et M :

$$x(n) = u(n-N) - u(n-M-1)$$

Définition (Signal fini/périodique) . Un signal $(x_n)_{n \in \mathbb{Z}}$ est dit *de longueur finie* s'il existe $N_1, N_2 \in \mathbb{Z}$ tels que :

$$\forall n \notin \llbracket N_1, N_2 \rrbracket, \quad x(n) = 0 \quad (1.1.7)$$

On dira alors que le signal a une longueur de $N_2 - N_1 + 1$. On dira aussi qu'il est périodique s'il existe $N \in \mathbb{N}^*$ tel que :

$$\forall n \in \mathbb{Z}, \quad x(n+N) = x(n) \quad (1.1.8)$$

Définition (Énergie et Puissance) . Pour tout signal discret $(x_n)_{n \in \mathbb{Z}}$, on définit :

- Son **énergie totale** :

$$E = \sum_{n=-\infty}^{+\infty} |x(n)|^2 \quad (1.1.9)$$

Si $E < \infty$, le signal est dit à énergie finie.

- Sa **puissance moyenne** :

$$P_{\text{moy}} = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=-N}^N |x(n)|^2 \quad (1.1.10)$$

Exemple Pour l'échelon unité $x(n) = u(n)$:

$$E = \sum_{n=0}^{+\infty} |1|^2 \longrightarrow +\infty \quad (\text{énergie infinie})$$

$$P_{\text{moy}} = \lim_{N \rightarrow \infty} \frac{1}{2N+1} \sum_{n=0}^N 1 = \lim_{N \rightarrow \infty} \frac{N+1}{2N+1} = \frac{1}{2}$$

1.1.2 Opérations élémentaires

Définition (Opérations) . Soit $(x_n)_{n \in \mathbb{Z}}$ un signal à valeurs dans $\mathbb{C}$. On définit les opérations suivantes :

- **Décalage** : Pour $n_0 \in \mathbb{Z}$, on a :

$$\forall n \in \mathbb{Z}, \quad y(n) = x(n - n_0)$$

Si $n_0 > 0$ alors y est retardé de n_0 , sinon si $n_0 < 0$ il est avancé.

- **Retournement** : $\forall n \in \mathbb{Z}, \quad y(n) = x(-n)$.
- **Sous-échantillonnage** :

$$y(n) = x(Mn)$$

où $M > 0$ est appelé *facteur de sous-échantillonnage*.

- **Sur-échantillonnage** :

$$\forall n \in \mathbb{Z}, \quad y(n) = x(n/N) \quad \Longleftrightarrow \quad y(n) = \begin{cases} x(n/N) & \text{si } n = 0, \pm N, \pm 2N, \dots \\ 0 & \text{sinon} \end{cases}$$

où $N > 0$ est le facteur de sur-échantillonnage.

Théorème (Décomposition) . Tout signal $(x_n)_{n \in \mathbb{Z}} \in \mathbb{C}^{\mathbb{Z}}$ peut être décomposé sous la forme :

$$\forall n \in \mathbb{Z}, \quad x(n) = \sum_{k \in \mathbb{Z}} x(k) \delta(n - k) \quad (1.1.11)$$

1.1.3 Systèmes discrets

Définition (Système) . Un système T est une transformation qui prend un signal $x(n)$ à son entrée et délivre un signal $y(n) = T[x(n)]$ à sa sortie.

On dira que T est *linéaire* si pour tous $\alpha_1, \alpha_2 \in \mathbb{C}$ et tous signaux x_1, x_2 , on a :

$$\forall n \in \mathbb{Z}, \quad T[\alpha_1 x_1(n) + \alpha_2 x_2(n)] = \alpha_1 T[x_1(n)] + \alpha_2 T[x_2(n)] \quad (1.1.12)$$

Propriété (Propriétés des systèmes à temps discret) . Soient $x, y \in \mathbb{C}^{\mathbb{Z}}$ deux signaux et T un système. On caractérise T par les propriétés :

- **Causalité** : Si pour tout $n_0 \in \mathbb{Z}$, la réponse $T[x(n_0)]$ ne dépend que des entrées et/ou sorties passées et présentes ($n \leq n_0$).
- **Invariance par décalage** : Si

$$T[x(n)] = y(n) \implies \forall n_0 \in \mathbb{Z}, \quad T[x(n - n_0)] = y(n - n_0) \quad (1.1.13)$$

Dans le cas contraire, le système est dit variant par décalage.

- **Stabilité (BIBO)** : Si pour toute entrée bornée, la sortie est bornée :

$$\exists A \in \mathbb{R}_+, \quad |x(n)| \leq A \implies \exists B \in \mathbb{R}_+, \quad |y(n)| \leq B \quad (1.1.14)$$

Un système linéaire et invariant par translation/décalage est désigné par l'acronyme **SLIT**.

Définition (Système physiquement réalisable) . Un système T est dit *physiquement réalisable* s'il est à la fois **stable** et **causal**.

Définition (Réponse impulsionnelle) . On appelle *réponse impulsionnelle* le signal $h(n)$ obtenu lorsque l'entrée est l'impulsion unitaire :

$$\forall n \in \mathbb{Z}, \quad h(n) = T[\delta(n)] \quad (1.1.15)$$

Proposition (Critères fondamentaux sur un SLIT) Soit un SLIT de réponse impulsionnelle $h(n)$:

- Le système est **stable** si et seulement si :

$$\sum_{n=-\infty}^{+\infty} |h(n)| < \infty \quad (1.1.16)$$

- Le système est **causal** si et seulement si :

$$\forall n < 0, \quad h(n) = 0 \quad (1.1.17)$$

1.2 Convolution et corrélation

Pour un SLIT, la décomposition de l'entrée donne :

$$y(n) = T[x(n)] = T \left[\sum_{k \in \mathbb{Z}} x(k) \delta(n-k) \right] = \sum_{k \in \mathbb{Z}} x(k) T[\delta(n-k)]$$

Puisque le système est invariant par décalage, $T[\delta(n-k)] = h(n-k)$, d'où :

$$y(n) = \sum_{k \in \mathbb{Z}} x(k) h(n-k) = (x * h)(n)$$

1.2.1 Convolution linéaire

Définition (Convolution temporelle) . Soient $x, y \in \mathbb{C}^{\mathbb{Z}}$. On définit le produit de convolution entre x et y par :

$$\forall n \in \mathbb{Z}, \quad (x * y)(n) = \sum_{k=-\infty}^{+\infty} x(k) y(n-k) \quad (1.2.1)$$

Propriété (Propriétés algébriques) . Pour tous signaux x, y, z :

- i) *Commutativité* : $x * y = y * x$.
- ii) *Associativité* : $(x * y) * z = x * (y * z)$.
- iii) *Distributivité* : $x * (y + z) = x * y + x * z$.

Propriété (Propriétés avec l'impulsion de Dirac) .

- i) Élément neutre : $(x * \delta)(n) = x(n)$.
- ii) Décalage : $x(n) * \delta(n - n_0) = x(n - n_0)$.
- iii) Composition d'impulsions : $\delta(n - n_0) * \delta(n - n_1) = \delta(n - n_0 - n_1)$.

Proposition (Signaux de longueur finie) Si x est de longueur N_x et y de longueur N_y , alors le signal convolué $z = x * y$ est de longueur finie au maximum égale à :

$$N_z = N_x + N_y - 1 \quad (1.2.2)$$

1.2.2 Convolution circulaire et Zero-Padding

Définition (Extension périodique) . Soit $x(n)$ un signal de longueur finie N . On définit son extension périodique $x_N(n)$ de période N par :

$$\forall n \in \mathbb{Z}, \quad x_N(n) = x(n \bmod N) = \sum_{p=-\infty}^{+\infty} x(n + pN) \quad (1.2.3)$$

Définition (Convolution circulaire) . Soient x et y deux signaux de même longueur finie N . La convolution circulaire est définie par :

$$\boxed{\forall n \in \llbracket 0, N-1 \rrbracket, \quad (x \circ y)(n) = \sum_{k=0}^{N-1} y(k)x_N(n-k)} \quad (1.2.4)$$

Proposition (Formulation matricielle) En posant $x = [x(0), \dots, x(N-1)]^T$ et $y = [y(0), \dots, y(N-1)]^T$, la convolution circulaire $z = x \circ y$ s'écrit sous forme d'un produit matrice-vecteur :

$$z = C \cdot x \quad (1.2.5)$$

où C est une matrice circulante construite à partir de y . Pour $N = 3$:

$$\begin{bmatrix} z(0) \\ z(1) \\ z(2) \end{bmatrix} = \begin{bmatrix} y(0) & y(2) & y(1) \\ y(1) & y(0) & y(2) \\ y(2) & y(1) & y(0) \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \end{bmatrix}$$

Théorème (Équivalence linéaire/circulaire par Zero-Padding) . Soient x et y de longueurs N_x et N_y . Pour obtenir la convolution linéaire exacte via une convolution circulaire :

1. On fixe la taille commune à $N = N_x + N_y - 1$.
2. On forme x_z en complétant x par $N_y - 1$ zéros, et y_z en complétant y par $N_x - 1$ zéros.
3. On a alors rigoureusement :

$$\boxed{\forall n \in \llbracket 0, N-1 \rrbracket, \quad (x_z \circ y_z)(n) = (x * y)(n)} \quad (1.2.6)$$

1.2.3 Corrélation à temps discret

Définition (Intercorrélation et Autocorrélation) . Soient $x, y \in \mathbb{C}^{\mathbb{Z}}$. La fonction d'intercorrélacion est définie par :

$$\forall n \in \mathbb{Z}, \quad C_{xy}(n) = \sum_{k=-\infty}^{+\infty} x(k)y(k-n) \quad (1.2.7)$$

Si $x = y$, on parle de fonction d'autocorrélation $C_{xx}(n)$.

Propriété (Propriétés de la corrélation) .

- i) Lien avec la convolution : $C_{xy}(n) = x(n) * y(-n)$.
- ii) Symétrie : $C_{xy}(n) = C_{yx}(-n)$, et pour l'autocorrélation : $C_{xx}(n) = C_{xx}(-n)$.
- iii) Maximum d'autocorrélation : $C_{xx}(0)$ est maximal et représente l'énergie du signal : $C_{xx}(0) = \sum |x(k)|^2 = E_x$.

Définition (Estimateurs pour signaux de longueur finie N) . Pour des signaux d'horizon N , on utilise :

- La forme biaisée :

$$C_{xy}(n) = \frac{1}{N} \sum_{k=0}^{N-n-1} x(k)y(k-n) \quad (1.2.8)$$

- La forme non biaisée :

$$C_{xy}(n) = \frac{1}{N-|n|} \sum_{k=0}^{N-n-1} x(k)y(k-n) \quad (1.2.9)$$

1.2.4 Formulaire de sommes classiques utiles

Proposition (Sommes finies et séries géométriques) • $\sum_{n=0}^{N-1} a^n = \frac{1-a^N}{1-a} \quad (a \neq 1)$

- $\sum_{n=0}^{+\infty} a^n = \frac{1}{1-a} \quad (|a| < 1)$
- $\sum_{n=0}^{N-1} na^n = \frac{(N-1)a^{N+1} - Na^N + a}{(1-a)^2}$
- $\sum_{n=0}^{+\infty} na^n = \frac{a}{(1-a)^2} \quad (|a| < 1)$
- $\sum_{n=0}^{N-1} n = \frac{N(N-1)}{2}, \quad \sum_{n=0}^{N-1} n^2 = \frac{N(N-1)(2N-1)}{6}$

Chapitre 2

Transformée de Fourier et TFD

Contents

2.1	Transformée de Fourier des signaux à temps discret (TFTD)	222
2.1.1	Définitions et conventions	222
2.1.2	Propriétés de la TFTD	223
2.2	Transformée de Fourier Discrète (TFD) et Algorithme FFT	224
2.2.1	Définition de la TFD	224
2.2.2	Algorithme de Fourier Rapide (FFT)	224
2.3	Troncature spectrale et Fenêtres d'apodisation	225

2.1 Transformée de Fourier des signaux à temps discret (TFTD)

2.1.1 Définitions et conventions

Définition (TFTD en convention fréquence) . Pour un signal $x(n)$ à temps discret, sa Transformée de Fourier à Temps Discret est la fonction continue $X(f)$ définie par :

$$X(f) = \sum_{n=-\infty}^{+\infty} x(n)e^{-2j\pi n f} \quad (2.1.1)$$

Remarques clés :

- Bien que $x(n)$ soit discret, $X(f)$ est une fonction continue de la fréquence f .
- $X(f) \in \mathbb{C}$, avec $X(f) = |X(f)|e^{j\text{Arg}(X(f))}$.
- $X(f)$ est périodique de période 1 : $X(f+1) = X(f)$.

Définition (TFTD inverse) . La reconstruction du signal temporel s'effectue sur une période fréquentielle unitaire :

$$x(n) = \int_{-1/2}^{+1/2} X(f)e^{2j\pi n f} df \quad (2.1.2)$$

On note symboliquement cette correspondance : $x(n) \xleftrightarrow{\mathcal{F}} X(f)$.

Remarque (Convention en pulsation ω) En posant la pulsation normalisée $\omega = 2\pi f$ ($\omega \in [-\pi, \pi]$) :

$$X(\omega) = \sum_{n=-\infty}^{+\infty} x(n)e^{-jn\omega}, \quad x(n) = \frac{1}{2\pi} \int_{-\pi}^{+\pi} X(\omega)e^{jn\omega} d\omega \quad (2.1.3)$$

$X(\omega)$ est alors périodique de période 2π . Le passage s'opère directement par $X(f) = X(\omega)|_{\omega=2\pi f}$.

2.1.2 Propriétés de la TFTD

Propriété (Propriétés fondamentales) . Soient $x(n) \xleftrightarrow{\mathcal{F}} X(f)$ et $y(n) \xleftrightarrow{\mathcal{F}} Y(f)$:

- **Linéarité** : $ax(n) + by(n) \xleftrightarrow{\mathcal{F}} aX(f) + bY(f)$.
- **Retournement temporel** : $x(-n) \xleftrightarrow{\mathcal{F}} X(-f)$.
- **Décalage temporel** : $x(n - n_0) \xleftrightarrow{\mathcal{F}} X(f)e^{-2j\pi n_0 f}$.
- **Modulation fréquentielle** : $x(n)e^{2j\pi n f_0} \xleftrightarrow{\mathcal{F}} X(f - f_0)$.
- **Convolution temporelle** : $(x * y)(n) \xleftrightarrow{\mathcal{F}} X(f) \cdot Y(f)$.
- **Multiplication temporelle** : $x(n) \cdot y(n) \xleftrightarrow{\mathcal{F}} (X * Y)(f) = \int_{-1/2}^{1/2} X(u)Y(f - u) du$.
- **Théorème de Parseval** : $\sum_{n=-\infty}^{+\infty} |x(n)|^2 = \int_{-1/2}^{1/2} |X(f)|^2 df$.

Proposition (Symétries de la TFTD)

Signal temporel $x(n)$	Spectre $X(f)$
Réel	Hermitien : $X(-f) = X^*(f)$ ($ X(f) $ pair, phase impaire)
Réel et pair	Réel et pair
Réel et impair	Imaginaire pur et impair
Imaginaire pur	Anti-hermitien
Imaginaire et pair	Imaginaire pur et pair
Imaginaire et impair	Réel et impair

Proposition (Paires usuelles de transformées)

$x(n)$	$X(f)$
$\delta(n)$	1
$\delta(n - n_0)$	$e^{-2j\pi n_0 f}$
1	$\delta(f)$ sur $[-1/2, 1/2]$
$e^{2j\pi f_0 n}$	$\delta(f - f_0)$
$\cos(2\pi f_0 n)$	$\frac{1}{2}[\delta(f - f_0) + \delta(f + f_0)]$
$\sin(2\pi f_0 n)$	$\frac{1}{2j}[\delta(f - f_0) - \delta(f + f_0)]$
$a^n u(n) \quad (a < 1)$	$\frac{1}{1 - ae^{-2j\pi f}}$

2.2 Transformée de Fourier Discrète (TFD) et Algorithme FFT

2.2.1 Définition de la TFD

On traite désormais des signaux réels ou complexes de durée finie N indexés sur $\llbracket 0, N-1 \rrbracket$.

Définition (TFD et TFD inverse) . La Transformée de Fourier Discrète (TFD) sur N points est donnée par :

$$X(m) = \sum_{n=0}^{N-1} x(n) e^{-2j\pi \frac{nm}{N}}, \quad m \in \llbracket 0, N-1 \rrbracket \quad (2.2.1)$$

Sa transformée inverse (ITFD) est définie par :

$$x(n) = \frac{1}{N} \sum_{m=0}^{N-1} X(m) e^{2j\pi \frac{nm}{N}}, \quad n \in \llbracket 0, N-1 \rrbracket \quad (2.2.2)$$

On note : $x(n) \xleftrightarrow{\text{TFD}} X(m)$.

Remarque (Échantillonnage et Résolution) La TFD correspond à l'échantillonnage de la TFTD continue $X(f)$ aux fréquences discrètes normalisées :

$$f_m = \frac{m}{N}, \quad m \in \llbracket 0, N-1 \rrbracket$$

La **résolution fréquentielle** d'une TFD à N points est donc :

$$\Delta f = \frac{1}{N} \quad (\text{ou } \Delta F = \frac{F_e}{N} \text{ en Hertz}) \quad (2.2.3)$$

Propriété (Propriétés spécifiques de la TFD) . Pour des séquences de longueur N :

- **Décalage circulaire temporel** : $x((n - n_0) \bmod N) \xleftrightarrow{\text{TFD}} X(m) e^{-2j\pi \frac{n_0 m}{N}}$.
- **Modulation** : $x(n) e^{2j\pi \frac{m_0 n}{N}} \xleftrightarrow{\text{TFD}} X((m - m_0) \bmod N)$.
- **Convolution circulaire** : $(x \circ y)(n) \xleftrightarrow{\text{TFD}} X(m) \cdot Y(m)$.
- **Produit temporel** : $x(n) \cdot y(n) \xleftrightarrow{\text{TFD}} \frac{1}{N} (X \circ Y)(m)$.

2.2.2 Algorithme de Fourier Rapide (FFT)

Proposition (Gain de complexité) Le calcul direct d'une TFD requiert N^2 multiplications et additions complexes. L'algorithme de la **FFT** (Fast Fourier Transform, e.g. Cooley-Tukey) exploite la décomposition par division de l'horizon pour $N = 2^p$ ($p \in \mathbb{N}$) et abaisse le coût algorithmique à :

$$\mathcal{O}(N \log_2 N) \quad \text{opérations complexes} \quad (2.2.4)$$

2.3 Troncature spectrale et Fenêtres d'apodisation

Traiter un signal numérique sur un horizon fini N revient à multiplier le signal théorique infini par une fenêtre temporelle de pondération $w(n)$:

$$x_{\text{tronqué}}(n) = x(n) \cdot w(n) \xleftrightarrow{\mathcal{F}} X(f) * W(f)$$

Ce produit de convolution par la réponse fréquentielle de la fenêtre $W(f)$ provoque un étalement de l'énergie :

- **Largeur du lobe principal** : régit la résolution fréquentielle (capacité à séparer deux raies proches).
- **Niveau des lobes secondaires** : régit la fuite spectrale (*spectral leakage*) vers les fréquences voisines.

Proposition (Comparatif des fenêtres usuelles d'apodisation sur N points)

Fenêtre	Définition temporelle $w(n), n \in \llbracket 0, N-1 \rrbracket$	Lobe principal	Atténuation
Rectangulaire	$w(n) = 1$	$\frac{2}{N}$	-13 dB
Triangulaire	$w(n) = 1 - \frac{ n-(N-1)/2 }{(N-1)/2}$	$\frac{4}{N}$	-25 dB
Hamming	$w(n) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right)$	$\frac{4}{N}$	-41 dB
Blackman	$w(n) = 0.42 - 0.5 \cos\left(\frac{2\pi n}{N-1}\right) + 0.08 \cos\left(\frac{4\pi n}{N-1}\right)$	$\frac{6}{N}$	-57 dB

Chapitre 3

Filtrage par Transformée de Fourier

Contents

3.1 Transformée en Z	226
3.1.1 Définition et Domaine de Convergence (DC)	226
3.1.2 Propriétés de la Transformée en Z	227
3.2 Description des SLIT et Fonctions de Transfert	227
3.2.1 Équation aux différences linéaires à coefficients constants	227
3.2.2 Fonction de transfert en Z et Réponse fréquentielle	227
3.2.3 Classification : Filtres RIF vs Filtres RII	228
3.3 Filtres sélectifs et Synthèse de filtres RIF	228
3.3.1 Filtres sélectifs idéaux	228
3.3.2 Synthèse des filtres RIF par la méthode des fenêtres	228

3.1 Transformée en Z

3.1.1 Définition et Domaine de Convergence (DC)

Définition (Transformée en Z) . La transformée en Z d'un signal discret $(x_n)_{n \in \mathbb{Z}}$ est la fonction de la variable complexe $z \in \mathbb{C}$ définie par :

$$X(z) = \sum_{n=-\infty}^{+\infty} x(n)z^{-n} \quad (3.1.1)$$

$X(z)$ n'est définie que sur son **Domaine de Convergence (DC)** (ou *Region of Convergence, ROC*), l'ensemble des points $z \in \mathbb{C}$ pour lesquels la série converge absolument.

Propriété (Topologie du Domaine de Convergence) . La forme du DC dépend de la nature temporelle du signal :

- **Signal causal** ($x(n) = 0, \forall n < 0$) : Le DC est l'extérieur d'un disque, $|z| > R_{\max}$.
- **Signal anti-causal** ($x(n) = 0, \forall n > 0$) : Le DC est l'intérieur d'un disque, $|z| < R_{\min}$.

- **Signal bilatère** (support infini des deux côtés) : Le DC est une couronne circulaire fermée $R_{\min} < |z| < R_{\max}$.

Exemple Pour le signal $x(n) = a^n u(n)$ avec $a \in \mathbb{C}$:

$$X(z) = \sum_{n=0}^{+\infty} a^n z^{-n} = \sum_{n=0}^{+\infty} (az^{-1})^n = \frac{1}{1 - az^{-1}} = \frac{z}{z - a}, \quad \text{pour } |az^{-1}| < 1 \iff |z| > |a|$$

3.1.2 Propriétés de la Transformée en Z

Propriété (Propriétés fondamentales) . Soient $x(n) \xleftrightarrow{Z} X(z)$ et $y(n) \xleftrightarrow{Z} Y(z)$:

- **Linéarité** : $\alpha x(n) + \beta y(n) \xleftrightarrow{Z} \alpha X(z) + \beta Y(z)$.
- **Théorème du retard** :

$$\boxed{x(n - k) \xleftrightarrow{Z} z^{-k} X(z)} \quad (3.1.2)$$

L'opérateur z^{-1} modélise l'élément unitaire de retard mémoire.

- **Convolution** : $(x * y)(n) \xleftrightarrow{Z} X(z) \cdot Y(z)$.

3.2 Description des SLIT et Fonctions de Transfert

3.2.1 Équation aux différences linéaires à coefficients constants

Tout système linéaire invariant par décalage discret est régi par une équation aux différences reliant l'entrée $x(n)$ et la sortie $y(n)$:

$$y(n) = \sum_{i=0}^N a_i x(n - i) - \sum_{j=1}^M b_j y(n - j) \quad (3.2.1)$$

3.2.2 Fonction de transfert en Z et Réponse fréquentielle

En appliquant la transformée en Z aux deux membres de l'équation :

$$Y(z) = \sum_{i=0}^N a_i z^{-i} X(z) - \sum_{j=1}^M b_j z^{-j} Y(z) \iff Y(z) \left[1 + \sum_{j=1}^M b_j z^{-j} \right] = X(z) \sum_{i=0}^N a_i z^{-i}$$

Définition (Fonction de transfert) . La fonction de transfert $H(z)$ du système s'écrit sous forme rationnelle :

$$\boxed{H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{i=0}^N a_i z^{-i}}{1 + \sum_{j=1}^M b_j z^{-j}}} \quad (3.2.2)$$

Remarque (Lien avec la réponse fréquentielle) Si le cercle unité $|z| = 1$ est inclus dans le domaine de convergence du système, la réponse fréquentielle harmonique s'obtient par l'évaluation en $z = e^{j2\pi f T_e}$ (avec $T_e = 1/F_e$) :

$$H(f) = H(z) \Big|_{z=e^{j2\pi f T_e}} = \frac{\sum_{i=0}^N a_i e^{-j2\pi f i T_e}}{1 + \sum_{j=1}^M b_j e^{-j2\pi f j T_e}} \quad (3.2.3)$$

3.2.3 Classification : Filtres RIF vs Filtres RII

Proposition (Comparaison structurelle) • **Filtres RIF (Réponse Impulsionnelle Finie / Non-récurrents)** :

- Tous les coefficients récurrents sont nuls : $\forall j \geq 1, b_j = 0$.
- Équation temporelle : $y(n) = \sum_{i=0}^N a_i x(n-i)$.
- Réponse impulsionnelle finie : $h(n) = a_n$ pour $0 \leq n \leq N$, et $h(n) = 0$ ailleurs.
- Fonction de transfert : $H(z) = \sum_{i=0}^N a_i z^{-i}$ (polynôme en z^{-1} , aucun pôle non nul).
- **Propriétés** : Stabilité inconditionnelle, possibilité de concevoir des filtres à phase rigoureusement linéaire.

• **Filtres RII (Réponse Impulsionnelle Infinie / Récurrents)** :

- Au moins un coefficient $b_j \neq 0$.
- La sortie dépend des valeurs de sortie passées (rétroaction).
- $H(z)$ admet des pôles non nuls.
- **Condition de stabilité** : Tous les pôles de $H(z)$ doivent être strictement situés à l'intérieur du cercle unité ($|p_k| < 1$).

3.3 Filtres sélectifs et Synthèse de filtres RIF

3.3.1 Filtres sélectifs idéaux

Un filtre sélectif sélectionne des composantes spectrales. Les 4 gabarits idéaux symétriques sur $[-1/2, 1/2]$ sont :

- **Passe-Bas (PB)** de fréquence de coupure f_c : $|H(f)| = 1$ si $|f| \leq f_c$, 0 sinon.
- **Passe-Haut (PH)** : $|H_{PH}(f)| = 1 - |H_{PB}(f)|$.
- **Passe-Bande** entre f_{c1} et f_{c2} : $|H(f)| = 1$ si $f \in [-f_{c2}, -f_{c1}] \cup [f_{c1}, f_{c2}]$, 0 sinon.
- **Coupe-Bande (CB)** : $|H_{CB}(f)| = 1 - |H_{PBande}(f)|$.

3.3.2 Synthèse des filtres RIF par la méthode des fenêtres

Les filtres idéaux ci-dessus possèdent une réponse impulsionnelle infinie et non causale de type sinus cardinal (ex. pour le passe-bas : $h_{id}(n) = 2f_c \text{sinc}(2\pi f_c n)$). Ils sont donc physiquement irréalisables.

Définition (Méthode des fenêtres) . Pour synthétiser un filtre RIF causal d'ordre N :

1. Calcul analytique de la réponse idéale infinie par TF inverse : $h_{id}(n) = \int_{-1/2}^{1/2} H_{id}(f) e^{j2\pi f n} df$.
2. Troncature et apodisation par une fenêtre finie $w(n)$ d'ordre N :

$$h(n) = h_{id}(n - N/2) \cdot w(n), \quad n \in \llbracket 0, N-1 \rrbracket \quad (3.3.1)$$

Le décalage de $N/2$ assure la causalité du filtre synthétisé.

Chapitre 4

Traitement d'images 2D

Contents

4.1	Numérisation, Représentation de l'Image 2D et Traitements Points-à-Points	230
4.1.1	Échantillonnage, Quantification et Modèle matriciel	230
4.1.2	Traitements Ponctuels	230
4.1.3	Histogramme	231
4.1.4	Convolution 2D	231
4.2	Premiers filtres spatiaux	232
4.2.1	Filtres de lissage	233
4.2.2	Filtres de réhaussement de contours (Dérivée seconde)	233
4.2.3	Opérateur Gradient et Filtres de Sobel (Dérivée première)	234
4.3	Analyse fréquentielle bidimensionnelle (TFD 2D)	234
4.3.1	Définition et conventions	234
4.3.2	Centrage du spectre et propriétés spatiales	235
4.4	Filtrage Sélectif et Réhaussement fréquentiel 2D	236
4.4.1	Filtres Passe-Bas 2D (Lissage fréquentiel)	236
4.4.2	Filtres Passe-Haut 2D et Réhaussement	236
4.5	Modélisation des Dégradations et Filtres de Débruitage	237
4.5.1	Modèles de dégradation	237
4.5.2	Filtres moyennneurs non linéaires (Restauration spatiale)	238
4.6	Bruit périodique, Problèmes Inverses et Défloutage	239
4.6.1	Restauration fréquentielle pour bruit périodique	239
4.6.2	Restauration d'une image floutée (Problèmes inverses)	240

Ce chapitre introduit la plupart des concepts du traitement d'images. Après avoir décrit les principales étapes de numérisation d'une image ainsi que les métriques usuelles de ses objets, nous introduisons des filtres plus évolués à l'aide de la convolution. La seconde partie de ce chapitre détaille les opérations de filtrage et de réhaussement possible grâce au domaine fréquentiel d'une image, caractérisé par son spectre. Nous concluons par une brève présentation des bruits périodiques et des problèmes inverses.

4.1 Numérisation, Représentation de l'Image 2D et Traitements Points-à-Points

Cette première section pose les généralités du traitement d'images numériques. Nous commençons par définir ce qu'est une image numérique ainsi que le processus d'échantillonnage. Après avoir abordé quelques opérations ponctuelles, nous introduisons la convolution ainsi que ses propriétés.

4.1.1 Échantillonnage, Quantification et Modèle matriciel

Nous présentons ici la méthode de représentation d'une image en deux dimensions. À l'origine, une image peut se représenter comme une application continue $i : (x, y) \rightarrow \mathbb{R}$. Pour stocker notre image, nous pouvons directement stocker la fonction i . Or il n'est pas possible de définir mathématiquement une fonction pour chaque image possible, on utilise donc le principe *d'échantillonnage spatial* : on discrétise l'image en *pixels* que l'on pourra stocker dans un emplacement mémoire fini.

Définition (Échantillonnage Spatial) . Soit $i : (x, y) \rightarrow \mathbb{R}$ une image continue. On appelle *processus d'échantillonnage* l'opération qui consiste à discrétiser celle-ci en une matrice I de taille $M \times N$, à l'aide de deux pas réguliers d_x, d_y tout en respectant la condition de Shannon :

$$i(m, n) = i(md_x, nd_y). \quad (4.1.1)$$

Le second processus consiste à projeter les valeurs continues de cette image discrétisée sur un ensemble discret de L niveaux de gris. Généralement on prends $L = 2^k$ niveaux de gris si l'on dispose de k bits pour coder chaque pixel. On appellera la résultante une *image numérique*.

Définition (Image matricielle et Pixel) . Une image numérique monochrome *codée* sur k bits est une matrice I de M lignes et N colonnes telle que pour tout $(x, y) \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket$, $I(x, y) \in \llbracket 0, 2^k - 1 \rrbracket$ représente un *pixel*.

En fonction des caractéristiques souhaitées de notre image, on peut coder la valeur de chaque pixel différemment. Il existe donc différents types d'images :

- Les *images binaires* : chaque pixel peut contenir que deux valeurs possibles : $I(x, y) \in \{0, 1\}$ (1 bit par pixel).
- Les *images à niveaux de gris* : $I(x, y) \in \llbracket 0, 255 \rrbracket$. Pour 256 valeurs différentes, il est nécessaire d'allouer 8 bits à chaque pixel en mémoire.
- Les *images en couleur (RGB)* : Chaque pixel est codé sous trois valeurs différentes correspondant à la quantité de couleur rouge, verte et bleue composant celui-ci :

$$I(x, y) = \begin{pmatrix} R(x, y) \\ G(x, y) \\ B(x, y) \end{pmatrix} \in \llbracket 0, 255 \rrbracket^3$$

Une telle image sera composée de 3 *canaux* (un pour chaque couleur) et sera stockée sous forme de *tenseur* de dimension $3 \times M \times N$.

4.1.2 Traitements Ponctuels

Chaque image étant stockée sous la forme d'une matrice, on peut dès à présent introduire quelques opérations possibles sur nos images. On parle d'opération point-à-point lorsque l'on effectue la même opération du chaque point de l'image de manière indépendante.

Proposition (Transformations ponctuelles usuelles) Une transformation ponctuelle applique une fonction de transfert $s = T(r)$ à chaque niveau de gris d'entrée $r \in \llbracket 0, L - 1 \rrbracket$ (avec $L = 256$ pour une image 8 bits) :

Transformation	Formule $s = T(r)$	Effet / Paramètres
Luminosité	$s = \text{tronc}(r + g)$	$g \in \mathbb{Z}$ ($g > 0$ éclaircit, $g < 0$ assombrit)
Contraste linéaire	$s = \text{tronc}(a[r - 127] + 127)$	$a > 1$ amplifie, $0 \leq a < 1$ réduit
Négatif	$s = (L - 1) - r = 255 - r$	Inverse la dynamique des intensités
Seuillage binaire	$s = \begin{cases} 1 & \text{si } r \geq \text{seuil} \\ 0 & \text{si } r < \text{seuil} \end{cases}$	Binarisation et segmentation du fond
Logarithmique	$s = c \log(1 + r)$	Étend les tons sombres, compresse les clairs
Loi de puissance (Gamma)	$s = c r^\gamma$	$\gamma < 1$ éclaircit, $\gamma > 1$ assombrit

4.1.3 Histogramme

Comme en statistiques, l'histogramme d'une image permet de représenter la quantité de chaque valeur possible des pixels sur un axe.

Définition (Histogramme, ddp et Fonction de répartition) . Pour une image I de taille $M \times N$ comportant $L = 256$ niveaux de gris, on définit son *histogramme* par l'application h telle que :

$$\forall g \in \llbracket 0, L - 1 \rrbracket, \quad h(g) = |\{r \in I \mid r = g\}|. \quad (4.1.2)$$

On définit sa *densité de probabilité* p par son histogramme normalisé par le nombre de pixels qu'elle contient :

$$p(g) = \frac{h(g)}{M \times N}, \quad \text{avec} \quad \sum_{g=0}^{L-1} p(g) = 1. \quad (4.1.3)$$

Enfin, sa *fonction de répartition* $\mathbb{P}$ en g est définie par la somme cumulée de la valeur de sa densité sur ses g premiers termes :

$$P(g) = \sum_{k=0}^g p(k) = \frac{1}{M \times N} \sum_{k=0}^g h(k). \quad (4.1.4)$$

Ces données permettent d'avoir un aperçu de la répartition des couleurs (ou niveaux de gris) sur une image. Son histogramme permet notamment de savoir si une image est plutôt sombre ou plutôt claire.

4.1.4 Convolution 2D

Définition (Convolution bidimensionnelle) . Soient deux signaux bidimensionnels A et B (représentant par exemple une image et un filtre). On définit le *produit de convolution 2D discret* entre A et B par :

$$(A * B)(k, l) = \sum_{x=-\infty}^{+\infty} \sum_{y=-\infty}^{+\infty} A(x, y) B(k - x, l - y). \quad (4.1.5)$$

Propriété (Propriétés de la convolution 2D) . La convolution possède plusieurs propriétés importantes à connaître car très utiles. Pour toutes images A, B, C on a :

- *Commutativité* : $A * B = B * A$.
- *Associativité* : $A * (B * C) = (A * B) * C$.
- *Distributivité* : $A * (B + C) = (A * B) + (A * C)$.

Lors du passage du masque sur les bords de l'image, les valeurs extérieures sont interpolées selon trois stratégies :

- **Zero-padding** : assignation de zéros en dehors de l'image (simple mais crée un artéfact de bordure sombre).
- **Extension périodique** : réplication de l'image par pavage toroïdal (en accord avec la TFD 2D).
- **Extension miroir (symétrie)** : réplication par réflexion par rapport au bord (atténue fortement les discontinuités de bordure).

Définition (Filtrage spatial par masque) . En pratique, on filtre une image d'entrée f à l'aide d'un *masque* (ou *noyau*) fini w de dimensions impaires $(2a + 1) \times (2b + 1)$ centré en $(0, 0)$. L'image filtrée g s'écrit alors :

$$g(x, y) = (f * w)(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (4.1.6)$$

De façon générale, si le masque n'est pas déjà normalisé, on divise par la somme de ses coefficients afin de conserver l'énergie moyenne de l'image :

$$g(x, y) = \frac{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)}{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)} \quad (4.1.7)$$

Remarque L'opération spatiale directe avec $f(x + s, y + t)$ est formellement une *corrélacion spatiale*. Pour correspondre exactement au produit de convolution théorique $f * w$, le masque w doit être retourné horizontalement et verticalement (rotation de 180°) avant l'application. Pour la majorité des masques de filtrage usuels (moyenneur, Laplacien, Gaussien), le noyau est symétrique par rapport à son centre, rendant convolution et corrélation strictement équivalentes.

4.2 Premiers filtres spatiaux

À partir de l'opération de convolution par masque discret, on peut concevoir une grande variété de filtres spatiaux pour modifier localement les propriétés de l'image.

4.2.1 Filtres de lissage

Les filtres de lissage atténuent les variations rapides d'intensité afin de réduire le bruit, au détriment de la netteté des contours :

- **Filtre moyennneur (linéaire)** : applique un masque uniforme normalisé de coefficients $w(s, t) = \frac{1}{(2a+1)(2b+1)}$. Pour une fenêtre standard 3×3 :

$$w = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (4.2.1)$$

Ce filtre réduit efficacement le bruit blanc gaussien mais floute les contours de l'image.

- **Filtre médian (non-linéaire)** : remplace la valeur du pixel central par la valeur médiane de l'ensemble des pixels ordonnés de son voisinage $\mathcal{V}$:

$$J(x, y) = \text{médiane}\{I(x + s, y + t) \mid (s, t) \in \mathcal{V}\} \quad (4.2.2)$$

Ce traitement est optimal pour éliminer le bruit impulsif (« poivre et sel ») sans étaler les arêtes nettes.

4.2.2 Filtres de réhaussement de contours (Dérivée seconde)

L'opération de réhaussement est principalement utilisée pour augmenter le contraste et la netteté d'une image, notamment pour corriger un flou spatial. Les contours correspondent à de fortes variations locales d'intensité, mises en valeur mathématiquement par des opérateurs différentiels. Le *Laplacien* est historiquement utilisé pour les processus de réhaussement.

Définition (Laplacien continu) . Soit $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ une fonction différentiable. On définit son *Laplacien* $\nabla^2 f$ comme la somme de ses dérivées partielles secondes :

$$\nabla^2 f(x, y) = \frac{\partial^2 f(x, y)}{\partial x^2} + \frac{\partial^2 f(x, y)}{\partial y^2} \quad (4.2.3)$$

Cette application donne une mesure quantitative de la courbure locale de f autour du point (x, y) . Tandis que le gradient donne l'orientation et la pente de ses variations, le Laplacien indique à quel point la valeur de la fonction en un point s'écarte de la moyenne de son voisinage. Une valeur positive signifie que le point est en dessous de cette moyenne locale, une valeur négative qu'il est au-dessus, et une valeur nulle indique une zone localement plane.

Sur une image numérique échantillonnée, on discrétise les dérivées secondes par des différences finies centrées :

$$\frac{\partial^2 f}{\partial x^2} \approx f(x + 1, y) + f(x - 1, y) - 2f(x, y) \quad (4.2.4)$$

$$\frac{\partial^2 f}{\partial y^2} \approx f(x, y + 1) + f(x, y - 1) - 2f(x, y) \quad (4.2.5)$$

En sommant ces deux composantes, on obtient l'expression discrète du Laplacien :

$$\nabla^2 f(x, y) = f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1) - 4f(x, y) \quad (4.2.6)$$

Proposition (Masques du Laplacien et Réhaussement spatial) Ici sont proposées différentes implémentations des masques du Laplacien :

- **Masques de convolution** : le Laplacien s'implémente directement par un noyau de convolution spatial à 4 voisins, ou à 8 voisins en intégrant les dérivées secondes diagonales :

$$\nabla_4^2 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \nabla_8^2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (4.2.7)$$

Le coefficient central négatif compare l'intensité du pixel courant à la somme de ses voisins.

- **Image réhaussée** : le Laplacien isole les contours mais élimine les composantes lentes (le fond continu de l'image). Pour restaurer la dynamique globale tout en accentuant les discontinuités, on soustrait le Laplacien à l'image originale :

$$g(x, y) = f(x, y) - \nabla^2 f(x, y) \quad (4.2.8)$$

Cette soustraction se condense en un masque spatial unique à coefficient central renforcé :

$$w_{\text{réhauss}} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad (4.2.9)$$

- **Masquage flou (*Unsharp Masking*)** : généralisation introduisant un coefficient $A \geq 1$ pour moduler le contraste du fond par rapport aux contours :

$$g(x, y) = Af(x, y) - \nabla^2 f(x, y) \quad (4.2.10)$$

4.2.3 Opérateur Gradient et Filtres de Sobel (Dérivée première)

Le gradient d'une fonction 2D est le vecteur pointant vers la plus forte variation locale :

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} G_x \\ G_y \end{bmatrix} \quad (4.2.11)$$

La norme de ce vecteur mesure l'intensité de la transition spatiale, approchée en calcul discret par $\|\nabla f\| \approx |G_x| + |G_y|$ pour limiter les calculs de racines carrées.

Définition (Opérateur de Sobel) . Le filtre de Sobel estime ce gradient à l'aide de deux masques orthogonaux de taille 3×3 combinant dérivation discrète et lissage transverse pour limiter la sensibilité au bruit :

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (4.2.12)$$

Le masque G_x met en évidence les contours verticaux, tandis que G_y détecte les contours horizontaux.

4.3 Analyse fréquentielle bidimensionnelle (TFD 2D)

Cette section introduit une opération fondamentale dans le domaine du traitement d'image qu'est la *Transformée de Fourier 2d*. Cette opération permet d'exprimer le contenu d'une image dans le *domaine spectral* et donne une répartition en deux dimensions de l'intensité des fréquences qui composent une image.

4.3.1 Définition et conventions

Définition (TFD 2D et TFD 2D Inverse) . Soit $f(x, y)$ une image numérique de dimensions $M \times N$, où $x \in \llbracket 0, M-1 \rrbracket$ et $y \in \llbracket 0, N-1 \rrbracket$. On définit la *Transformée de Fourier Discrète 2D* de l'image $f(x, y)$, notée $F(u, v)$, par :

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cdot \exp \left(-2j\pi \left(\frac{ux}{M} + \frac{vy}{N} \right) \right) \quad (4.3.1)$$

pour tout $u \in \llbracket 0, M-1 \rrbracket$ et $v \in \llbracket 0, N-1 \rrbracket$.

La *Transformée de Fourier Discrète 2D Inverse* (TFDI) permet la reconstruction exacte de l'image spatiale :

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \cdot \exp \left(2j\pi \left(\frac{ux}{M} + \frac{vy}{N} \right) \right) \quad (4.3.2)$$

Proposition (Spectre d'amplitude, Densité spectrale et Phase) La Transformée de Fourier d'un signal (image ou son) vit dans le domaine *spectral* (i.e complexe). Chacun de ses termes se décompose donc en deux sous-termes :

$$F(u, v) = \underset{\text{partie réelle}}{\text{Re}(u, v)} + j \underset{\text{partie imaginaire}}{\text{Im}(u, v)} .$$

On peut alors définir différentes métriques pour ce spectre :

- Le **spectre d'amplitude** (module) : $|F(u, v)| = \sqrt{R(u, v)^2 + I(u, v)^2}$.
- Le **spectre de phase** : $\phi(u, v) = \arctan \left(\frac{I(u, v)}{R(u, v)} \right)$.
- La **densité spectrale de puissance (DSP)** : $P(u, v) = |F(u, v)|^2 = R(u, v)^2 + I(u, v)^2$.

La phase $\phi(u, v)$ transporte l'essentiel de l'information structurelle et géométrique de l'image (contours, bords des objets). La reconstruction d'une image à partir de sa phase seule ($f' = \text{Re}[\mathcal{F}^{-1}\{e^{j\phi}\}]$) préserve les formes perceptibles, alors que la reconstruction à partir du module seul ($f' = \mathcal{F}^{-1}\{|F|\}$) détruit l'intelligibilité spatiale.

4.3.2 Centrage du spectre et propriétés spatiales

Proposition (Centrage du spectre – fftshift) Par défaut, l'origine fréquentielle continue $F(0, 0)$ est positionnée dans le coin supérieur gauche ($u = 0, v = 0$). Pour décaler la fréquence nulle au centre du spectre ($u = M/2, v = N/2$), on multiplie l'image spatiale par $(-1)^{x+y}$ avant transformation :

$$\mathcal{F} [f(x, y) \cdot (-1)^{x+y}] = F \left(u - \frac{M}{2}, v - \frac{N}{2} \right) \quad (4.3.3)$$

Proposition (Relation d'incertitude et orientation) La transformée de Fourier induit des comportements caractéristiques sur la forme des object et leur positions :

- **Principe d'incertitude** : si $\Delta x \Delta y$ est l'extension spatiale d'un motif et $\Delta u \Delta v$ son étalement fréquentiel, alors :

$$\Delta x \Delta y \cdot \Delta u \Delta v \geq \frac{1}{16\pi^2} \quad (4.3.4)$$

Un motif spatial très étroit possède un large spectre fréquentiel, et réciproquement.

- **Orientation** : une famille de lignes ou un contour incliné selon un angle θ dans l'espace engendre des composantes fréquentielles dirigées orthogonalement ($\theta + \pi/2$) dans le plan de Fourier.

4.4 Filtrage Sélectif et Réhaussement fréquentiel 2D

Lorsque l'on utilise le *spectre* d'une image, on parle souvent sur *spectre d'amplitude*. Ce dernier représente les basses fréquences au centre du spectre $((M/2, N/2))$ et les hautes fréquences sur les contours de celui-ci. On peut alors effectuer des opérations simples sur les pixels en considérant leur distance au centre du spectre $D(u, v)$: la distance euclidienne d'une fréquence (u, v) au centre du spectre :

$$D(u, v) = \sqrt{\left(u - \frac{M}{2}\right)^2 + \left(v - \frac{N}{2}\right)^2} \quad (4.4.1)$$

On définit l'opération de filtrage de la même façon que pour les signaux 2D :

Définition (Filtrage) . Soit $H \in \mathcal{M}_{M \times N}(\mathbb{R})$ une matrice de même dimension que l'image I à filtrer et J sa transformée de fourier. L'image filtrée I_f par H est définie par :

$$\forall (x, y) \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket, \quad I_f(x, y) = \text{Re} [\text{TF}^{-1}(H \cdot J)](x, y). \quad (4.4.2)$$

4.4.1 Filtres Passe-Bas 2D (Lissage fréquentiel)

Un filtre passe-bas atténue les hautes fréquences (bruit, contours) au profit des variations lentes. Il permet d'atténuer les contours d'une image en supprimant les bords de son spectre. Soit D_0 la fréquence de coupure :

- **Passe-bas idéal** :

$$H(u, v) = \begin{cases} 1 & \text{si } D(u, v) \leq D_0 \\ 0 & \text{si } D(u, v) > D_0 \end{cases} \quad (4.4.3)$$

Provoque de sévères oscillations spatiales (phénomène de Gibbs / effet de cerclage) en raison de la discontinuité abrupte.

- **Passe-bas de Butterworth d'ordre n** :

$$H(u, v) = \frac{1}{1 + \left[\frac{D(u, v)}{D_0}\right]^{2n}} \quad (4.4.4)$$

Transition progressive qui élimine le phénomène de cerclage pour les ordres faibles ($n \leq 2$).

- **Passe-bas Gaussien** :

$$H(u, v) = e^{-\frac{D^2(u, v)}{2D_0^2}} \quad (4.4.5)$$

Transition parfaitement lisse sans aucune oscillation spatiale.

4.4.2 Filtres Passe-Haut 2D et Réhaussement

Les filtres passe-haut préservent les contours et éliminent la composante continue. Ils sont définis comme les opposés des filtres passe-bas. Ainsi, à chaque filtre passe-bas, lui correspond sont filtre passe-haut :

$$H_{\text{PH}}(u, v) = 1 - H_{\text{PB}}(u, v) \quad (4.4.6)$$

On en déduit les formes idéales, de Butterworth et Gaussienne.

Remarque Les filtres passe-haut sont généralement utilisés dans le *réhaussement* d'images.

Proposition (Réhaussement Fréquentiel) Dans le domaine fréquentiel, on peut alors appliquer n'importe quel filtre passe-haut pour accentuer les contours d'une image. Pour cela, on peut affiner les effets du filtre à l'aide de paramètres $a \geq 0$ et $b > a$ tels que :

$$\boxed{H_{\text{réhauss}}(u, v) = a + b H_{\text{PH}}(u, v).} \quad (4.4.7)$$

Typiquement $0,25 \leq a \leq 0,5$ et $1,5 \leq b \leq 2$. Le paramètre a conserve le fond de l'image (basses fréquences), tandis que b amplifie les contours.

4.5 Modélisation des Dégradations et Filtres de Débruitage

Dans cette section, on s'intéresse à des images bruitées : on parle de *dégradation*. Après avoir défini la notion de *bruit* sur une image et comment les simuler, nous verrons différents procédés pour les altérer/supprimer par un processus de *débruitage*.

4.5.1 Modèles de dégradation

Définition (Phénomène de bruitage) . Soit f un signal (image ou son), h une *fonction de dégradation* et n un *terme de bruit*. La dégradation g de f par un *système linéaire invariant avec bruit additif* est défini comme le signal :

$$\underbrace{g = h * f + n}_{\text{Domaine Spatial}} \iff \underbrace{G = HF + N}_{\text{Domaine Fréquentiel}} \quad (4.5.1)$$

où G, H et F sont leurs transformées de Fourier respectives. On dira que h est la *fonction de dégradation* et n le *terme de bruit*.

On identifiera souvent le bruit à un processus aléatoire que l'on peut caractériser par une densité de probabilité. Une métrique intéressante pour quantifier l'influence du bruit sur un signal est d'utiliser le rapport de leur puissances.

Définition (RSB – *Signal-to-Noise-Ratio*) . Soit f un signal (non bruité) et n un bruit. On définit le *ratio signal/bruit* par le rapport de leur deux puissances :

$$RSB = \frac{P_s}{P_b} = \frac{\sum_x |f(x)|^2}{\sum_x |n(x)|^2}. \quad (4.5.2)$$

Remarque Dans le cas d'un bruit centré $\mathbb{E}(n) = 0$ de variance σ_n^2 , sa puissance coïncide exactement avec sa variance : $P_b = \mathbb{E}(n^2) = \sigma_n^2$.

Proposition (Densités de probabilité usuelles des bruits) Soit z la variable aléatoire représentant l'intensité bruitée du pixel. Les modèles statistiques standards sont résumés ci-dessous :

Type de bruit	Densité de probabilité $p(z)$	Moyenne $\bar{z}$	Variance σ^2
Gaussien	$\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(z-\bar{z})^2}{2\sigma^2}\right)$	$\bar{z}$	σ^2
Uniforme ($a \leq z \leq b$)	$\frac{1}{b-a}$	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
Rayleigh ($z \geq a$)	$\frac{2}{b}(z-a) \exp\left(-\frac{(z-a)^2}{b}\right)$	$a + \sqrt{\frac{\pi b}{4}}$	$\frac{b(4-\pi)}{4}$
Gamma / Erlang ($z \geq 0$)	$\frac{a^b z^{b-1}}{(b-1)!} e^{-az}$	$\frac{b}{a}$	$\frac{b}{a^2}$
Exponentiel ($z \geq 0$)	ae^{-az}	$\frac{1}{a}$	$\frac{1}{a^2}$
Poivre et Sel	$\begin{cases} P_p & \text{si } z = p \text{ (noir)} \\ P_s & \text{si } z = s \text{ (blanc)} \\ 0 & \text{sinon} \end{cases}$	—	—

4.5.2 Filtres moyennneurs non linéaires (Restauration spatiale)

Dans cette sous-section, on considère uniquement des dégradation à base de bruit soit de la forme :

$$g = f + n \quad \Longleftrightarrow \quad G = F + N. \quad (4.5.3)$$

Grâce à ces modèles plus simples, il suffirait de soustraire le bruit pour retrouver l'image de départ mais sans la connaissance de ce dernier, on doit se contenter d'opérations plus simples. L'opération la plus simple serait de remplacer chaque pixel par une moyenne de ses pixels voisins. Cette méthode permet de rétablir l'image de départ pour la plupart des modèles de bruits classiques mais a tendance à flouter l'image.

Proposition (Filtres Moyennneurs) Soit S_{xy} une fenêtre d'analyse de taille $m \times n$ définie pour tout (x, y) . Différents filtres moyennneurs existent :

- **Moyennneur géométrique :**

$$\hat{f}(x, y) = \left[\prod_{(s,t) \in S_{xy}} g(s, t) \right]^{\frac{1}{mn}} \quad (4.5.4)$$

Ce filtre lisse l'image mais préserve mieux les détails qu'une moyenne arithmétique naïve.

- **Moyennneur harmonique :**

$$\hat{f}(x, y) = \frac{mn}{\sum_{(s,t) \in S_{xy}} \frac{1}{g(s, t)}} \quad (4.5.5)$$

filtre efficace sur du bruit sel ou gaussien mais inefficace sur un bruit de type poivre (pixels noirs).

- **Moyennneur contra-harmonique d'ordre R :**

$$\hat{f}(x, y) = \frac{\sum_{(s,t) \in S_{xy}} g(s, t)^{R+1}}{\sum_{(s,t) \in S_{xy}} g(s, t)^R} \quad (4.5.6)$$

Si $R > 0$, élimine le bruit poivre ; si $R < 0$, élimine le bruit sel (ne traite pas les deux simultanément). Pour $R = 0$, on retrouve le filtre arithmétique ; pour $R = -1$, le filtre harmonique.

Pour aller plus loin, on peut essayer de définir un filtre qui s'adapte localement au bruit appliqué. On considère toujours un signal bruité de la forme $g(x, y) = f(x, y) + n(x, y)$ tel que le bruit n possède une espérance nulle et une variance connue σ_n^2 .

Définition (Filtre adaptatif local) . Soit g un signal construit à partir d'un signal f bruité à partir d'un filtre n centré de variance σ_n^2 . On définit le *filtre adaptatif local* par :

$$\hat{f}(x, y) = g(x, y) - \frac{\sigma_n^2}{\sigma_L^2} [g(x, y) - m_L]. \quad (4.5.7)$$

Avec m_L , la *moyenne locale* sur un voisinage S_{xy} de (x, y) et σ_L^2 la variance locale :

$$m_L = \frac{1}{mn} \sum_{(s,t) \in S_{xy}} g(s, t), \quad \sigma_L^2 = \frac{1}{mn} \sum_{(s,t) \in S_{xy}} (g(s, t) - m_L)^2. \quad (4.5.8)$$

Remarque La plupart de ces filtres produisent un aspect flou sur l'image du fait de la moyennisation des pixels. Pour y remédier, on peut utiliser un principe de réhaussement de l'image (voir précédemment).

4.6 Bruit périodique, Problèmes Inverses et Défloutage

Dans cette dernière partie, on s'intéresse à des problèmes plus complexes tels que la suppression de bruits périodiques en reprérant des motifs dans le spectre d'une image. On s'intéresse aussi à la notion de *problèmes inverses* et au défloutage.

4.6.1 Restauration fréquentielle pour bruit périodique

Dans certains processus de capture d'un signal, on peut s'exposer à l'émission d'un bruit *périodique* (signal électromagnétique, son, etc...). Ces interférences périodiques sont facilement reconnaissables dans le domaine spectral car elles se traduisent par des pics impulsionnels isolés et symétriques par rapport à l'origine du spectre.

Pour retirer ces interférences, on applique des filtres réjecteur de bande sur la distance D_0 de l'origine à partir de laquelle apparaît ces motifs dans le spectre.

Proposition Le filtre *réjecteur idéal* annule brutalement les fréquences d'un anneau d'épaisseur W autour de D_0 :

$$H(u, v) = \begin{cases} 0 & \text{si } D_0 - \frac{W}{2} \leq D(u, v) \leq D_0 + \frac{W}{2} \\ 1 & \text{sinon.} \end{cases} \quad (4.6.1)$$

Tandis que le filtre *réjecteur gaussien* applique une transition douce sans créer d'effets de cerclage :

$$H(u, v) = 1 - \exp \left(-\frac{1}{2} \left[\frac{D^2(u, v) - D_0^2}{D(u, v) \cdot W} \right]^2 \right). \quad (4.6.2)$$

Remarque On peut aussi récupérer les fréquences responsables du bruit pour l'analyser en utilisant un *filtre passe-bande complémentaire* simplement obtenu par soustraction :

$$H_{PB} = 1 - H_{\text{Réjecteur}}. \quad (4.6.3)$$

4.6.2 Restauration d'une image floutée (Problèmes inverses)

Une image dégradée par un flou linéaire (défocalisation, bougé de capteur) et parasitée par un bruit additif s'écrit dans le domaine spatial et fréquentiel sous la forme :

$$g(x, y) = h(x, y) * f(x, y) + n(x, y) \iff G(u, v) = H(u, v)F(u, v) + N(u, v). \quad (4.6.4)$$

où $h(x, y)$ représente la réponse impulsionnelle du système de flou (*Point Spread Function* ou PSF) et $n(x, y)$ le terme de bruit.

Si l'on tente d'inverser directement le système à l'aide de l'opérateur $1/H(u, v)$, l'estimation fréquentielle devient :

$$\hat{F}(u, v) = \frac{G(u, v)}{H(u, v)} = F(u, v) + \frac{N(u, v)}{H(u, v)}. \quad (4.6.5)$$

Cette inversion naïve pose un problème dit *mal posé* : la fonction de transfert de flou $H(u, v)$ s'atténue et tend vers zéro aux moyennes et hautes fréquences. La division par des valeurs proches de zéro amplifie alors démesurément le terme de bruit $N(u, v)/H(u, v)$, dégradant totalement l'image restaurée au lieu de la rendre nette.

Pour stabiliser l'inversion, plusieurs méthodes de régularisation sont envisageables.

Proposition (Filtres inverses régularisés) Pour éviter la divergence du terme de bruit, on peut borner l'inversion de deux manières :

- Le *filtre pseudo-inverse tronqué*, qui applique un seuil minimal $T > 0$ sur l'amplitude de $H(u, v)$:

$$\hat{F}(u, v) = \begin{cases} \frac{G(u, v)}{H(u, v)} & \text{si } |H(u, v)| \geq T \\ G(u, v) & \text{sinon.} \end{cases} \quad (4.6.6)$$

- Le *filtre inverse limité aux basses fréquences*, qui tire profit du fait que la composante continue $H(0, 0)$ est importante (moyenne spatiale positive de la PSF). L'inversion est alors strictement restreinte à un disque de rayon D_0 centré sur l'origine fréquentielle :

$$\hat{F}(u, v) = \begin{cases} \frac{G(u, v)}{H(u, v)} & \text{si } D(u, v) \leq D_0 \\ G(u, v) & \text{sinon.} \end{cases} \quad (4.6.7)$$

Théorème (Filtre de Wiener) . Le *filtre de Wiener* propose une solution optimale au sens des moindres carrés en minimisant l'erreur quadratique moyenne globale $\mathbb{E}[|f(x, y) - \hat{f}(x, y)|^2]$. Son expression fréquentielle est donnée par :

$$\hat{F}(u, v) = \left[\frac{1}{H(u, v)} \frac{|H(u, v)|^2}{|H(u, v)|^2 + K} \right] G(u, v) \quad (4.6.8)$$

où la constante K modélise le rapport des densités spectrales de puissance du bruit et du signal :

$$K \approx \frac{P_n(u, v)}{P_f(u, v)} = \frac{1}{\text{RSB}(u, v)}. \quad (4.6.9)$$

Remarque Le facteur d'atténuation $\frac{|H(u, v)|^2}{|H(u, v)|^2 + K}$ effectue un arbitrage continu entre déconvolution et filtrage du bruit :

- En basses fréquences ($|H(u, v)|^2 \gg K$), le signal utile domine le bruit : le terme correcteur tend vers 1 et le filtre se comporte comme l'inverse direct $1/H(u, v)$ pour déflouter l'image.

- En hautes fréquences ($|H(u, v)|^2 \ll K$), le bruit domine le signal : le terme correcteur tend vers 0, ce qui coupe le filtre et empêche l'explosion du bruit.
- Dans le cas idéal sans bruit ($K = 0$), on retrouve exactement le filtre inverse classique.

Chapitre 5

Analyse du son et Reconnaissance vocale

Contents

5.1	Architecture des Systèmes de Reconnaissance Vocale (RAP)	242
5.1.1	Sources de variabilité et chaîne de traitement	242
5.2	Paramétrisation Acoustique : Coefficients MFCC	243
5.3	Modélisation Statistique et Modèles de Mélanges de Gaussiennes (GMM)	243
5.3.1	Cadre Bayésien et Détection d'Activité Vocale (VAD)	243
5.3.2	Mélange de Gaussiennes (GMM) et Algorithme EM	243
5.4	Alignement Temporel Dynamique (DTW)	244
5.5	Modèles de Markov Cachés (HMM / MMC)	245
5.5.1	Formalisme et Hypothèses	245
5.5.2	Algorithme de Viterbi	245
5.6	Modèles Connexionnistes (Réseaux de Neurones Récurrents)	245
5.6.1	Perceptron et Réseau Récurrent Simple (RNN)	245
5.6.2	Cellule LSTM (Long Short-Term Memory)	246
5.6.3	Cellule GRU (Gated Recurrent Unit)	246

5.1 Architecture des Systèmes de Reconnaissance Vocale (RAP)

5.1.1 Sources de variabilité et chaîne de traitement

Un système de Reconnaissance Automatique de la Parole (RAP) extrait une séquence de mots à partir d'un signal acoustique de parole. La difficulté réside dans les variabilités acoustiques (bruit, coarticulation), prosodiques (durée, intonation), lexicales et syntaxiques.

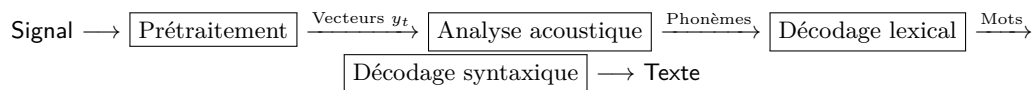


FIGURE 5.1 – Pipeline classique d'un système de reconnaissance de formes vocales.

5.2 Paramétrisation Acoustique : Coefficients MFCC

L'analyse de parole s'effectue sur des trames courtes et stationnaires de durée ≈ 20 ms avec un recouvrement de 50%.

Définition (Algorithme d'extraction des MFCC) . L'obtention des coefficients *Mel-Frequency Cepstral Coefficients* (MFCC) suit la cascade :

1. **Préaccentuation** : filtre passe-haut $y(n) = x(n) - \alpha x(n-1)$ (avec $\alpha \approx 0,97$) pour compenser la chute spectrale d'environ -6 dB/octave due au rayonnement des lèvres.
2. **Fenêtrage de Hamming** : limitation temporelle réduisant la fuite spectrale.
3. **Spectre TFD (FFT)** : calcul de la transformée de Fourier rapide sur chaque trame.
4. **Banc de filtres de Mel** : banc de filtres triangulaires espacés linéairement sous 1000 Hz et logarithmiquement au-delà (échelle psychoacoustique de Mel) :

$$\text{Mel}(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right) \quad (5.2.1)$$

5. **Compression logarithmique** : calcul du logarithme de l'énergie en sortie de chaque filtre.
6. **Transformée en Cosinus Discrète (DCT)** : décorrélation spectrale équivalente à une TFD inverse.

On retient en pratique les $n_0 \leq 13$ à 30 premiers coefficients cepstraux (le premier coefficient c_0 , reflet de l'énergie globale, étant souvent traité séparément).

5.3 Modélisation Statistique et Modèles de Mélanges de Gaussiennes (GMM)

5.3.1 Cadre Bayésien et Détection d'Activité Vocale (VAD)

Pour classer une observation $y \in \mathbb{R}^d$ parmi K classes, la règle du Maximum a Posteriori (MAP) s'écrit :

$$s^*(y) = \arg \max_k P(k|y) = \arg \max_k \frac{P(y|k)P(k)}{P(y)} = \arg \max_k P(y|k)P(k) \quad (5.3.1)$$

Pour la Détection d'Activité Vocale (VAD), l'observation scalaire est l'énergie court-terme $y = \frac{1}{N} \sum s_n^2$. Le test binaire Parole / Non-Parole se réduit à une comparaison à un seuil optimal y^* séparant les deux gaussiennes $\mathcal{N}(m_P, \sigma_P^2)$ et $\mathcal{N}(m_{NP}, \sigma_{NP}^2)$.

5.3.2 Mélange de Gaussiennes (GMM) et Algorithme EM

Pour modéliser la variabilité acoustique d'un son continu, la densité d'émission est décomposée en un mélange de M composantes gaussiennes :

$$P(y|k) = \sum_{i=1}^M \alpha_k^i \mathcal{N}(y; m_k^i, \Sigma_k^i) \quad \text{avec} \quad \sum_{i=1}^M \alpha_k^i = 1 \quad (5.3.2)$$

Théorème (Algorithme Expectation-Maximization (EM)) . L'estimation des paramètres $\{\alpha_i, m_i, \Sigma_i\}$ s'effectue itérativement :

1. **Initialisation** : tirage initial des moyennes par partitionnement de Lloyd / K -means (Vector Quantization - VQ), $\Sigma_i = I$ et $\alpha_i = 1/M$.
2. **Étape E (Estimation)** : calcul des probabilités a posteriori d'appartenance de chaque observation y_t à la gaussienne i :

$$p_k^i(t) = \frac{\alpha_k^i \mathcal{N}(y_t; m_k^i, \Sigma_k^i)}{\sum_{j=1}^M \alpha_k^j \mathcal{N}(y_t; m_k^j, \Sigma_k^j)} \quad (5.3.3)$$

3. **Étape M (Maximisation)** : mise à jour des paramètres du mélange :

$$\bar{\alpha}_k^i = \frac{1}{T} \sum_{t=1}^T p_k^i(t), \quad \bar{m}_k^i = \frac{\sum_{t=1}^T p_k^i(t) y_t}{\sum_{t=1}^T p_k^i(t)}, \quad \bar{\Sigma}_k^i = \frac{\sum_{t=1}^T p_k^i(t) (y_t - \bar{m}_k^i)(y_t - \bar{m}_k^i)^T}{\sum_{t=1}^T p_k^i(t)} \quad (5.3.4)$$

5.4 Alignement Temporel Dynamique (DTW)

La méthode DTW (*Dynamic Time Warping*) mesure la dissemblance entre une séquence d'observation $A = (a_1, \dots, a_N)$ et une référence $B = (b_1, \dots, b_M)$ en absorbant les variations non-linéaires de débit d'élocution.

Définition (Chemin de déformation et contraintes) . Un chemin d'alignement est une suite $C = (c_1, \dots, c_K)$ de cases $c_k = (i_k, j_k) \in \llbracket 1, N \rrbracket \times \llbracket 1, M \rrbracket$ minimisant le coût cumulé :

$$D(A, B) = \min_C \frac{\sum_{k=1}^K w_k d(a_{i_k}, b_{j_k})}{\sum_{k=1}^K w_k} \quad (5.4.1)$$

Le chemin est soumis à trois conditions rigoureuses :

- **Coïncidence des extrémités** : $c_1 = (1, 1)$ et $c_K = (N, M)$.
- **Monotonie temporelle** : $i_k \geq i_{k-1}$ et $j_k \geq j_{k-1}$.
- **Continuité** : pas de saut temporel ($i_k - i_{k-1} \leq 1$ et $j_k - j_{k-1} \leq 1$).

Propriété (Équations de récurrence de Bellman) . Soit $g(i, j)$ le coût cumulé minimal pour atteindre la case (i, j) :

- **Schéma symétrique usuel** ($w_{\text{diag}} = 2, w_{\text{lat}} = 1$) :

$$g(i, j) = \min \begin{cases} g(i-1, j) + d(i, j) \\ g(i-1, j-1) + 2d(i, j) \\ g(i, j-1) + d(i, j) \end{cases} \quad (5.4.2)$$

Le coût final $g(N, M)$ est normalisé par le facteur constant $N + M$.

- **Schéma asymétrique** (alignement projeté sur l'axe i) :

$$g(i, j) = \min \begin{cases} g(i-1, j) + d(i, j) \\ g(i-1, j-1) + d(i, j) \\ g(i, j-1) \end{cases} \quad (5.4.3)$$

Normalisé par la longueur temporelle de l'observation N .

5.5 Modèles de Markov Cachés (HMM / MMC)

5.5.1 Formalisme et Hypothèses

Un MMC du 1er ordre est défini par le triplet $\Lambda = (\Pi, A, B)$:

- L'ensemble fini d'états cachés $Q = \{q_1, \dots, q_N\}$.
- Le vecteur des probabilités initiales $\Pi = (\pi_i)_{1 \leq i \leq N}$ avec $\pi_i = P(q_1 = i)$.
- La matrice de transition $A = (a_{ij})_{1 \leq i, j \leq N}$ avec $a_{ij} = P(s_t = j \mid s_{t-1} = i)$ vérifiant $\sum_j a_{ij} = 1$.
- Les densités de probabilité d'émission $B = \{b_j(o_t)\}$ où $b_j(o_t) = P(o_t \mid s_t = j)$, modélisées par des GMM.

5.5.2 Algorithme de Viterbi

L'algorithme de Viterbi extrait la séquence d'états optimale $S^* = (s_1^*, \dots, s_T^*)$ ayant produit la suite d'observations $O = (o_1, \dots, o_T)$. Pour éviter les sous-dépassements de capacité (*underflow*), on travaille en log-probabilités.

Théorème (Récursion de Viterbi) . Soit $\delta_t(j) = \max_{s_1, \dots, s_{t-1}} \ln P(s_1, \dots, s_t = j, o_1, \dots, o_t \mid \Lambda)$:

1. **Initialisation** ($t = 1$) :

$$\delta_1(i) = \ln \pi_i + \ln b_i(o_1), \quad \psi_1(i) = 0 \quad (5.5.1)$$

2. **Récursion temporelle** ($2 \leq t \leq T$) :

$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) + \ln a_{ij}] + \ln b_j(o_t) \quad (5.5.2)$$

$$\psi_t(j) = \arg \max_{1 \leq i \leq N} [\delta_{t-1}(i) + \ln a_{ij}] \quad (5.5.3)$$

3. **Terminaison et Rétro-traçage (*Backtracking*)** :

$$s_T^* = \arg \max_{1 \leq i \leq N} \delta_T(i), \quad s_t^* = \psi_{t+1}(s_{t+1}^*) \quad \text{pour } t = T-1, \dots, 1 \quad (5.5.4)$$

5.6 Modèles Connexionnistes (Réseaux de Neurones Récurrents)

5.6.1 Perceptron et Réseau Récurrent Simple (RNN)

Dans un RNN, la sortie d'un état caché à l'instant $t-1$ est réinjectée en entrée à l'instant t :

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (5.6.1)$$

Bien qu'adapté aux séquences, le RNN simple souffre de disparition ou explosion du gradient (*vanishing/exploding gradient*) sur les longues dépendances temporelles.

5.6.2 Cellule LSTM (Long Short-Term Memory)

La cellule LSTM compense la perte de gradient grâce à une mémoire interne linéaire C_t (état de cellule) régulée par trois portes sigmoïdales :

$$\text{Porte d'oubli : } F_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (5.6.2)$$

$$\text{Porte d'entrée : } I_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (5.6.3)$$

$$\text{Candidat cellule : } \tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \quad (5.6.4)$$

$$\text{Mise à jour mémoire : } \boxed{C_t = F_t * C_{t-1} + I_t * \tilde{C}_t} \quad (5.6.5)$$

$$\text{Porte de sortie : } O_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (5.6.6)$$

$$\text{État caché délivré : } \boxed{h_t = O_t * \tanh(C_t)} \quad (5.6.7)$$

5.6.3 Cellule GRU (Gated Recurrent Unit)

La cellule GRU simplifie le LSTM en fusionnant l'état de cellule et l'état caché, réduisant le nombre de paramètres à deux portes :

$$\text{Porte de mise à jour (update) : } Z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (5.6.8)$$

$$\text{Porte de réinitialisation (reset) : } R_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (5.6.9)$$

$$\text{État candidat : } \tilde{h}_t = \tanh(W[h_t * h_{t-1}, x_t] + b) \quad (5.6.10)$$

$$\text{État final : } \boxed{h_t = (1 - Z_t) * h_{t-1} + Z_t * \tilde{h}_t} \quad (5.6.11)$$

